TÉCNICO LISBOA

CNV/CC&V MEIC/METI/MECD
Computação em Nuvem e Virtualização

inesc id lisboa



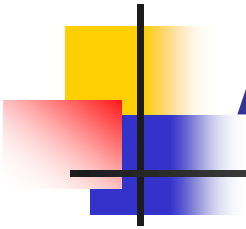
Course Presentation

Prof. Luís Veiga

IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© Luís Veiga 2015-24



Agenda

- Instructors
- Dates and Schedule
- Goals
- Context
- Syllabus
- Bibliography
- Evaluation and Grading
- Homework
- Summary

Instructor *(Lectures)*



- Prof. Luís Antunes Veiga (Associate Professor / Senior Researcher)
  - Técnico Lisboa / INESC-ID - Distributed Parallel and Secure Systems
    - Utility/Cloud Computing (VM scheduling, Docker), Edge Clouds
    - Extension of VMs (Jikes RVM, OpenJDK), Garbage Collection
    - Cloud Storage (HBase), Distributed Data Processing (Map-Reduce, Spark, Flink)
    - Data Replication, Mobile Middleware, Adaptability, Context-Awareness
    - Supervision: 10 PhD completed, 60+ MSc completed
  - FCT Projects: NovaVM (virtual machines), ContexTWA (mobile sensing with cloud data processing)
  - EU Projects: CloudStars (cloud/data/edge), MoTiV (smart cities, mobility detection),
  - Service: ACM EuroSys SC, ACM Middleware SC, IFIP WG6.1 ExCom

http://www.gsd.inesc-id.pt/~lveiga

  - Search:Luís Veiga→ Fénix, Google Scholar, ResearchGate, DBLP
    - do not confuse with the Civil Eng., poet or photographer.

Interested on MSc/PhD on these topics? Let me know.

Instructor *(Labs)*



- Afonso Fernandes
  - Técnico Lisboa / INESC-ID GAIPS
  - Early Stage Researcher at INESC-ID GAIPS
    - Working on a project to develop new Deep Reinforcement Learning methods for Combinatorial Optimization
  - Bachelor in Computer Science and Engineering
  - Master in Computer Science and Engineering
    - Specializations in Artificial Intelligence and Cybersecurity
    - Thesis theme: Robustness of multimodal generative models
  - Past: Computer Programmer Analyst at qubIT
    - Research & development of cryptographic tools and microservices

Instructor *(Labs)*



- Serhii Ivanenko
  - PhD Student (Técnico Lisboa)
  - Early Stage Researcher at INESC-ID
    - Distributed, Parallel and Secure Systems Group
    - Working on a project related to Serverless and language runtimes
      - In collaboration with Oracle Labs
      - Under supervision of Prof. Rodrigo Bruno (https://rodrigo-bruno.github.io/)
  - BSc in Software Engineering (Lviv Polytechnic National University)
  - MSc in Computer Science and Engineering (Técnico Lisboa)
    - Specializations in Software Engineering and Cybersecurity
    - Thesis: Serverless Persistent Resources
  - Past: Student Intern at Oracle Labs
    - Contributed to GraalJS and GraalVM Native Image

Course E-mail

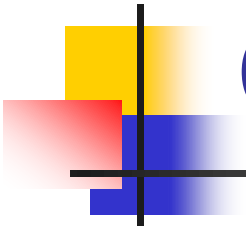
- Course E-mail:
- **meic-cnv@tecnico.ulisboa.pt**
  - Subject: CNV-23-24 - ...
  - Identification: name, student number, lab shift, group number and campus (A / T)
  - Any mail pelase use subject CNV-23-24
    - (very important!)

Dates & Schedule *(Alameda)*

- **Dates :**
  - 1st Exam: June 18th
  - 2nd Exam: July 2nd
  - Project:
    - Project submission deadline: May 31st
    - Checkpoint midway: May 17th
    - Visualization and discussion following weeks
- **Schedule:**
  - Theory classes:
    - Tuesday: **13h30-15h30 (EA2)**
    - Friday : **10h00-12h00 (EA5)**
  - Lab classes:
    - Wednesday: **15h30-18h30 (Lab 1)**
    - Thursday: **12h30-15h30** and **15h30-18h30 (Lab 1)**
    - Friday: **14h00-17h00 (Lab 2)**
    - Group enrollment via web to open soon
      - **Groups with 3 students!**

Dates & Schedule *(Taguspark)*

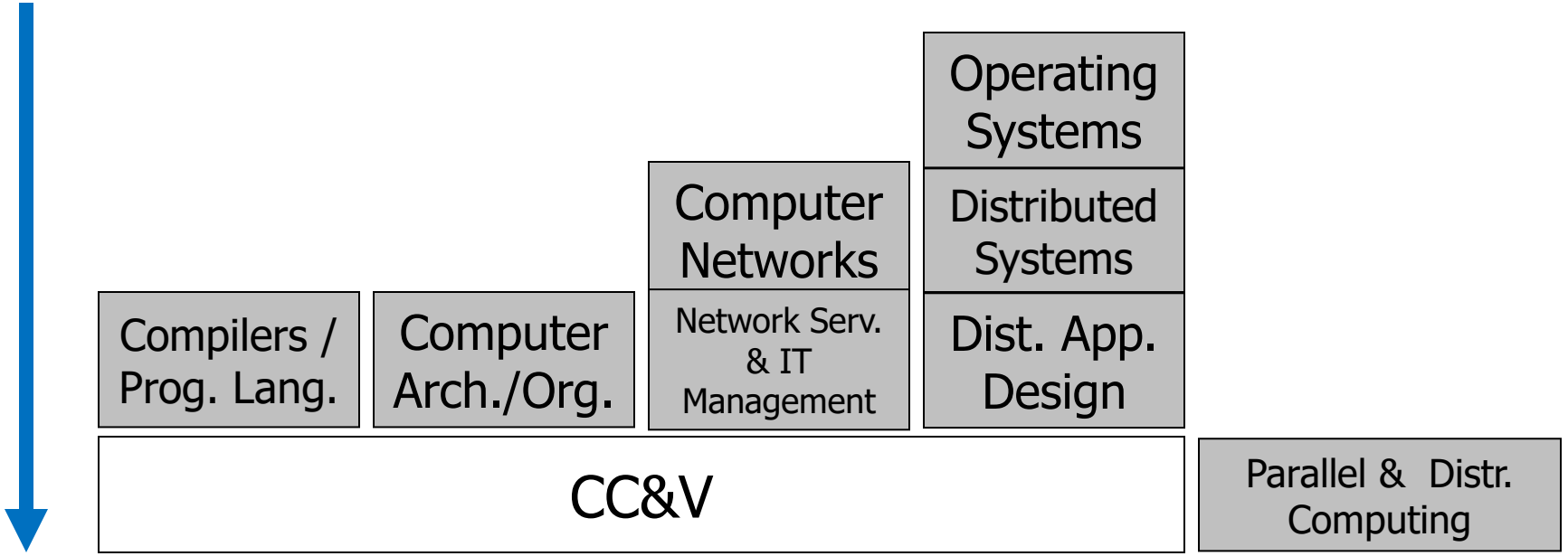
- **Dates :**
  - 1st Exam: June 18th
  - 2nd Exam: July 2nd
  - Project:
    - Project submission deadline: May 31st
    - Checkpoint midway: May 17th
    - Visualization and discussion following weeks
- **Schedule:**
  - Theory classes:
    - Tuesday: **08h30-10h30 (1-24)**
    - Thursday: **15h30-17h30 (1-24)**
  - Lab classes:
    - Wednesday: **08h30-11h30 (1-17)**
    - Friday: **08h30-11h30 (1-17)**
    - Group enrollment via web to open soon
      - **Groups with 3 students!**



Goals

- Attain an integrated perspective of cloud computing and virtualization:
  - their conjunction and synthesis, in the design of modern distributed and large-scale computing systems, addressing its technology, and underlying mechanisms and algorithms.
- Understand a vertical approach to the various technologies for virtualization and cloud computing, that:
  - enhance applications and services with increased flexibility, better resource utilization and economic efficiency, and higher scalability and adaptability.
- Be able to develop scalable and reliable applications and systems:
  - for cloud computing, on top of virtualization infrastructures and platforms, and employing cloud application models.
- Be able to assess decisions, solutions, and tradeoffs:
  - involved in the development, usage and management of virtualization infrastructures for cloud computing
- Welcome: *Feedback*, ideas, and imagination in projects.

Related Topics



Historical Context for CC&V

Computing

- Multiprocessing
- Distributed Processing
- Parallel Processing
- DSM
- Cluster
- Grid
- Cycle-sharing / P2P
- Cloud
- Edge / Fog / µClouds

1960s

Networking

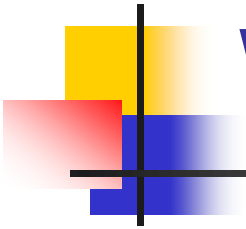
- No network
- Ethernet/Internet
- Large bandwidth LANs
- Large bandwidth WANs

2020s

- Optic fiber, FTTH

What is cloud computing?

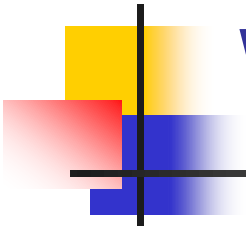
- **Solution for really large scale issues.**
- Good *vapourware* :
  - more HW, larger data sets.
- Examples:
  - Web specific problems:
    - crawling, indexing, search.
  - Data mining and analytics.
  - Genome datamining.
  - High energy physics, Astronomy.
  - E-Science, Data Science



What is cloud computing?

- **Highly efficent scalable architectures.**
- Virtualization and open API Web Services are very important and key enablers.
- Data-center driven with economies-of-scale.

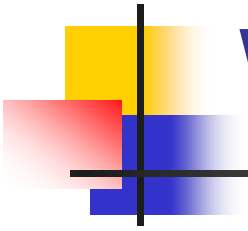
- Examples:
  - Google, 15k to 450k (typically ~250k) PCs.
  - Microsoft, Illinois container data center (~225k servers).



What is cloud computing?

- **Lots of Data.**

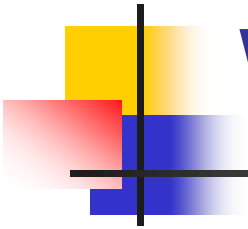
- Internet Archive: 10 PB (2012).
- Google handles > 100 PB/day.
- CERN's LHC generates 10-15 PB/year.
- Big-Data / Stream Processing:
  - Data analytics, fraud detection, network monitoring.



What is virtualization?

- **Why is virtual better than real?**
  - why Virtual memory, Java VM?
  - why KVM, Virtual Private Network (VPN) ?

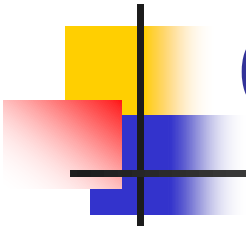
- **Advantages:** why accept inherent overhead?
  - portability, flexibility, reliability, management
  - only available choice with legacy systems
  - allows optimization techniques
  - reduce overhead and provide higher efficiency



What is virtualization?

- **Virtualization is everywhere.**
- In today's and future IT

- **Key-enabler of:**
  - cloud computing, resource sharing,
  - infrastructure and platform management,
  - software productivity and reliability,
  - support mobile and embedded devices,
  - security, reliability, energy-awareness, ...



CC&V for the end-user

- Utility computing,
  ***Infrastructure as a Service* (IaaS)**:
  - *"I just want to rent CPU cycles."*
  - E.g.: Amazon EC2 (System-level Virtualization)
- ***Platform as a Service* (PaaS):**
  - *"Give me an API and I'll handle the implementation."*
  - E.g.: Google App Engine (Process-level Virtualization)
- *Software as a Service* (SaaS):
  - *"I want a finished application available off-site."*
  - E.g.: Gmail.

CC&V at large for study and research

- **Great way to learn about:**
  - systems, distributed systems,
  - programming languages, architectures
- Lots of interesting technology out there

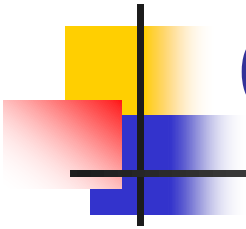






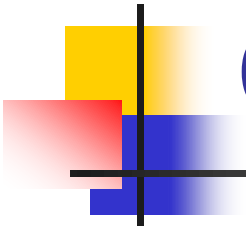






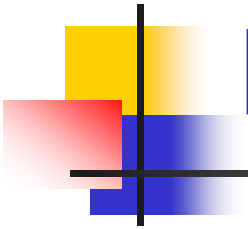
Core Syllabus

- *Part I: Introduction and Basics*
  - intro to cloud computing and virtualization
  - core emulation techniques
- *Part II: Infrastructure-as-a-Service*
  - system-level virtualization
  - cloud execution and monitoring
- *Part III: Platform-as-a-Service*
  - Java VM architecture and implementation
  - cloud platforms and programming models
  - Function-as-a-Service/serverless case-studies



Core Syllabus

- *Part IV: Data-and-Storage as Services (Big-Data)*
  - cloud storage
  - cloud file systems
- *Part V: Scalable Computing (Big-Data Processing)*
  - Map-Reduce, workflows/dataflows
  - stream processing
- *Part VI: Cross-cutting Issues*
  - cloud data center design and operation
    - VM migration and energy efficiency



Bibliography

- Main (books):
  - **[VM] Virtual Machines** - Versatile Platforms for Systems and Processes, Jim Smith and Ravi Nair, Morgan Kaufmann.
  - **[CC] Cloud Computing** - Theory and Practice, Dan C. Marinescu, Morgan Kaufmann.
- Additional:
  - Class slides and reference papers.
  - Selected conference papers for reference/presentation (made available in pdf).
  - Java VM Specification.

Evaluation

- **Exam** (35%)
  - Minimum grade: 8 points
- **Project** (45%)
  - Groups of 3 students
  - Minimum grade: 8 points
  - General Topic:
    - sample cloud processing system with variable CPU workloads
    - involving cloud API and virtualization technology
- Past Projects:
  - *Cloud systems for: Face recognition, Sudoku solver, Maze solver, Map colouring, Radar scanner, CloudEco workloads…*
  - *manage and monitor VMs elastically, schedule and balance requests*

Evaluation (cont.)

- **Paper Presentation** (15%)
  - Group presentation of CC&V-related paper (15 min.)
  - Typically, in Lab classes
  - List of papers provided by the course

- **Class Participation** (5%) – Only for Bonus
  - Show up for classes
    - interest and participation
    - ask questions, give feedback to groups presenting papers!!
  - Grade: lecture and presentations attendance
  - Counts to raise Final Grade, otherwise reverts to Exam (then 40%)

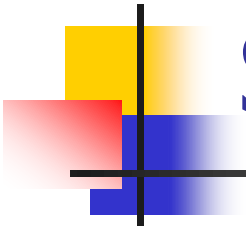
TPC/Homework...

- Group enrollment lab/project/presentations
  - Fénix, 3 students per group
  - Students must follow the course in the campus enrolled (lectures, labs, project, exam)
    - Exceptions to this can only be handled manually at the end according to the notice and email sent to all students.
  - Students without group should find colleagues
    - We publish list with partial groups, students looking for groups
  - Students with project completed before, mail us!!
    - Relevant to assess shift placements

Summary

- Goals...*what is sought...*
  - *Attain an integrated perspective of cloud computing and virtualization: their conjunction and synthesis, in the design of modern distributed and large-scale computing systems, addressing its technology, and underlying mechanisms and algorithms.*
  - *Understand a vertical approach to the various technologies for virtualization and cloud computing, that enhance applications and services with increased flexibility, better resource utilization and economic efficiency, and higher scalability and adaptability.*
  - *Be able to develop scalable and reliable applications and systems for cloud computing, on top of virtualization infrastructures and platforms, and cloud application models.*
  - *Be able to assess decisions, solutions, and tradeoffs involved in the development, usage and management of virtualization infrastructures for cloud computing*

Summary (cont.)



- Labs...*what is used...*
  - *Pin Tool (tool for native code instrumentaton by Intel), Javassist (byte-code instrumentation tool in Java), Amazon Web Services (AWS)*
- Project...*what is experimented...*
  - *Cloud APIs and virtualization technology*
- *Evaluation...what is expected...*
  - *Exam (35%), Project (45%), Paper presentation (15%),*
  - *Class participation (5%)*

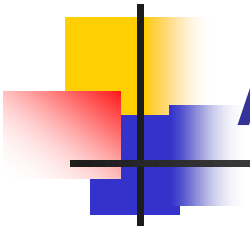


Introduction to Cloud Computing and Virtualization

Prof. Luís Veiga
IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

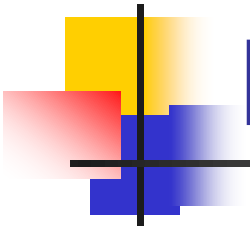
© LV, JG 2015-24



Agenda

- Introduction to Cloud Computing
  - Net-Centric Computing and Content
  - Computing as Utility
  - Cloud Ecosystem
  - Advantages and Challenges
- Introduction to Virtualization
  - Well-defined Interfaces: Abstraction and Virtualization
  - Computer System Interfaces
    - Instruction-Set Architecture
    - Application Binary Interface
  - Machine perspective: System VM and process VM

Network-centric computing



- Information processing more efficient on large farms of computing and storage accessible on Internet
  - Grid computing (early 1990s)
    - targeted primarily at scientific computing
  - Utility computing (2005-2006)
    - by IT companies and targeted at enterprise computing
- focus of utility computing
  - business model for providing computing services
  - it often requires a cloud-like infrastructure
- Cloud computing: *a path to utility computing*
  - embraced by major IT companies
  - e.g. Amazon, Google, Microsoft, IBM, HP, Oracle, …

Net-centric computing & content

- E.g. Data-intensive
  - large scale science/eng. simulations require large volumes of data.
- E.g. Network-intensive
  - transfer large volumes of data requires high bandwidth networks.
- Low-latency networks
  - for data streaming, parallel computing, …
- Systems used with *thin clients* with limited resources
  - e.g., wireless devices such as smart phones and tablets.
  - e.g., wired devices such as terminals with desktop virtualization.

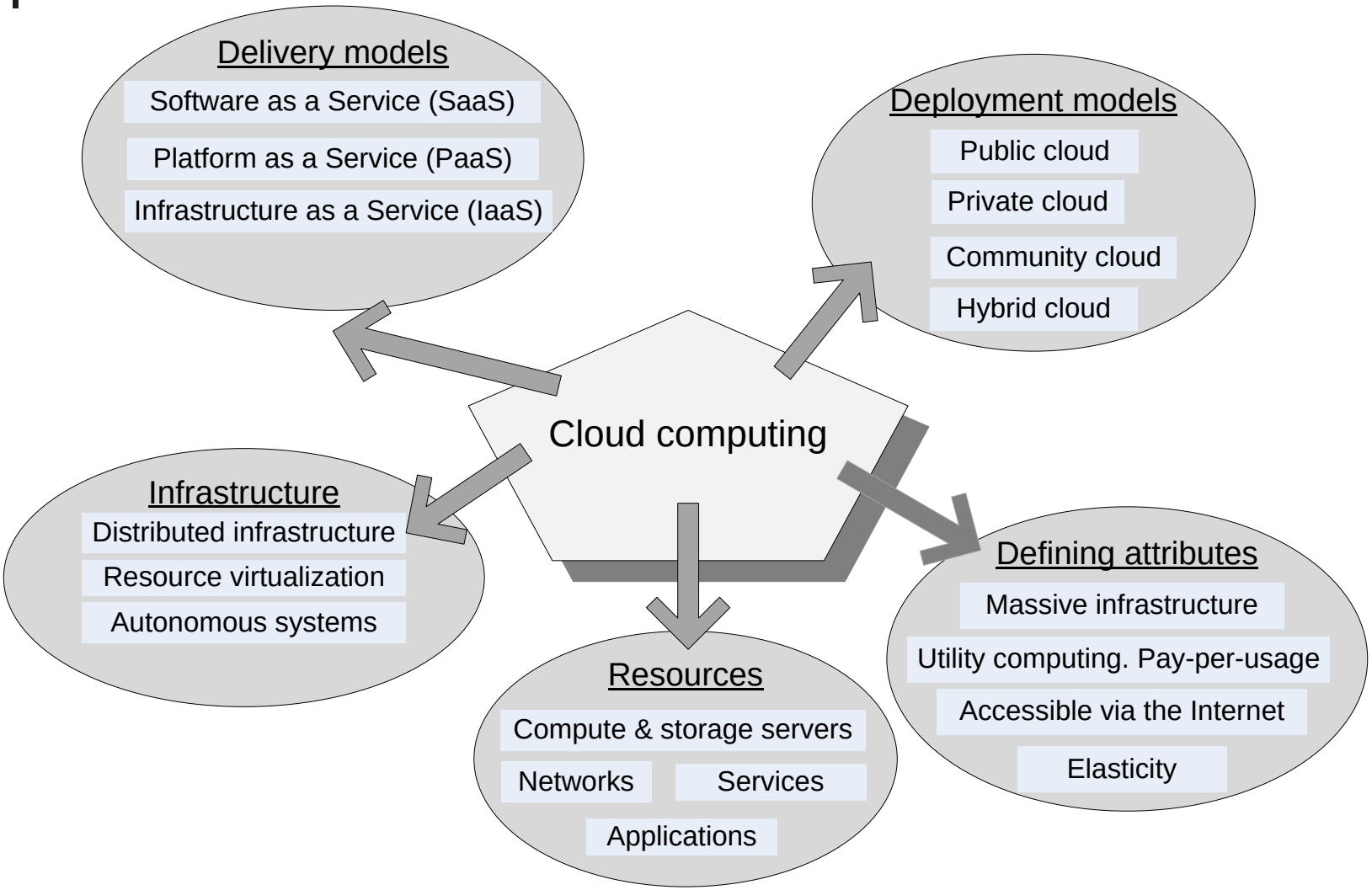
Cloud: Computing as an Utility

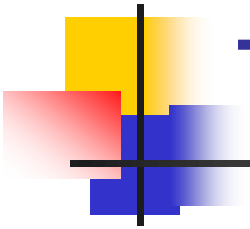
- Computing as an utility
- (as with water, electricity, internet):
  - Illusion of infinite resources on demand:
    - 1 CPU x 100 h = 100 CPU x 1h
  - Removal of any *a priori* commitment:
    - no (or very low) initial costs.
    - reduce CapEx (capital expenditure),
    - convert to OpEx (operational expenditure).
  - Small granularity payments:
    - computing paid by CPU hour, web request, storage access, and/or bandwidth used.

Cloud Computing

- Use Internet technologies to offer scalable, elastic services.
  - *elastic computing* →
    - ability of dynamically acquiring computing resources
    - adapt to support a variable workload.
- Resources used for these services
  - can be metered
  - users charged only for the resources used
- Service providers
  - ensure maintenance and security
  - can operate more efficiently and lower costs
    - due to specialization and centralization.

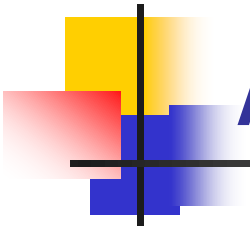
The Cloud Ecosystem





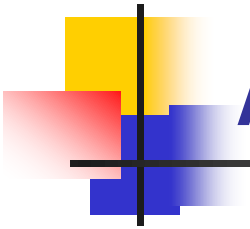
Types of clouds

- Public Cloud
  - infrastructure available to general public or a large industry group.
  - owned by the organization selling cloud services.
- Private Cloud
  - infrastructure is operated solely for an organization.
- Community Cloud
  - infrastructure shared by several organizations.
  - supports a community or consortium that has shared concerns.
- Hybrid Cloud
  - composition of two or more clouds (public, private, or community)…
  - bound by standardized technology enabling data and application portability.



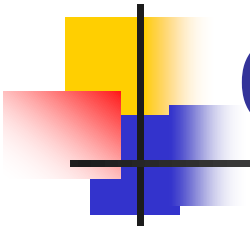
Advantages

- Resources are shared
  - CPU cycles, storage, network bandwidth,… GPUs, etc.
- When multiple applications share a system:
  - their peak demands for resources are not synchronized
  - multiplexing leads to a higher resource utilization.
- Resources can be aggregated
  - to support data-intensive applications.
- Data sharing
  - facilitates collaborative activities



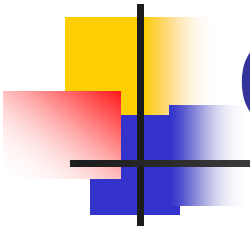
Advantages

- Eliminates initial investment costs
  - of private computing infrastructure
  - (and maintenance and operation costs)
- Cost reduction
  - concentration of resources creates opportunities
  - e.g. for pay-as-you-go computing
- Elasticity
  - accommodate workloads with very large peak-to-average ratios
- User convenience
  - virtualization allows users to operate in familiar environments



Challenges

- Availability of service
  - what happens when the service provider cannot deliver?
- Vendor lock-in
  - diversity of services, data organization, interfaces at providers
  - customer hooked to one provider -> hard to move to another
- Data confidentiality and auditability
- Data transfer bottleneck
  - many applications are data-intensive
  - geo-distributed caching, Content Distribution/Delivery Networks

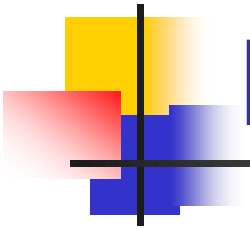


Challenges

- Performance unpredictability
  - Consequence of resource sharing.
  - Q: How to support elasticity, ability to scale up and down quickly?
  - Q: How to use resource virtualization and performance isolation for QoS guarantees?
- Resource management
- Security and confidentiality
- Addressing such challenges offers research opportunities!!

Virtualization technology *...then and now...*

- Past
  - Virtual memory (1957-1969)
  - IBM mainframes - System/360 (1964/65)
    - CP-40 operating system, limited VMs, fixed memory
    - CP-67/CMS Multiple OS, all CPU instructions virtualized
    - VM/370 (1972-)
- Present
  - VMWare, VirtualPC, Xen, Hyper-V, VirtualBox, QEMU, Bochs, KVM, z/VM for IBM zSeries
  - Solaris Zones, cgroups, LXC, Docker, Swarm, Kubernetes
  - Java VM, .NET CLR, Python, PHP Zend, Ruby, Node.js
- *Hot topics*
  - server farms, server consolidation
  - virtual clustering, *cycle-sharing,*
  - grid, utility computing, P2P Grids, micro-clouds, community clouds
  - cloud computing (and fog and edge clouds)
    - Amazon Elastic Compute Cloud, Openstack, Cloudstack, Eucalyptus,...

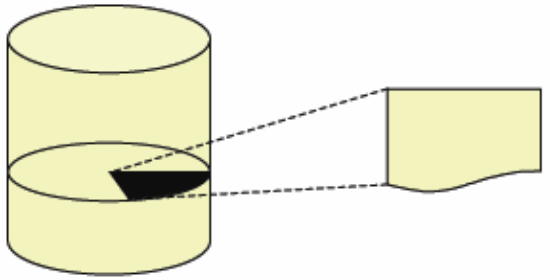


Motivation for Virtualization

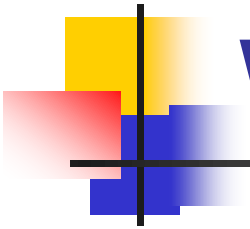
- why Virtual memory?
  - relocatable, sharing/protection, larger address space, working set
- why VirtualBox, VMWare, Xen, KVM, QEMU ?
  - side-by-side Linux and Windows, virtual appliances (OS+applications)
  - reliability, checkpointing, migration, support for legacy systems
- why Java VM, .NET, Python, PHP ?
  - interoperability, application sharing, mobile code, sandboxing, garbage collection, code optimization, development productivity
- why Virtual Private Network (VPN) ?
  - communication security on top of unsecure network

Abstraction

- Complexity in computer systems
  - Hardware:
    - chips, transistors, gates, signals, radio, disk plates,...
  - Software:
    - CPU instructions, OS, libraries, compilers, languages,...
    - lines of code, functions, methods, classes, components, packages, ...



- Abstraction hides details
  - key to manage complexity
  - simplified interface to underlying resources
  - E.g., file on hard-disk
    - mapped to collection of sectors, tracks
    - not addressed directly by user or programmer (uses File *streams*)
  - Approach
    - **levels of abstraction+ well-defined interfaces**



Well-defined Interfaces

- Examples
  - Instruction sets: x86, ARM, Apple M1, PowerPC (PPC)
  - OS system calls: Windows, Linux, MacOS
  - Libraries: POSIX, OpenGL, DirectX
  - Byte-code formats: JVM bytecode, MSIL (.NET bytecode)

- Advantages
  - decoupling
  - independence from changes in internal details
    - e.g., applications and OS, compilers and CPU, ...
  - vendor independence
  - parallel development (e.g., H/W and S/W)
  - diversity: competing abstractions favour innovation

Well-defined Interfaces

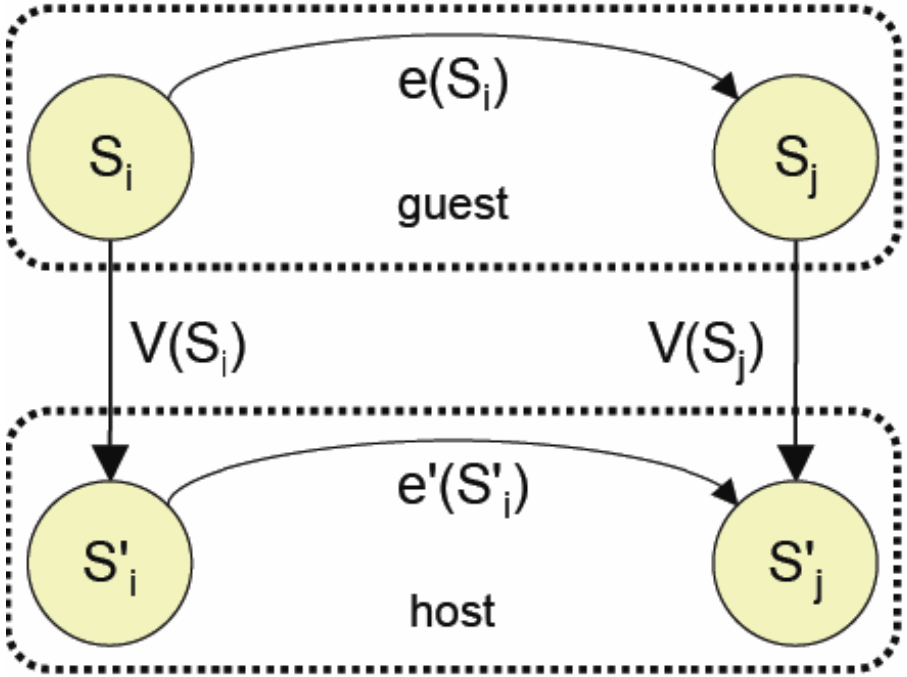
- Disadvantages
  - interfaces are confining
  - may reduce interoperability
    - subsystems designed for one interface do not work with others
    - e.g., different instruction-sets, # CPUs, OSs, executable formats
    - assumption that OS has full control over hardware
  - very restrictive specially in networked environment
  - limited flexibility, lock-in, security and failure isolation

- Solution?
  - Keep advantages,
    - address complexity, foster innovation and development productivity
  - Avoid disadvantages
    - preserve interoperability and allow flexibility

  - Virtualization: **mapping of interfaces**
    - e.g., H/W-S/W, applications-OS, languages-systems

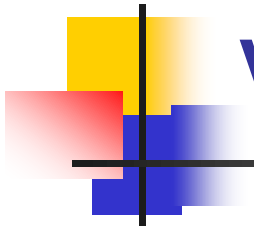
Virtualization

- isomorphism
  - *virtual* **guest**
  - *real* **host**
  - state-mapping
    - V() maps states
  - emulation
    - sequences of operations



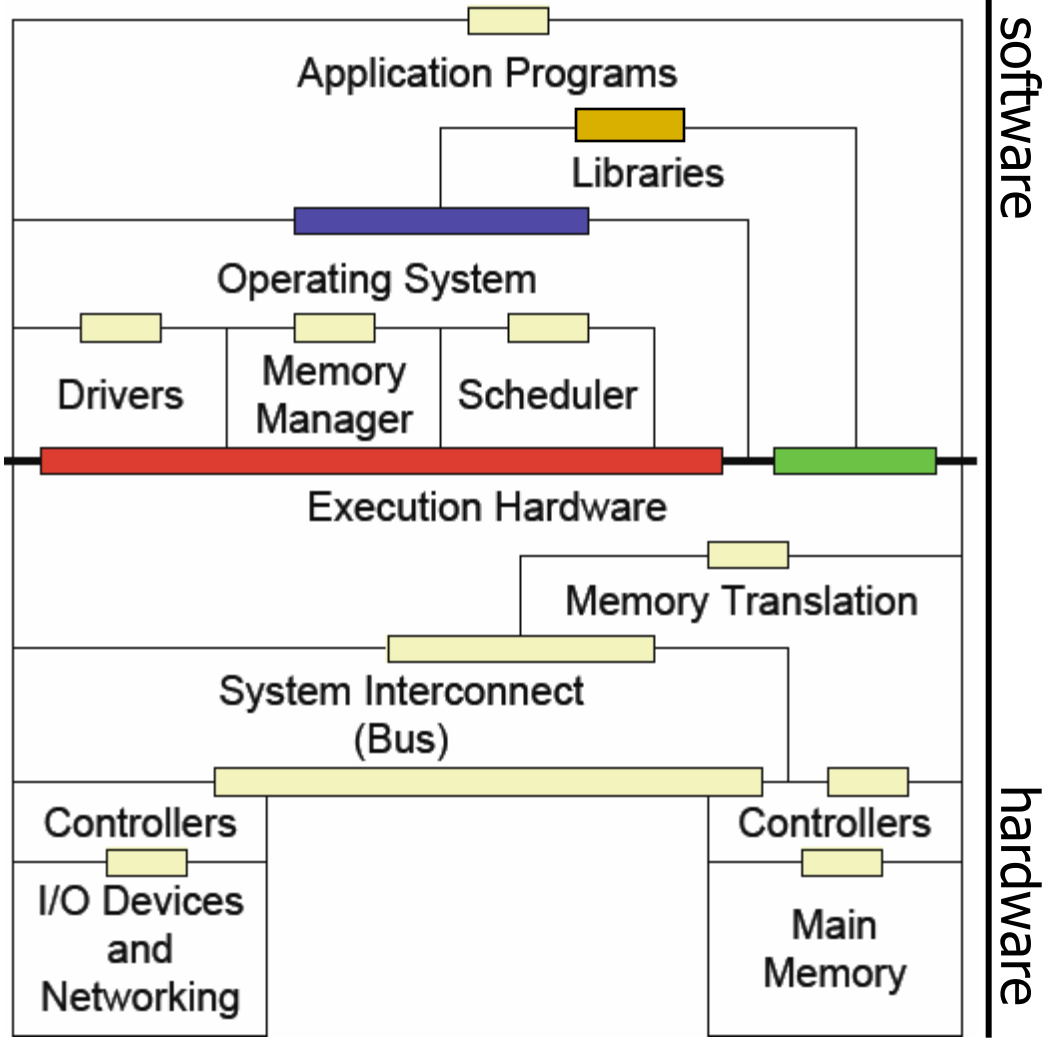
- virtualization vs abstraction
  - abstraction is also an isomorphism
  - virtualization does not necessarily hide details
    - e.g., maintains similar state size and number of operations

Virtualization (Examples)



- Virtual hard-disks as large files on real hard-disk
  - Virtual sectors → parts on real file stored in real sectors
  - Virtual r/w → file r/w backed by r/w on real disk
  - No abstraction, same level of detail
- VMWare or VirtualBox with Linux guest running on Win host
- VMs to execute Win/x86 binaries
  - e.g., running Windows games in Linux using Wine/WineHQ
- Byte-codes running on *imaginary real* Java VM
- Unifies techniques developed separately
  - operating systems, language, compiler and hardware designers

Computer System Interfaces



Relevant Interfaces

- **ISA**: Instruction-Set Architecture
  - system+user ISA
  - user ISA

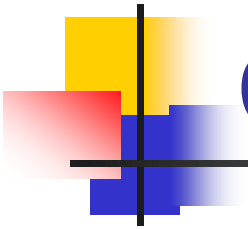


- **ABI**: Application Binary Interface
  - system calls
  - user ISA



- **API**: Application Programming Interface
  - library calls
  - user ISA





Computer System Interfaces

- Instruction Set Architecture (ISA)
  - division between hardware and software
  - components
    - user-level ISA
      - CPU instructions visible to applications
    - system-level ISA
      - CPU instructions visible only to supervisor software
        - e.g., operating system
      - can also (and mostly often does) include user-level ISA
  - decoupling/confinement trade-off
    - software built for a specific ISA runs on ***any*** hardware that implements the same ISA, but ***only*** on those
      - i.e., platform portability

Computer System Interfaces

- Application Binary Interface (ABI)
  - mediates access of application software to the system where it executes
    - hardware resources
    - core/basic services
  - components
    - user-level ISA
    - system-calls to the OS
      - OS functions performed by request of applications
      - validate security, resource management, …
  - decoupling/confinement trade-off
    - application binaries compiled run unmodified on ***any*** system with the same ISA and OS, but ***only*** there…
      - no recompilation, i.e., binary portability

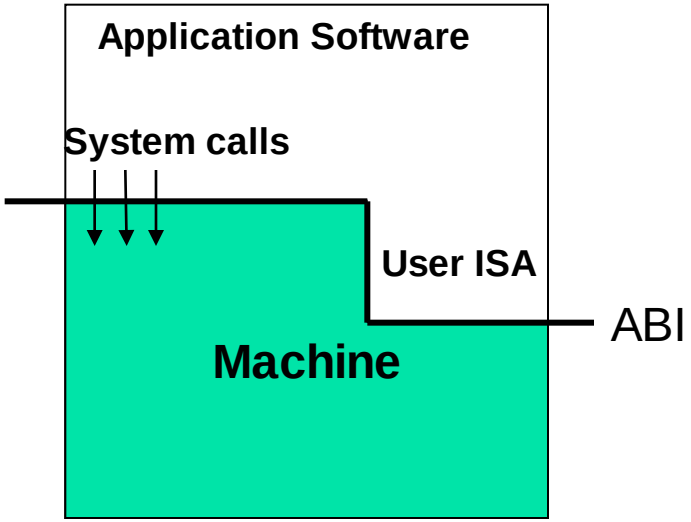
Machine: Matter of Perspective

- Process Perspective (i.e. *runtime*)
  - *the machine:* OS and user-level H/W
    - Logical memory adress space assigned to the process
    - User-level registers and instructions for code execution
    - I/O visible through OS, via system calls
    - Transient: exists only during process execution
  - ABI provides interface between process and machine

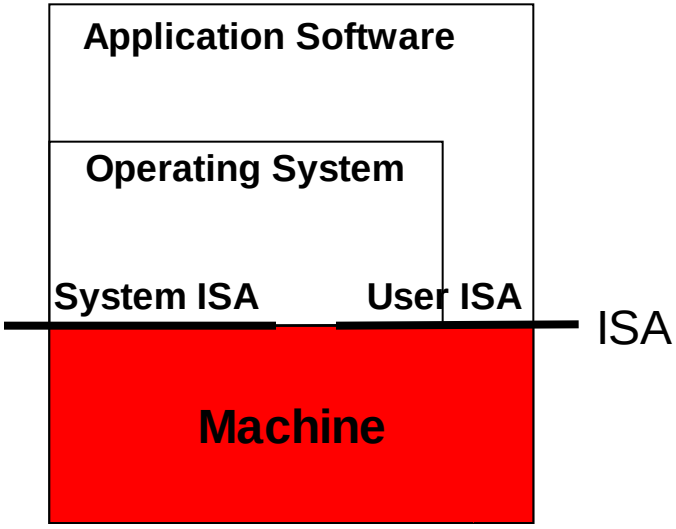
- System Perspective (i.e. *VMM virtual machine monitor*)
  - *the machine:* underlying hardware alone
    - system:full execution environment=many processes+users
    - all processes share file system and I/O resources
    - OS is part of the system
    - Persistent: survives processes and reboots
  - ISA provides interface between system and machine

Machine: Matter of Perspective

- Process Perspective
  - OS entity
  - ABI interfaced
    - i.e. emulates an ABI
  - transient
  - *(enabler for PaaS)*



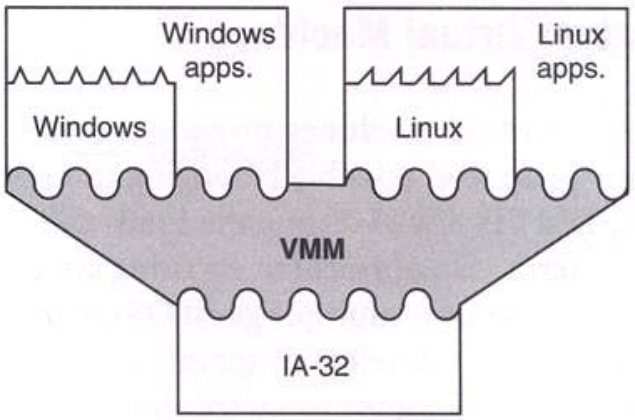
- System Perspective
  - hardware
  - ISA interfaced
    - i.e emulates an ISA
  - persistent
  - *(enabler for IaaS)*



(Key) Example of Process VMs

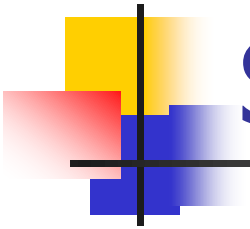
- High-level language (HLL) virtual machines
  - address difficulty of full cross-platform portability for any ISA
  - do not correspond to any pre-existing real platform
    - Java, .NET CLR, Python, node.JS, Pascal P-code, Lisp
  - designed for ease of portability
    - match features of high-level programming language
    - minimize H/W specifics and OS specifics
    - deployment: portable intermediate code (*virtual* ISA)
      - *bytecodes* instead of binary object native code
  - VM executes portable intermediate code
    - employing interpretation, pre-compilation or JIT-compilation

System VMs



- ***classic*** approach **(Type-1)**
  - VMM on top of hardware
    - initial solution, used in IBM mainframes
    - revisitied in *Windows Hyper-V*
  - Pro/Cons
    - completely transparent, most efficient
    - installed from scratch
      - must include (all/relevant) device drivers

- ***hosted*** VM **(Type-2)**
  - VMM on top of host OS
    - VMWare, Xen, VirtualPC, VirtualBox
  - Pro/Cons
    - installed like applications, leverages guest-OS drivers and lower-level services
    - possible loss of efficiency due to additional software layers



Summary

- Introduction to Cloud Computing
  - Net-Centric Computing and Content
  - Computing as Utility
  - Cloud Ecosystem
  - Advantages and Challenges
- Introduction to Virtualization
  - Well-defined Interfaces: Abstraction and Virtualization
  - Computer System Interfaces
    - Instruction-Set Architecture
    - Application Binary Interface
  - Machine perspective: System VM and process VM

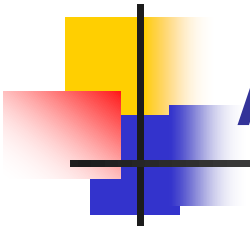


Core Emulation Techniques
(Interpretation & Binary Translation)

Prof. Luís Veiga
IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© LV, JG 2015-24

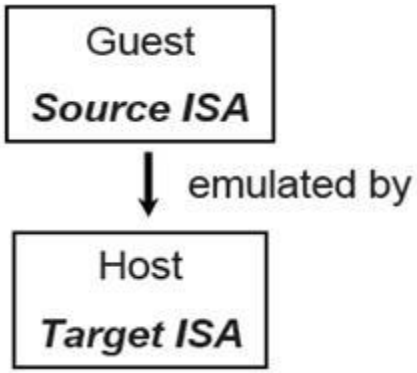


Agenda

- Basic Interpretation
  - architected state, context blocks
  - decode and dispatch interpreter
- Threaded Interpretation
  - indirect threaded interpretation
  - basic predecoding
  - direct threaded interpretation
- Binary Translation
  - code translation
  - state mapping
  - incremental predecoding and translation
  - control transfer optimization
    - translation chaining

Emulation

- Emulation:
  - *"the process of implementing the interface and functionality of one system (or subsystem) on a system with different interface and functionality"*
    - e.g., an Android phone emulator running within a window on a PC
- Instruction set emulation
  - key aspect of many virtual machine implementations
  - VM must support binary compiled for one ISA and run on another
  - **host** platform: host hardware + target ISA
    - implements emulation
      - instructions, registers, memory architecture
    - reproduces expected software behavior
  - **guest**: source ISA
    - implementation of user applications

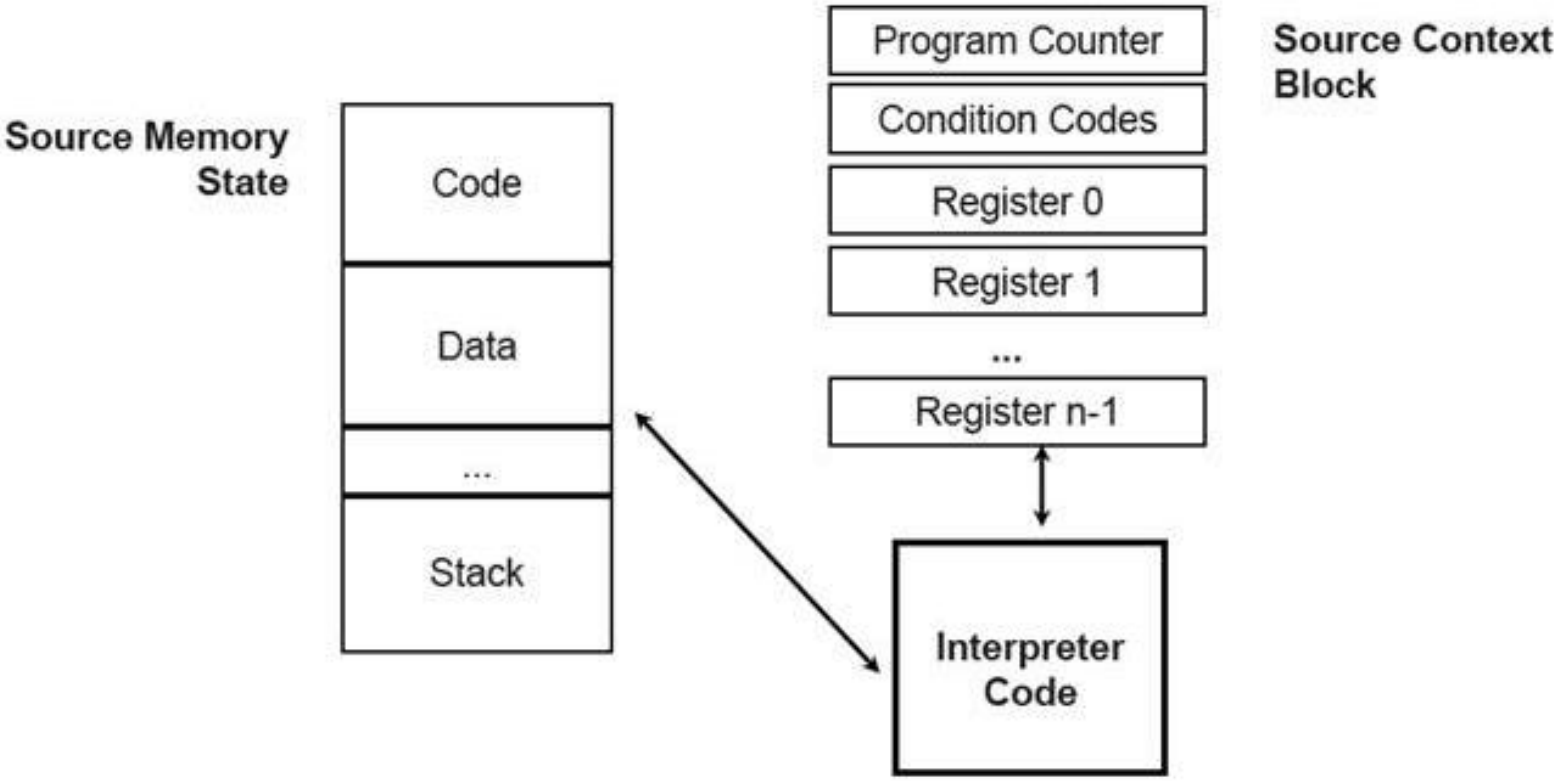


Basic Interpretation

- Initially used for programming languages
  - e.g., Lisp, BASIC, etc.
- VM Application
  - interpretation of binaries (machine code)
    - emulate instructions
    - manage complete architected (emulated) state
      - includes registers, main memory
      - guest program, guest data, guest stack
    - maintain table of *context blocks*
      - general purpose registers
      - program counter
      - ….
      - *(all CPU registers and state information)*

Basic Interpretation

- Interpreter Overview
  - interpreter code for instruction intepretation
  - architected source state in interpreter's data memory



Decode-and-dispatch interpreter

- Interpretation loop
  - stepping through the program
  - read and modify state according to instruction
  - **decode** an instruction (i.e. fetch each field in specific set of bits)
  - **dispatch** it to an **interpretation routine** based on **type** of instruction
  - type of instruction binary is obtained from representation
- Code loop for interpreting PPC ISA

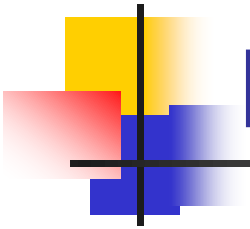
```
while (!halt && interrupt){
    inst=code[PC];
    opcode=extract(inst,31,6);
    switch(opcode){
        case LoadWordAndZero: LoadWordAndZero(inst);
        case ALU: ALU(inst);
        case Branch: Branch(inst);
              . . . . .
}
```

Decode-and-dispatch interpreter

- E.g. emulation of PPC instruction for memory access
  - LWZ (load-word-and-zero): basic PPC load instruction (equivalent to x86 MOV M[...])
  - loads a 32-bit quantity into 64-bit register and zeros higher bits

- E.g. Interpreter routine for LWZ in PPC ISA
  - extract arguments, load guest memory into guest reg, advance PC

```
LoadWordAndZero(inst){
    RT=extract(inst,25,5);   // target reg: start bit - field
  width
    RA=extract(inst,20,5);   // source reg as base address
    displacement=extract(inst,15,16); // offset
    source=regs[RA];
    address=source+displacement;
    regs[RT]=(data[address]<<32)>>32
    PC=PC+4;
}
```



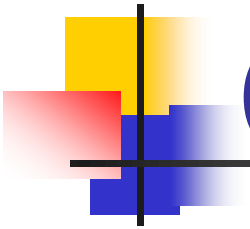
Decode-and-dispatch interpreter

- Advantages
  - low memory requirements, zero start-up time
  - easy *(and tiresome)* to implement

- Disadvantages
  - slow steady-state performance
  - inefficient w.r.t. branches and code cache *(slower execution)*

  - source instructions parsed on each emulated execution

- Start-up: *cost of emulating instruction for the first time*
- Steady-state: *rate of instruction emulation long-term*



(Indirect) Threaded Interpretation

- Idea:
  - avoid inneficient control flow of interpreter loop code

- (Indirect) Threaded Interpretation
  - reduce number of branches in decode-dispatch loop

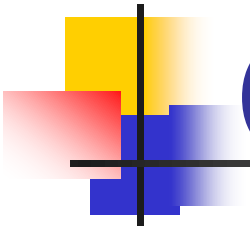
  - by trading off (code) space for emulation speed
  - duplicating/replicating decode-dispatch code
    - at every instruction emulation routine

(Indirect) Threaded Interpretation

- append portion of dispatch code to end of each emulation routine
  - can eliminate a few branches (*switch, call-ret, loop repeat*)
  - dispatch resorts to table (i.e., *indirect*)

```
void LoadWordAndZero(inst) {
        RT = extract(inst, 25, 5);
        RA = extract(inst, 20, 5);
        displacement = extract(inst, 15, 16);
        if (RA == 0) source = 0;
        else source = regs[RA];

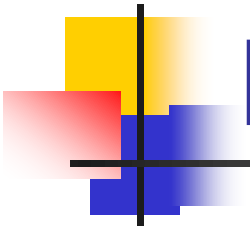
        address = source + displacement;
        regs[RT] = (data[address] << 32) >> 32;
        PC = PC + 4;
        if (halt || interrupt) goto exit;
        inst = code[PC];
        opcode = extract(inst, 31, 6);
        extended_opcode = extract(inst, 10, 10);
        routine = dispatch[opcode, extended_opcode];
        goto *routine;
```



(Indirect) Threaded Interpretation

- Advantages:
  - low memory requirements
    - but higher than decode-and-dispatch interpretation
  - zero start-up time

- Disadvantages:
  - steady-state performance is still slow
    - but better than decode-and-dispatch interpretation
    - still requires analysis for every instruction execution
    - still employs indirect branch and access to centralized table



Basic Predecoding

- Idea:
  - why repetitive instruction analysis and field extraction?
    - for each dynamic instance (execution) of same instruction
    - save opcode and fields for later executions

- Basic Predecoding
  - parses an instruction and saves it in an *intermediate format*
    - easily and efficiently accessible fields
    - simplifies and speeds up interpretation

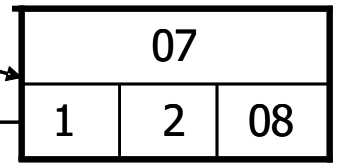
Basic Predecoding

- E.g. Predecoding for PPC ISA
  - Example for **lwz** instruction
    - op-code (e.g. 07) and three operands

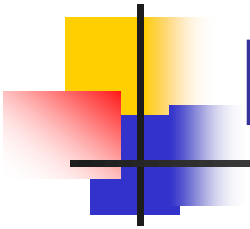
```
LoadWordAndZero(inst)
    RT=extract(inst,25,5);
    RA=extract(inst,20,5);
    RB=extract(inst,15,5);
```

```
RT= code[TPC].dest
RA= code[TPC].src1;
displacement=code[TPC].src2
```

```
lwz r1, 8(r2);load word and zero
```



Intermediate form / format

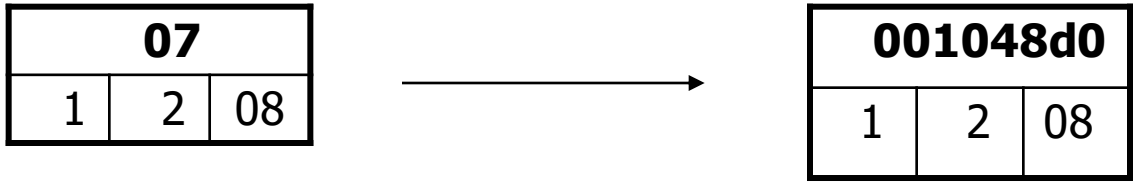


Basic Predecoding

- Advantages
  - Speed-up in interpretation
    - no redundant field extraction, faster interpretation
- Disadvantages
  - higher memory requirements (dual code representation)
  - new sources of overhead and complexity
    - separate target PC and synchronization with source PC

Direct Threaded Interpretation

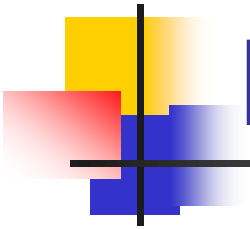
- Basic predecoding is indirect & still requires dispatch table
  - Requires additional memory access before jump
- How to eliminate this indirection?
  - Replace instruction code with actual address of interpreter routine
  - Dispatch table is removed (from emulation control flow)




```
If (halt || interrupt) goto exit;
opcode= code[TPC].op;
routine=dispatch [opcode];
goto  *routine;
```

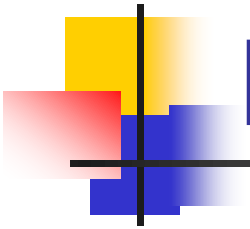
→

```
If (halt || interrupt) goto exit;
routine= code[TPC].op;
goto  *routine;
```



Direct Threaded Interpretation

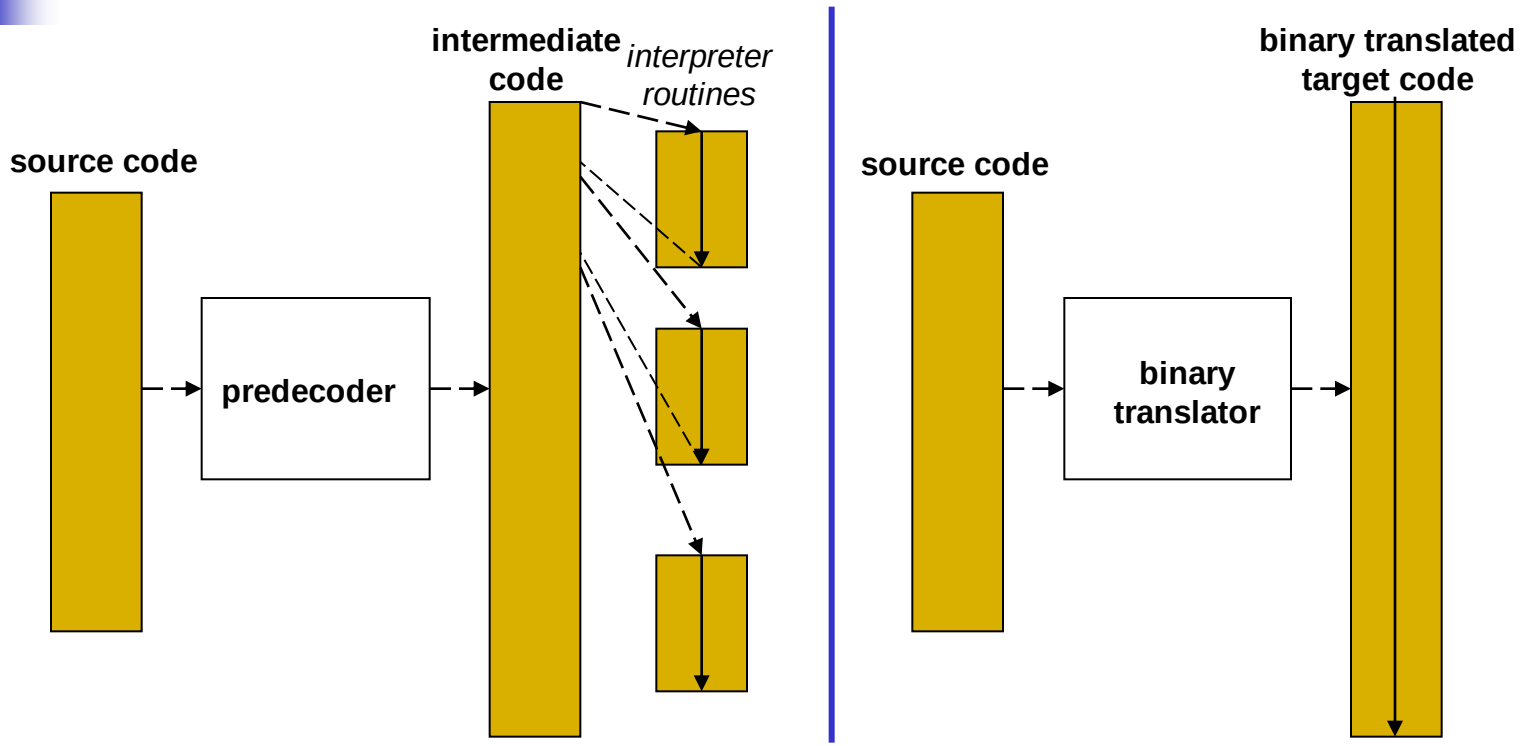
- Advantages
  - greater speed-up in interpretation
    - fewer jumps/branches
    - fewer memory accesses
- Disadvantages
  - portability issues
    - intermediate format dependent on actual routine addresses
      - exact location of routine in memory, not relocatable
    - when porting emulator to different architecture
      - interpreter code must be regenerated



Binary Translation

- (Another) Process of converting
  - different from other emulation techniques
  - from source binary directly into target binary
- Goal
  - enhance performance
    - remove interpreter routine overhead
- How? → *simple binary translation*
  - map each source instruction to
  - its own customized target code
  - execute translated code directly on the host
    - (almost) as if a native binary in the target architecture

Predecoding and Binary Translation



- both convert source machine code into another form
- but predecoding & DT interpretation still require constant overhead
  - calling interpreter routine for each & every instruction emulated
    - each type of source instruction maps to a routine
    - poor code locality, possible parameter passing, many jump targets
  - **binary translation**: converted code is executed directly

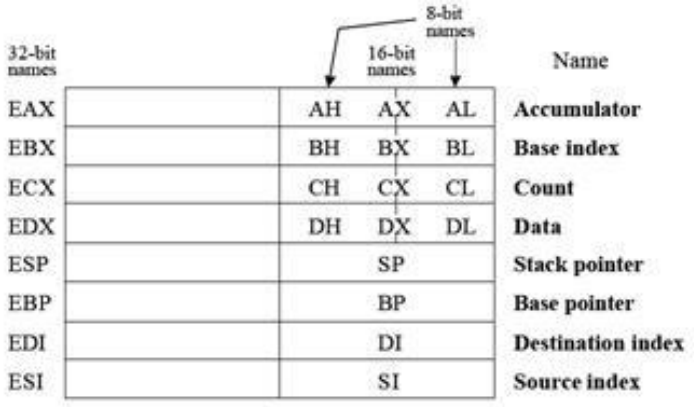
Binary Translation

- E.g., simple translation: IA-32 source binary to PPC target binary

  - **IA-32 code**

```
addl    %edx,4(%eax)
movl    4(%eax), %edx
add %eax,4
```

```
addl    edx,[eax+4]
movl    [eax+4],edx
add eax,4
```



  - **possible PPC translation**

```
r1 points to IA-32 register context block
r2 points to IA-32 memory image
r3 contains IA-32 source ISA PC value
```

```
lwz   r4,0(r1)        ;load %eax from register block
addi  r5,r4,4         ;add 4 to %eax
lwzx  r5,r2,r5        ;load operand from memory
lwz   r4,12(r1)       ;load %edx from register block
add   r5,r4,r5        ;perform add
stw   r5,12(r1)       ;put result into %edx
addi  r3,r3,3         ;update PC (3 bytes)

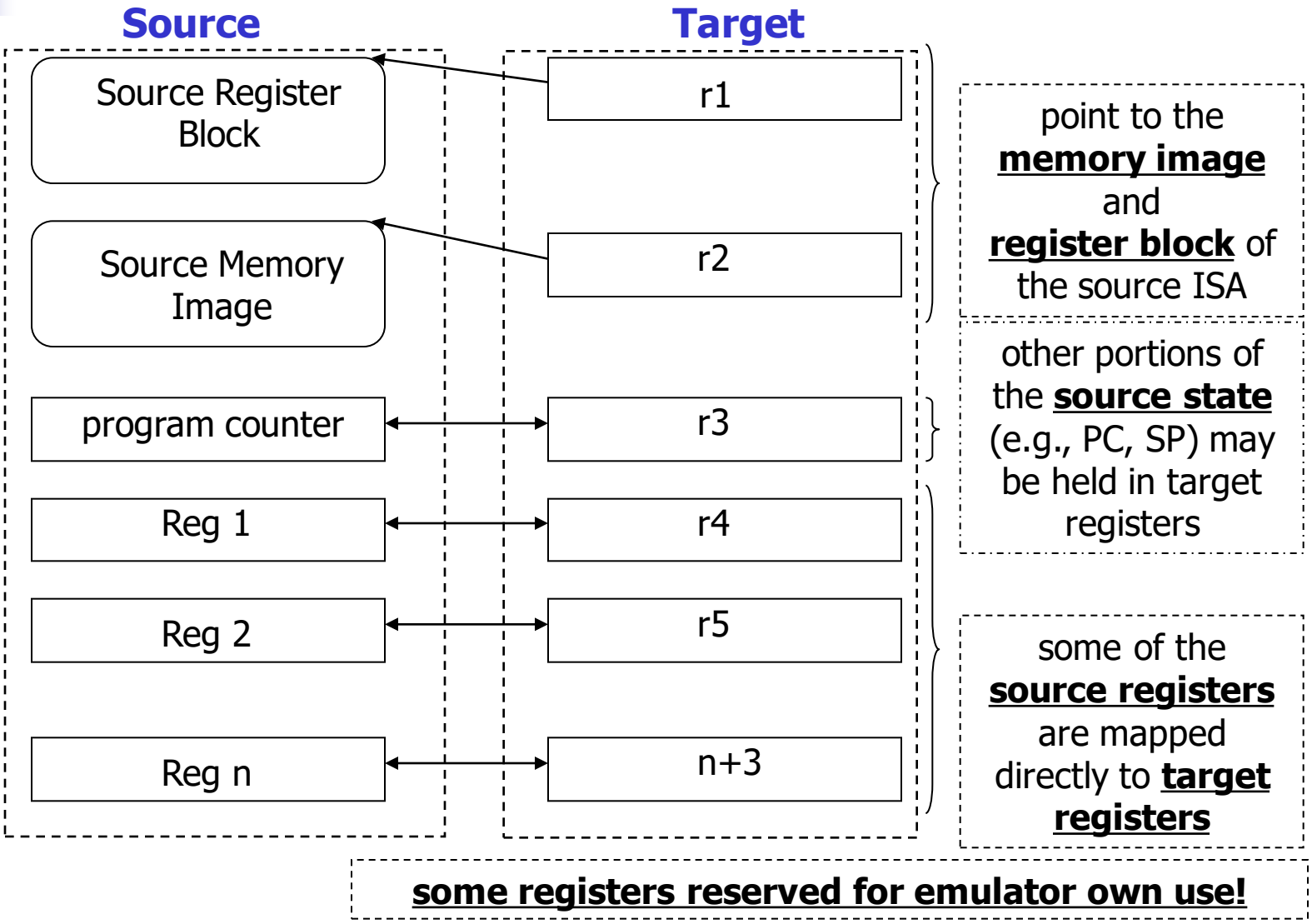
lwz   r4,0(r1)        ;load %eax from register block
addi  r5,r4,4         ;add 4 to %eax
lwz   r4,12(r1)       ;load %edx from register block
stwx  r4,r2,r5        ;store %edx value into memory
addi  r3,r3,3         ;update PC (3 bytes)

lwz   r4,0(r1)        ;load %eax from register block
addi  r4,r4,4         ;add immediate
stw   r4,0(r1)        ;place result back into %eax
addi  r3,r3,3         ;update PC (3 bytes)
```

Binary Translation & State Mapping

- Simple binary translation
  - one-by-one instruction translation
  - always loads and saves registers from context block
    - many more memory accesses in target code w.r.t. source
  - does not leverage information about
    - source registers used in consecutive source instructions
    - target registers used to simulate source registers
- How to improve?: → using *state-mapping*
  - avoid constant access to register context in memory
  - How?: remember which target registers hold values of source registers between instructions
    - direct access to target registers in target code
    - i.e., register state-mapping

State Mapping



Binary Translation with State Mapping

- **IA-32 code**

```
addl    %edx,4(%eax)
movl    4(%eax), %edx
add %eax,4
```

```
addl    edx,[eax+4]
movl    [eax+4],edx
add eax,4
```

- **PPC code with state mapping**

```
r1 points to IA-32 register context block
r2 points to IA-32 memory image
r3 contains IA-32 ISA PC value
r4 holds IA-32 register %eax ...
r7 holds IA-32 register %edx
```

```
addi  r16,r4,4     ;add 4 to %eax
lwzx  r17,r2,r16   ;load operand from memory
add   r7,r17,r7    ;perform add of %edx

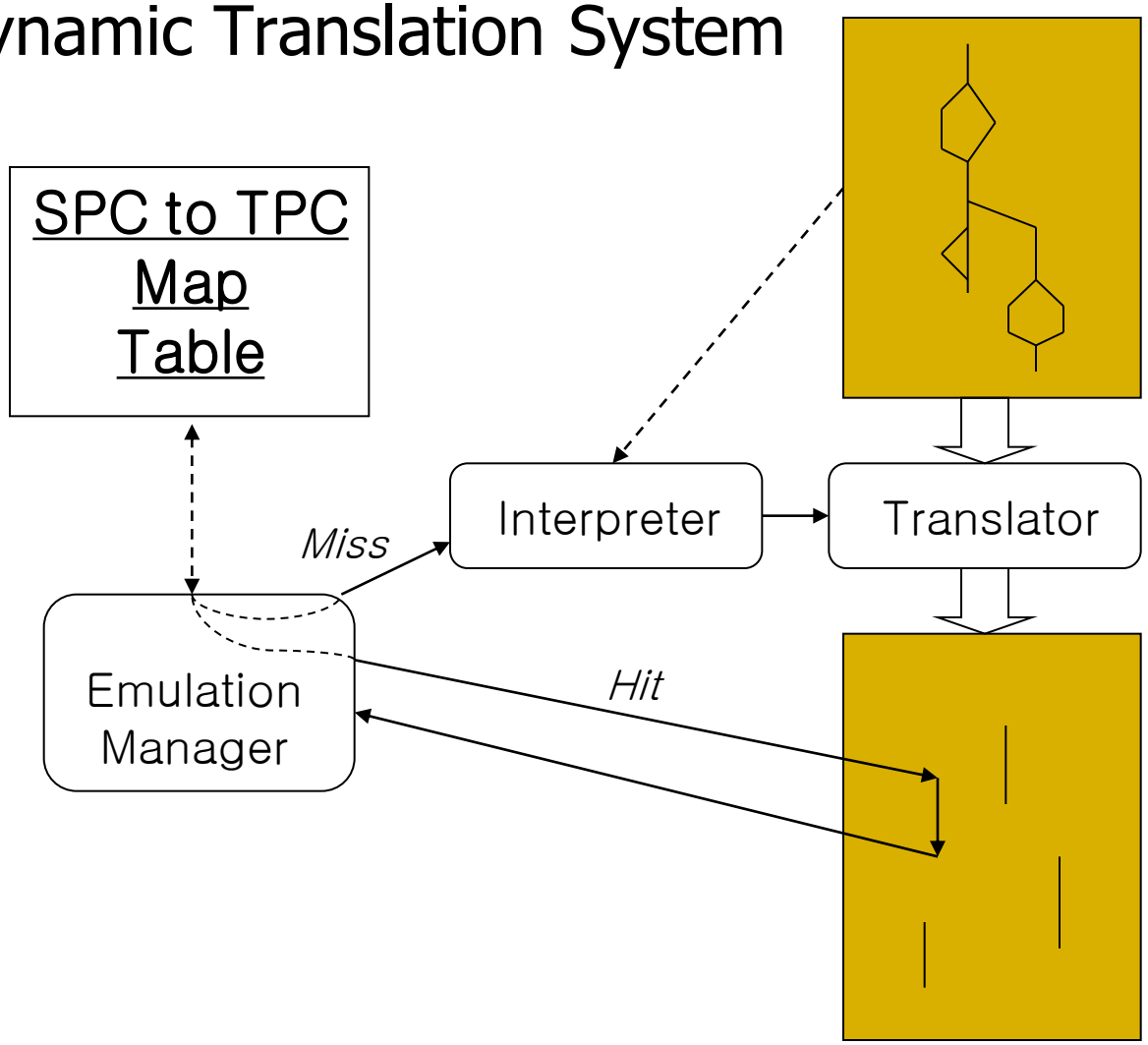
addi  r16,r4,4     ;add 4 to %eax
stwz  r7,r2,r16    ;store %edx value into memory

addi  r4,r4,4      ;increment %eax
addi  r3,r3,9      ;update PC (9 bytes)
```

- Performance enhancement
  - reduce target code size from 16 to 7 instructions
  - no redundant access to register context block (stored in memory)
  - reduce total memory accesses from 9 to 2 (same as source code!)

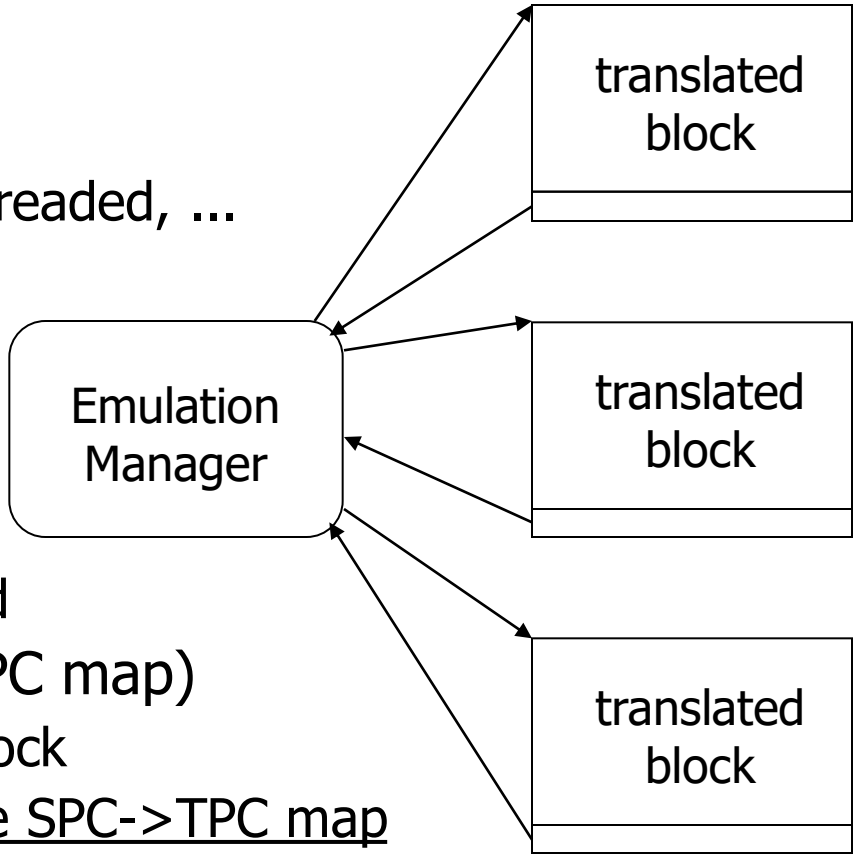
Incremental Predecoding and Translation

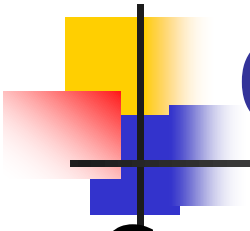
- Overview of Dynamic Translation System



Incremental Predecoding and Translation

- Control Flow with translated blocks
  - load source binary into memory
  - initially interpret
    - decode and dispatch, indirect threaded, ...
  - dinamically perform translation
    - translated code into *code cache*
    - lookup SPC→TPC mapping
  - when jump/branch found
    - (dynamic) basic block completed
  - EM looks-up target (via SPC->TPC map)
    - execute next (dynamic) basic block
    - if absent, translate it and update SPC->TPC map
    - eventually, all emulated code is translated
      - except control flow among blocks handled by EM



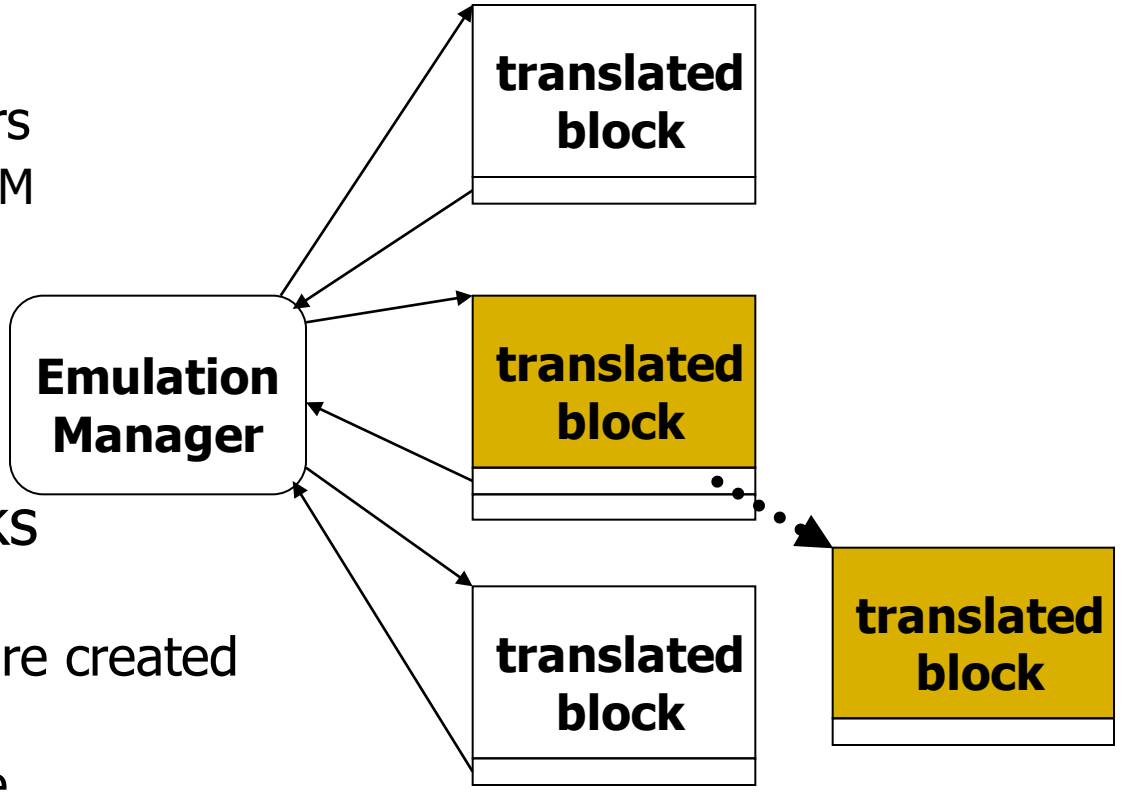


Control Transfer Optimizations

- Goal:
  - reduce/eliminate overhead of control flow emulation of the guest
  - e.g., look-up of translated blocks in branches/jumps, routine calls, …
- E.g. optimization:
  - Translation Chaining

Control Transfer Optimizations

- Translation Chaining
  - analogous to threading w.r.t. interpreters
  - goal:
    - avoid indirect transfers
      - among blocks via EM
    - improve performance
  - joins translated blocks
    - linked into chains
    - dinamically, as they are created
    - executed in sequence
    - no branch to EM code
    - how to achieve this ?
      - direct branch/jump among translated blocks



Interpretation vs Translation

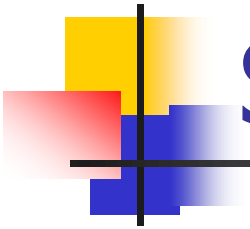
	Decode-and-Dispatch	Indirect Threaded Interpreter	Basic Predecoding	Direct Threaded Interpreter	Binary Translation
Memory requirements	Low	Low (higher than DnD)	High	High	High
Start-up performance	Fast	Fast	Slow	Slow	Very Slow
Steady-state performance	Slow	Slow (slightly better than DnD)	Medium	Medium	Fast
Code portability	Good	Good	Good	Medium	Poor

Reference: Static vs Dynamic Basic Blocks

- **Static basic blocks**: code sequences with one entry and one exit
  - begin and end with control transfer instructions, or targets of such instructions
- **Dynamic basic blocks**: determined by actual flow of control at run time
  - often larger and in greater numbers than SBB
  - begin just after a taken branch, ends with next branch
  - an instruction may belong to several DBB
  - redundant, unused ones dropped over time

```
          add…
          load…                 block 1
          store…
loop:     load…
          add…                  block 2
          store
          brcond skip
          load…                 block 3
          sub…
skip:     add…
          store                 block 4
          brcond loop
          add…
          load…
          store…                block 5
          jump indirect
          …
          …
```

```
          add…
          load…
          store…
loop:     load…                 dblock 1
          add…
          store
          brcond skip
          load…
          sub…                  dblock 2
skip:     add…
          store
          brcond loop
loop:     load…
          add…
          store                 dblock 3
          brcond skip
skip:     add…
          store                 dblock 4
          brcond loop
          …
          …
```



Summary

- Basic Interpretation
  - architected state, context blocks
  - decode and dispatch interpreter
- Threaded Interpretation
  - indirect threaded interpretation
  - basic predecoding
  - direct threaded interpretation
- Binary Translation
  - code translation
  - state mapping
  - incremental predecoding and translation
  - control transfer optimization
    - translation chaining

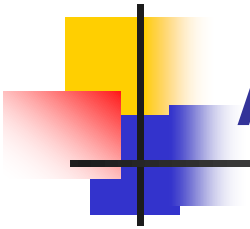


System Virtualization
(Cloud Resource Virtualization)

Prof. Luís Veiga
IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© LV, JG 2015-24



Agenda

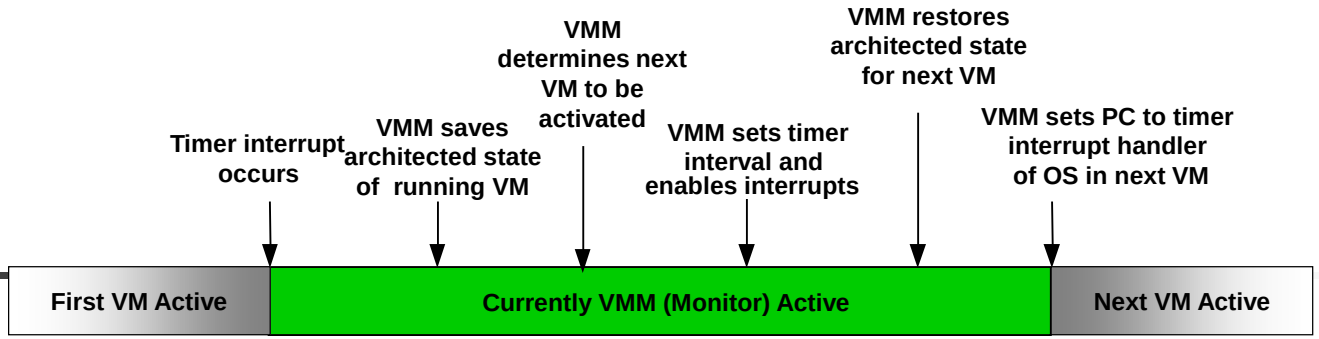
- Overview: system VM environment, resource control
- Resource Virtualization – Processor
  - ISA virtualizability – model and analysis
  - handling problem instructions
  - hybrid VM approach (VMWare)
- Resource Virtualization – Memory
  - virtual memory support in VM
  - virtualizing page tables, shadow page tables
- Performance Enhancement and Hardware Support
  - paravirtualization (Xen)
  - hardware support for virtualization (Intel VT-x)

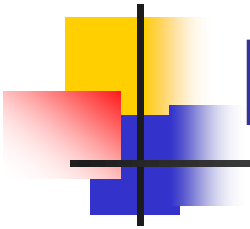
System VM Overview

- VMM main activity
  - schedule and manage
    - allocation of HW resources to various guest VMs
      - each one executing their own guest OS
  - point of control for shared physical resources, e.g.
    - CPU registers (user and system-level instructions)
    - *"actual"* physical memory in system
    - all H/W and the various I/O devices attached to system
  - efficient system operation
    - VMM operation should have *top* privileges
      - i.e. higher than *actual* privileges of supported guest VMs
        - guest OS in guest VMs believes it controls HW
        - *actual* privilege <> *perceived/believed* privilege

System VM Overview

- Resource Control
  - VMM must maintain overall control of all hardware resources
    - assign them to VMs and guarantee to get them back
    - some resources only accessible to OS
    - e.g.: the CPU interval timer/trigger that drives CPU scheduling
      - set up by OS, triggers on zero, OS regains control
      - OS hands CPU over to another process (or another guest VM)
  - approach in System VMs:
    - VMM intercepts all privileged instructions
    - VMM intercepts all accesses to privileged resources
      - traps-&-emulates the operation of these resources at all times





Processor Virtualization

- Processor Virtualization
  - Key issues (besides core emulation):
    - emulate system-level software correctly
      - execute guest OS expected functionality in each guest VM
      - prevent any guest VMs from controlling the entire system
  - Conditions for ISA virtualizability:
    - Model, architecture, properties, theorem
    - Handling problem instructions

Processor Virtualization

- Basic design goal for system VMs (identical guest/host ISA)
  - significant fraction of instructions execute directly on native H/W
  - emulate remaining instructions

- *well-behaved* (efficiently virtualizable) vs *ill-behaved* ISA
  - *well-behaved ISA*:
    - trap occurs naturally when an instruction needs to be emulated
    - handler jumps to appropriate interpreter routine
    - interprets single instruction, returns control back to original program
  - *ill-behaved ISA*:
    - difficulty in isolating instructions needing emulation (code discovery)
    - complexity: algorithms/data structures for emulation

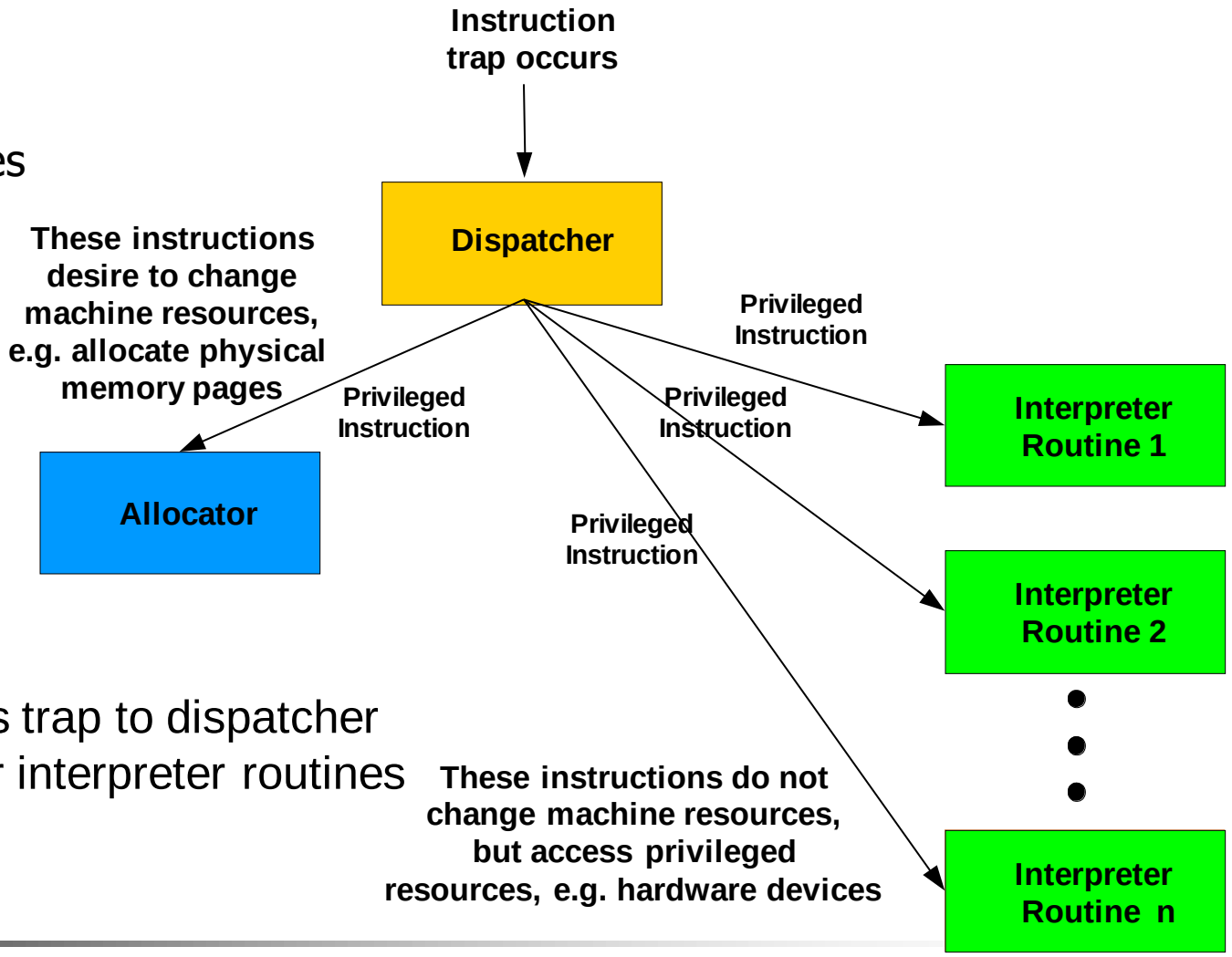
Processor Virtualization

- **Can I build a VMM for a particular ISA?**
- **Conditions for ISA virtualizability**
  - based on paper by *Popek and Goldberg (1974)*
  - following discussion restricted to native system VMs
    - only the VMM runs in system mode
    - all other software (including guest OS) runs in user mode
  - assumptions:
    - hardware: processor and uniformly addressable memory
    - processor operates in one of two modes: system or user mode
    - some subset of the instruction set is available only in system mode (*system-ISA*) and traps otherwise
    - memory addressing is done relative to contents of a relocation register (e.g., segment registers, location of page tables, etc.)

Processor Virtualization

Conditions for ISA virtualizability: Model (Architecture)

- VMM components:
  - dispatcher
  - allocator
  - interpreter routines



In well-behaved ISA:

all relevant instructions trap to dispatcher which calls allocator or interpreter routines

Processor Virtualization

- Conditions for ISA virtualizability: Model (analysis)
  - **privileged** instruction**:**
    - traps if in user mode and does not trap if machine in system mode
    - e.g., set CPU Timer (SPT, IBM System/370)
      - updates CPU interval timer to start/resume CPU time slicing
        - if CPU in system mode, traps if not:VMM manages each VM CPU timer
    - direct execution of privileged instructions
      - must be disallowed for guest OS
      - OS in a VM must not change H/W resources directly affecting other VMs
      - thus: **all guest-OS code forced to execute in user mode and trap**
  - **not privileged:** never traps
    - even if behaving differently in 2 modes (system and user)
      - e.g., does not trap but may execute with reduced functionality,
      - common source of problems (e.g., IA-32/x86, see ahead)

Processor Virtualization

- Conditions for ISA virtualizability: Model (analysis)
  - **sensitive** instructions**:** those that interact with hardware
    - **control-sensitive:**
      - attempt to change configuration of system resources
      - e.g., Set CPU Timer
    - **behavior-sensitive:**
      - behavior/results produced depend on resource configuration
      - e.g., Pop Stack into Flags Register (POPF, IA-32)
        - pops CPU status flag registers from stack held in memory
        - system mode: updates/restores all CPU status flags
        - user mode: updates/restores only user-level CPU flags
  - **innocuous** instructions: neither control nor behavior-sensitive

Processor Virtualization

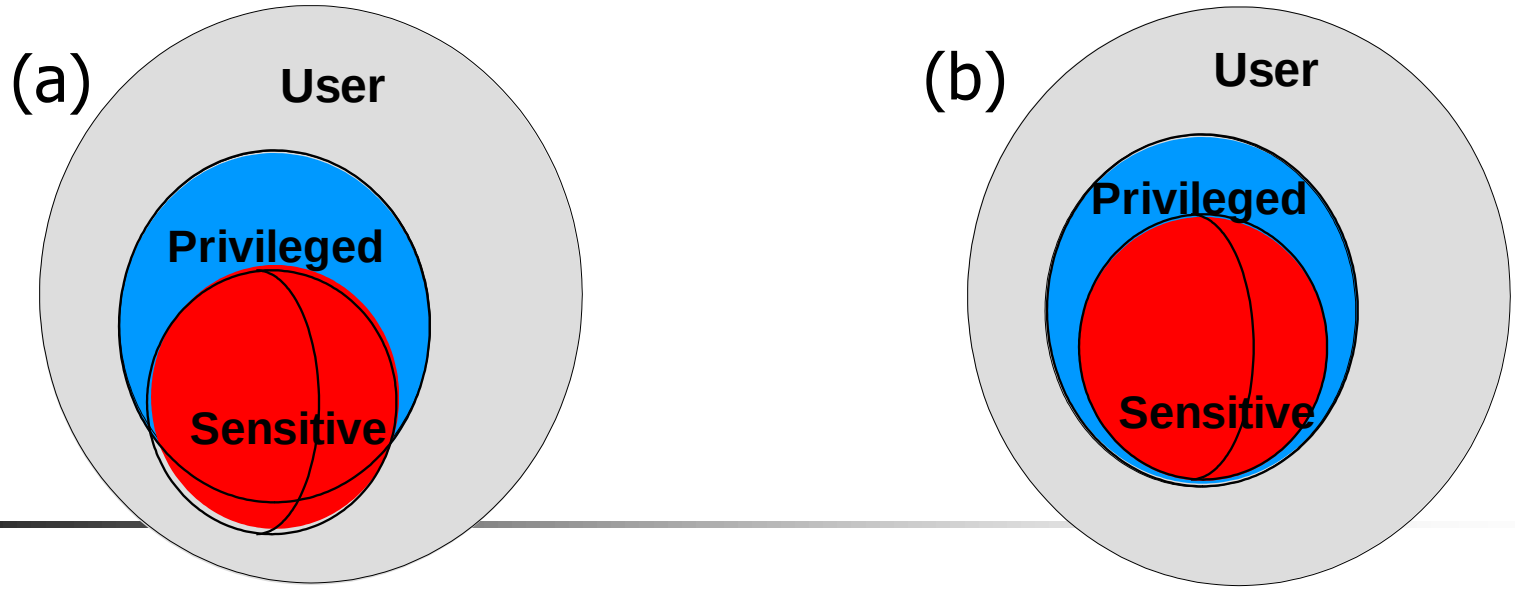
- Conditions for ISA virtualizability: Properties
  - Popek/Goldberg: properties of true *efficient* VMM
    - efficiency
      - all innocuous instructions must be executed natively on hardware
    - resource control:
      - it should not be possible for guest software:
      - to directly change configuration of any system resources
    - equivalence:
      - any program executing on a virtual machine:
        - must behave in identical manner as if running natively
      - with only a few exceptions, e.g.:
        - slower performance to the emulation overhead
        - limitations on the extent of resources available (memory, disk)

Processor Virtualization

- Conditions for ISA virtualizability: Theorem
  - **Theorem 1 - (*efficient*) VMM construction:**
    - *(…) a virtual machine monitor may be constructed if the set of **sensitive instructions** for that computer **is a subset** of the set of **privileged instructions.***
  - implications:
    - all non-privileged instructions can be executed natively
    - an efficient VM implementation can be constructed if
      - instructions that could interfere with correct/efficient functioning of VMM always trap in user mode
        - in order to force control back to VMM.
        - which ones? sensitive instructions.
    - theorem is useful because it suggests
      - convenient assessing of ISA determining feasibility of efficient VMM

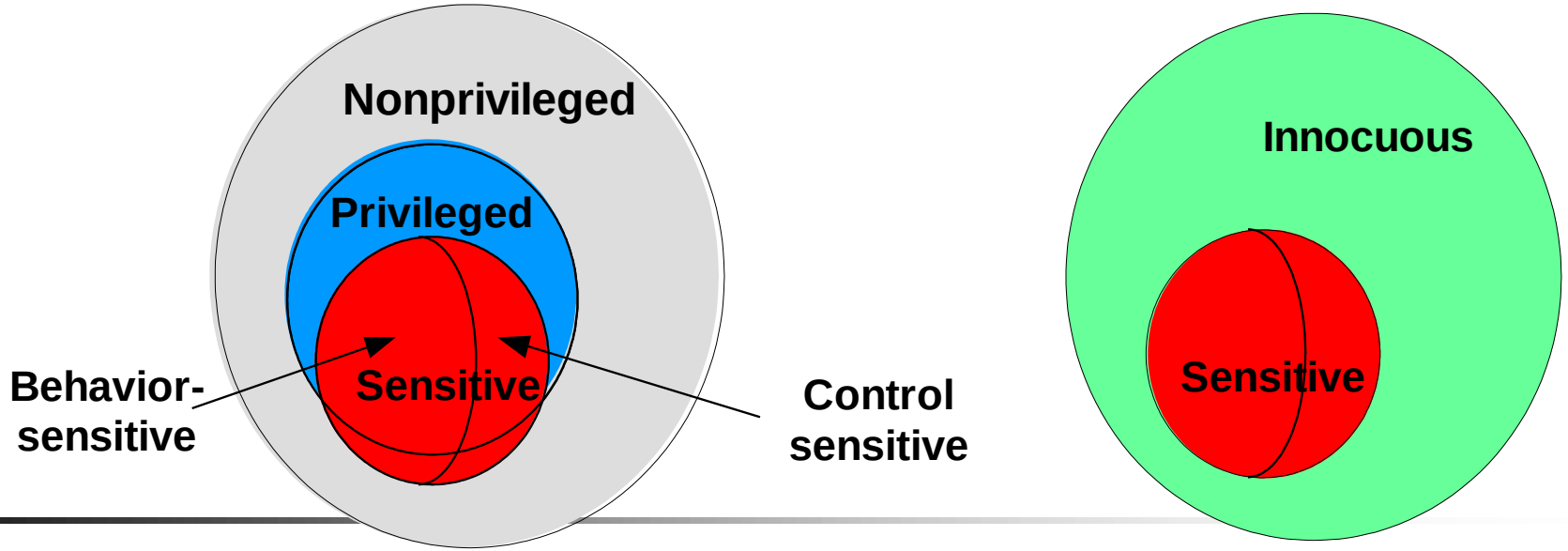
Processor Virtualization

- Conditions for ISA virtualizability
  - e.g., (*efficient*) VMM construction:
    - (a) does not satisfy theorem condition
      - contains **critical instructions:** sensitive yet not privileged
    - (b) satisfies condition → efficiently virtualizable
      - all sensitive instructions trap, control back to VMM
        - emulates them with overall VMM information



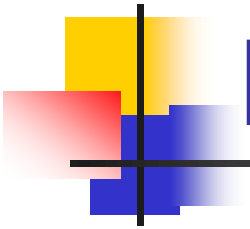
Processor Virtualization

- Conditions for ISA virtualizability:
  - e.g. problem Instructions
  - **sensitive** subset of **privileged**? Not necessarily, e.g., POPF
    - system mode: updates/restores all CPU status flags
    - user mode: updates/restores only user-level CPU flags
      - i.e., fails silently in guest OS (in user mode) no trap to VMM
    - correct handling: different if executed by guest OS or application



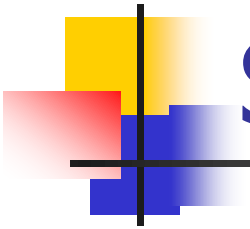
Processor Virtualization

- Handling Problem Instructions
  - e.g., POPF instruction is sensitive but not privileged
    - **critical instruction** (sensitive but not privileged)
      - does not generate a trap in user-level
      - but has different, limited functionality w.r.t. system-level
      - violates the virtualizability condition of Theorem 1
    - IA-32 includes several (~17) such critical instructions
  - Yet, System VM implementation possible...
    - ...inspite of the existence of *critical* instructions...
    - ...given that x86 system VMs are...mostly everywhere



Processor Virtualization

- Handling Problem Instructions
  - Popek/Goldberg assume all non-provileged instructions executed natively
    - Q: how to handle / emulate critical instructions ?
  - Solution: Hybrid virtual machine system (e.g., VMWare)
    - virtual machine system in which
      - some non-privileged instructions must be specially emulated
        - e.g., critical instructions not executed directly
      - using techniques related to binary translation
        - VMM intercepts and scans guest code before execution
        - discovers all critical instructions
        - replaces/translates them with explicit trap to the VMM



System Virtual Machine

- Resource Virtualization: Memory
  - Virtual memory support in System VM environment
  - Virtualizing architected page tables:
    - page tables are usually managed by the (guest) OS
      - believing it is in direct control of the physical memory
    - VMM intercepts this:
      - sole entity managing the physical memory
      - provides illusion of control to each guest OS

Memory Virtualization

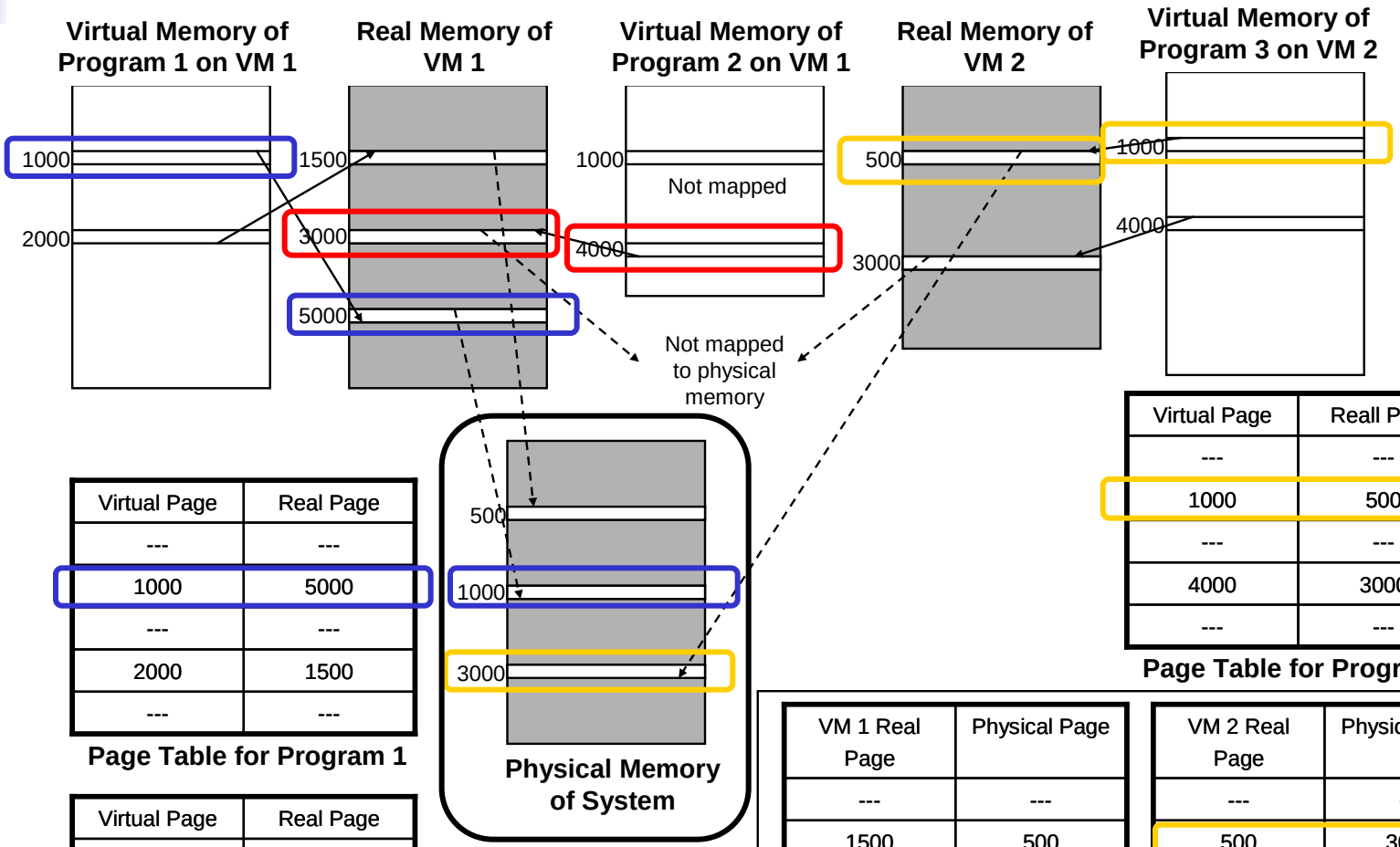
- Virtual memory support in System VM
  - Virtual memory in native system:
    - most ISA support address translation:
      - virtual to real addresses
      - e.g., using page tables
    - in native execution:
      - real addresses → physical addresses

Memory Virtualization

- Virtual memory support in System VM
  - Virtual memory in system VM:
    - *real* memory and physical memory are different
      - *real* : guest VM *illusion* of physical memory supported by VMM
      - physical memory: the *actual* hardware memory
    - guest memory space requires further mapping to host H/W
      - *real* addresses must still be translated to physical addresses
      - new level in the memory hierarchy
        - **virtual address → *real* address → physical address**
    - VMM maintains *real map table:* *real* pages → physical pages
      - VMM maintains own swap space
      - distinct from the swap spaces of each of the guests

Memory Virtualization

Memory Mapping of Guest VMs



Virtual Page	Real Page
---	---
1000	5000
---	---
2000	1500
---	---

Page Table for Program 1

Virtual Page	Real Page
---	---
1000	Not mapped
---	---
4000	3000
---	---

Page Table for Program 2

Virtual Page	Reall Page
---	---
1000	500
---	---
4000	3000
---	---

Page Table for Program 3

VM 1 Real Page	Physical Page
---	---
1500	500
---	---
3000	Not mapped
---	---
5000	1000
---	---

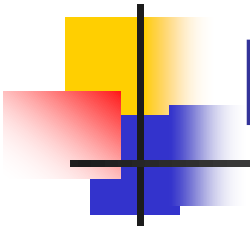
Real Map Table for VM1

VM 2 Real Page	Physical Page
---	---
500	3000
---	---
3000	Not mapped
---	---

Real Map Table for VM2

Memory Virtualization

- Virtual memory support in System VM
  - contemporary platforms (most ISA)
    - page translation supported by *page table* and *TLB*
    - one of them is OS-managed depending on ISA
  - OS-managed (*architected*) page table:
    - structured defined by ISA, e.g., IA-32
    - maintained by OS and hardware cooperation
      - page table pointer register (e.g., PTPR in IBM, CR3 in IA-32)
    - TLB is maintained and used only by H/W
      - not directly visible to OS
    - when TLB miss (*page fault*)
      - H/W MMU walks page table to find an entry
      - if there is none → page not mapped to real memory
      - there is page fault and OS takes over to load/map page



Memory Virtualization

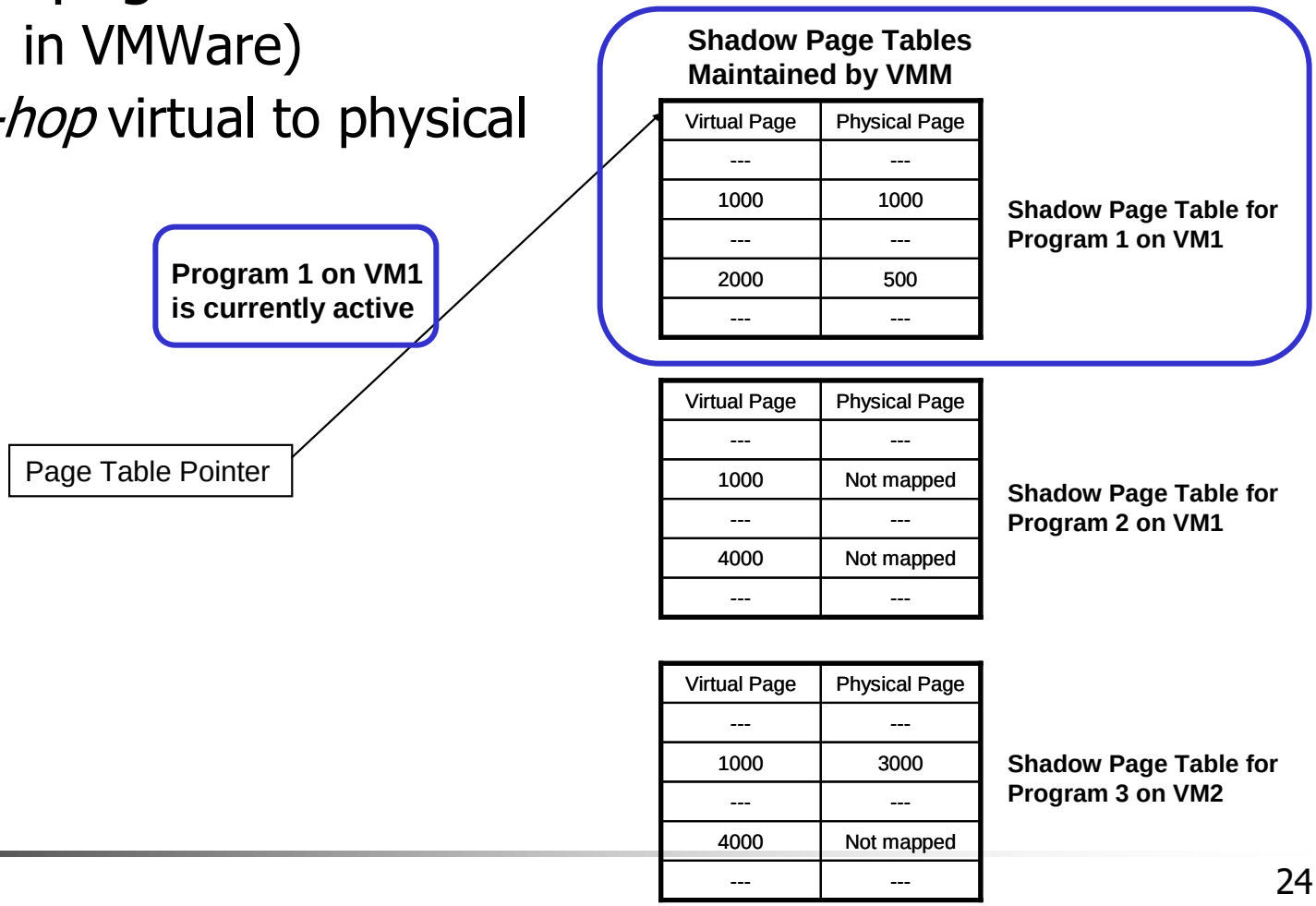
- Virtualizing architected page tables
  - OS in each guest VM maintains own page tables:
    - virtual → *real*
  - VMM maintains optimized mapping information
    - shadow page tables
      - one for each process in each guest VM
    - direct virtual-to-physical mapping
      - to avoid extra level of indirection
    - used by H/W to translate virtual addresses fast
      - (and to keep TLB up-to-date)

Memory Virtualization

- Virtualizing architected page tables
  - **(using shadow page tables for efficiency)**
- approach:
  - page table pointer register (PTPR) is also virtualized
    - e.g., page table/directory base register, (e.g. CR3 on IA-32)
  - VMM manages real PTPR and keeps *virtual* PTPR for each VM
    - when guest VM activated, real PTPR points to shadow page table
  - when guest OS attempts to access/modify (its) PTPR:
    - it is intercepted by the VMM
      - trapped because privileged or previously patched instruction
    - VMM accesses *virtual* PTPR while updating the real PTPR if needed
      - read: return guest *virtual* PTPR
      - write: update *virtual* PTPR and real PTPR to guest *shadow table*

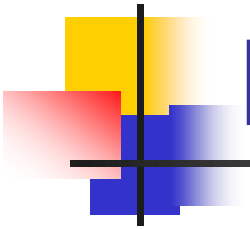
Memory Virtualization

- Virtualizing architected page tables
  - *shadow* page tables
    - (e.g. in VMWare)
    - *one-hop* virtual to physical



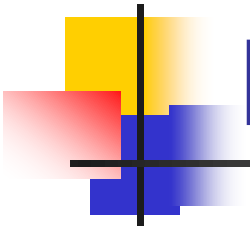
Performance Enhancement

- System VMs improve hardware utilization
  - sharing resources among multiple users/applications
  - illusion to guest OS they own all machine resources
- Difficulties:
  - expectation of identical performance to full machine
  - but there are still many sources of overhead:
    - e.g. setup overhead, emulation (e.g. sensitive instructions),
    - context switching (VMM→guest VM→guest OS)
    - state saving/restoring CPU/VM state, bookkeeping (e.g., timers)
- Q: How to enhance performance:
  - use knowledge of guest OS structure (*paravirtualization*)
  - develop H/W extensions (*e.g, Intel VT-x*)



Paravirtualization

- Paravirtualization
  - insight: possible to get performance improvements
    - if (guest) OS is aware of native or guest VM execution
    - guest OS can issue fewer privileged instructions
      - smaller degradation in performance
    - guest OS can relegate some functions to VMM
      - providing additional info unavailable/difficult to VMM
      - *handshaking*
        - normalized instructions/messages guest OS → VMM
        - e.g., hypercalls in *Xen* and *Windows Hyper-V.*
  - advantages
    - eliminating effort duplication / overhead by guest OS/VMM

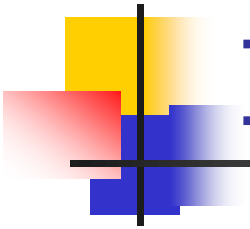


Paravirtualization

- Paravirtualization
  - VM performance improved with guest OS modified
    - e.g., Xen *hypervisor* with paravirtualized Linux
      - provides (different) interface to guest OS
      - workaround features difficult to virtualize in IA-32
  - modified machine-dependent parts of guest OS:
    - replace some guest OS instructions with *hypercalls*
      - including critical but also privileged
      - also improving latency and overall efficiency
    - eliminates need for complex virtualization
      - code detection/discovery, code patching
      - shadow table maintenance
    - guest OS receive traps as queued events sent by Xen

Intel VT-x *(Vanderpool)*

- Background/Motivation
  - IA-32/x86 is notoriously difficulty to virtualize
    - code discovery/patching in VMWare
    - guest OS source modification with hypercalls in Xen
      - lower virtualization overhead of sensitive instructions
      - still context switching among guest OS and Xen
      - programming/development effort in guest OS adaptation
  - Inspiration: IBM VM *world* (VM/370 ... z/VM)
    - CPU hardware enhancements to improve virtualization



Intel VT-x *(Vanderpool)*

- Intel VT-x
  - H/W enhancements for IA-32 VMM implementation
    - new CPU modes
    - new instructions
    - new internal data structures
  - available/active in *most recent* CPUs
    - e.g, Xeon, i5, i7, i9, …

Intel VT-x *(Vanderpool)*

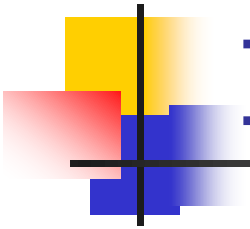
- VMX CPU mode operation
  - VMX root mode, VMX non-root mode
    - all x86 privilege levels (i.e. four) available in both modes

- VMX root mode
  - full privilege access to H/W
    - similar to conventional unprotected x86 operation
  - access to new VMX instructions
  - intended solely for VMM operation

Intel VT-x *(Vanderpool)*

- VMX CPU mode operation
  - VMX non-root mode
    - limited control of global resources
      - even when in privilege level 0 (e.g., *system* ISA)
    - intended for applications, guest OS, VM instances
    - code is executed in original privilege level
      - *standard* guest OS can run unmodified in level 0
        - still under VMM control
      - guest applications run in level 3
    - *eliminates need for paravirtualization but still allows it*

Intel VT-x *(Vanderpool)*



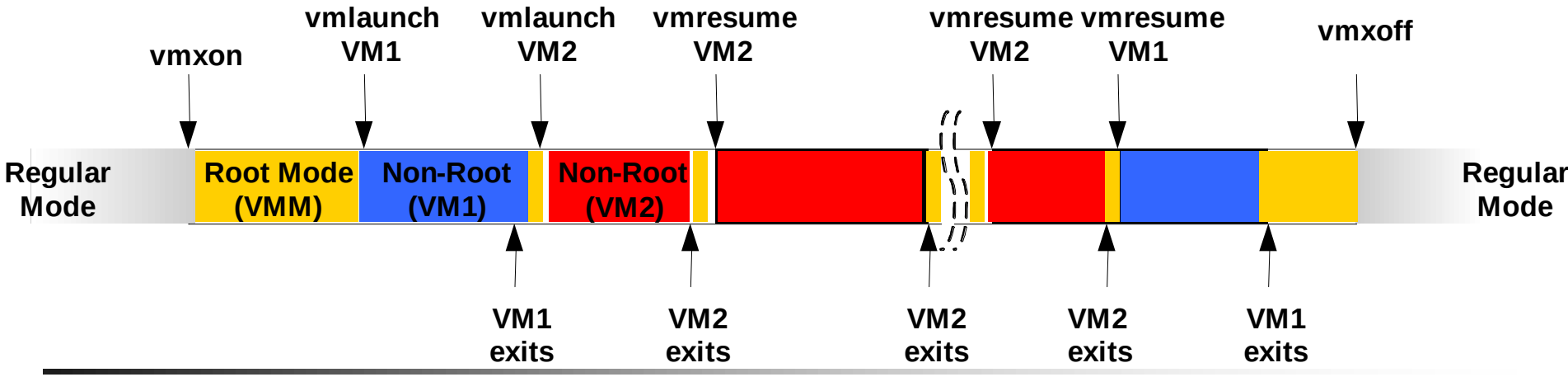
- VT-x CPU instructions:
  - VMX CPU operation mode management
    - vmxon (start VMX)
      - boot up in root mode, initializes VMM environment
    - vmxoff (shutdown VMX)
      - returns CPU to normal operation mode
      - removes state associated with guest VMs
  - Guest VM scheduling
  - Guest VM state management
  - *Hypervisor* call (special dedicated instruction)

Intel VT-x *(Vanderpool)*

- VT-x CPU instructions
  - Guest VM scheduling
    - vmlaunch: initiates guest VM
      - executed by VMM in root mode
      - CPU starts executing guest VM code in non-root mode
      - if guest code attempts to modify resources reserved by VMM
        - causes control return to to VMM code,
        - registers reason of guest VM interception,
        - executes VMM code in root mode
    - vmresume: re-schedules a pre-existing VM instance
      - causes return to guest in non-root operation mode

Intel VT-x *(Vanderpool)*

- E.g., guest VM scheduling under VT-x
  - System VM operation
    - vmxon → vmxoff
  - guest VMs: VM1, VM2 launched and rescheduled
  - switch among guest VMs
    - never direct, always through VMM → root operation mode
    - access to reserved resources, interrupt, or hypercall



Intel VT-x *(Vanderpool)*

- VT-x CPU instructions
  - *Hypervisor* cal (or *hypercall)*
    - vmcall:
    - only instruction executable in non-root mode
    - purpose:
      - explicitly signal a condition to VMM
      - request a VMM service
    - parameter information is implementation dependent
      - guest OS must know *interface/contract* of each hypervisor
        - e.g. *Xen, Hyper-V*

Intel VT-x *(Vanderpool)*

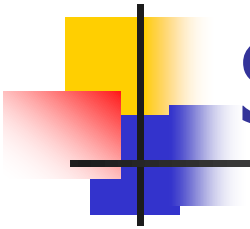
- Guest VM state management
  - guest VM switching requires state saved/restored accordingly
  - major overhead, explicit memory load/store instructions
  - previously, guest VM architected (differently) by each VMM

  - VT-x provides H/W implementation of guest VM state
    - VMCS (virtual machine control structure) encapsulates VM state
      - manipulated by H/W and in root mode
    - H/W loads, maps, saves, restores guest VM state from memory

  - VMCS design:
    - fully specified, yet, opaque, machine/CPU-independent
    - read/written with special VT-x instructions, e.g. vmread/vmwrite

Intel VT-x *(Vanderpool)*

- Advantages: promise faster VM and VMM operation
  - new operation mode for VMM
  - guest code without access to critical resources
    - runs natively, no need for code scanning & patching
  - save/restore guest VM state
    - major source of overhead in software-based system VM
      - even with optimizations such as register-register mapping
    - mostly performed via H/W in VT-x using VMCS
      - allows H/W to know of active guest VMs
        - H/W even knows which one is executing
  - safety/extensibility
    - VMCS opaque, H/W implemented, accessible only via operations



Summary

- Overview: system VM environment, resource control
- Resource Virtualization – Processor
  - ISA virtualizability – model and analysis
  - handling problem instructions
  - hybrid VM approach (VMWare)
- Resource Virtualization – Memory
  - virtual memory support in VM
  - virtualizing page tables, shadow page tables
- Performance Enhancement and Hardware Support
  - paravirtualization (Xen)
  - hardware support for virtualization (Intel VT-x)

Reference: Processor Virtualization

- Conditions for ISA virtualizability:
  - Handling instructions without problems
  - e.g, emulating SPT (*set CPU timer*) instruction:
    - e.g., guest OS scheduling time-slices from guest processes
    - instruction is privileged→ generates trap
    - VMM examines contents to be loaded into CPU timer
      - if within time-slice of guest VM, allow it
      - otherwise, adjust it (trim it)
    - meanwhile:
      - VMM keeps time differences in internal table
      - allows proper restoration when guest VM is activated again

Reference: Memory Swapping in VMs

- Handling memory page faults in system VMs:
  - i.e., no entry in VMM page table of guest application
    - virtualizing architected page tables managed by guest OS
  - page **mapped** in virtual page table of guest app?
    - page fault should be handled solely by VMM
      - *(...guest OS is being tricked...)*
      - VMM previously moved real page to its own swap space
      - e.g. when total real memory of VMs > physical memory
    - VMM brings real page back into physical memory
    - VMM updates real map table and affected shadow table(s)
    - guest OS is not informed of any page fault
      - guest OS did not experience any fault
      - *(...guest OS is being tricked...)*

Reference: Memory Swapping in VMs

- Handling memory page faults in system VMs:
  - i.e., no entry in VMM page table of guest application
    - virtualizing architected page tables managed by guest OS
  - page ***not* mapped** in virtual page table of guest app?
    - guest OS should handle its own fault in *virtualized* manner
    - guest OS issues instruction to modify its page table
    - VMM intercepts this request
      - privileged access to write-protected page table
    - VMM updates guest page table and
      - updates mapping in appropriate *shadow* page table
    - returns control to guest VM
    - *(…guest OS believes it is in control…)*

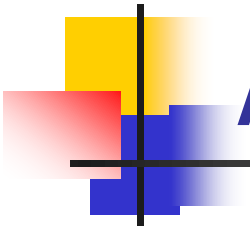


IaaS Cloud Execution and Monitoring (Part A)

Prof. Luís Veiga
IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© LV, JG 2015-24 ©Figures: Cloud Computing - Theory and Practice, Dan C. Marinescu, MK.



Agenda

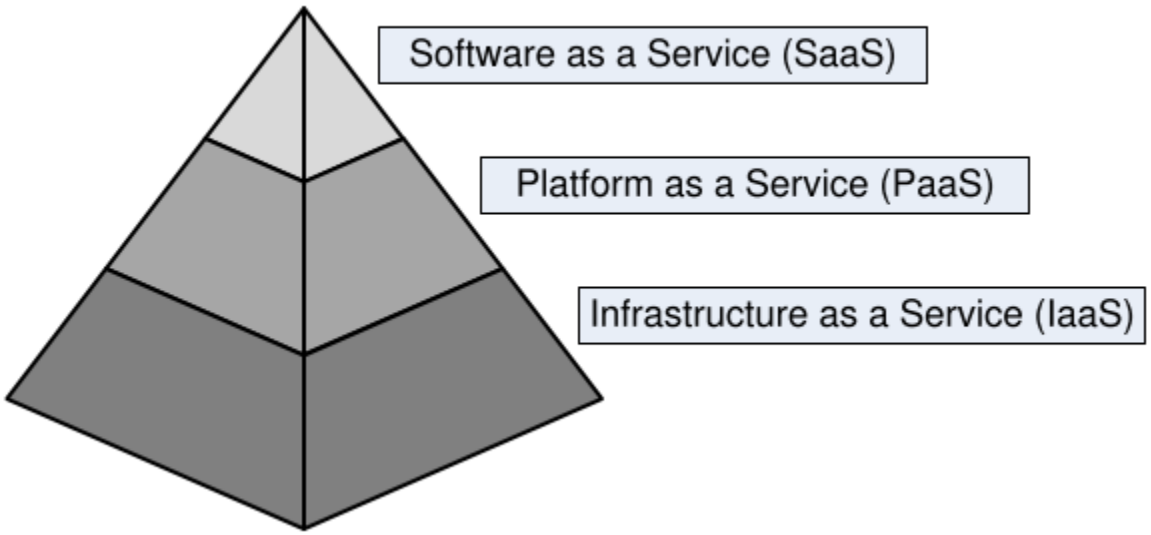
- Cloud Execution Platforms and Models:
  - Overview: existing cloud infrastructure
  - Amazon Web Services
    - AWS Main Architecture, AWS Ecosystem
  - AWS EC2 - Elastic Compute Cloud (public)
- Cloud Monitoring:
  - AWS
    - CloudWatch, Elastic Load Balancing, AutoScale

Cloud Computing Overview

- The cloud computing infrastructure at Amazon, Google, and Microsoft (when all 3 became players c. mid 2012).
  - Amazon is a pioneer in Infrastructure-as-a-Service (IaaS).
  - Google's efforts were focused on Software-as-a-Service (SaaS) and Platform-as-a-Service (PaaS).
  - Microsoft became involved in Azure which blends PaaS and IaaS.
- Private clouds are an alternative to public clouds. Open-source cloud computing platforms such as…:
  - OpenStack, CloudStack
  - Eucalyptus, OpenNebula, Nimbus
  - Docker Swarm
  - Google Kubernettes
- …can be used as a control infrastructure for a private cloud.

Cloud Computing Overview

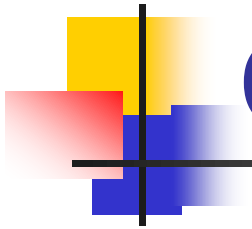
- Cloud Delivery Models (review)



Cloud Computing Overview

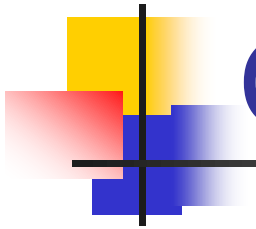
- Software-as-a-Service (SaaS)
  - only runs applications developed by service provider
  - services use thin client interface such as a web browser
  - users do not control underlying cloud infrastructure
    - i.e., network, servers, operating systems, storage, or even individual application capabilities.
- E.g. enterprise services:
  - group-ware and collaborative, supply chain, customer relationship management, desktop software, financial management, search.
- E.g. Web 2.0 applications:
  - social networking, blogs, wiki services, portal services.

Cloud Computing Overview

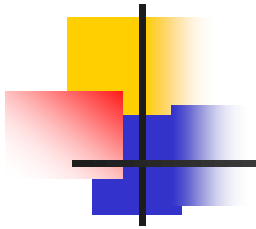


- Platform-as-a-Service (PaaS)
  - offers capability to deploy consumer-created or acquired applications
  - uses programming languages and tools supported by the provider.
  - user does not control the underlying cloud infrastructure
  - user has control over:
    - deployed applications
    - application hosting environment configurations.
- E.g., Google App Engine
  - Java, Python applications

Cloud Computing Overview

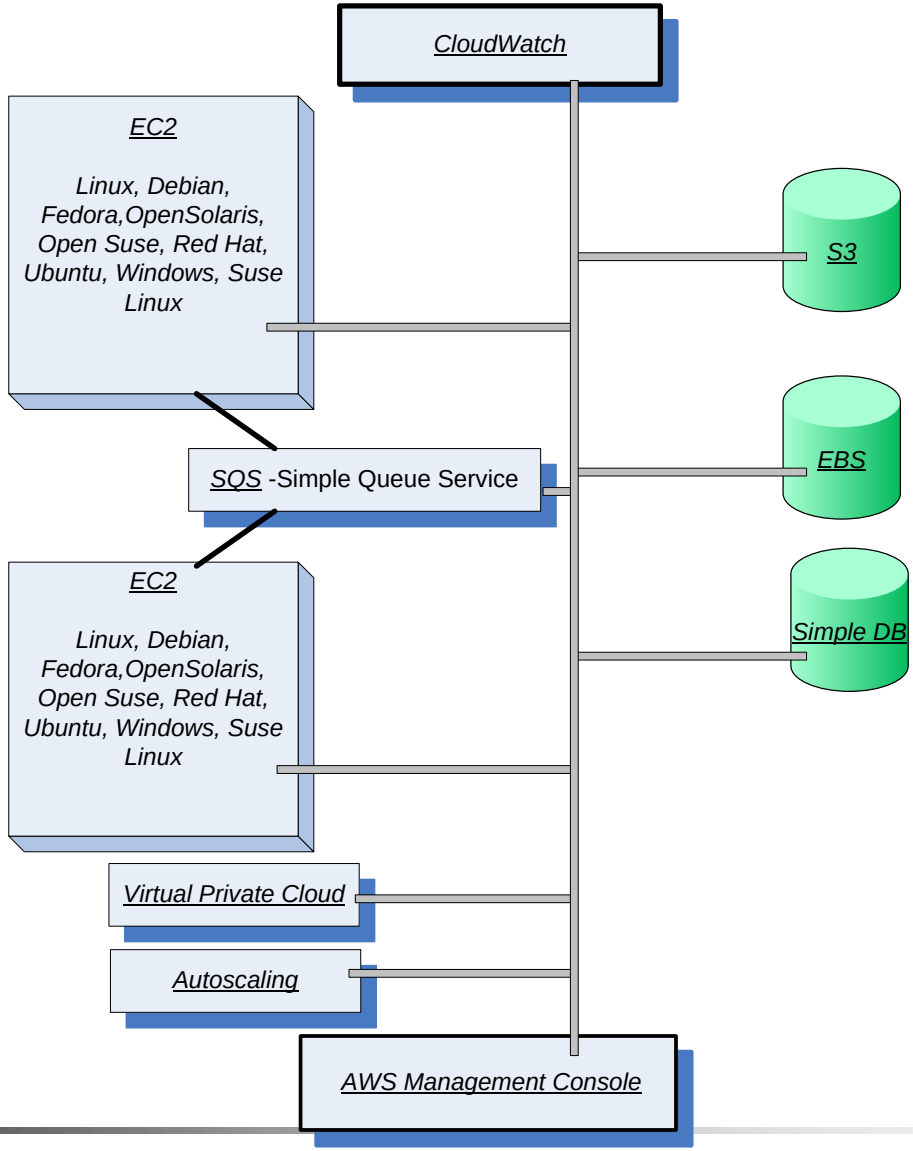


- Infrastructure-as-a-Service (IaaS)
  - capability to provision processing, storage, networks, …
  - user able to deploy and run arbitrary software
  - user has control over:
    - operating systems, storage, deployed applications, middleware, libraries, platforms.
  - user does not control underlying cloud infrastructure
- E.g. AWS Elastic Compute Cloud



Amazon
AWS
Amazon Web Services

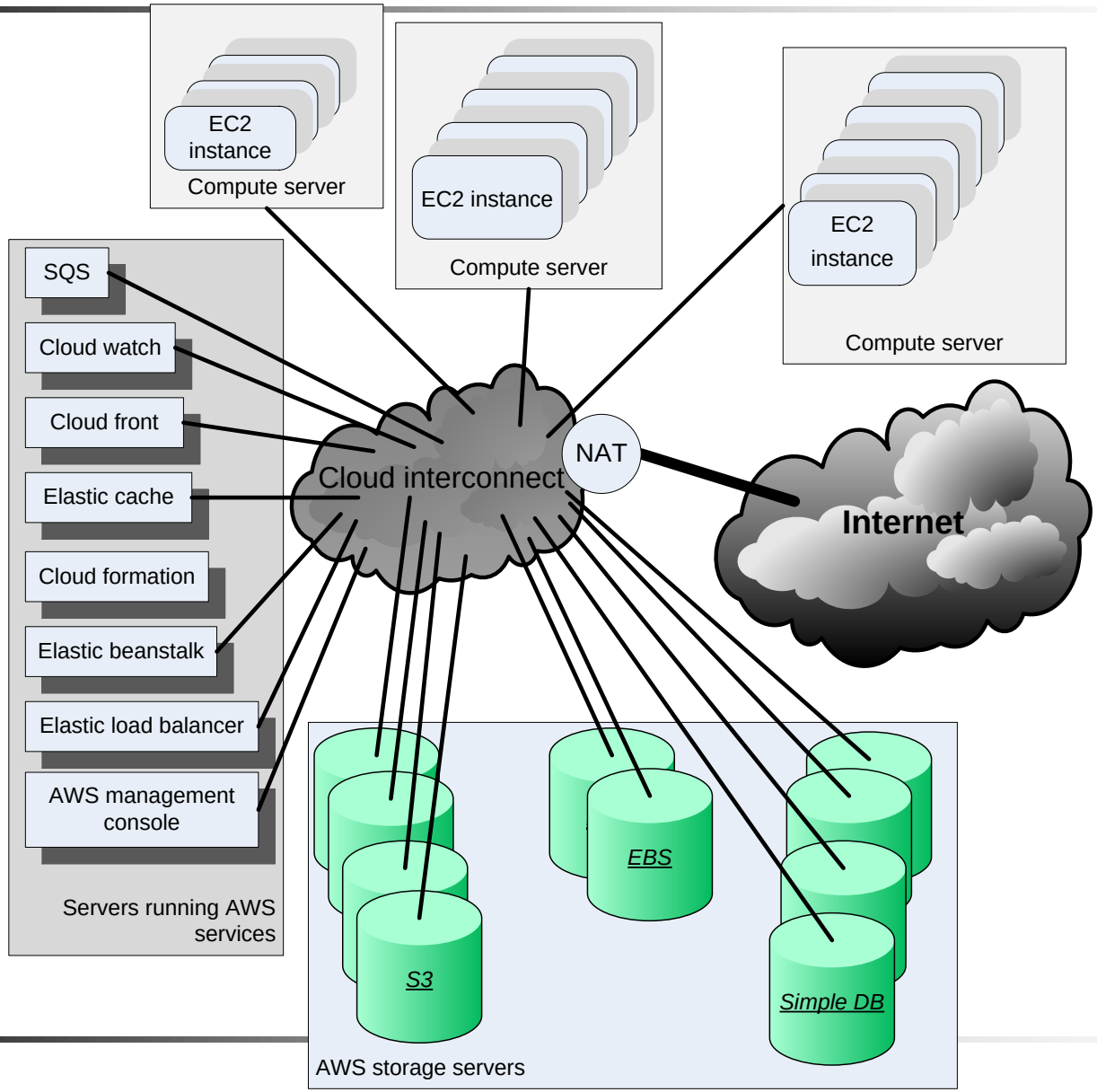
AWS Main Architecture



AWS Main Architecture

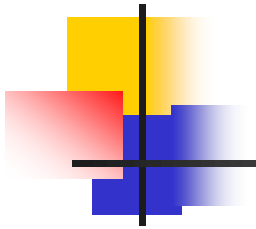
- AWS Management Console
  - allows users to access the services offered by AWS
- Elastic Cloud Computing (EC2)
  - allows user to launch a variety of operating systems
- Simple Queuing Service (SQS)
  - allows multiple EC2 instances to communicate
- Storage services
  - Simple Storage Service (S3), Simple DB, Elastic Block Storage (EBS), DynamoDB
- Cloud Watch: supports performance monitoring
- Auto Scaling: supports elastic resource management

AWS Overall Ecosystem



AWS Overall Ecosystem

- Elastic Cache
  - in-memory caching for web applications
- CloudFront
  - web service for content delivery (CDN)
- Elastic Load Balancer
  - automatically distributes incoming requests over multiple instances.
- CloudFormation
  - allows the creation of templates of a stack/collection describing the infrastructure for an application.
- Elastic Beanstalk
  - handles application automatically deployment, capacity provisioning, load balancing, auto-scaling, and monitoring.



Amazon EC2:
Elastic Compute Cloud

AWS Regions and AZs

- Amazon offers cloud services through a network of data centers on several continents.
- In each *Region:*
  - there are several Availability Zones
  - AZs are interconnected by high-speed networks
- An *Availability Zone* is:
  - a data center consisting of a large number of servers.
- Regions do not share resources and communicate through the Internet.

AWS Regions and AZs

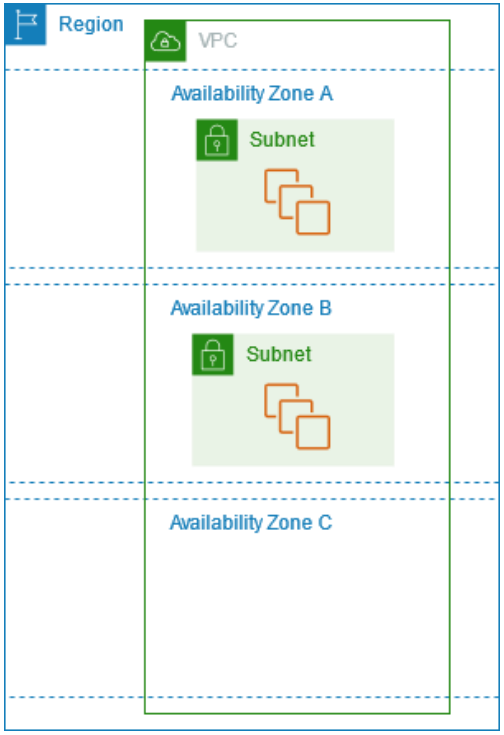
- Examples of current AWS Regions (green) and projected in the near future (red)



AWS Regions and AZs

- E.g., Regions and Availability Zones

Region	Location	Availability Zones
US West	Oregon	us-west-2a/2b/2c
US West	North California	us-west-1a/1b/1c
US East	North Virginia	us-east-1a/2a/3a/4a
Europe	Ireland	eu-west-1a/1b/1c
South America	Sao Paulo, Brazil	sa-east-1a/1b
Asia/Pacific	Tokyo, Japan	ap-northeast-1a/1b
Asia/Pacific	Singapore	ap-southeast-1a/1b



- Further location towards network edge:
  - **Wavelength Zones:** enables applications with ultra-low latency to mobile devices and end users using carriers' 5G networks.
  - **AWS Outposts (on customer premises, AWS-managed):** extends AWS infrastructure, services, APIs to customer premises, using local pool of compute and storage for lower latency and data processing needs.

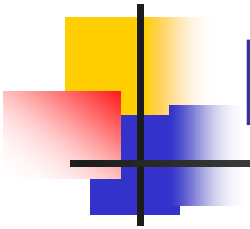
AWS instances

- An instance is a virtual server (system VM) with a well specified set of resources including:
  - CPU cycles, main memory, secondary storage, communication and I/O bandwidth.
- The user chooses:
  - The region and the availability zone where this virtual server should be placed.
  - An instance type from a *limited* menu of instance types.
- When launched, an instance is provided with a DNS name; this name maps to a:
  - private IP address: for internal communication within the internal EC2 communication network.
  - public IP address: for communication outside the internal Amazon network, e.g., for communication with the user that launched the instance.

AWS instances (cont.)

- Network Address Translation (NAT)
  - maps external IP addresses to internal ones.
- The public IP address is assigned only for the lifetime of an instance.
- An instance can request an elastic IP address, rather than a public IP address.
  - the elastic IP address is a static public IP address allocated to an instance from the available pool of the availability zone and paid for.
  - an elastic IP address is not released when the instance is stopped or terminated and must be released when no longer needed.

Elastic Compute Cloud



- On-demand virtual server provisioning service
  - provisions VM instances requested by user/programs
- Uses disk images created by the users and then transferred, or pre-built by Amazon
  - AMI, Amazon Machine Images
- Low granularity payments
  - e.g. hourly, 5-minute, 1-minute periods
- Access via Web Services
  - key enabler for adoption
  - with language-specific SDKs on top of the WS APIs

Instances (detail)

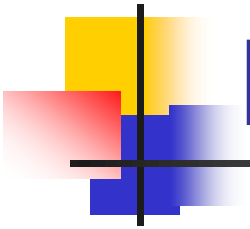
- Instances:
  - (system) guest virtual machines running particular image (AMI).
- CPU performance is measured
  - using *"EC2 Compute Unit"* (empirical unit designed by Amazon) ~ 1.0 to 1.2 GHz 2007 AMD Opteron.
- The virtual machines do not match the physical machines but have clearly defined characteristics:
  - Small: 32-bit, 1 ECU, 1,7GB RAM, 160 GB HD temporary.
  - Large: 64-bit, 2x2 ECU, 7,5 GB RAM, 850 GB HD temp.
  - XL: 64-bit, 4x2 ECU, 15 GB RAM, 1690 GB HD temp.
- Other types: micro, high memory/CPU, cluster,...

Instances (detail)

- E.g. Instance Types

Instance Name	API Name	Platform (32/64-bit)	Memory (GB)	Max *EC2* Compute Units	I-Memory (GB)	I/O (M/H)
StdM		32 and 64	0.633	1 VC; 2 CUs		
StdS	`m1.small`	32	1.7	1 VC; 1 CU	160	M
StdL	`m1.large`	64	7.5	2 VCs; 2 × 2 CUs	85	H
StdXL	`m1.xlarge`	64	15	4 VCs; 4 × 2 CUs	1,690	H
HmXL	`m2.xlarge`	64	17.1	2 VCs; 2 × 3.25 CUs	420	M
Hm2XL	`m2.2xlarge`	64	34.2	4 VCs; 4 × 3.25 CUs	850	H
Hm4XL	`m2.4xlarge`	64	68.4	8 VCs; 8 × 3.25 CUs	1,690	H
HcpuXL	`c1.xlarge`	64	7	8 VCs; 8 × 2.5 CUs	1,690	H
Cl4XL	`cc1.4xlarge`	64	18	33.5 CUs	1,690	H

Examples of AMIs



- OSs:
  - Red Hat Linux, Windows Server, OpenSolaris, Ubuntu, Debian
- DBs:
  - DB2, SQL Server, mySQL
- High performance computing:
  - Hadoop, Condor, OpenMPI
- Web Servers:
  - Apache, IIS
- And many more…
- **Beware of user submitted AMIs!!**

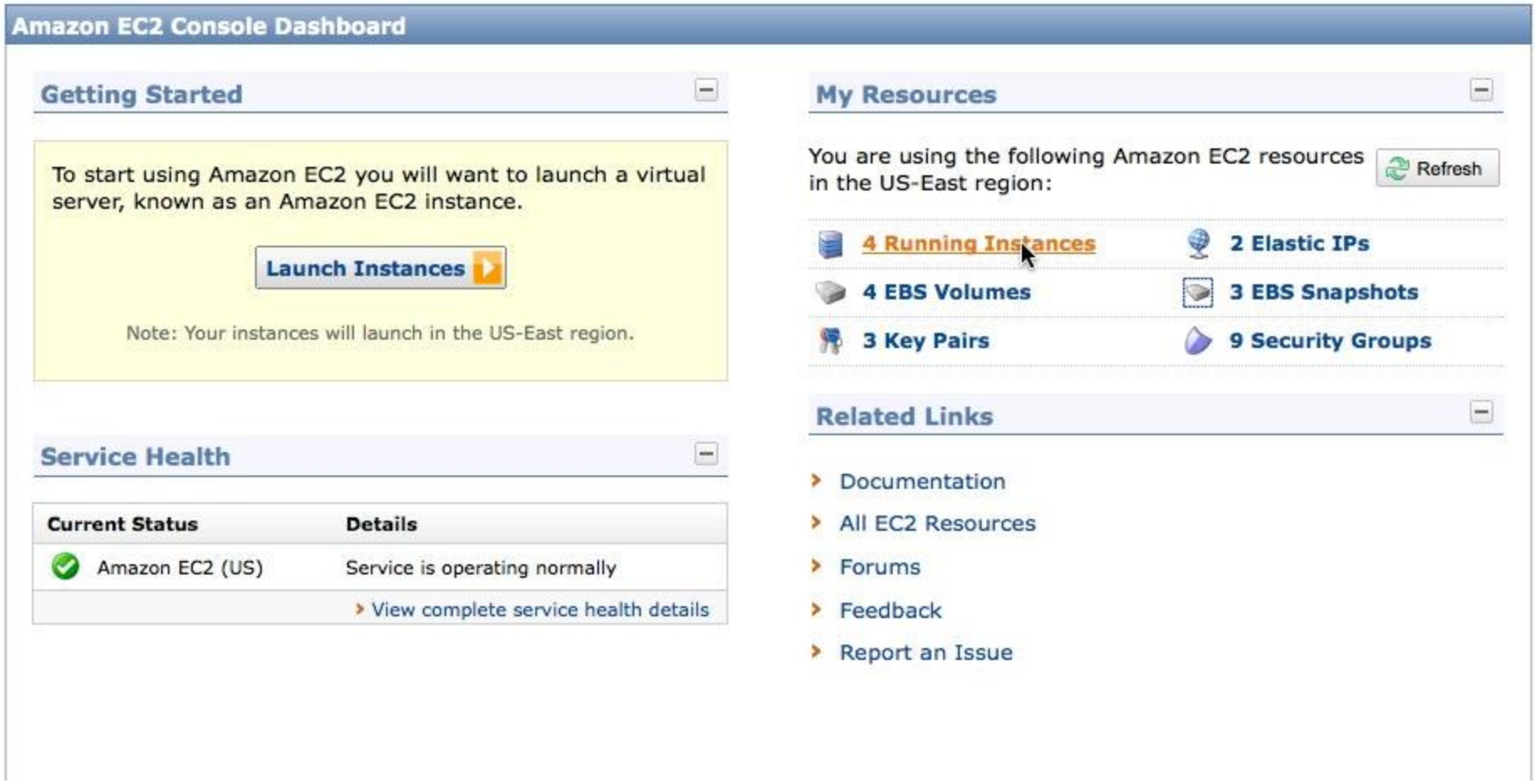
EC2: WS Access

- Several WS protocols are available:
  - REST: Each resource has a specific URI (HTTP path)
  - the HTTP operations represent specific resource manipulations:
    - GET: read the resource's description
    - PUT: change the resource
    - POST: create sub-resources
    - DELETE: eliminate the resource
- SOAP: HTTP+XML
  - In SOAP,
  - HTTP is just a container for an XML message that encapsulates the complete description of the operation.

EC2: Use

- There are many more interfaces to AWS services:
  - Several language SDKs (mappings to API):
    - Java, C#, Python,…
  - Command line tools
  - Browser extensions
  - Web App: Amazon Management Console
- …

Amazon Management Console



EC2: Example (1)

- The use of EC2 requires:
  - creating a pair of public/private keys and
  - associating them with the virtual machine

- CreateKeyPair:
  - creates a pair of public/private keys.
- DescribeKeyPairs:
  - list the user's keys.
- DeleteKeyPair:
  - eliminates a pair of keys.

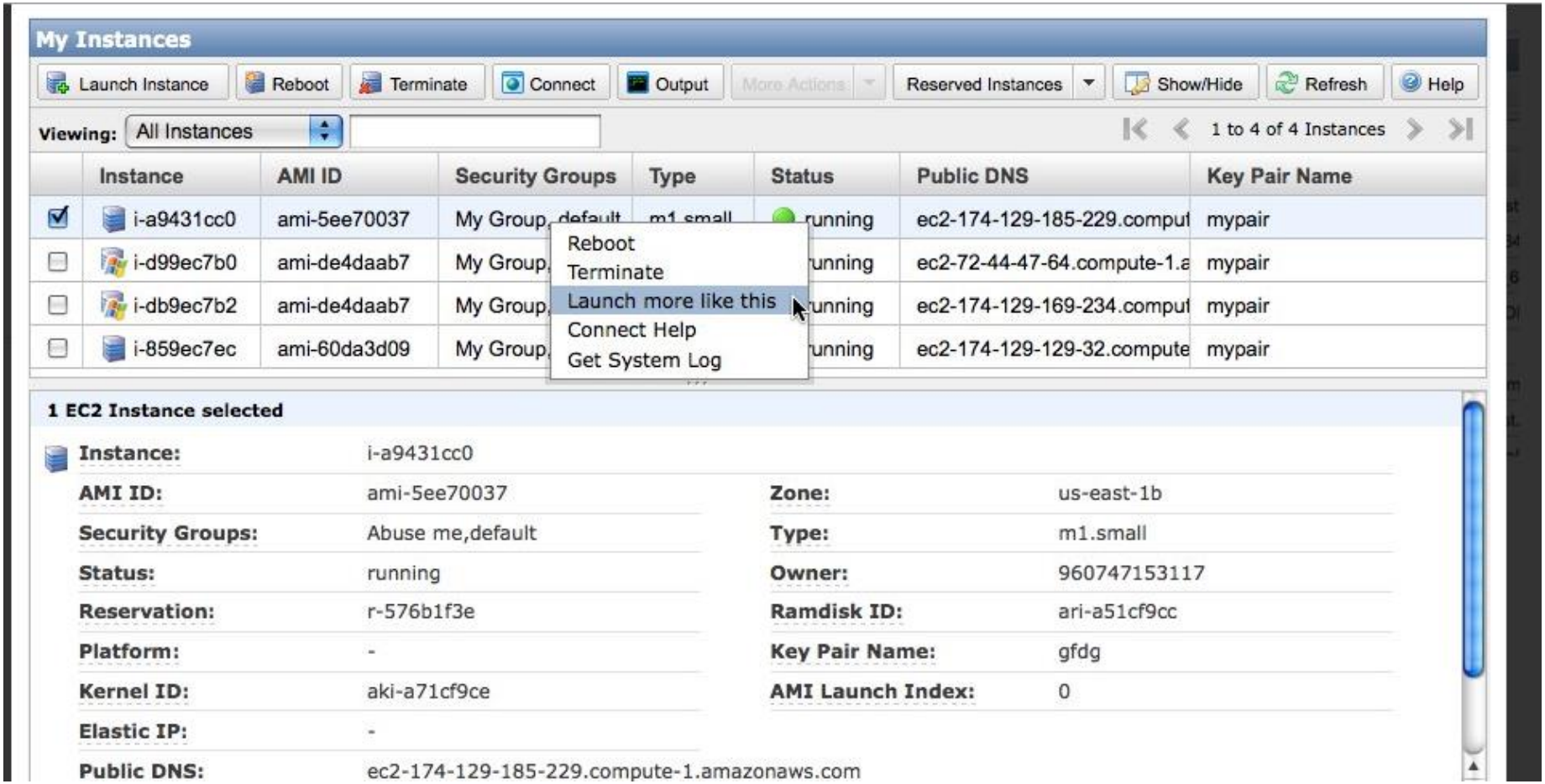
EC2: Example (2)

- To start instances: RunInstances
  - ImageId: id of the AMI to be started.
  - MinCount: minimum number of instances.
  - MaxCount: expected number of instances.
  - KeyName: key to access the instance.
  - SecurityGroup:
    - list of security groups (network address based ACLs).
  - InstanceType: S, L, XL, etc. where AMI boots/runs.
  - Region and Availability Zone to instantiate VM.

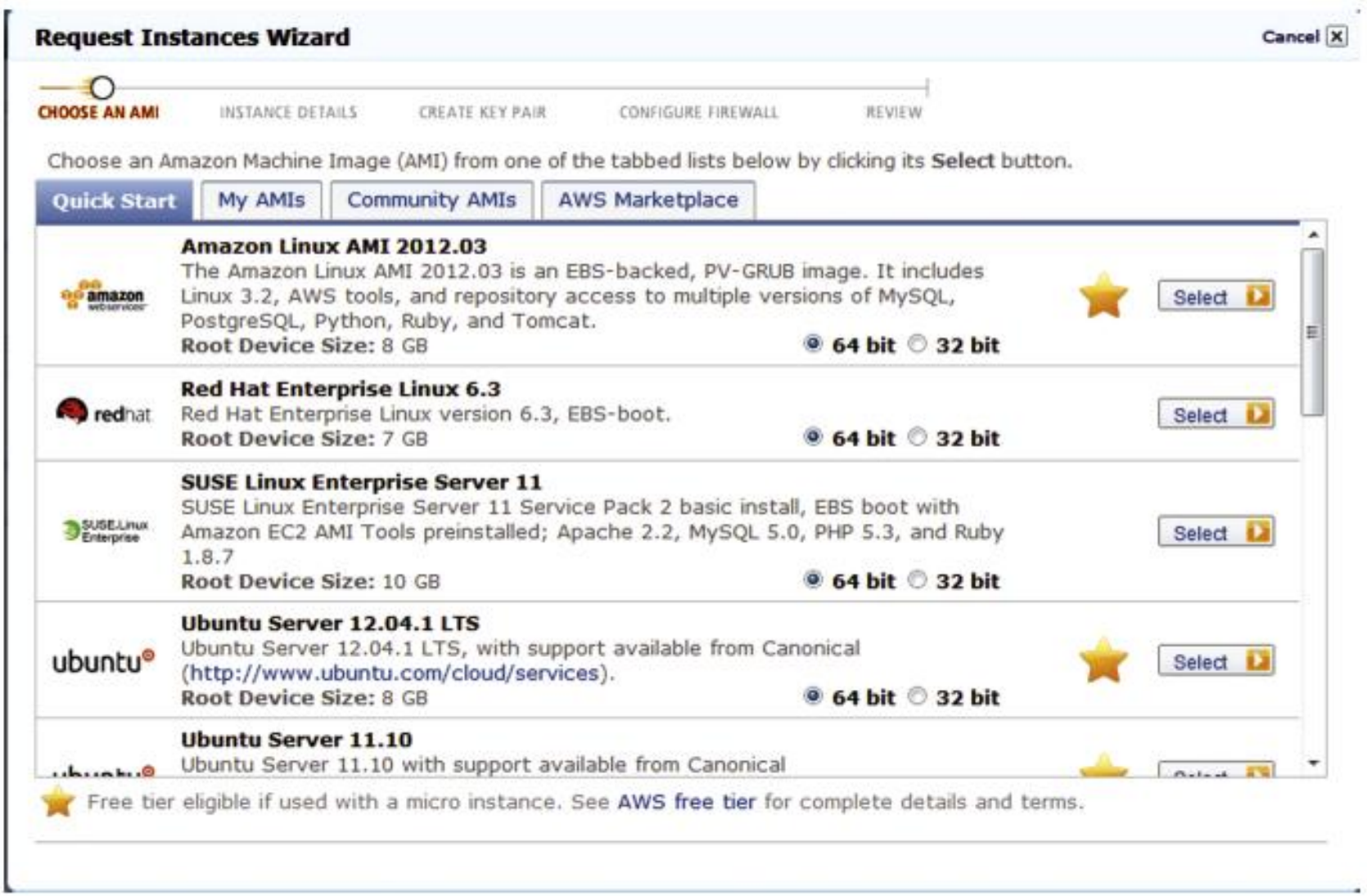
EC2: Example (3)

- Once the instances are started:
- DescribeInstances describes their state:
  - ImageId;
  - State:
    - pending,
    - running,
    - shutting down,
    - stopped,
    - terminated;
  - Index in the list of instances;
  - Public and private (inside AWS) DNS name;

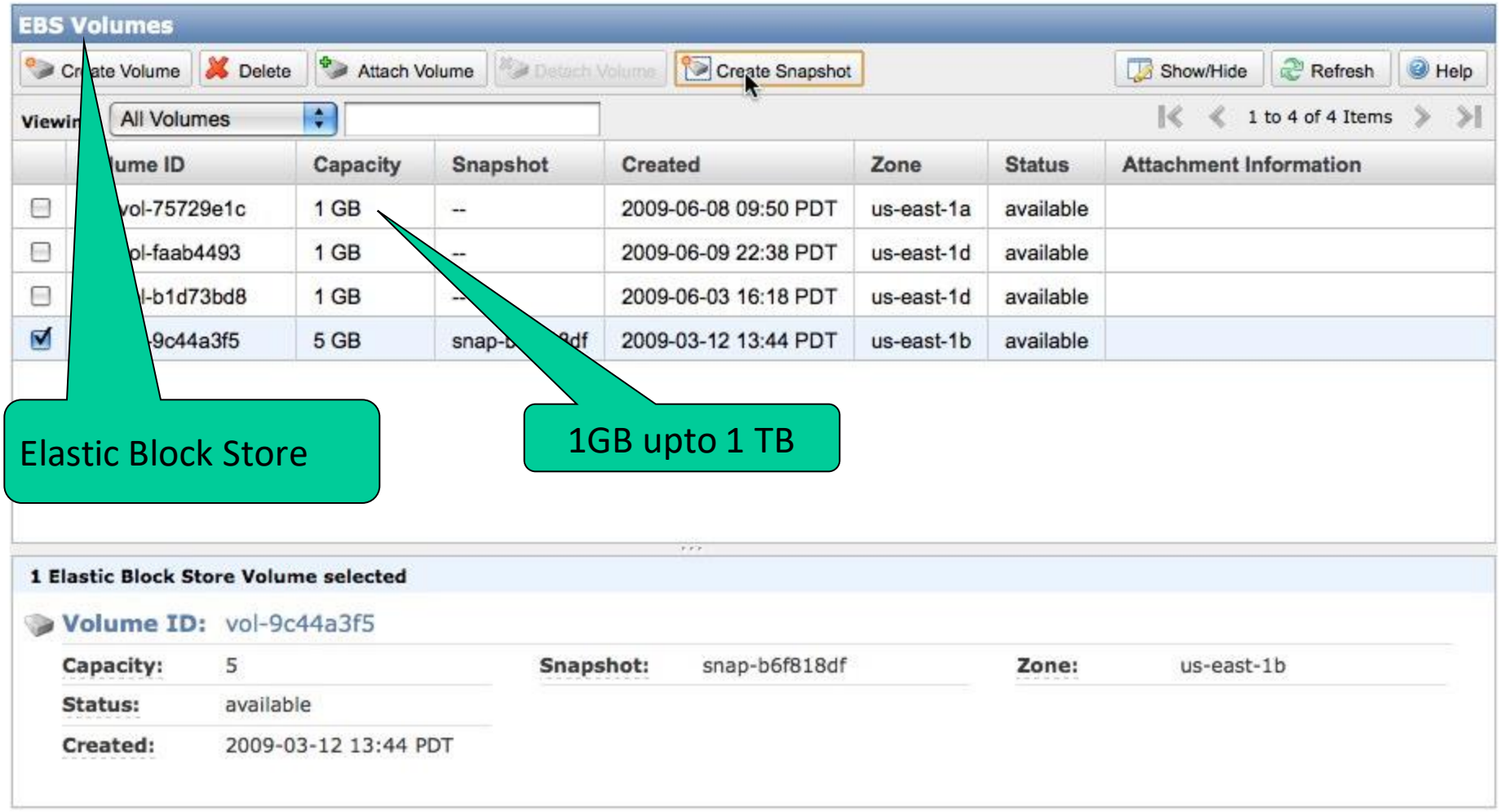
Amazon Management Console (1)



Amazon Management Console (2)



Amazon Management Console (3)



Instances (IP addresses)

- Each instance has:
  - a public <IP address, DNS name> pair and
  - a private one.
- IP addresses are dynamic and:
  - require that a dynamic DNS service be used.
- Delays in the update of DNS names:
  - may appear to be faults.

- Alternatively:
  - one may add a front-end server with a fixed address outside AWS, redirecting to instance public address

Instances (Networking)

- E.g. outbound networking cost (US West Region)

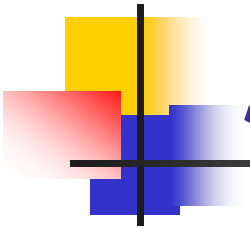
Amount of Data	Charge $
First 1 GB	0.00
Up to 10 TB	0.12
Next 40 TB	0.09
Next 100 TB	0.07
Next 350 TB	0.05

Security Groups (1)

- *Security Groups* are:
  - groups of network rules
  - to control the access to EC2 instances.
- They only limit inbound traffic.
  - Recall outbound traffic is unrestricted (and paid for).

- There are IP rules and group rules:
  - IP rules limit inbound traffic based on:
    - Protocol
    - Destination port
    - Source IP

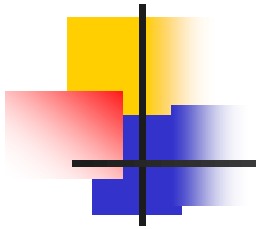
Security Groups (2)



- Group rules limit access (to instances):
  - from other EC2 instances or from outside world.
- Group rules limit access:
  - by port, protocol and security group.
- The security group is just a label:
  - that can be added/removed from each instance.
- Each instance can be
  - part of several security groups.
- Each security group
  - can have up to 100 security rules.

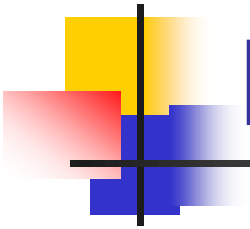
Security Groups: Operations

- CreateSecurityGroup
- DeleteSecurityGroup
- DescribeSecurityGroup
- AuthorizeSecurityGroupIngress
  - GroupName: security group name
  - IpProtocol: tcp, udp, icmp
  - FromPort: first port of the allowed range
  - ToPort: last port of the allowed range
  - CidrIP: network mask of source addresses
- RevokeSecurityGroupIngress:
  - similar parameters to AuthorizeSecurityGroupIngress

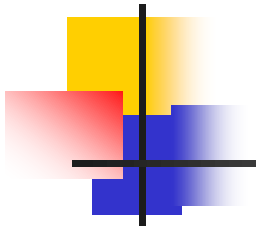


Monitoring in the Cloud

Monitoring in Cloud



- Both crucial and hard due to the scale.
- Goals:
  - detect faulty/failed nodes and applications.
  - detect overloaded applications.
  - detect load imbalances.
  - correct these problems by:
    - issuing commands to the nodes.
    - restarting nodes.
    - moving/migrating applications.
    - reporting problems.
- Least open part of commercial cloud platforms.



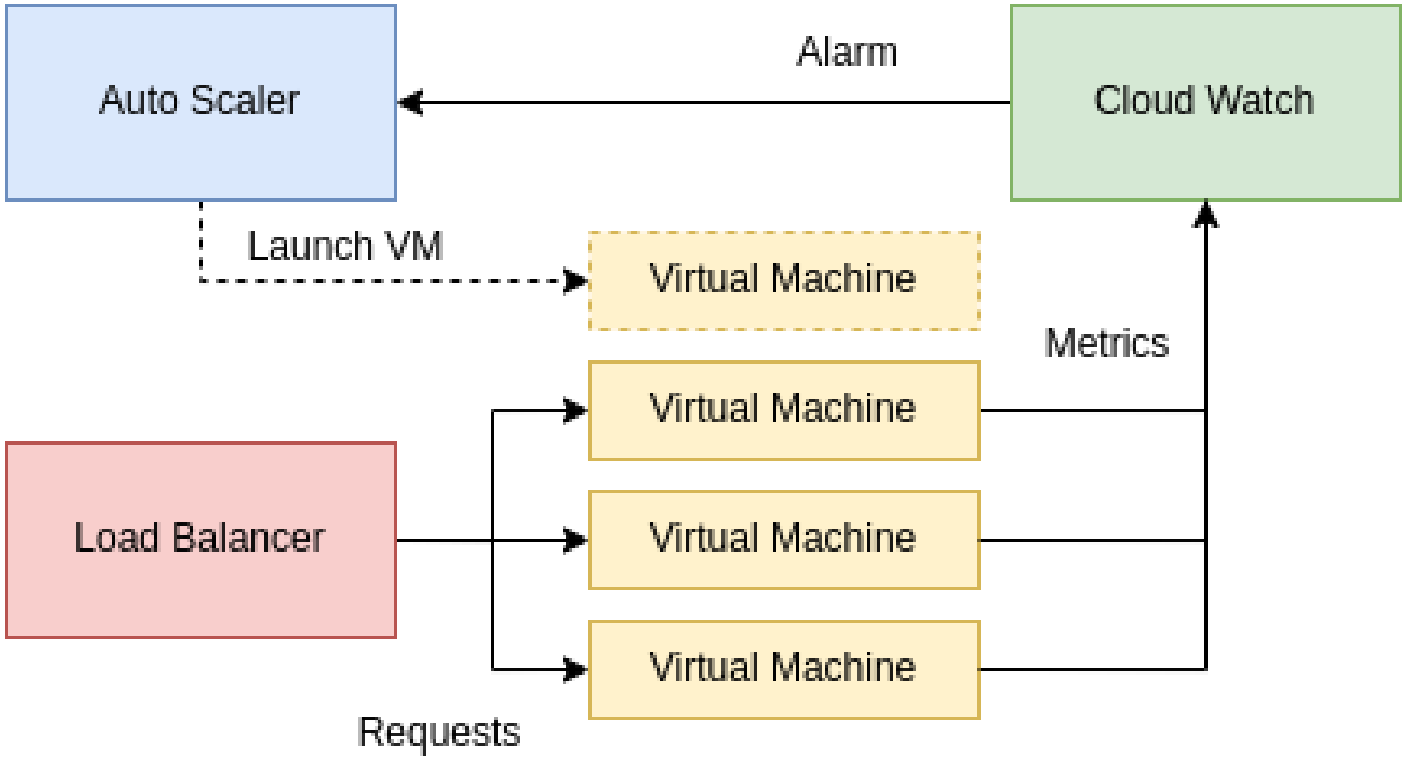
Monitoring in AWS

Amazon CloudWatch

- Elastic Compute Cloud's monitoring mechanism.
- Goal:
  - allow users and services to check the status of AWS running nodes.
- Interface:
  - set of VM- and application-probing Web Services.
- Monitoring Data:
  - gathered by each physical node hypervisor and
  - aggregated at CloudWatch server nodes.

Amazon CloudWatch

- E.g., Load Balancing and VM Scaling



Amazon CloudWatch

- AWS CW collects and tracks metrics:
  - to optimize performance of applications
  - to increase efficiency of resource utilization

- When launching an AMI, user can start CloudWatch and specify type of monitoring:
  - Basic Monitoring
    - free of charge; collects data at 5-minute intervals.
  - Detailed Monitoring
    - subject to charge; collects data at 1-minute interval.

AWS CloudWatch:Operations

- ListMetrics, lists metrics available to each client:

```
<ListMetricsResponse>
   <ListMetricsResult>
    <Metrics>
     <member>
   <MeasureName>NetworkIn</MeasureName>
   <Namespace>AWS/EC2</Namespace>
  </member>
     ...
```

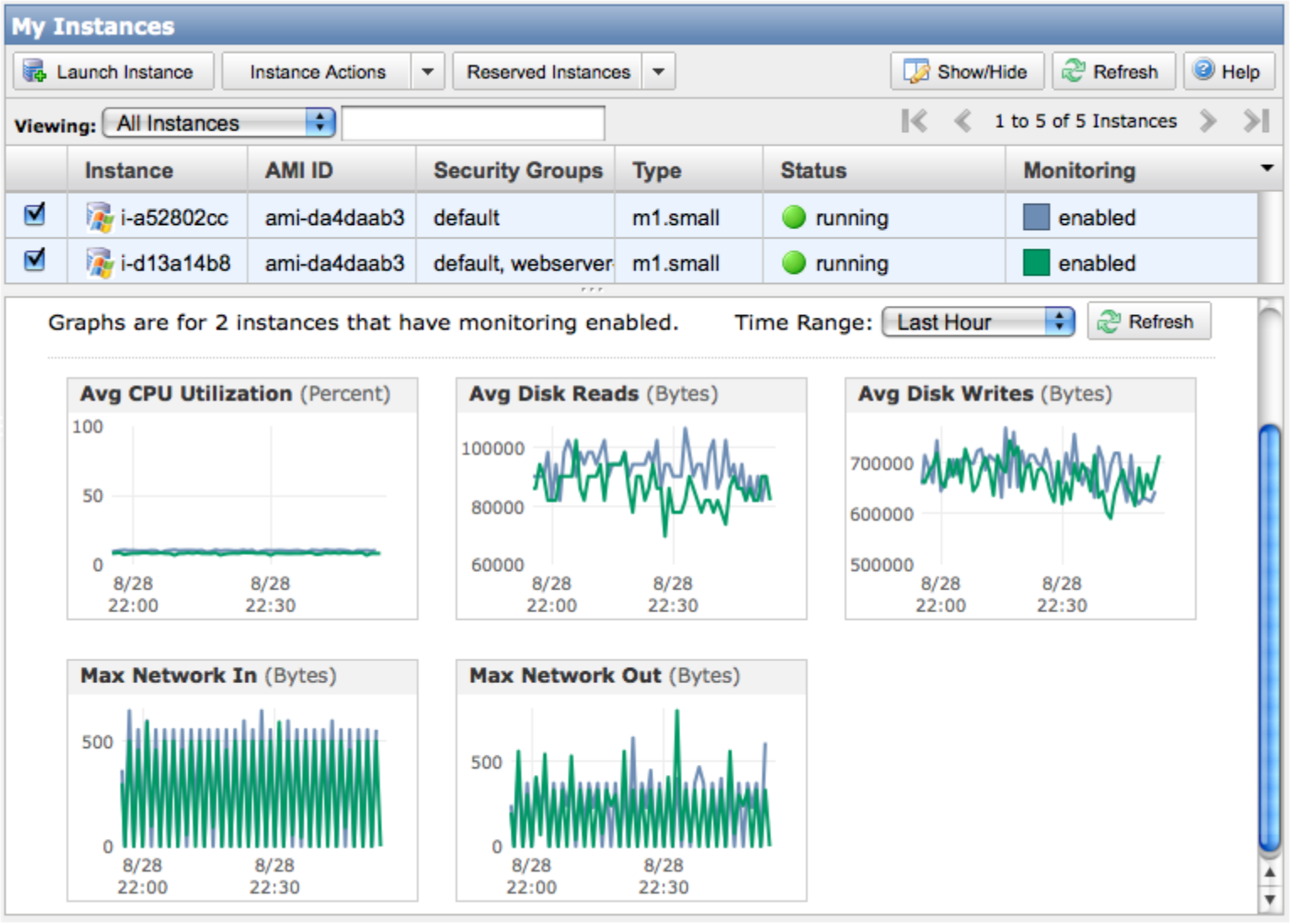
Amazon CloudWatch

- Definitions of provided measures:
  - Measure: measuring events (e.g. disk read usage)…
    - …that are aggregated per minute.
  - Metric: time-ordered series of data-points ACW returns.
  - Unit: unit in which a measure is expressed:
    - e.g., bytes, counts, seconds, percentage.
  - Statistic: a metric's functional characteristic:
    - min, max, average, sum.
  - Namespace:
    - name of the AWS area/service generating the metric.
  - Dimension: aggregation scale
    - e.g., by instance, datacenter or AutoScaling group.

AWS CloudWatch: Operations

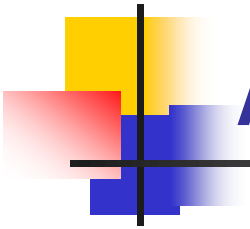
- GetMetricStatistics, parameters:
  - MeasureName:
    - a measure name returned by ListMetrics.
  - StartTime,
  - EndTime:
    - date/time of the first and last values requested.
  - Dimension:
    - e.g., ImageId, InstanceId, InstanceType,
    - e.g. AutoScalingGroupName
  - Returns: a set of DataPoints.

Amazon CloudWatch



Elastic Load Balancing

- AWS Load balancer (ELB).
  - Redirects inbound requests to a set of nodes.
  - Supports HTTP, HTTPS, TCP, SSL requests.
- Supports grouping all of a session's requests:
  - Groups requests using load balancer or app cookies.
    - session affinity, sticky sessions
- Monitors node's status.
  - Removes faulty nodes from load sharing group.



Auto Scaling

- Auto Scaling allows:
  - automatically scaling Amazon EC2
  - scale according to user-specified conditions

- With Auto Scaling:
  - set of instances can be automatically adjusted:
    - in order to meet demand variation.
    - careful to limit costs and resource waste.

Auto Scaling: Example (1)

- Creating a Launch Configuration:

```
as-create-launch-config MyLaunchConfiguration --
image-id myAMI --instance-type m1.small
```

- Creating an Auto Scaling Group:

```
as-create-auto-scaling-group MyAutoScalingGroup
--launch-configuration MyLaunchConfiguration --
availability-zones us-east-1a --min-size 2 --
max-size 20 --load-balancers MyLoadBalancer
```

Auto Scaling: Example (2)

- Creating activation conditions:

```
as-create-or-update-trigger MyTrigger --auto-
scaling-group MyAutoScalingGroup --namespace
"AWS/EC2" --measure CPUUtilization --statistic
Average --dimensions "AutoScalingGroupName=
MyAutoScalingGroup" --period 60 --lower-threshold 40
--upper-threshold 80 --lower-breach-increment "-1" -
-upper-breach-increment "1" --breach-duration 600
```

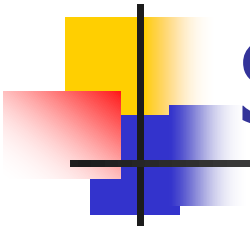
- Check activation conditions:

```
as-describe-triggers MyAutoScalingGroup

(response) TRIGGER MyTrigger MyAutoScalingGroup
NoData AWS/EC2 CPUUtilization Average 60
```

Auto Scaling: Open Issues

- Will AWS respond in time:
  - EC2 startup time?
    - dependent on user class, VM type and infrastructure load.
- Are we reacting to:
  - real traffic?
  - or to a DoS (denial of service) attack?
- Is it going to be profitable:
  - to react to the traffic increase?
- Is Auto Scaling a crutch for bad planning?



Summary

- Cloud Execution Platforms and Models:
  - Overview: existing cloud infrastructure
  - Amazon Web Services
    - AWS Main Architecture, AWS Ecosystem
  - AWS EC2 - Elastic Compute Cloud (public)
- Cloud Monitoring:
  - AWS
    - CloudWatch, Elastic Load Balancing, AutoScale

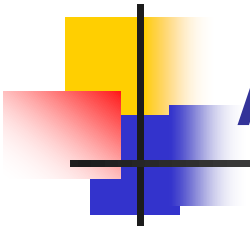


IaaS Cloud Execution and Monitoring (Part B)

Prof. Luís Veiga
IST / INESC-ID Lisboa

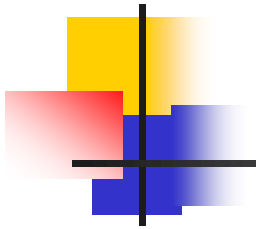
https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© LV, JG 2015-24 ©Figures: Cloud Computing - Theory and Practice, Dan C. Marinescu, MK.



Agenda

- Cloud Execution Platforms and Models:
  - Microsoft Azure (public)
    - Microsoft Azure Fabric Controller
  - OpenStack (private)
    - Nova scheduler
  - User vs. provider responsibilities, SLAs
- Scaling a Web Server with the Cloud:
  - *"Handling Flash Crowds from Your Garage"*



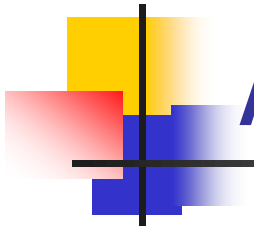
Microsoft Azure

Windows Azure



- Flexible application hosting platform
- Transparent application management
  - just provide code and service model
- Includes large-scale storage
- Used by Microsoft for its own cloud services
  - e.g. Microsoft Live

Azure: Goal Properties

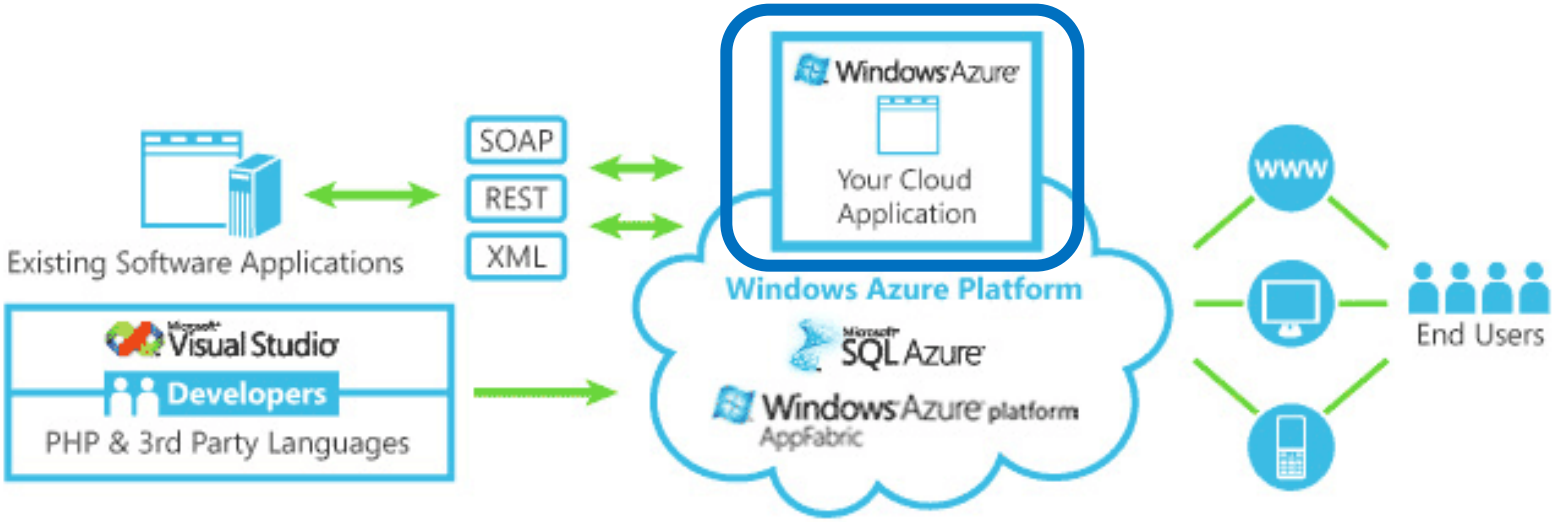


- (Similar to other platforms.)

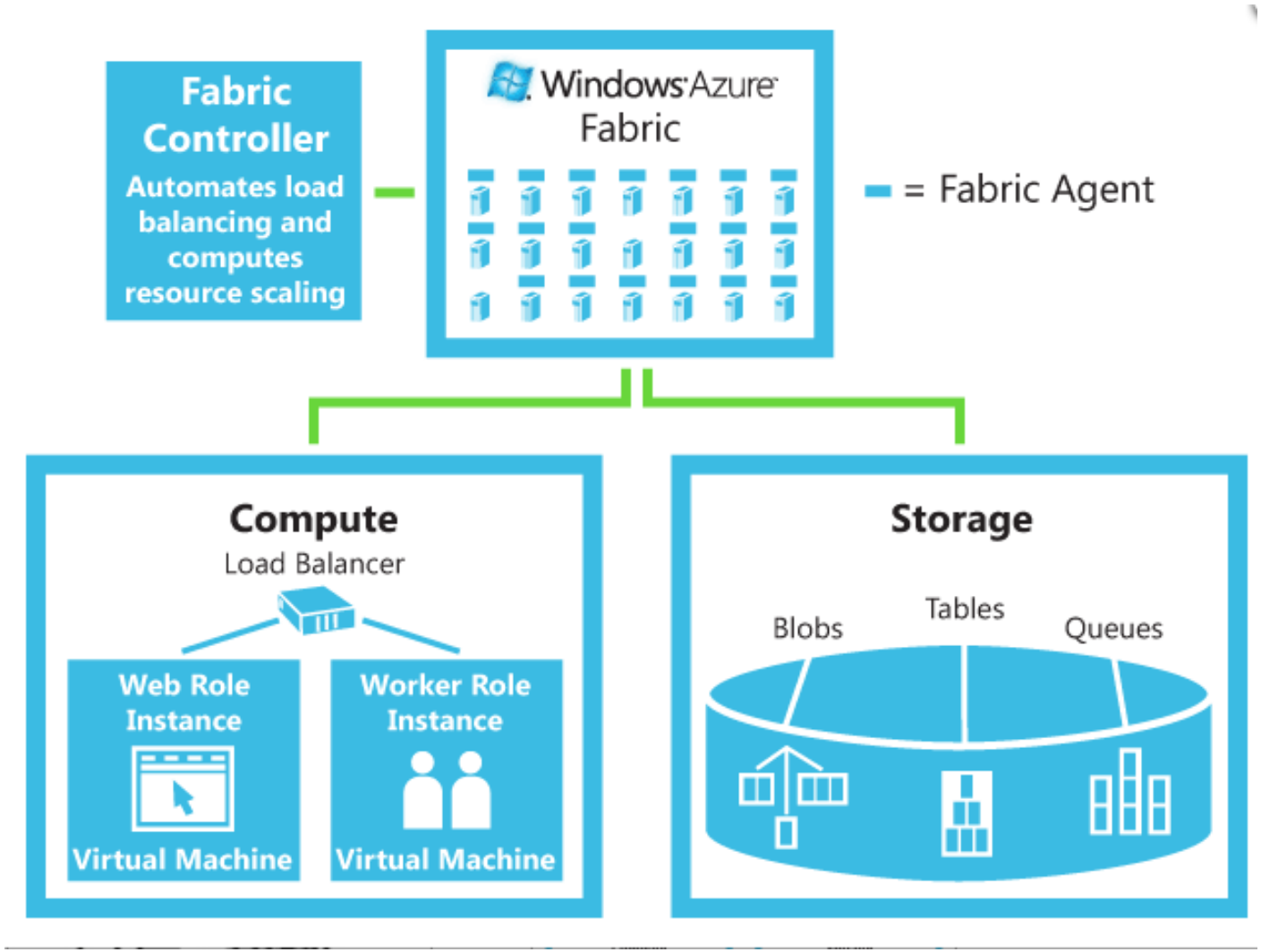
- High availability
- Performance
- Scalability
- Resource sharing

- Support familiar programming models
- Hide system administration further:
  - manage service (not servers),
  - manage data (not databases).

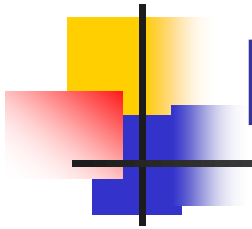
Windows Azure



Windows Azure

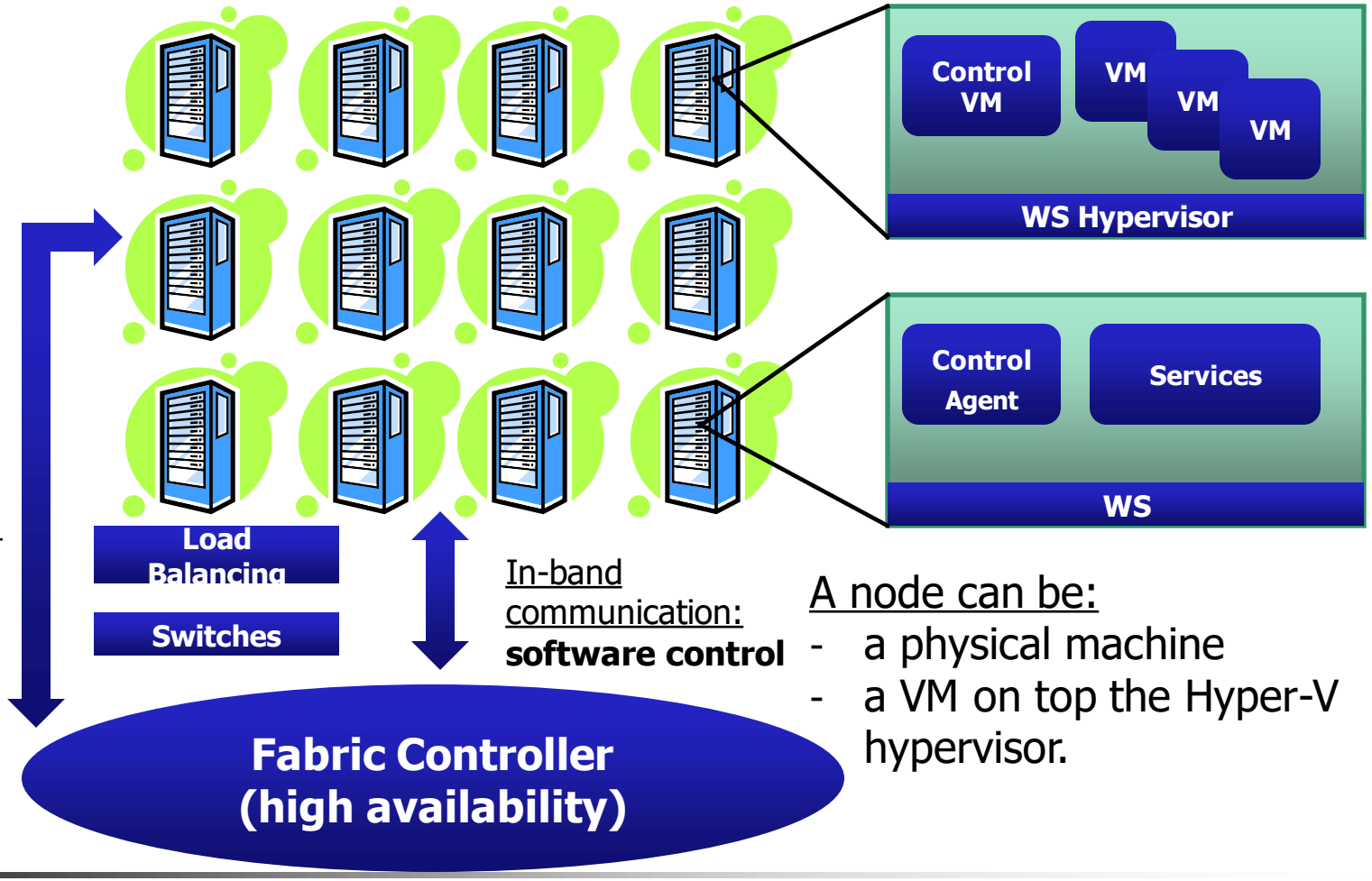


Nodes: Windows HPC Server



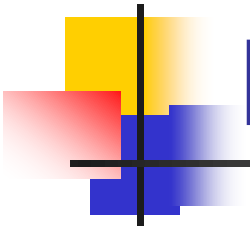
- Each node is running Windows HPC Server
- Windows HPC Server is Windows Server with:
  - Cluster manager:
    - organizes nodes into head nodes and compute nodes
    - performs software updates
  - Scheduler:
    - starts jobs in compute nodes
  - Cluster monitor:
    - gathers information about the cluster's status
  - Replication of the head nodes

Azure Fabric & Nodes



A node can be:

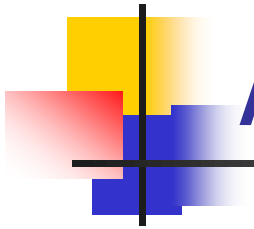
- a physical machine
- a VM on top the Hyper-V hypervisor.



Hyper-V

- Microsoft's own hypervisor
  - based on Xen and supports Xen's OS versions
  - paravirtualization
    - guest-OS (Win) explicitly modified to invoke VMM directly
    - Pros: no binary translation, no patching, no overhead
    - Cons: no transparency,
      - needs adaptation of each individual guest-OS to each individual hypervisor. **Q:** Remember why ?
- Compared to VMWare:
  - based on a micro-kernel design
  - no inclusion of third party code for better security
  - free with Windows Server

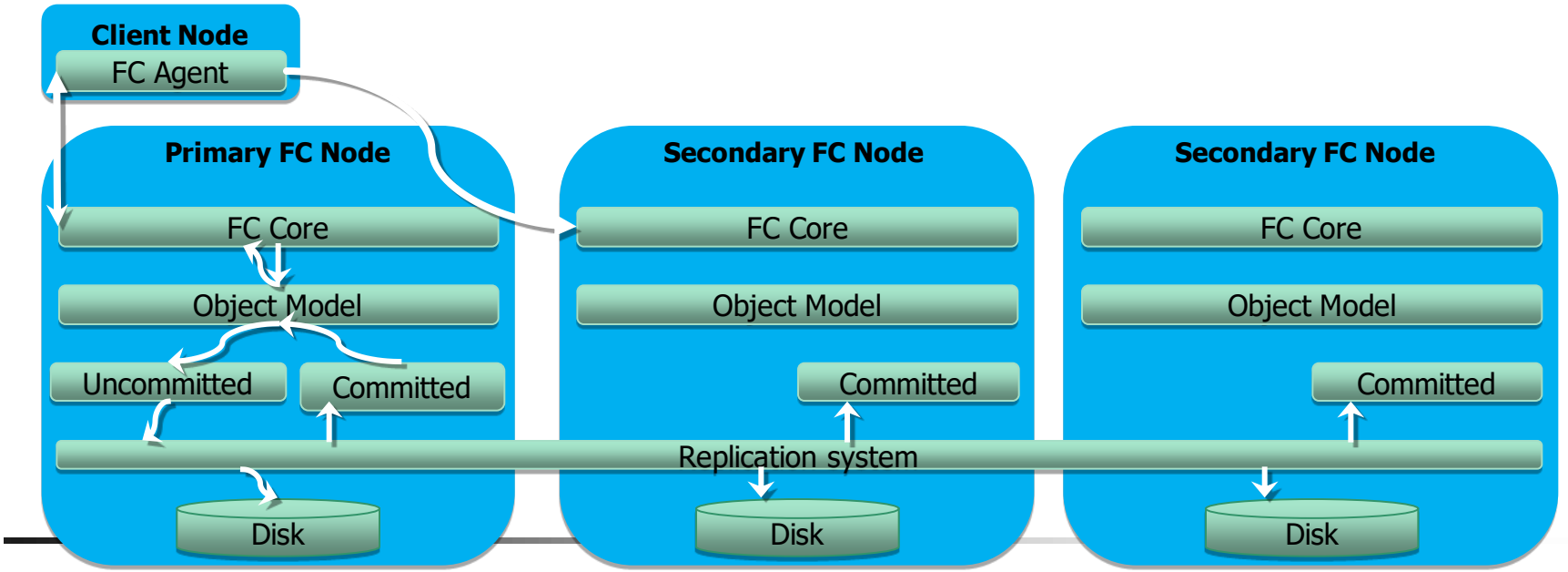
Azure Fabric Controller

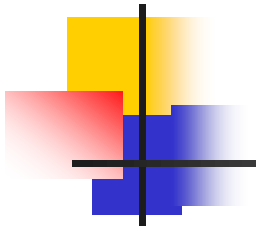


- Manages all the hardware in Azure Fabric nodes
  - talks to a specific daemon on each worker node
  - distributes services among available processors
- Monitors the system:
  - adapts to the hardware needs
  - monitors inactive hardware
- Starts instances of applications/services
  - and keeps applications running by reacting to faults
- Maintains (replicated) state machine on how to lead:
  - each node from current state to its goal state.
  - if it fails, applications are restarted on other nodes.

Azure Fabric Controller

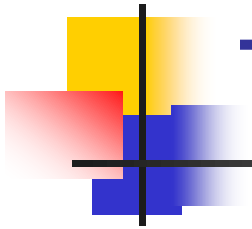
- The controller is a 5 to 7 node cluster:
  - state is replicated passively.
  - a new leader is elected in case of a fault
  - running applications do not fail if the controller fails (**Q:** what about new app launches?)





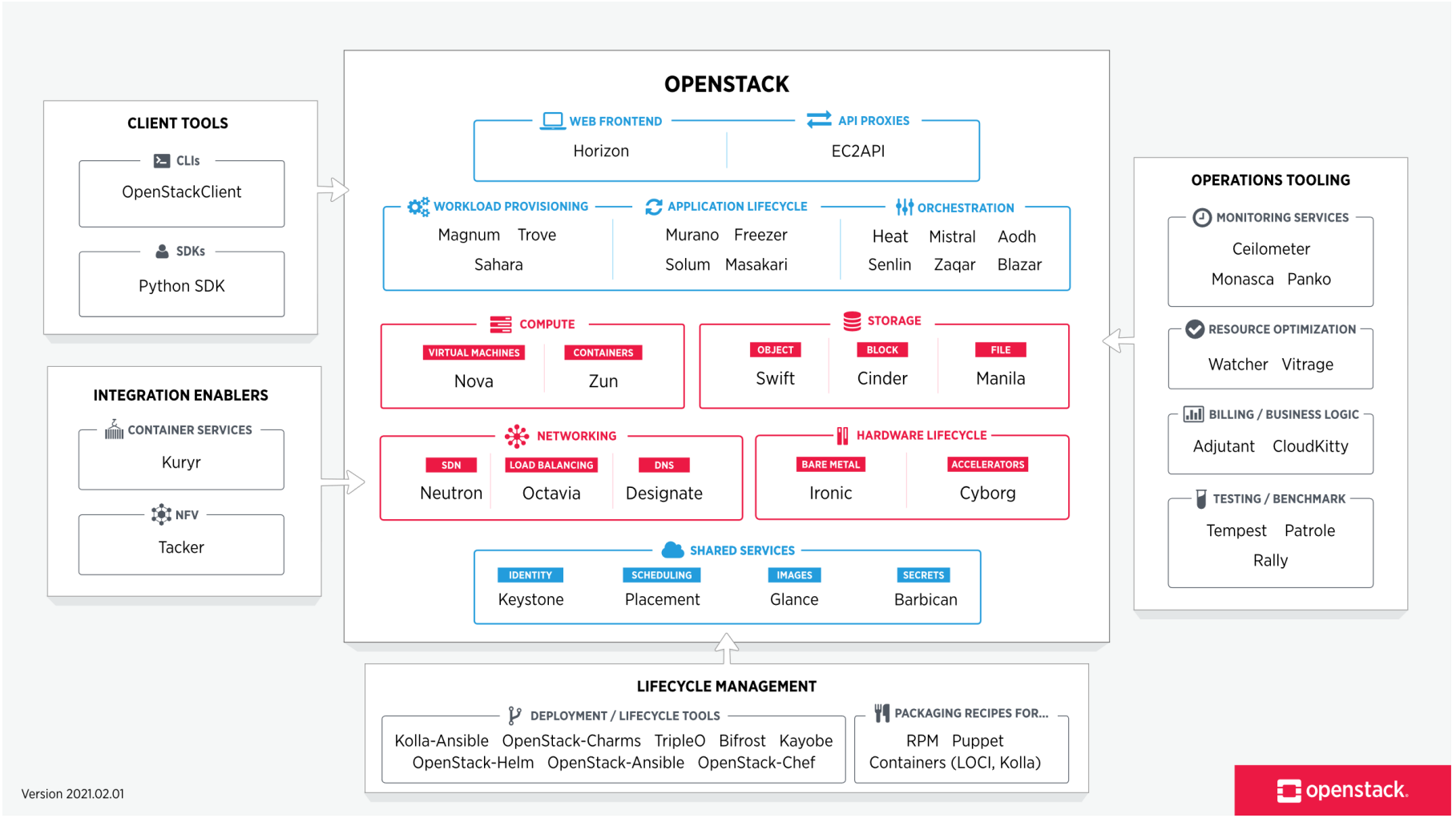
OpenStack

The OpenStack Project



- Aims to provide open source standard for the Cloud: *the cloud's "Linux"*
  - founded by NASA and Rackspace in 2010.
  - large open-source development
- Predominantly IaaS.
  - set of interconnected services accessed via:
    - REST, web portal and CLI.
  - largely compatible with AWS:
    - EC2 (Compute – Nova)
    - S3 (Storage – Swift).
- Largely modular extensible architecture.

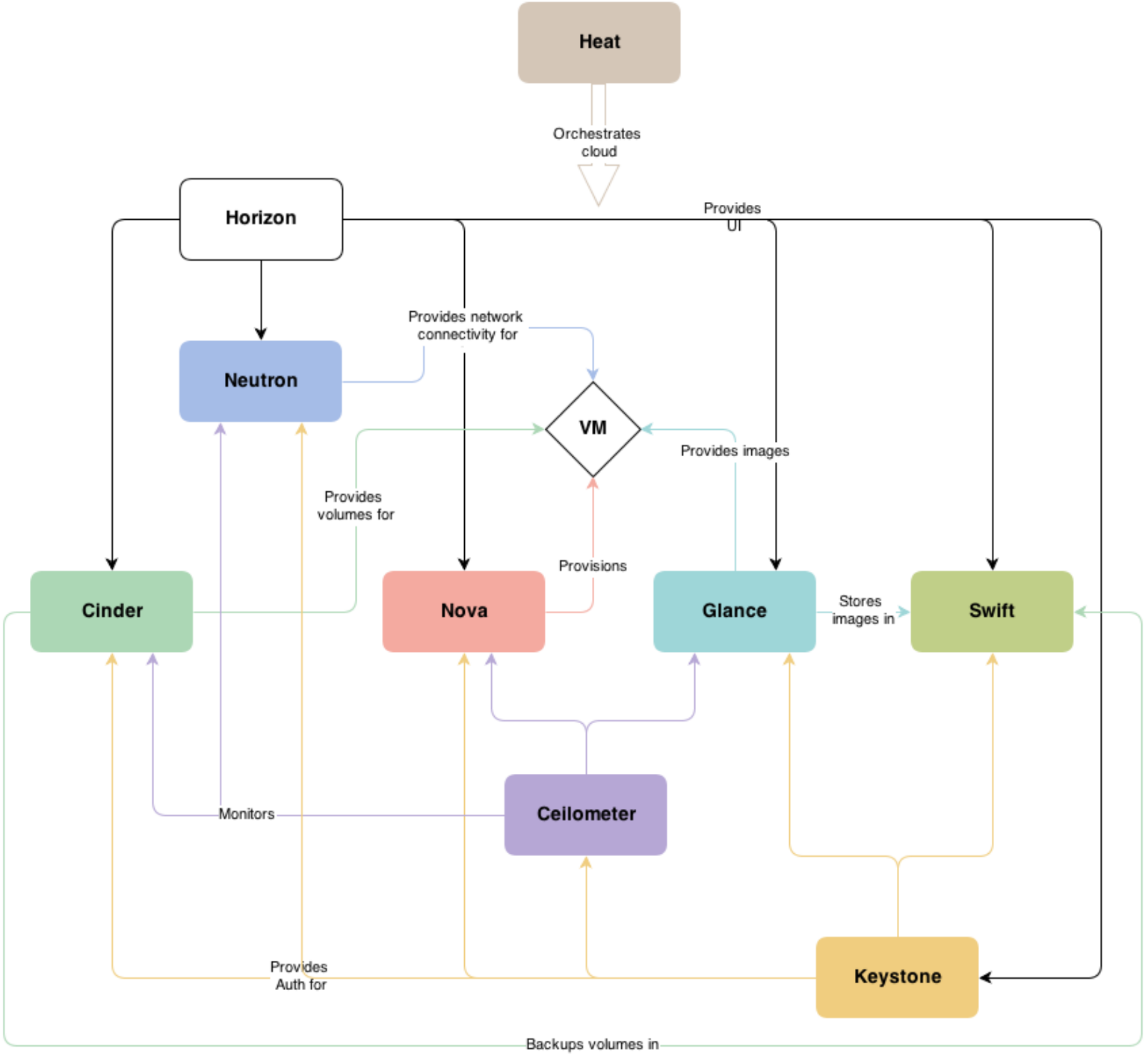
The OpenStack Ecosystem

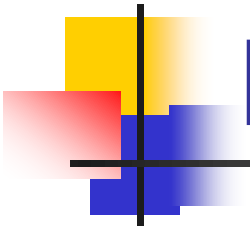


OpenStack Main Components

- Compute (Nova)
- Object Storage (Swift)
- Block Storage (Cinder)
- Networking (Neutron)
- Dashboard (Horizon)

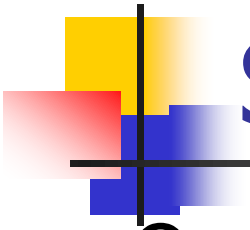
- Identity (Keystone)
- Disk Image (Glance)
- Monitoring/Telemetry (Ceilometer)
- Orchestration (Heat)


Heat
Orchestrates cloud
Horizon
Provides UI
Provides network connectivity for
Neutron
VM
Provides images
Provides volumes for
Provisions
Cinder
Nova
Glance
Stores images in
Swift
Monitors
Ceilometer
Provides Auth for
Keystone
Backups volumes in



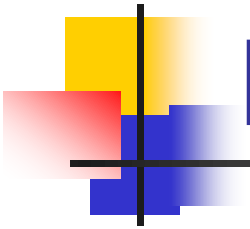
Nova

- Computation component of OpenStack.
  - main role: *Fabric Controller*.
  - written in Python.
- Connects to **hypervisors** on **compute nodes**:
  - Hyper-V, VMWare, Xen, to suit organizations...
- Starts VMs based on:
  - Glance (image) and
  - Cinder (block).
- Deployment requires plugging in a **scheduler** to make decisions on VM allocation/placement
  - also customizable in Python code



Swift

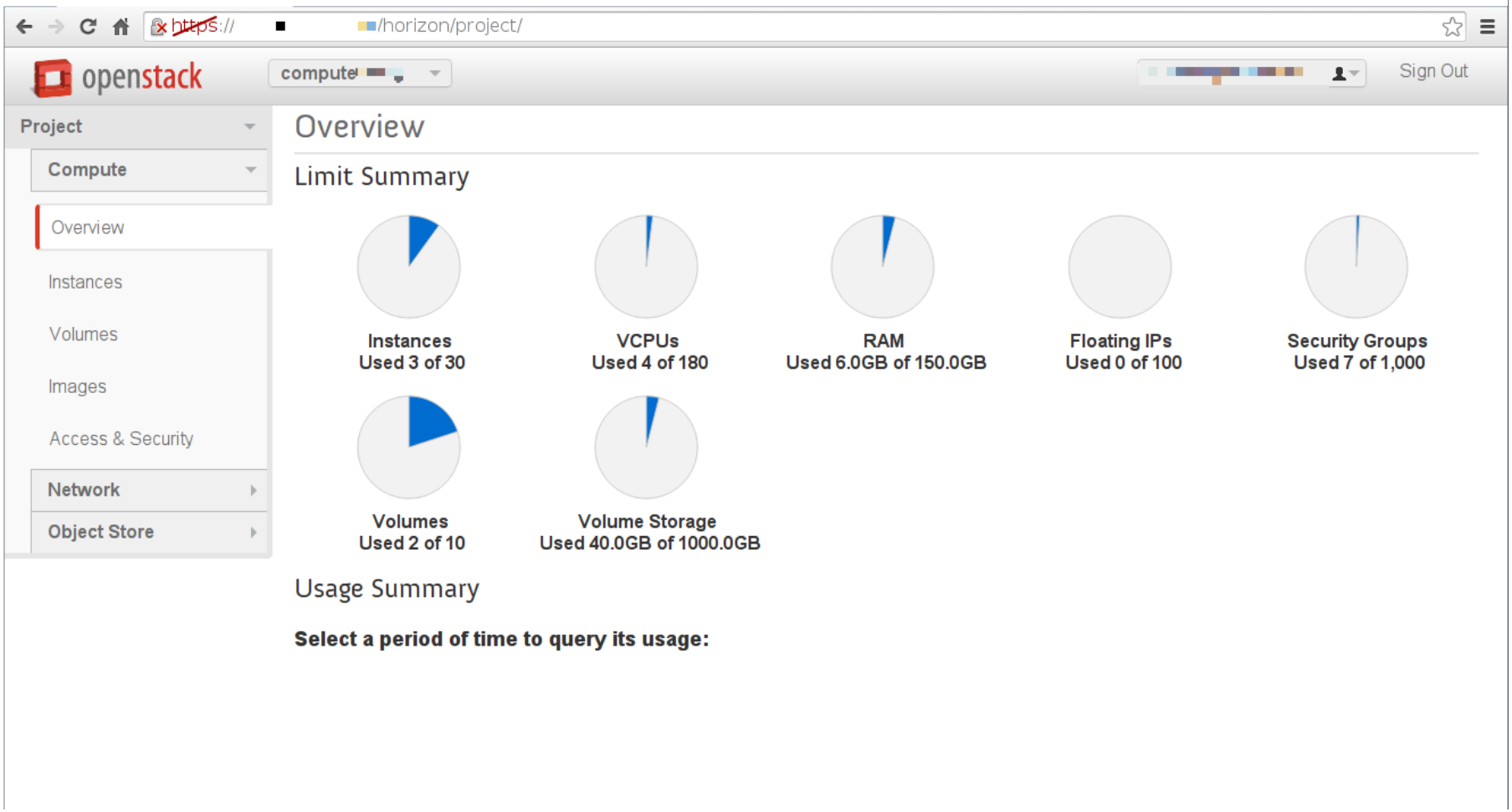
- OpenStack's storage support.
- Used by:
  - Glance (disk image service).
  - Cinder (block storage service).
- Hierarchical structure based on:
  - accounts, containers and objects.
  - each account and container is a SQLite file.
  - objects are binary streams.
- Architecture is based on:
  - 3 DHTs (distributed hash tables).
  - one per level, with references to each other.



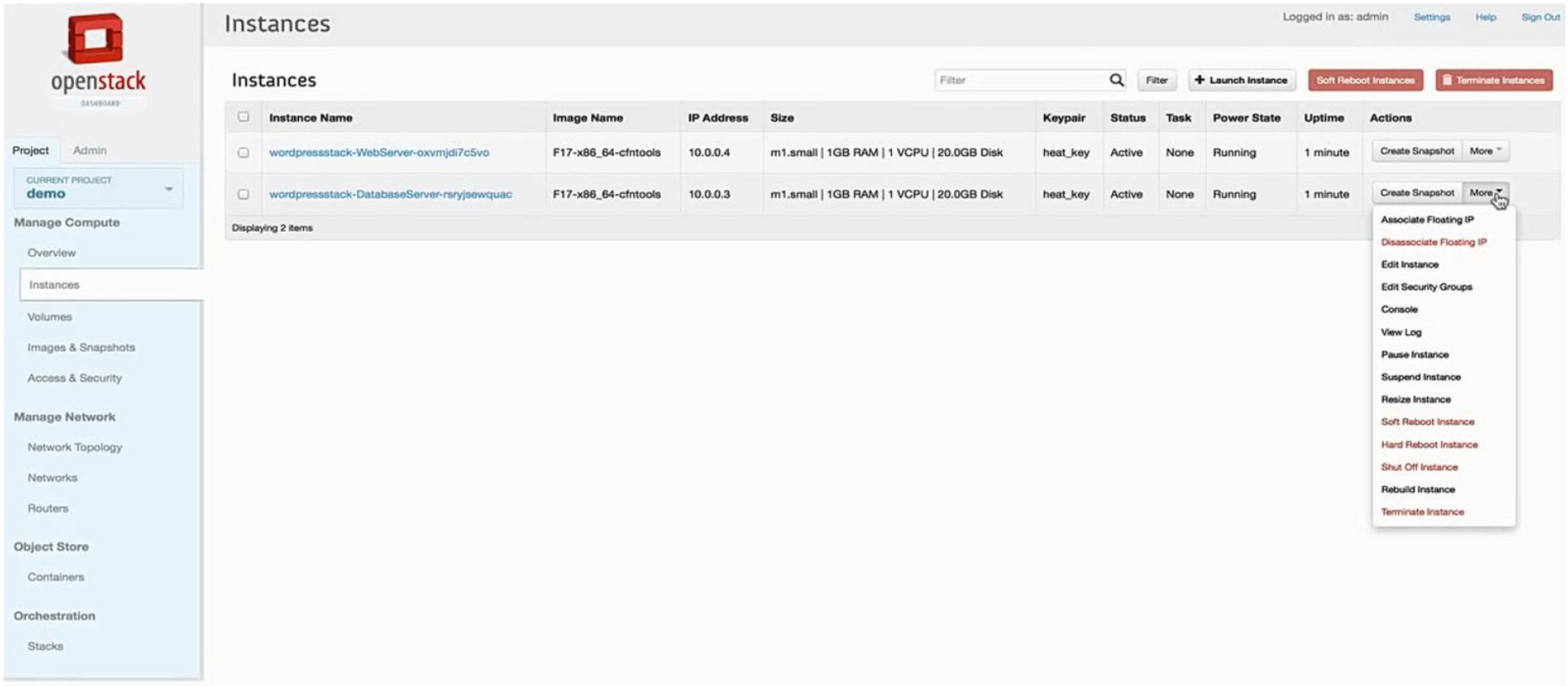
Neutron

- Manages network connections between nodes.
  - including floating IP (elastic IP in OpenStack)
- Network design is based on:
  - networks, subnets, ports and rules.
- Usual REST operations:
  - list, create, delete,...
  - over basic concepts/objects/resources.

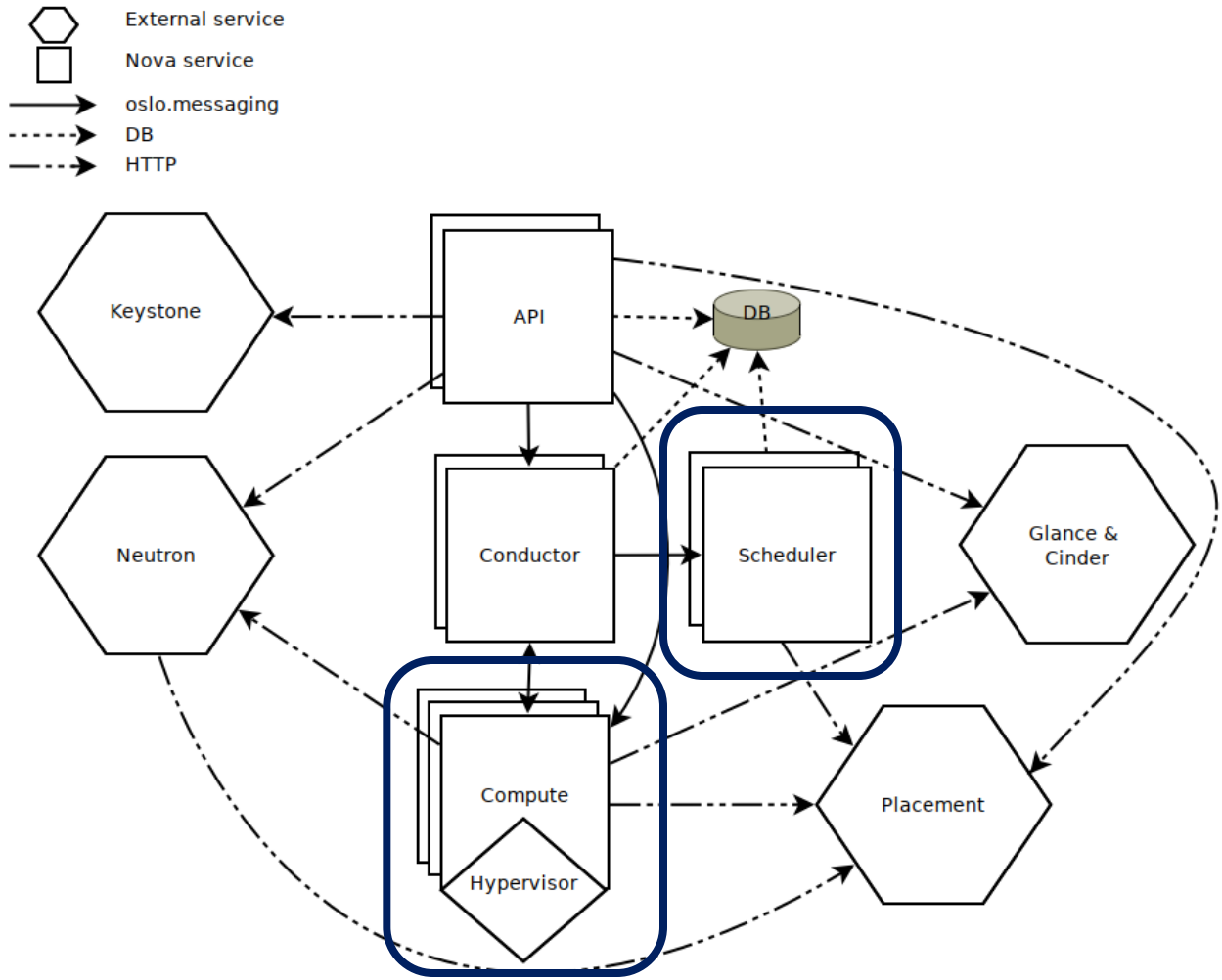
Horizon


https://
/horizon/project/
openstack
compute
Sign Out
Project
Compute
Overview
Instances
Volumes
Images
Access & Security
Network
Object Store
Overview
Limit Summary
Instances
Used 3 of 30
VCPUs
Used 4 of 180
RAM
Used 6.0GB of 150.0GB
Floating IPs
Used 0 of 100
Security Groups
Used 7 of 1,000
Volumes
Used 2 of 10
Volume Storage
Used 40.0GB of 1000.0GB
Usage Summary
Select a period of time to query its usage:

Horizon


openstack
DASHBOARD
Project
Admin
CURRENT PROJECT
demo
Manage Compute
Overview
Instances
Volumes
Images & Snapshots
Access & Security
Manage Network
Network Topology
Networks
Routers
Object Store
Containers
Orchestration
Stacks
Instances
Logged in as: admin
Settings
Help
Sign Out
Instances
Filter
Filter
+ Launch Instance
Soft Reboot Instances
Terminate Instances
Instance Name
Image Name
IP Address
Size
Keypair
Status
Task
Power State
Uptime
Actions
wordpressstack-WebServer-oxvmjdi7c5vo
F17-x86_64-cfntools
10.0.0.4
m1.small | 1GB RAM | 1 VCPU | 20.0GB Disk
heat_key
Active
None
Running
1 minute
Create Snapshot
More
wordpressstack-DatabaseServer-rsryjsewquac
F17-x86_64-cfntools
10.0.0.3
m1.small | 1GB RAM | 1 VCPU | 20.0GB Disk
heat_key
Active
None
Running
1 minute
Create Snapshot
More
Displaying 2 items
Associate Floating IP
Disassociate Floating IP
Edit Instance
Edit Security Groups
Console
View Log
Pause Instance
Suspend Instance
Resize Instance
Soft Reboot Instance
Hard Reboot Instance
Shut Off Instance
Rebuild Instance
Terminate Instance

Nova - Architecture

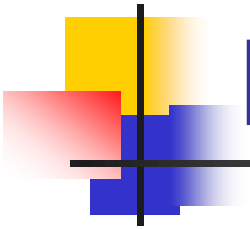


Nova - Architecture

- Resources can be divided in:
  - Regions and Availability Zones
- CPU overcommit factor ratio: 16 to 1
  - i.e., each core be shared on average by 16 VMs
- Memory overcommit factor/ratio: 1.5 to 1
  - (physical) memory is much less elastic than CPU
  - memory is used/occupied most of the time
  - seldom idle
  - cost of swapping is too high

Nova Scheduler

- Main operation
  - receive requests to create/launch VM instances
  - decide VM allocation/scheduling/placement
    - i.e., which (physical) host should be selected
    - instruct local hypervisor (e.g Xen, KVM) to launch VM
  - store decisions made in DB (Nova shared state)
    - multiple Nova scheduler instances for faster throughput
      - avoiding scheduling to become a global bottleneck
- Interacts with other components
  - collect information about resources
  - obtain VM images
  - communication through queues



Nova Scheduler

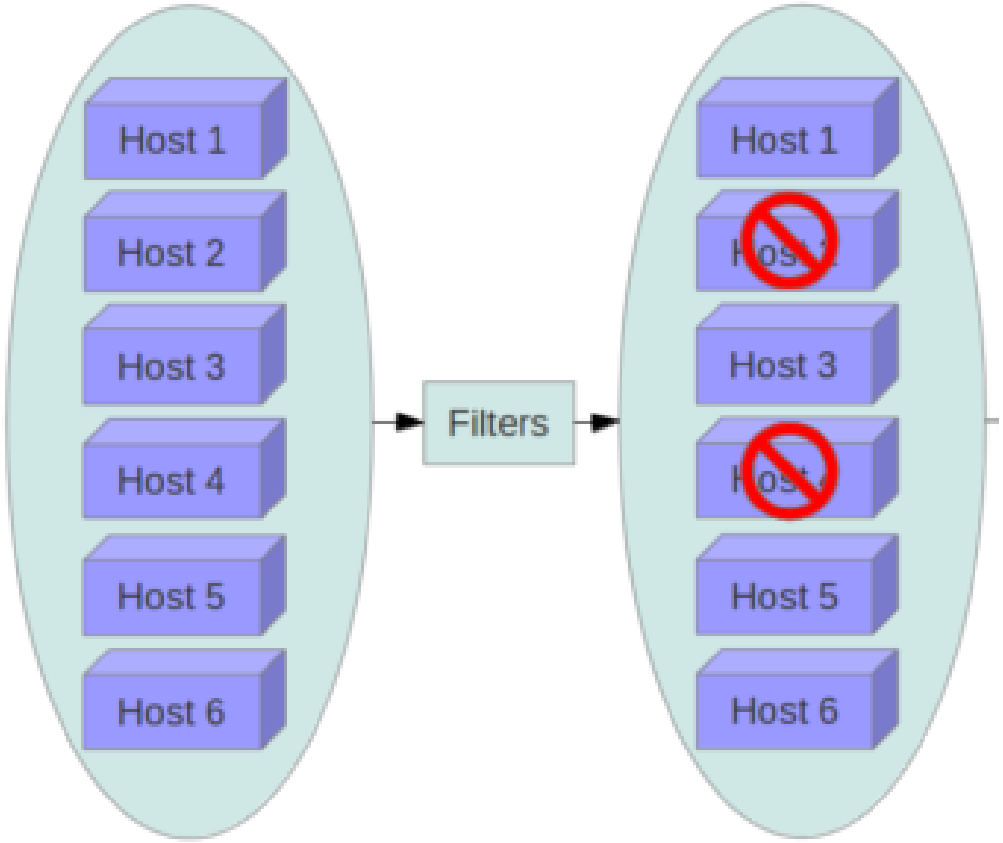
- Nova Scheduler → *Filter Scheduler*
  - coded and extensible in Python

- Filter Scheduler algorithm
  - operates in phases:
    - get state of all compute nodes
    - **filtering** – finds suitable hosts
    - **weighing** – chooses best hosts
  - driven by user-defined criteria
  - allows expressing:
    - **requirements, preferences, policies**

Nova Scheduler - Filtering

- Filtering phase:
  - determine the set of suitable hosts
    - based on user **requirements**
    - e.g., with enough minimum resources
    - e.g., with specific hypervisors
  - By:
  - iterating list of hosts
  - applying predicate filters (Python functions)
    - matches/compares filter predicate to state of host
    - returns true (allow/pass) or false (reject/refuse)
  - providing list of suitable hosts to next phase

Nova Scheduler - Filtering



Nova Scheduling – e.g. Filters

- ComputeFilter
  - passes all hosts that are operational and enabled.
- AvailabilityZoneFilter
  - filters hosts by availability zone, passes hosts of AZs specified.
- CoreFilter
  - only passes hosts with sufficient number of CPU cores.
- RamFilter
  - only passes hosts with sufficient RAM to host the instance.
- DifferentHostFilter (sets of instances)
  - true if host to place instance on is different from all hosts used by a set of instances.
- SameHostFilter (sets of instances)
  - true if host to place an instance on already hosts other instances of the same group.

Nova Scheduling – e.g. Filters

- **E.g.,** host state variables to check and assess in custom filters
- free_ram_mb
  - (compared with a number, values like ">= 4096")
- free_disk_mb
  - (compared with a number, values like ">= 10240")
- host
  - (compared with a string, values like: "<in> compute")
- hypervisor_type
  - (compared with a string, values like: "s== QEMU")
- num_instances
  - (compared with a number, values like: "<= 10")
- vcpus_total
  - (compared with a number, values like: "= 48", ">=24")
- vcpus_used
  - (compared with a number, values like: "= 0", "<= 10")

Nova Scheduler - Weighing

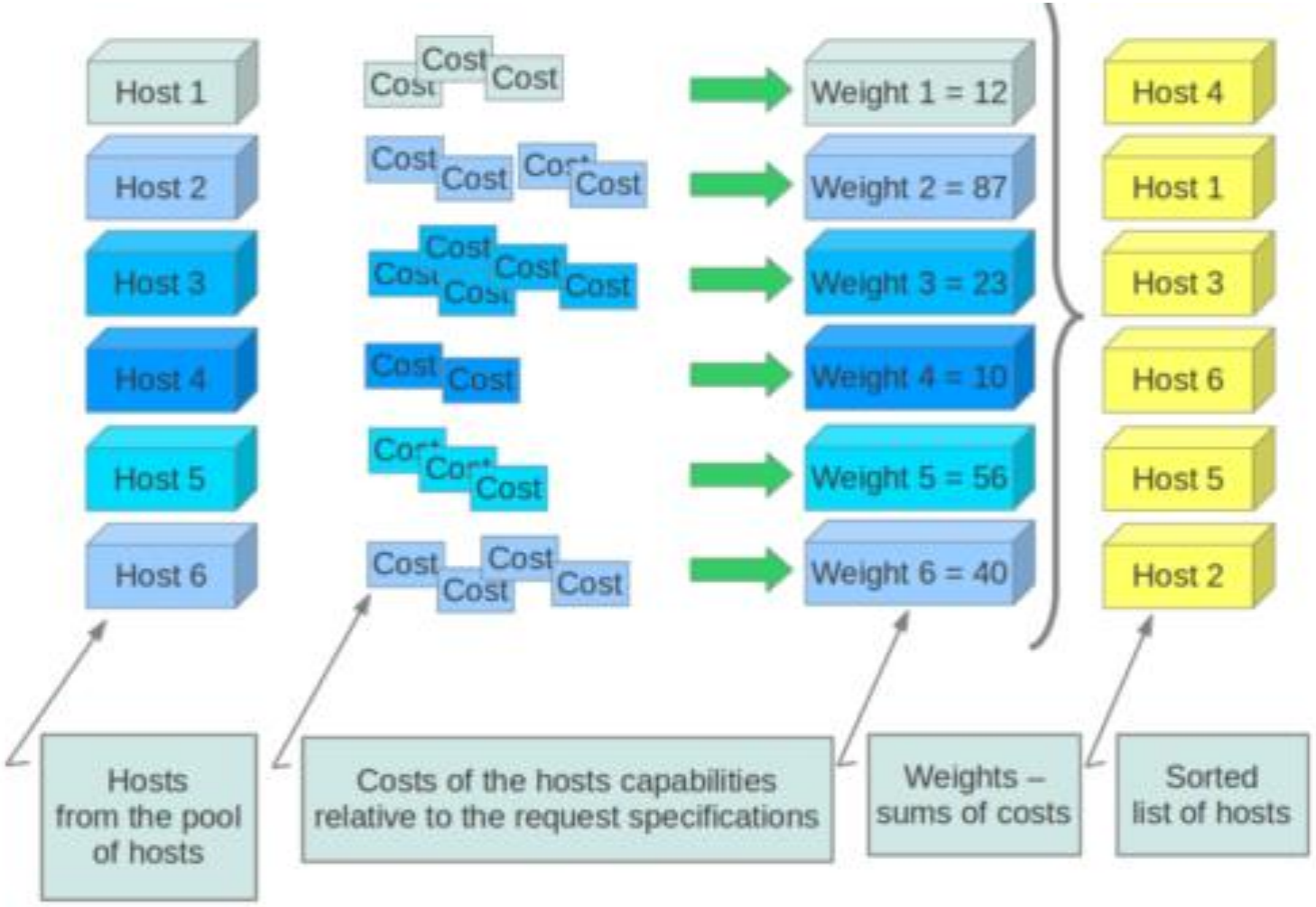
- Weighing phase:
  - determine the best hosts to start the instances
    - based on **preferences** (e.g. more memory, CPU)
    - based on overall **policies or strategies**
      - e.g., spread/balance or stack/consolidate
  - By, repeatedly:
  - assigning different weights to all hosts in list
    - according to their relative quality
    - all weights are normalized before combined
  - weights can have (relative) multipliers
    - prioritize each one against others (e.g. free memory)

Nova Scheduler - Weighing

- Weighing phase:
  - for each host considered:
    - Nova *virtually* consumes resources on it.
    - prospective result determines weight of host in selection
      - e.g., the amount of remaining free memory, etc.
    - winning have better status:
      - e.g. more free cores, free memory, combination of both…
  - final weight for each host:

weight = w1_multiplier * norm(w1) + w2_multiplier * norm(w2) + ..

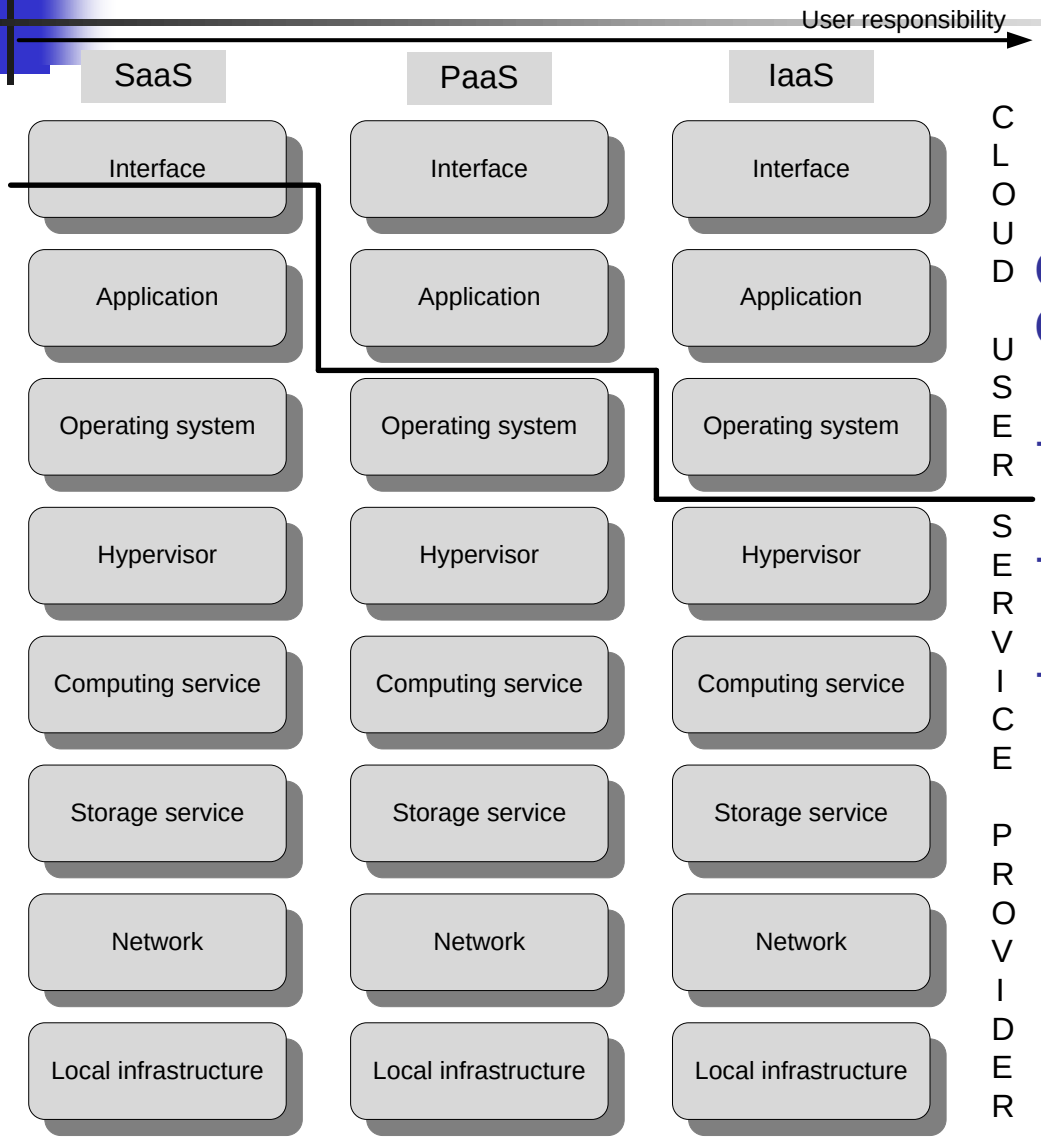
Nova Scheduler - Weighing



Nova Scheduler – E.g. weights

- **RAMWeigher:**
  - compute weight based on available RAM on compute node. Sort with largest weight winning.
  - if multiplier is negative, the host with least RAM available will win (useful for stacking hosts, instead of spreading).
- **CPUWeigher:**
  - compute weight based on available vCPUs on the compute node. Sort with the largest weight winning.
  - if multiplier is negative, the host with least CPUs available will win (useful for stacking hosts, instead of spreading).
- **DiskWeigher:**
  - hosts weighted and sorted by free disk space with largest winning.
  - if multiplier is negative, the host with less disk space available will win (useful for stacking hosts, instead of spreading).
- **MetricsWeigher (generic):**
  - can compute weight based on compute node host's various metrics.

Responsibility sharing between user and CSP



Q: who should handle (user, CSP) in Iaas, PaaS ?

- DDoS attack on network?
- security exploit on guest OS?
- bug in application code?

Service Level Agreement (SLA)

- SLA: a negotiated contract between the customer and CSP
  - can be legally binding or informal.
- Specifies the services that the customer receives:
  - rather than how the cloud service provider delivers the services.
  - allows CSP to manage infrastructure /optimize operation within SLA

Service Level Agreement (SLA)

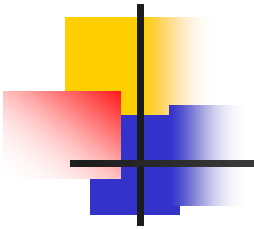
- Objectives:
  - identify and define the customer's needs and constraints including the level of resources, security, timing, and QoS.
  - provide framework for understanding
  - critical aspect of this framework is clear definition of classes of service and the costs.
  - simplify complex issues.
  - clarify the boundaries between the responsibilities of clients and CSP in case of failures.
  - reduce areas of conflict.
  - encourage dialog in the event of disputes.
  - eliminate unrealistic expectations.

Service Level Agreement (SLA)

- E.g. (taken from AWS EC2 SLA)

- General Service Commitment: *AWS will use commercially reasonable efforts to make the Included Services each available for each* ***AWS region with a Monthly Uptime Percentage of at least 99.99%, in each case during any monthly billing*** *cycle (the "Service Commitment"). In the event any of the Included Services do not meet the Service Commitment, you will be eligible to receive a Service Credit.*

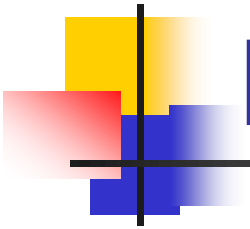
- Service Credits (perc. of payments already done):
- **Monthly Uptime Percentage - Service Credit Percentage**
- < 99.99% but >= 99.0% - 10%
- < 99.0% but >= 95.0% - 30%
- < 95.0% -100%



Case-study

Scalability:
What is the Best Approach for Cloud Computing Clients?

"Handling flash crowds from your garage"



Flash Crowds!

- We have seen several examples of scalability in cloud platforms, but:
  - what about the clients?
  - what if we have a server running an application and it needs to scale quickly?
  - how do we adapt the front-ends?

- Three main requirements:
  - the system must scale to a very large size.
  - the system must scale quickly.
  - off-peak operation must be cheap.

Available Tools (1)

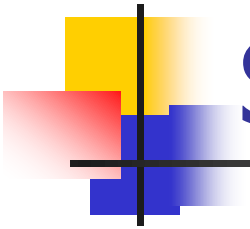
- Data storage services:
  - Pros: they are cheap and
    - they scale transparently for the user.
  - Cons: mostly only solve problem of static content.

- Virtual servers at ISP:
  - Pros: before the cloud, it was already possible to rent virtual servers at ISPs
    - even at different geographical locations.
  - Cons: It only solves the latency/bandwidth problem.
    - mostly, computation of distributed applications doesn't scale easily (no fast instance launching)

Available Tools (2)

- Cloud computing services.

- External DNS services:
  - prevents the service from facing a bottleneck on the DNS requests.



Scalable Architectures (1)

- What is the best approach to matching a large set of clients with a multi-server service?

- Hyp. 1: Use only a storage service (e.g., S3)
  - good for servers with a large percentage of static content.

Scalable Architectures (2)

- Hyp. 2: Cluster with DNS load balancing
  - rent several machines (e.g. EC2).
  - add machines to the DNS record.
    - by default, addresses are used in round-robin fashion.
    - causes delays to the clients who cached the DNS record
  - in general the issue is the large number of clients
    - and not large number of requests from the same client

Scalable Architectures (3)

- Hyp. 3: HTTP Redirection
  - having a server redirect the initial client request to a set of back-end servers.
  - subsequent requests don't go through the redirection.

Scalable Architectures (4)

- Hyp. 4: L4 or L7 Rerouting
  - a front-end server analyzes the request's:
    - source (4 OSI level 4 – e.g. TCP) or
    - content (OSI level 7 – e.g. HTTP)
    - reroutes each request to corresponding back-end server.
  - requires *high-performance* server (*middlebox*) or switch
    - client does not see the redirection.
    - similar to Amazon Elastic Load Balancing.
    - (also implemented as Network Function Virtualization)

- Hyp. 5: Hybrids of the 4 previous hypothesis.

M: repl. front-end *N: repl. back-end*	Storage Service	HTTP Redirection	L4/L7 Rerouting	DNS Load Balancing
Applicability				
Scale Limitations				
Client Affinity				
Scale-up Time				
Scale-down Time				
Front-end Fault: **New Sessions**				
Front-end Fault: **Ongoing Sessions**				
Replicated Front-end Fault: **New Sessions**				
Replicated Front-end Fault: **Ongoing Sessions**				
Back-end Fault: **New Sessions**				
Back-end Fault : **Ongoing Sessions**				

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redirection**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations				
Client Affinity				
Scale-up Time				
Scale-down Time				
Front-end Fault: **New Sessions**				
Front-end Fault: **Ongoing Sessions**				
Replicated Front-end Fault: **New Sessions**				
Replicated Front-end Fault: **Ongoing Sessions**				
Back-end Fault: **New Sessions**				
Back-end Fault : **Ongoing Sessions**				

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redirection**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate		
Client Affinity				
Scale-up Time				
Scale-down Time				
Front-end Fault: **New Sessions**				
Front-end Fault: **Ongoing Sessions**				
Replicated Front-end Fault: **New Sessions**				
Replicated Front-end Fault: **Ongoing Sessions**				
Back-end Fault: **New Sessions**				
Back-end Fault : **Ongoing Sessions**				

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redir.**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate		
Client Affinity				
Scale-up Time				
Scale-down Time				
Front-end Fault: New Sessions				
Front-end Fault: Ongoing Sessions				
Replicated Front-end Fault: New Sessions				
Replicated Front-end Fault: Ongoing Sessions				
Back-end Fault: New Sessions				
Back-end Fault : Ongoing Sessions				

Redirecting clients (specially if it's done only when a session begins), is very cheap even if the front-server is receiving back-end status reports and running a load balancing algorithm.

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redir.**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate	Request arrival rate	Unlimited
Client Affinity				
Scale-up Time				
Scale-down Time				
Front-end Fault: New Sessions				
Front-end Fault: Ongoing Sessions				
Replicated Front-end Fault: New Sessions				
Replicated Front-end Fault: Ongoing Sessions				
Back-end Fault: New Sessions				
Back-end Fault : Ongoing Sessions				

The UDP-based DNS response has only 512 bytes (up to 25 back-end servers). Most ISP complete the request using TCP if there are more than 25. However, some DNS clients only use the first reply.

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redir.**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate	Request arrival rate	Unlimited
Client Affinity	N/A	Coherent	Coherent	Incoherent
Scale-up Time				
Scale-down Time				
Front-end Fault: New Sessions				
Front-end Fault: Ongoing Sessions				
Replicated Front-end Fault: New Sessions				
Replicated Front-end Fault: Ongoing Sessions				
Back-end Fault: New Sessions				
Back-end Fault : Ongoing Sessions				

Coherent, but in the case of L4 there are growing hurdles to success: NAT, proxies,...

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redir.**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate	Request arrival rate	Unlimited
Client Affinity	N/A	Coherent	Coherent	Incoherent
Scale-up Time	Immediate	VM start-up time	VM start-up time	VM start-up time + DNS TTL
Scale-down Time				
Front-end Fault: New Sessions				
Front-end Fault: Ongoing Sessions				
Replicated Front-end Fault: New Sessions				
Replicated Front-end Fault: Ongoing Sessions				
Back-end Fault: New Sessions				
Back-end Fault :				

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redir.**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate	Request arrival rate	Unlimited
Client Affinity	N/A	Coherent	Coherent	Incoherent
Scale-up Time	Immediate	VM start-up time	VM start-up time	VM start-up time + DNS TTL
Scale-down Time	Immediate	Session duration	Session duration	Days
Front-end Fault: New Sessions				
Front-end Fault: Ongoing Sessio				
Replicated Front- Fault: New Sessions				
Replicated Front-end Fault: Ongoing Sessions				
Back-end Fault: New Sessions				
Back-end Fault :				

In L4 it is difficult to identify when sessions finish (e.g. webmail).

There are DNS clients that ignore DNS records' TTL and take days to invalidate their DNS cache.

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redir.**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate	Request arrival rate	Unlimited
Client Affinity	N/A	Coherent	Coherent	Incoherent
Scale-up Time	Imme		art-up time	VM start-up time + DNS TTL
Scale-down Time	Immediate	Session duration	Session duration	Days
Front-end Fault: New Sessions	N/A	Total Failure	Total Failure	Significant Fault
Front-end Fault: Ongoing Sessions				
Replicated Front-end Fault: New Sessions				
Replicated Front-end Fault: Ongoing Sessions				
Back-end Fault: New Sessions				
Back-end Fault :				

The front-end of the storage service. Not the web server.

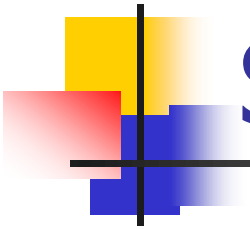
M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redir.**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate	Request arrival rate	Unlimited
Client Affinity	N/A	Coherent	Coherent	Incoherent
Scale-up Time			ime	VM start-up time + DNS TTL
Scale-down Time			tion	Days
Front-end Fault: New Sessions	N/A	Total Failu	Total Failure	Significant Fault
Front-end Fault: Ongoing Sessions	N/A	Has no effect	Total Failure	Rare Effect
Replicated Front-end Fault: New Sessions	Improbable	Long delay for 1/m sessions?	Long delay for 1/m sessions?	Small Delay
Replicated Front-end Fault: Ongoing Sessions				
Back-end Fault: New Sessions				
Back-end Fault :				

If there is load balancing of the redirection servers, one has to wait for the client to try another server. It should take max 5 s but in some Linux implementations it takes up to 3 min!

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redir.**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate	Request arrival rate	Unlimited
Client Affinity	N/A	Coherent	Coherent	Incoherent
Scale-up Time	Immediate	VM start-up time	VM start-up time	VM start-up time + DNS TTL
Scale-down Time	Immediate	Session duration	Session duration	Days
Front-end Fault: **New Sessions**	N/A	Total Failure	Total Failure	Significant Fault
Fro **Ongo**			Total Failure	Rare Effect
Replicated Front-end Fault: **New Sessions**	Improbable	for 1/m ssions?	Long delay for 1/m sessions?	Small Delay
Replicated Front-end Fault: **Ongoing Sessions**	Improbable	Has no effect	1/m sessions fail.	Some sessions have small delay.
Back-end Fault: **New Sessions**	Improbable	Has no effect	Has no effect	Long delay for 1/n sessions.
Back-end Fault : **Ongoing Sessions**	Improbable	User recoverable fault	Occasional fault	Long delay for 1/n sessions.

e.g., in S3 1% of first write attempts fail, but immediate retries succeed.

M: repl. front-end *N: repl. back-end*	**Storage Service**	**HTTP Redir.**	**L4/L7 Rerouting**	**DNS Load Balancing**
Applicability	Static HTTP	HTTP	Everything	Everything
Scale Limitations	Scales very well	Client arrival rate	Request arrival rate	Unlimited
Client Affinity	N/A	Coherent	Coherent	Incoherent
Scale-up Time	Immediate	VM start-up time	VM start-up time	VM start-up time + DNS TTL
Scale-down Time	Immediate	Session duration	Session duration	Days
Front-end Fault: **New Sessions**	N/A	Total Failure	Total Failure	Significant Fault
Front-end Fault: **Ongoing Sessions**	N/A	Has no effect	Total Failure	Rare Effect
Replicated Front-end Fault: **New Sessions**	Improbable	Long delay for 1/m sessions?	Long delay for 1/m sessions?	Small Delay
Replicated Front-end Fault: **Ongoing Sessions**	Improbable	Has no effect	1/m sessions fail.	Some sessions have small delay.
Back-end Fault: **New Sessions**	Improbable	Has no effect	Has no effect	Long delay for 1/n sessions.
Back-end Fault : **Ongoing Sessions**	Improbable	User recoverable fault	Occasional fault	Long delay for 1/n sessions.



Summary

- Cloud Execution Platforms and Models:
  - Microsoft Azure (public)
    - Microsoft Azure Fabric Controller
  - OpenStack (private)
    - Nova scheduler
  - User vs. provider responsibilities, SLAs

- Scaling a Web Server with the Cloud:
  - *"Handling Flash Crowds from Your Garage"*

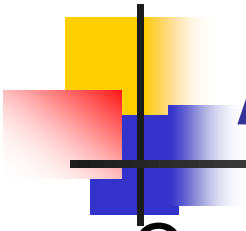


High-level Language VMs Architecture
(Java VM Architecture)

Prof. Luís Veiga
IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

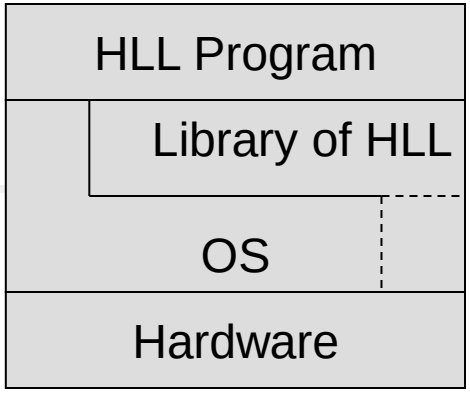
© LV, JG 2015-24



Agenda

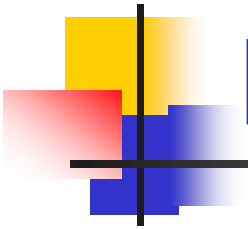
- Overview of HLL VMs
- Pascal P-code VM
  - memory architecture, instruction set
- Java VM Architecture
  - data types, data storage
  - byte-code (virtual) instruction set
    - data movement, control flow, method invocation, object creation
  - exceptions/errors
  - threads and synchronization

HLL VM Overview



- Process VM approach
  - *after-the-fact,* pragmatic
  - neither host nor guest designed with portability in mind

- Design new guest-ISA/interface VM-based
  - *step-back, before-the-fact*, portability as primary goal
  - generic ISA
    - free of implementation-specific features/resources
    - defined for user programs, not real machines
  - higher-level system interface
    - consolidates/aggregates features easily supported by most OS
  - support for features of HLL (e.g., OO languages)

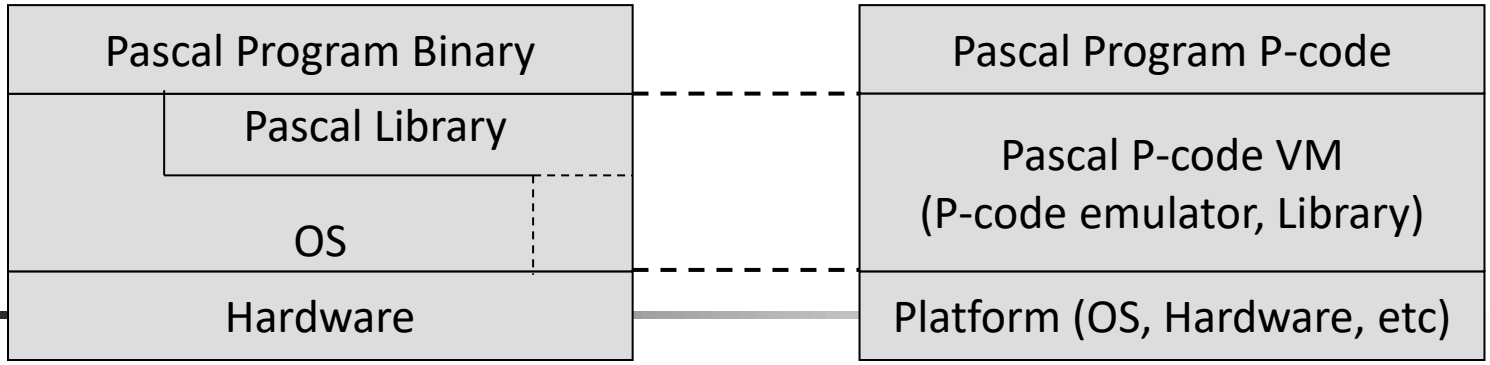


HLL VM Overview

- HLL VM vs traditional Process VMs
  - support for virtual ISA (V-ISA) vs. real ISA
  - interface based on standard libraries vs. OS sys-calls
    - e.g., file system, network, etc.
  - V-ISA includes:
    - instruction set
    - data specification (*metadata*)
      - important for platform independence
      - dominates overall specification
    - more of a *data set architecture*
      - than instruction set architecture

Pascal P-Code VM

- P-code for highly portable Pascal (c. 1975-1978)
  - simple, stack-based virtual-ISA (just like Java bytecode)
- Development
  - requires only one Pascal compiler
    - applications distributed as a P-code version
- Execution system
  - P-code emulator/interpreter
  - library routines for each ABI: host-OS interface, I/O
- Porting Pascal reduced to implementing the VM
  - much simpler than writing another Pascal compiler from scratch!



The Pascal P-Code VM

- P-code Instruction Set
  - Op-code and offset with operand relative location
  - Push and pop values
    - from/to stack and memory areas
  - ALU functions operate on stack
  - Typed instructions
    - e.g., addi (add integer), addr (add real)

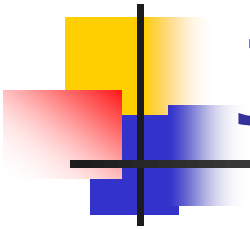
```
lodi    0   3   //  load int variable from current stack frame (nest 0 depth)
                //  offset 3 from top of mark stack
ldci    1       //  push int constant 1
addi            // add two integers at top of stack
stri    0   3   // store int variable back to location 3 of current stack frame
```

Java VM Architecture

- Java virtual-ISA
  - bytecode (instruction) definitions
  - set of pre-defined data-types
  - metadata: data definitions and inter-relationships
    - formalized in class file specification
- Java VM Architected State
  - For each running thread:
    - implicit VM registers
      - PC, local variable pointer, operand stack pointer,
      - stack pointer, current frame pointer, constant pool base
    - stack: locals, operands, method parameters and return values
  - Global:
    - heap: objects, arrays
    - class file contents, e.g., constant pool
      - holds immediate values and other constant information

Java VM Architecture

- Data types specification
  - primitive types: int, char, byte, short, float, double
  - reference type: hold reference value to object or *null*
  - objects carry data declared by class
    - composition/aggregation of primitive fields and references
    - array is a special object with ISA support (bytecodes, exceptions)
- Avoiding implementation details or constraints
  - no fixed representation or sizes for data, only ranges…
    - e.g., byte type only states range of values (–128 and +127)
    - internal references can be pointers, handles, indexes, …
  - no direct data addressing (i.e. no arbitrary pointers)
    - located via references, as fields of objects in heap
    - employ offsets using constant pool pointer, stack pointer

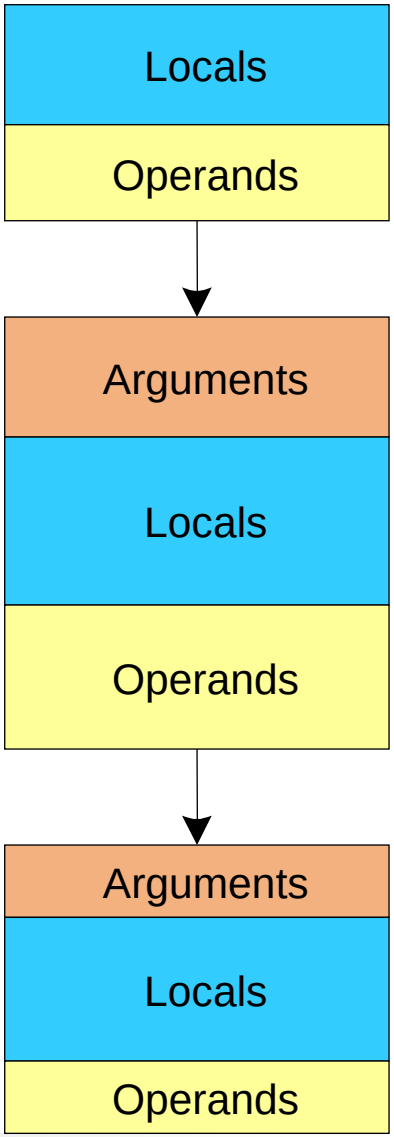


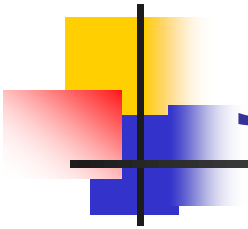
Java VM Architecture

- Data Storage
  - Stack
  - Type and Method Area
  - Heap
  - Constant Pool
  - Memory Hierarchy

Java VM Architecture

- Data Storage: Stack *(Typed)*
  - each JVM thread has its own stack
  - on method invocation
    - new frame is allocated on the stack
  - arguments and local variables
    - numbered from 0 (**zero->this for non-static**)
  - operand stack used for all computations
  - method calling
    - caller pushes object ref and arguments
    - become first locals (as variables) of the callee
  - other frame data
    - return address, data for constant pool resolution, and exception table
    - can be placed between locals and operands





Java VM Architecture

- Data Storage: Type and Method Area
  - one method area per JVM
    - created on JVM start-up
    - analogous to *'text/code'* segment of conventional binaries
  - type definitions
    - (class or interface), inheritance
    - class variables and instance field information
  - method tables: method prototype information of each class
    - bytecode of methods, line numbers, exception tables
    - (stored in streams/attributes area at end of class)
  - all references to names/IDs target constant pool

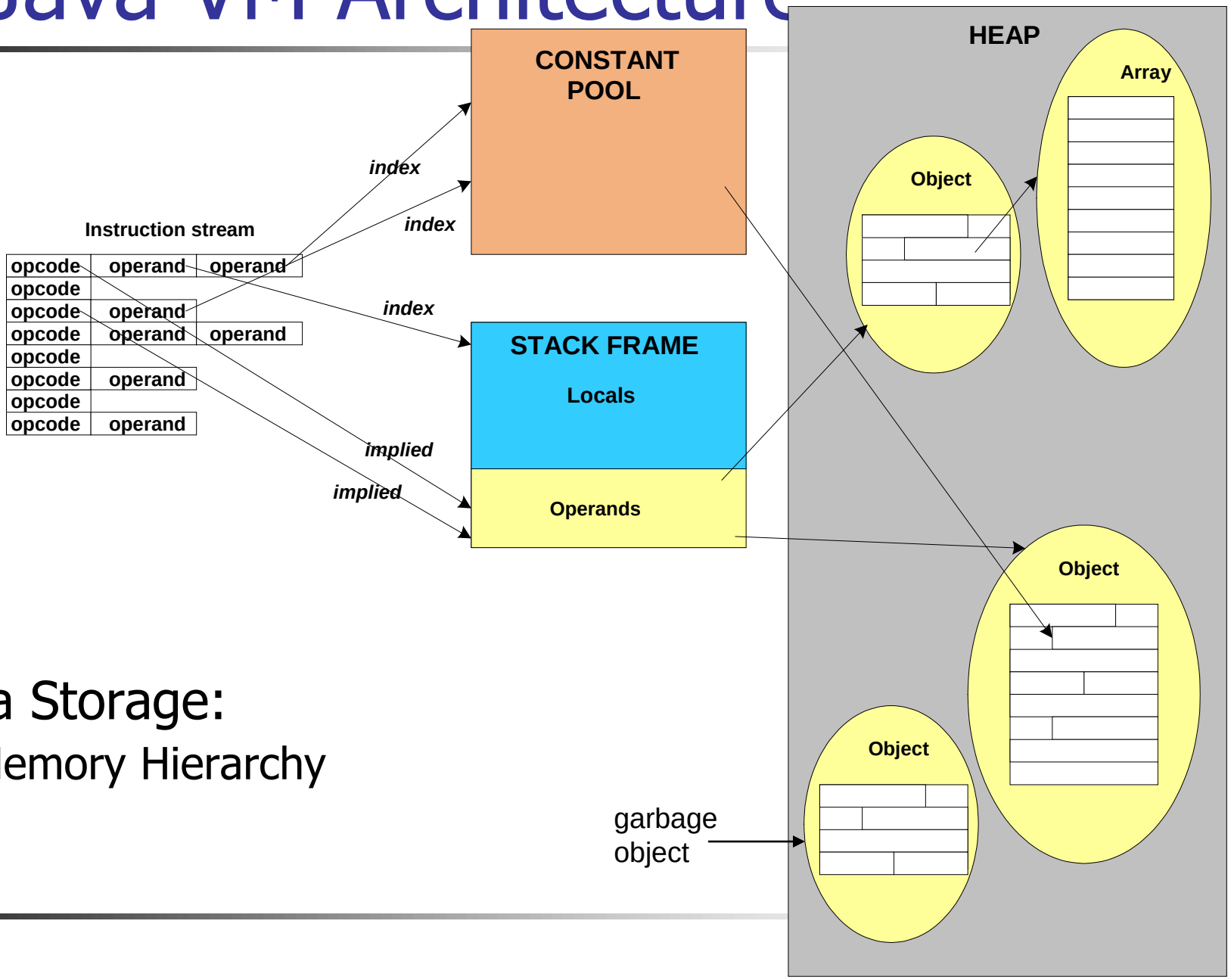
Java VM Architecture

- Data Storage: Heap
  - memory for objects, arrays, and class instances
  - one and only one heap per Java application
    - all threads share it, requires explicit synchronization
  - JVM instruction (new) allocate objects on heap
    - no explicit instruction to release memory
    - GC (Garbage Collector) collects garbage
  - internal object representation
    - fully implementation dependent

Java VM Architecture

- Data Storage: Constant Pool (CP)
  - collection of all symbolic data within a class
    - analogous to symbol table, beginning of class file
    - each constant pool is private to its class
  - instructions often need constants
    - e.g., integer operands, addresses, strings, etc.
  - many constants not stored with instructions
    - saved in CP and instructions include indexes to CP
    - e.g.,
      - `ldc #2` → 12345678
      - `invokespecial #4` → java/lang/object/<init>()V
  - pooling advantages:
    - allows sharing/saving, smaller code foot-print,
    - fixed, pre-determined size for each instruction type

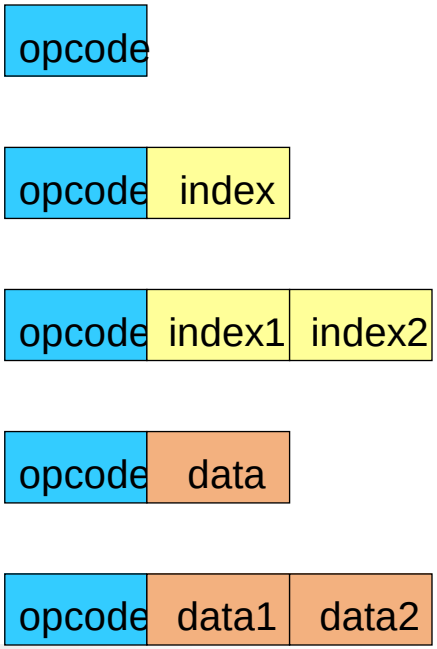
Java VM Architecture



- Data Storage:
  - Memory Hierarchy

Java VM Architecture

- Instruction Set: Formats
  - opcode byte + zero or more operands (depending on op-code)
    - operands: index, immediate data, or PC-relative offset
  - operands fetched from:
    - instruction operands, current constant pool,
    - local variables, operand stack
  - each primitive type has its own instruction set
    - e.g., iadd, fadd, …
    - operand types in stack
      - must match the opcode (verified)
    - Java-bytecode ISA: a typed stack machine
  - code representation
    - defined for class file, not memory image

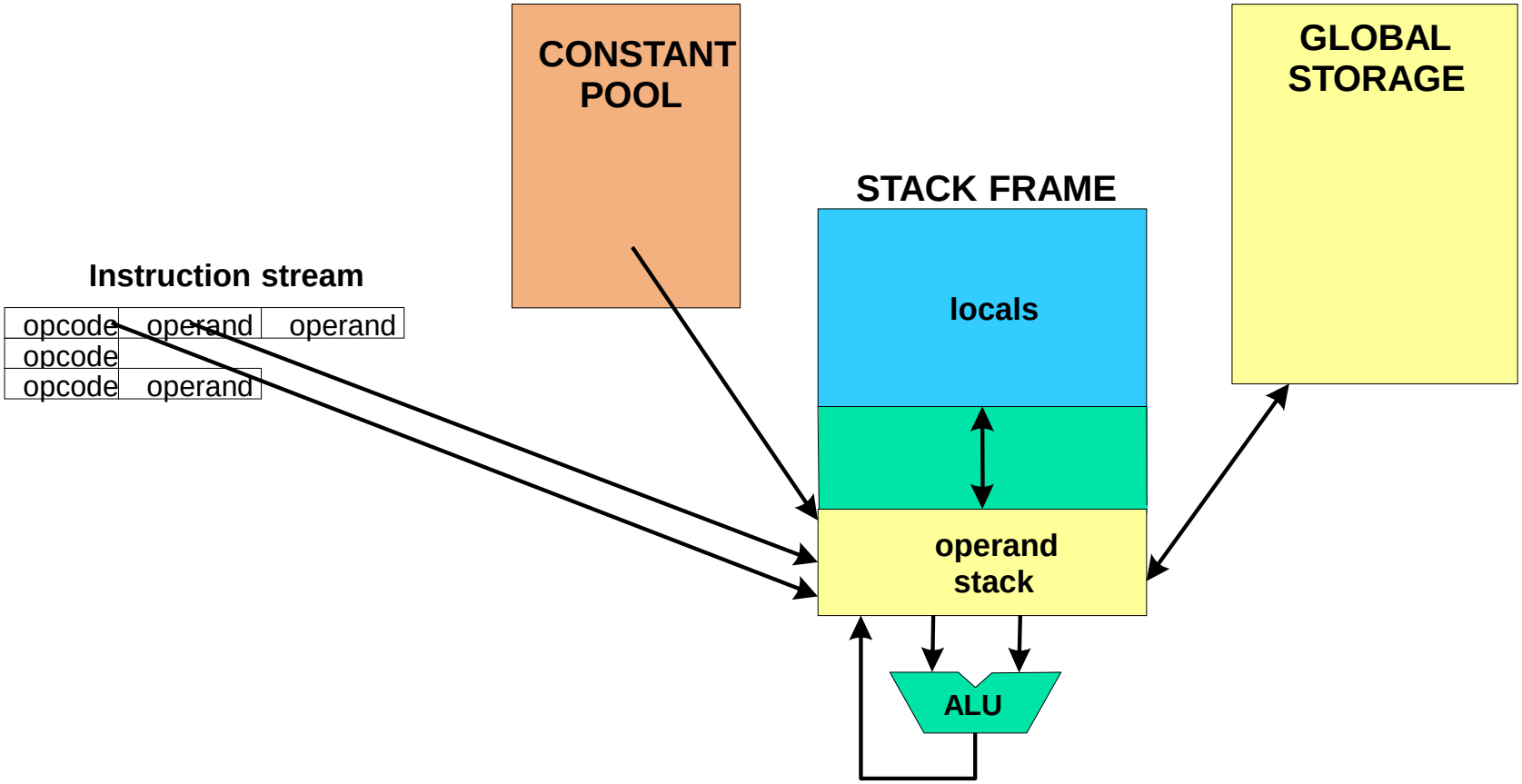


Java VM Architecture

- Instruction Set: Instruction Types
  - data-movement instructions
    - pushing constants onto the operand stack
    - moving local variable contents to and from the operand stack
    - manipulating object references and fields
    - generic stack instructions
  - functional instructions
    - type conversion, arithmetics, logical, shift
  - control flow instructions
    - jumps, conditional jumps, intra-method subroutines
    - method invocation, method return
  - others
    - object and array instantiation (also use stack)
    - monitors for synchronization (also use stack)
    - nop

Java VM Architecture

- Instruction-Set: Data-Movement
  - all data movement takes place via stack



Java VM Architecture

- Instruction-Set: Data-Movement
  - pushing constants onto the stack
    - `aconst_null`, `iconst_1`,
    - `ldc` (via constant pool), `bipush` (direct immediate)
    - there can be different instructions for the same function
  - move values between operand stack and locals in stack frame
    - `aload_0, iload_1, aload 5, istore_2, astore 5`
      - optimized byte-codes for offsets 0-3
  - field access and check instructions
    - `getfield, putfield` - move data between object and stack
    - `getstatic, putstatic` - move data between stack and class data area
    - `checkcast, instanceof` - check if an object is an instance of a type
  - object creation
    - `new` allocates new object and saves the reference on the operand stack
      - proper creation requires following explicit constructor invocation
  - stack manipulation instructions
    - `pop, dup, swap, …`

Java VM Architecture

- Data-Movement: Constant immediates and pool

```
int i = 100;
int j = 1000000;
long l1 = 1;
long l2 = 0xffffffff;
double d = 2.2;
```

  - Java VM bytecodes

```
PC offset instruction
 0 bipush 100  // push byte or a small int with bipush
 2 istore_1

 3 ldc #1      // push int constant 1000000; a larger int value uses ldc
 5 istore_2

 6 lconst_1    // a tiny long value uses short and fast lconst_1 bytecode
 7 lstore_3    // long local occupies two words

 8 ldc2_w #6   // push long 0xffffffff (that is, an int -1);
               // any long constant value can be pushed using ldc2_w
11 lstore 5

13 ldc2_w #8  // push double constant 2.200000;
16 dstore 7
```

Java VM Architecture

- Data-Movement Example Byte-code
  - Java method
    - fields:
      - sides [ ] field is #2 in constant pool (not necessarily class offset)

```
public int perimeter() {
   return 2*(this.sides[0]+this.sides[1]);
}
```

  - Java VM bytecodes

```
PC offset instruction
    0:   iconst_2          // push multiplier 2 on stack
    1:   aload_0           // load this ref on stack
    2:   getfield   #2;    // get field sides ref via const pool, push on stack
    5:   iconst_0          // push index zero on stack
    6:   iaload            // load array element from array

    7:   aload_0           // load this ref on stack
    8:   getfield   #2;    // get field sides ref via const pool, push on stack
   11:   iconst_1          // push index one on stack
   12:   iaload            // load array element from array

   13:   iadd              // perform sides[0]+sides[1]
   14:   imul              // perform 2*(sides[0]+sides[1])
   15:   ireturn           // returns, leaves result in stack
```

Java VM Architecture

- Instruction-Set: control flow
  - branches and jumps
    - ifeq (top stack zero?), if_icmpeq (top stack positions ==?), ifnull
    - goto, jsr (with stacked return address)
    - PC-relative branch to a constant offset (mostly, no indirection)
  - all control paths can be easily tracked and verified
    - no code discovery problem, everything in code stream is code/instructions

```
int i = 0;
while (i < 100) {
        i++; }
```

```
0 iconst_0      // init local variable i
1 istore_1
2 goto 8        // eval condition before loop
5 iinc 1 1      // loop body
8 iload_1       // eval loop condition
9 bipush 100
11 if_icmplt 5
14 return
```

Java VM Architecture

```
switch (i) {
        case 0:  return 0;
        case 1:  return 1;
        case 2:  return 2;
        default: return -1;
    }
```

- Instruction-Set: control flow
  - switch/case
    - tableswitch
    - used when cases of switch can be efficiently represented as implicit indices into a range table of target offsets.
      - 4-byte range and target values and padding (8 ints in lookupswitch example)

```
 0 iload_1              // Push local variable 1 (argument i)
 1 tableswitch 0 to 2:        // Valid indices are 0 through 2
        0: 28           // If i is 0, continue at 28
        1: 30           // If i is 1, continue at 30
        2: 32           // If i is 2, continue at 32
        default:34      // Otherwise, continue at 34
28 iconst_0             // i was 0; push int constant 0...
29 ireturn              // ...and return it
30 iconst_1             // i was 1; push int constant 1...
31 ireturn              // ...and return it
32 iconst_2             // i was 2; push int constant 2...
33 ireturn              // ...and return it
34 iconst_m1            // otherwise push int constant -1...
35 ireturn              // ...and return it
```

Java VM Architecture

```
switch (i) {
  case -100: return -1;
  case    0: return 0;
  case  100: return 1;
  default:   return -1;
}
```

- Instruction-Set: control flow
  - switch/case
    - lookupswitch, explicit case values in rows
    - used when cases of switch are sparse, and implicit table representation of tableswitch becomes inefficient in terms of space. (4-byte match/target value pairs x 4 and padding)

```
 0   iload_1            // Push local variable 1 (argument i)
 1   lookupswitch 3:    // number of (sparse) cases
       -100: 36
       0: 38
       100: 40
       default:42
36 iconst_m1            // i was -100; push int constant minus 1...
37 ireturn
38 iconst_0             // i was 0; push int constant zero...
39 ireturn
40 iconst_1             // i was 100; push int constant 1...
41 ireturn
42 iconst_m1            // otherwise; push int constant minus 1...
43 ireturn
```

Java VM Architecture

- Control Flow: Method invocation
  - `Invokevirtual` – for class instance methods
    - indexes constant pool location with description of called method
      - method address (via table index), number and type of arguments, # of locals, maximum operand stack depth
    - check if arguments match,
      - operand stack must contain *num_args* words of parameters
      - `objectref` (reference to object where method is invoked on)
      - *param* words must match order and type required by method
    - allocate stack frame of appropriate size
      - push `objectref` word for `this`
      - push arguments as local variables in frame
    - jump to entry point of called method
      - return PC is saved on the stack (in frame data area),
      - can not be accessed directly (only through method return)

Java VM Architecture

- Control Flow: Method invocation
  - `invokeinterface`
    - invoking interface method (interface reference)
  - `invokespecial`
    - instance initialization methods (constructors)
    - private methods
    - methods of a superclass of the current class
  - `invokestatic`
    - invoking static methods
    - no push of objectref this
    - *local* in position zero is first actual parameter
- Returning Values
  - `ireturn:` makes return after "popping" an integer
    - removes the method stack frame, then push result back to stack
  - `return:` for void methods
  - freturn, dreturn, lreturn for other types

Java VM Architecture

- Instruction-Set: control flow
  - e.g., class instance method invocation and return

```
int add12and13() {
    return addTwo(12, 13);
}
```

  - Java VM bytecodes

```
PC instruction
0  aload_0              // push local variable 0 (this)
1  bipush 12            // push int constant 12
3  bipush 13            // push int constant 13
5  invokevirtual #4     // method Example.addtwo(II)I
8  ireturn              // return int on top of operand stack;
                        // it is the int result of addTwo()
```

  - static methods:
    - do not require *aload_0* (no this object ref to push)

Java VM Architecture

- Instruction-Set: object creation

```
Object create() {
    return new Object();
}
```

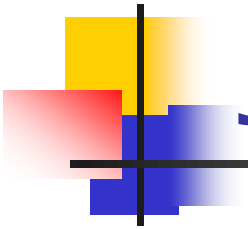
  - Java VM bytecodes

```
PC instruction
0   new #1                  // allocates object Class java.lang.Object
                            // and leaves reference in stack

3   dup                     // duplicates reference in stack

4   invokespecial #4        // method java.lang.Object.<init>()V
                            // consumes one reference in stack

7   areturn                 // return reference to initialized object
```



Java VM Architecture

- Exceptions and Errors
  - exception handling in Java
    - employing `try` and `catch` blocks
  - all exceptions must be handled somewhere
    - if no catch block in method raising exception
      - stack frame popped until exception handler found
  - an exception is thrown via `throw` instruction

Java VM Architecture

- Exceptions and Errors: Throwing Exceptions

```
void cantBeZero(int i) throws TestExc
{
        if (i == 0) {
                throw new TestExc(); }
}
```

```
PC   instruction
 0   iload_1            // push argument 1 (i)
 1   ifne 12            // if i==0, allocate instance & throw
 4   new #1             // create instance of TestExc
 7   dup                // one reference to the constructor
 8   invokespecial #7   // method TestExc.<init>()V
11   athrow             // initial reference is thrown
12   return             // never get here if we threw TestExc
```

Java VM Architecture

- Exceptions and Errors: Exception Tables
  - use exception table to specify an exception handler
  - exceptions identified by table in class file
    - address range where checking is in effect
    - target if/when exception is thrown
      - operand stack is emptied, ensure predictable structure
    - type of exception to be catched
  - if no table entry in current method
    - pop stack frame and check calling method
    - default handlers at main()
    - e.g., if an arithmetic exception is thrown
      - between bytecode 8 and 12, jump to bytecode 96

From	To	Target	Type
8	12	96	Arithmetic Exception

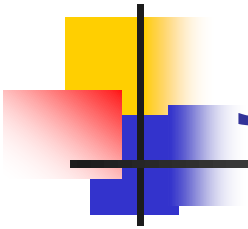
Java VM Architecture

- Exceptions and Errors: Catching Exceptions

```
try { tryItOut(); }
catch (TestExc1 e) {
        handleExc(e); }
catch (TestExc2 e) {
        handleExc(e); } }
```

```
PC   instruction
 0  aload_0           // Begin try block
 1  invokevirtual #5 // Method Example.tryItOut()V
 4  return            // End of try block; normal return
 5 astore_1          // Begin handler TestExc1; store thrown value in local 1
 6 aload_0           // Push this
 7 aload_1           // Push thrown value
 8 invokevirtual #7  // Invoke handler handleExc(LTestExc1;)V
11 return            // Return after handling TestExc1
12 astore_1          // Begin handler TestExc2; store thrown value in local 1
13 aload_0           // Push this
14 aload_1           // Push thrown value
15 invokevirtual #7  // Invoke handler handleExc(LTestExc2;)V
18 return            // Return after handling TestExc2
```

Exception table:	From	To	Target	Type
	0	4	5	Class TestExc1
	0	4	12	Class TestExc2



Java APIs

- Java Threads
  - java.lang.Thread class and Runnable interface
  - Thread execute run() method during its lifetime
  - Synchronization through monitor
    - required when we have a *synchronized* method
    - supported by:
      - *monitorenter* and *monitorexit* bytecode
      - architected locks associated with each synchronized object (assumed in JVM spec, implemented in Java VMs)
    - other synchronization support:
      - object methods wait(), notify(), notifyAll()

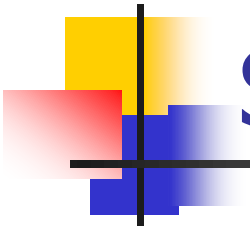
Java APIs

```
void onlyMe(Foo f)
{
        synchronized(f) {
                doSomething();
        }
}
```

- Java Threads
  - e.g., of bytecodes in synchronization block

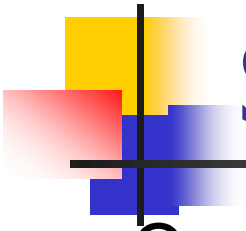
```
PC   instruction
 0   aload_1            // push f
 1   astore_2           // store it in local variable 2
 2   aload_2            // push local variable 2 (f)
 3   monitorenter       // enter the monitor associated with f
 4   aload_0            // holding the monitor, pass this and...
 5   invokevirtual #5   // call Example.doSomething()V
 8   aload_2            // push local variable 2 (f)
 9   monitorexit        // exit the monitor associated with f
10   return             // return normally
11   aload_2            // in case of any throw, end up here
12   monitorexit        // be sure to exit monitor...
13   athrow             // ...then rethrow the value to the invoker

Exception table:        From    To      Target          Type
(implicitly created)    4       8       11              any
```



Summary of Virtual ISA Features

- Metadata in V-ISA allows verification/interoperability
  - regular ISA does not have it
  - compilers use data structures and then throw them away
    - they are only implicit in binary code
- Memory architecture is more abstract
  - conceptually unbounded in size, no explicit visible addressing,
  - memory contents available to computation
  - but not explicilty addressed for read or write
  - strict limitation in memory address manipulation
- No *explicit* registers in bytecode
  - leads to simpler emulation
- Minimal OS dependence through libraries



Summary

- Overview of HLL VMs
- Pascal P-code VM
  - memory architecture, instruction set
- Java VM Architecture
  - data types, data storage
  - byte-code (virtual) instruction set
    - data movement, control flow, method invocation, object creation
  - exceptions/errors
  - threads and synchronization

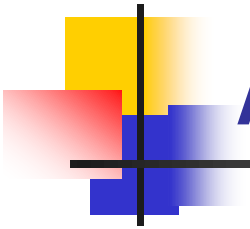


High-level Language VMs Implementation
(Java VM Implementation)

Prof. Luís Veiga
IST / INESC-ID Lisboa

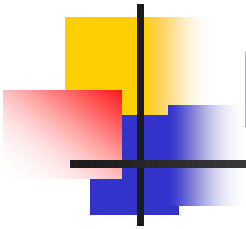
https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© LV, JG 2015-24



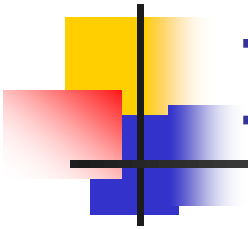
Agenda

- HLL VM Implementation Overview
- Implementing Security
  - sandobox model, intra-process protection
  - bytecode correctness, security policy enforcement
- Garbage Collection
  - mark-and-sweep, compacting GC, copying GC
  - generational GC
  - incremental and concurrent GC
- High Performance Emulation
  - optimization framework
  - optimizations: inlining, virtual methods, short-lived objects, array checks



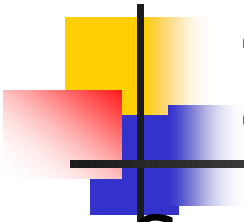
HLL VM Implementation Overview

- Role of a HLL VM implementation
  - incarnate the VM specification
  - provide an execution environment
  - carry out operations specified by programs
- Major components
  - Security
    - handled in Class Loader and during Emulation
  - Memory system
    - garbage collector, GC algorithms
  - Emulation engine
    - bytecode interpreter, *Just-In-Time (JIT)* compiler/optimizer



Implementing Security

- Implementing Security
  - sandbox model
    - intra-process protection
    - bytecode correctness
  - security policy enforcement



Implementing Security

- Sandbox Model
  - default security model in Java
  - creates barrier around Java execution environment
  - user applications *(possibly dynamically downloaded)*
    - always executed within the bounds of the sandbox
    - cannot affect resources on the outside of sandbox
  - basic rules of sandbox
    - JVM code and data protection
      - enforced by **static and dynamic checks**
      - app interacts w/ JVM/host only via trusted API/library
    - protection of files and other resources
      - enforced by Security Manager

Implementing Security

- IntraProcess Protection
  - **static (load time) checking**
    - consistency checks:
      - types and valid references
      - method calls and arguments
    - integrity checks:
      - correct data usage, **operand stack tracking**
      - control transfer
        - to ensure jump/branch never leaves valid code region
  - **dynamic (run time) checking**
    - check null pointer
    - check array bounds
    - check dynamic casting or conversion

Implementing Security

- Operand Stack Tracking
  - performed at class loading for validity check
    - leverage static typed checking
    - avoid costly constant/repetitive checking during runtime
    - all control flows can/must be determined at loading time
    - class loader can also check for the following:
      - stack limits, types of operands to JVM instructions
      - accesses or assignments to local variables
  - overall goal:
    - ensure correct, predictable and consistent app behavior
    - allow detection of invalid programs
      - may crash/hijack the VM in undefined/unpredictable way
      - with incorrect and/or inconsistent behavior across JVMs

Implementing Security

- Operand Stack Tracking
  - Overall approach:
    - analyze bytecodes of methods sequentially
    - track potential data movement, types, stack configuration
    - resulting from execution (but without actually executing)

- Correctness criteria:
  - operand stack at any given point in program has:
    - same number of operands
    - of the same types
    - in the same order
  - regardless of control flow path getting/leading there

Implementing Security

- Operand Stack Tracking
  - e.g., of valid code sequence
  - stack structure (number and type of elements) predictable
    - any point in code
    - regardless of prior branching behavior upstream in the code
    - always the same at the end of code

```
          iload   A          //push int. A from local mem.
          iload   B          //push int. B from local mem.
          ifne else1         // branch if B ne 0
          iload   C          // push int. C from local mem.
          goto   endelse1
   else1  iload   F          //push F
endelse1  iadd               // add from stack; result to stack
          istore D           // pop sum to D
```

  - Error in book: `if_cmpne` should read as `ifne`

Implementing Security

Operand Stack Tracking

- e.g., of invalid code sequence
- stack structure at skip1 and in iadd depends on path followed
  - if B is non zero, just method arguments and/or locals
  - if not, arguments, locals and C left on stack as an operand
  - at second branch, either one or two operands left
- exact stack content structure in *iadd* and at end is unknown
  - with only static analysis (check *error* in *iadd* when B<>0; E=0)
  - with only static analysis (check *error* in *istore* when B=0; E<>0)

```
        iload   B          // push int. B from local mem.
        ifne    skip1       // branch if B ne 0
        iload   C          // push int. C from local mem.
skip1   iload   D          // push D
        iload   E          // push E
        ifne    skip2       // branch if E ne 0
        iadd               // add from stack; result to stack
skip2   istore  F          // pop to F
```

Implementing Security

Security Policy Enforcement

- Security Manager: class in java.lang API for Java
  - all methods accessing outside world (in Java Base Class Library):
    - explicit ask/check for permitions.
  - check if the requested operation is permitted by policy in effect
    - returns if permitted, throws SecurityException if not permitted
    - includes **checkXXXPermission() methods**
      - e.g., File..., Socket..., PropertyPermission,
  - attached when application initiated
    - cannot be changed, deleted, or replaced
  - user can specify policy criteria/permission/rules:
    - e.g., which files and access, network ports, statec in policy file.
  - apps limited to privileges of user (JVM is user process)
  - limitation: only *qualitative* protection (objects and operations)
    - not *quantitative* protection
    - e.g., over-allocate memory, excess threads, recursion, CPU usage…

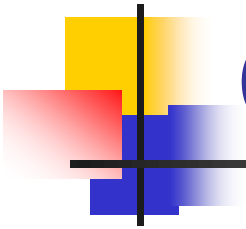
Garbage Collection

- Object oriented programming environment
  - objects freely created, used and discarded
    - by application ***(mutator)***
  - illusion of unbound memory to the programmer
  - no worry with explicit memory allocation/release
  - improved programming productivity and reliability

- In reality:
  - memory resources are not unbounded
  - program can run out of memory
  - to prevent or forestall memory exhaustion
  - use GC ***(collector)***:
    - collect objects that are no longer accessible/reachable
    - reuse/reclaim memory for new objects

Garbage Collection

- Garbage Collector
  - essential part of JVM implementation
    - though not a part of JVM specification
  - overhead of GC
    - time spent in GC is not spent in computing
  - potential of GC
    - improved memory locality, reliable, faster memory management
  - tradeoffs considered when implementing GC
    - garbage collection time
      - pause time
    - object-allocation time
      - application performance
    - object-access time
      - application performance
    - memory efficiency
      - heap space usage

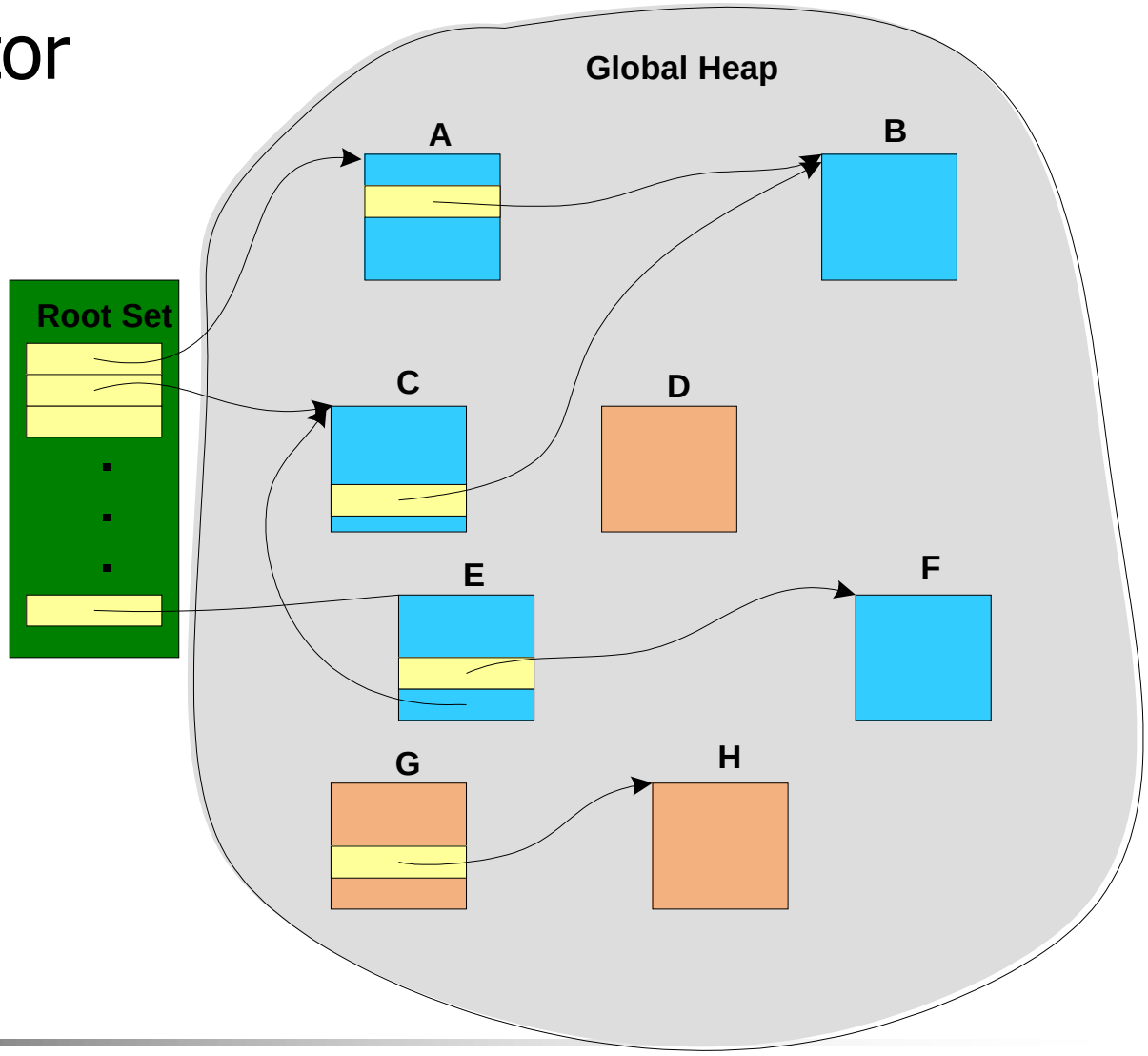


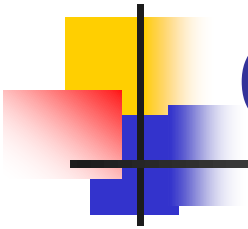
Garbage Collection

- Garbage Collector
  - root-set
    - set of references pointing to objects held in global heap
  - reachable/live objects
    - reachable from transitive closure of root-set
      - directly or indirectly via references in other live objects
  - unreachable/dead/garbage objects
    - outside of the transitive closure of root-set
      - cannot be reached from any chain of references starting from the root set
  - two main families
    - ***tracing*** (mostly in local GC, in study here)
    - *reference counting* (mostly in distributed computing)

Garbage Collection

- Garbage Collector
  - root-set
    - thread stacks
    - local vars
    - arguments
    - operands
    - statics
    - constants
  - live objects
    - A, B, C, E, F
  - garbage
    - D, G, H

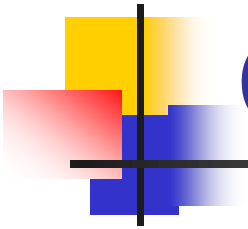




Garbage Collection

- GC algorithms
  - mark-and-sweep
  - compacting GC
  - copying GC
  - generational GC
  - incremental and concurrent GC

- Promise of successively improved *performance*



Garbage Collection

- Mark-and-Sweep
  - mark phase
    - start with the root references
      - on stack, static objects, constant pool…
    - trace and mark all reachable objects
      - e.g., special bit in object header, or entry in global bitmap
    - when traced object found, stop (re-)tracing along this path
  - sweep phase
    - all unmarked objects are collected
    - make reclaimed space available for reuse
    - e.g., combine garbage into a linked list of free objects

Garbage Collection

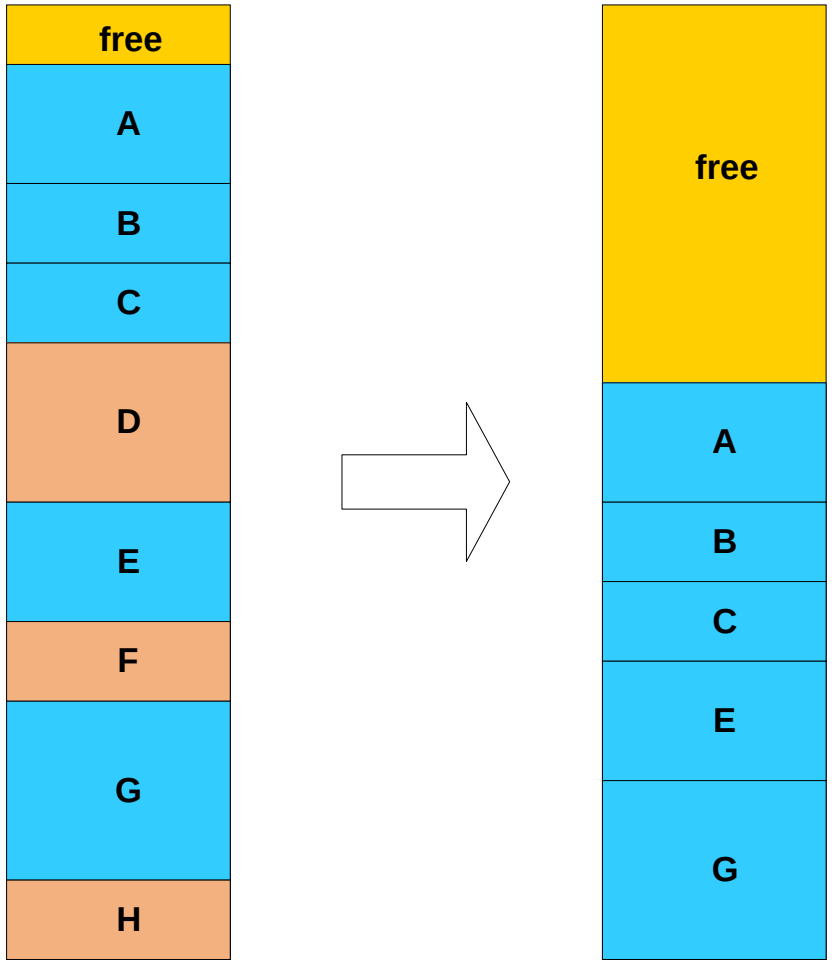
- Mark-and-Sweep
  - advantages
    - relatively fast
    - relatively short garbage collection time
  - disadvantages
    - non-contiguous free space
    - leads to memory fragmentation
    - long object allocation time
      - finding adequate block in free blocked linked-list
      - similar issues in the implementation of *malloc()/free()* funtions
  - optimizations
    - use range of fixed object sizes, e.g., powers of two
    - multiple segregated linked-lists, one for each block size
    - allocate new objects from list with best-fit
    - occasionally rebalance space in each category

Garbage Collection

- Compacting GC
  - insight
    - consolidate all garbage into large contiguous region
    - create new objects from there fast
      - no need for free-list management, just advance a pointer
    - consolidate live objects
      - improve memory locality, and cache performance
  - mark phase
    - find reachable (live) objects
  - compact phase
    - compute new locations for live objects
    - move objects
    - update (*patching)*
      - references stored in objects (fields) in order to point to new locations of referenced objects (that were moved)

Garbage Collection

- Compacting GC
  - E.g.,
    - before GC
      - live:
        - A, B, C, E, G
      - garbage:
        - D, F, H
    - after GC
      - consolidated free space



Garbage Collection

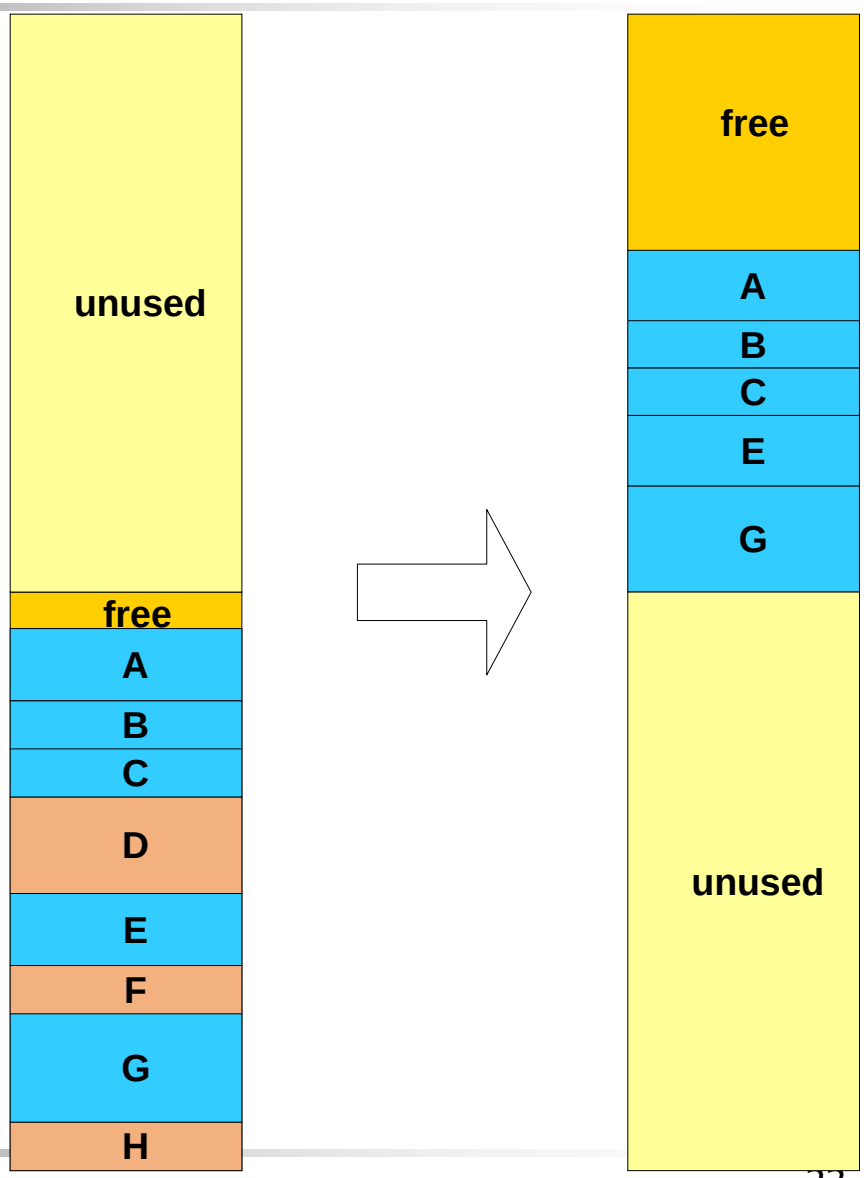
- Compacting GC
  - advantages
    - shorter object allocation time
    - improved memory locality
      - live objects are contiguous in memory
  - disadvantages
    - longer garbage collection time
    - multiple passes through the heap
      - marking
      - computing new locations
      - moving objects (memory transfers)
      - updating (patching) references (to moved objects)
    - too much overhead if fully executed every single time
      - following approach and GC algorithms aim at reducing this
      - otherwise, may use compacting only periodically

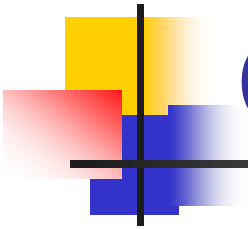
Garbage Collection

- Copying GC
  - approach: combine mark and sweep/compact
    - divide heap into two halves (semi-spaces)
      - i.e., from-space, to-space
    - allocate in from-space
    - perform collection when from-space is full
  - GC operation:
    - copy live objects into to-space
    - flip/switch roles of from-space and to-space
  - advantages
    - relatively faster than compacting collectors
    - reduces number of passes through heap
      - combines two/more GC passes through the heap
  - disadvantages
    - higher memory requirements
      - one semi-space unused, most of the time

Garbage Collection

- Copying GC
  - e.g.
    - copy live objects
      - from-space
      - in to-space
    - flip spaces
      - just conceptually,
      - only change pointers
    - all from-space is now
      - unused contiguous space



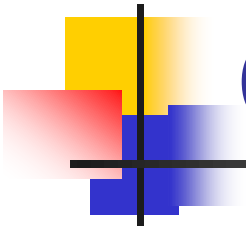


Garbage Collection

- Generational GC
  - insight
    - object lifetimes have bimodal distribution
      - very long life time: garbage-collected infrequently
      - very short life time: garbage-collected frequently
    - most objects have short lives
    - objects should be grouped according to their age
  - goal:
    - avoid repeated scan (and copy) of long-lived objects
  - approach:
    - divide heap into two sub-heaps
      - ***nursery*** and ***tenured/mature*** spaces

Garbage Collection

- Generational GC
  - *nursery*
    - holds younger objects, namely newly created objects
    - perform GC more frequently
    - objects surviving a certain number of collections
      - move from nursery into tenured space
  - *tenured / mature* space
    - contains long-lived objects
    - perform GC infrequently
      - objects here become garbage less often
  - spaces can be managed in different ways
    - nursery: smaller region, copying collectors
    - tenured space: larger region, mark and sweep collectors
      - occasionally compact
    - most efficient approach:
      - widely employed in HLL VMs, 2+ regions

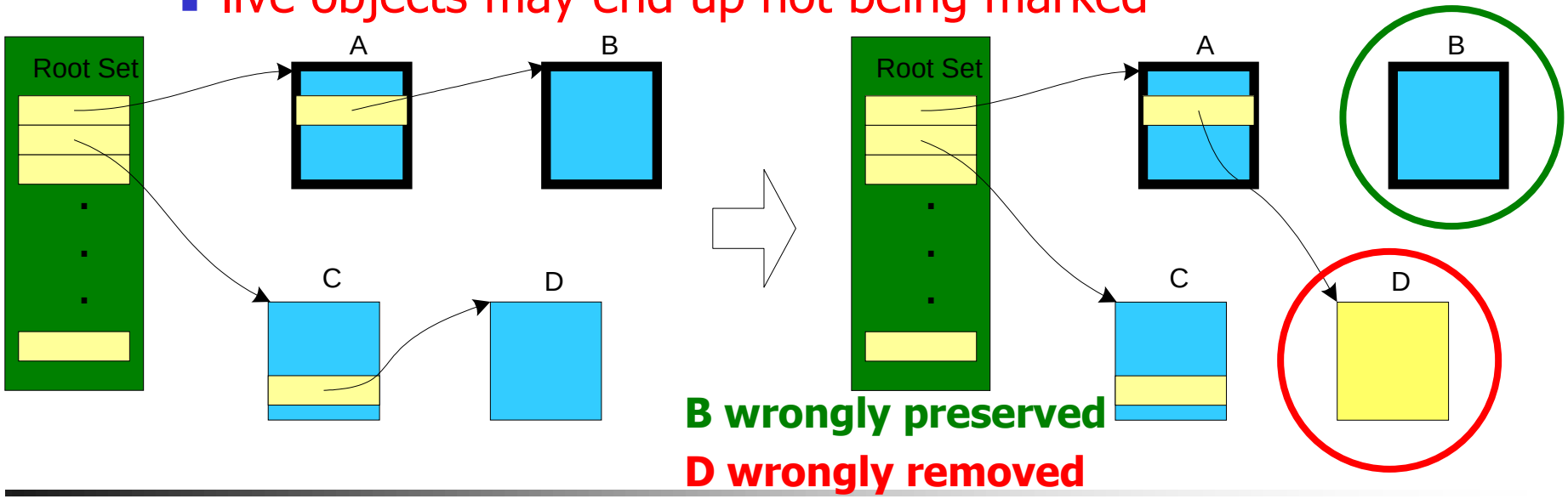


Garbage Collection

- Stop-and-collect methods
  - all algorithms described earlier
- problems:
  - program may pause for arbitrary long time
  - hard to meet real-time requirement or responsiveness
  - solution
    - incremental collectors
      - spread out collection time by doing collection incrementally
    - concurrent (parallel) collectors
      - if multiple processors are available, allocate one (some) to GC
  - key requirement
    - synchronization between application *(mutator)* and GC
      - pointers among objects may be temporarily inconsistent

Garbage Collection

- Incremental and Concurrent Mark-Sweep
  - problem with GC/application (mutator) concurrency
    - objects already marked **(bold)** may be modified by application
      - E.g., A
    - objects already marked may become garbage
    - live objects may end up not being marked



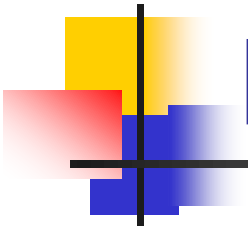
Garbage Collection

Incremental and Concurrent Mark-Sweep

- solutions: tri-color GC, read/write barriers
  - tri-color marking GC
    - white: suspected to be garbage
    - black: found to be alive
    - gray: reached by GC, but descendents not yet scanned
  - read-barrier:
    - when object not yet scanned (white) is read by *mutator*
      - immediately scan object (enclosed references could be copied elsewhere, later deleted and never considered)
      - colour it as gray
  - write-barrier: when scanned (black) object is written by *mutator*
    - trap (intercept) or record modification (save obj. address)
      - re-analyse object, e.g., put at end of scanning list
      - written object could now contain *the* only reference to some object (e.g., a new one)

Garbage Collection: Summary

Collector	Collection Time	Object Allocation Time	Access Time	Memory Efficiency
Mark and Sweep	Good	Poor **(best-fit search)**	Good	Medium **(fragmentation)**
Mark and Sweep (multi-list)	Good	Medium	Good	Medium
Compacting	Poor **(compact, patching)**	Good **(contiguous free space)**	Good	Good **(improved locality)**
Copying	Medium to Poor **(patching)**	Good **(contiguous free space)**	Good	Poor **(unused space)**



High-Performance Emulation

- High Performance Emulation
  - generic optimization framework
    - staged emulation
  - Java VM optimizations
    - bytecode optimizations
    - low-level optimizations

Optimization Framework

- Integration of Instruction Emulation into overall VM
  - basic framework to achieve high-performance emulation
    - combination of several emulation methods:
      - interpretation, binary translation and optimization
  - goal: minimize emulation overhead

- Performance Tradeoff
  - *startup time (S)*
  - *steady state performance (T)*

- Metrics:
  - *e.g., overall time for emulating an instruction N times*
  - *e.g., time to emulate first N program instructions*

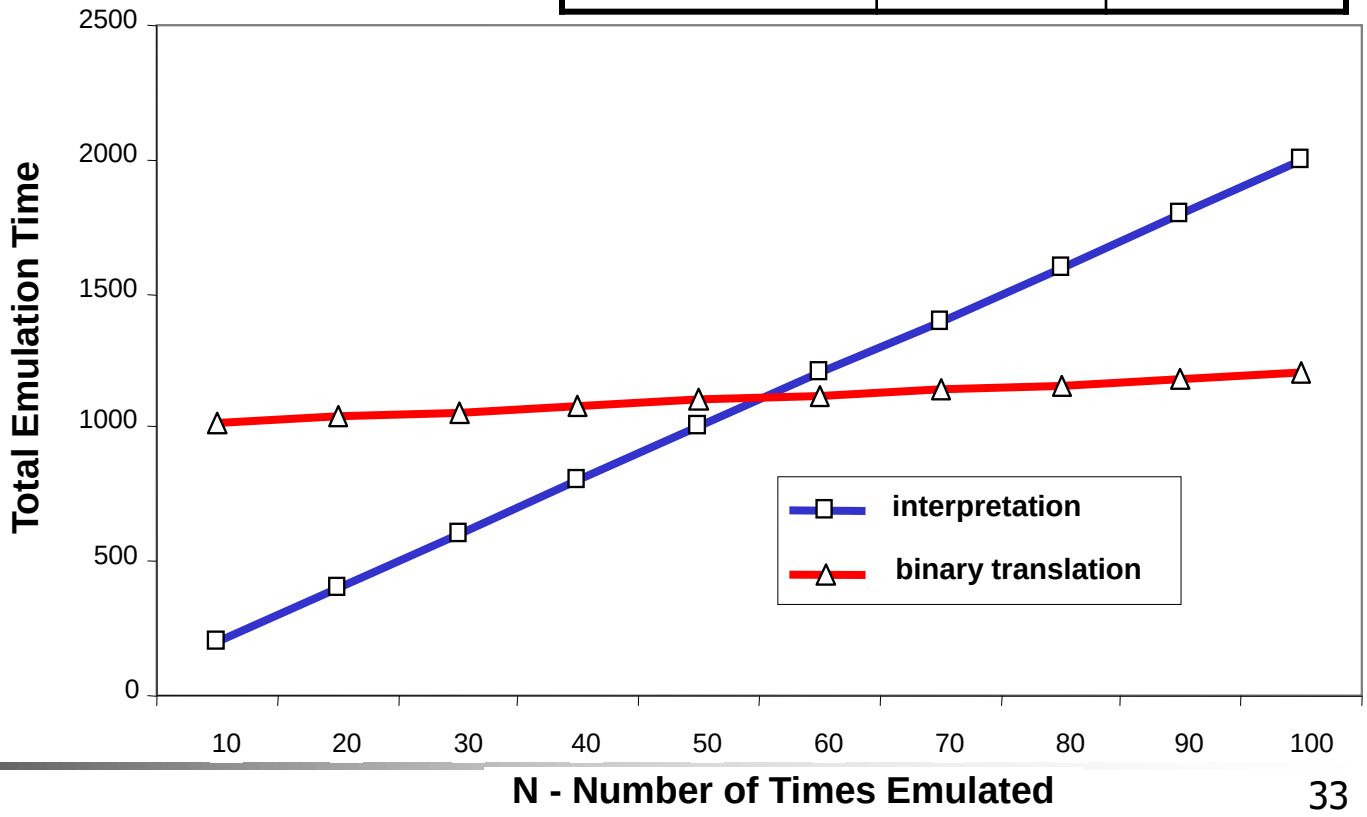
Optimization Framework

- Performance Tradeoff
  - *startup time (S)*
    - one-time cost of converting code prior to emulation
      - e.g., predecoded threaded interpretation, binary translation
  - *steady state performance (T)*
    - average rate of instruction emulation
      - continuing overhead per emulated instruction
  - Interpretation vs Binary translation
    - low startup, high steady state cost **vs.**
    - high startup, low steady state cost
- Overall time for emulating an instruction N times
  - Total = S + N x T

Optimization Framework

- Performance Trade-Off
  - if N < 55,
    - interpretation is better
  - if N > 55,
    - binary translation is better

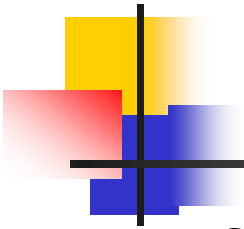
	S	T
Interpretation	0	20
Binary Translation	1000	2



Optimization Framework

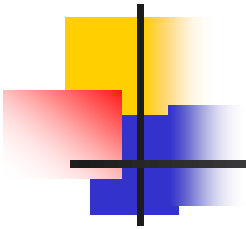
Staged Emulation

- Q: how to predict how many times instruction is emulated?
  - in advance, which method is optimal?..without *guessing...*
- A: minimize cost with available knowledge
  - start with *minimal fixed cost*
    - low startup cost → interpretation
  - gather profile data
    - e.g., number/frequency of executions
  - when *threshold reached*
    - e.g., trade-off (or break-even) reached
  - switch to aiming at *minimal marginal cost*
    - low steady-state cost
    - binary translation of code blocks
  - higher tresholds reached
    - further optimize translated block (apply more optimizations)
- E.g., Oracle Hotspot JVM, Jikes RVM (no interpretation)



High-Performance Emulation

- Performance
  - important consideration in HLL VMs (Java VM, .NET CLR,…)
- Specific Issues
  - frequent use of addressing indirection
    - in data and code
    - object references, methods, field accesses
  - frequent use of small methods
    - high relative overhead of invocation
- Challenges
  - overall, make an object-oriented program go the fastest
  - gains must compensate run-time optimization overhead



High-Performance Emulation

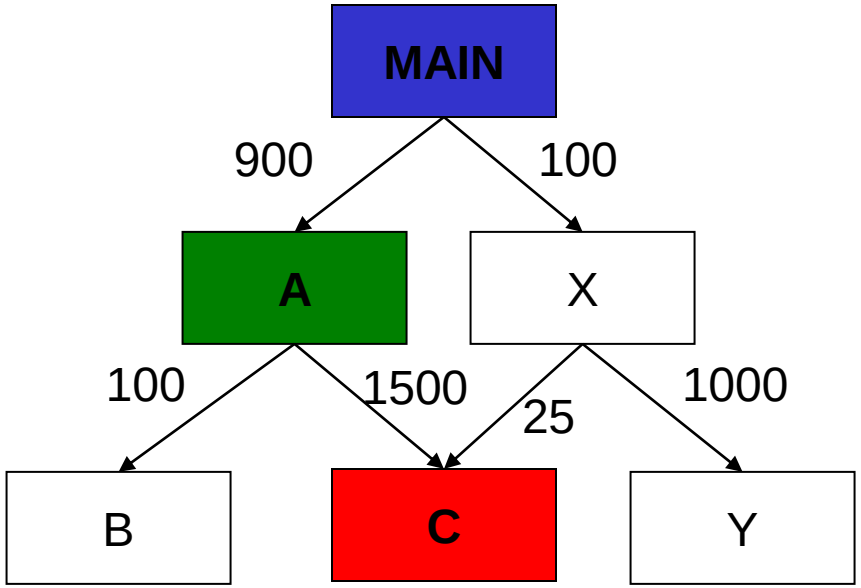
- Optimization: Method Inlining
  - replace method call with actual code in the method
  - small methods
    - calling/parameter fetch sequence larger than method code
    - very frequent in object-oriented languages
    - should always be inlined
  - larger methods
    - apply cost-benefit analysis
    - benefit based on profile data
  - performance gain
    - function of method size and call frequency
    - ideally: hottest & smallest methods

High-Performance Emulation

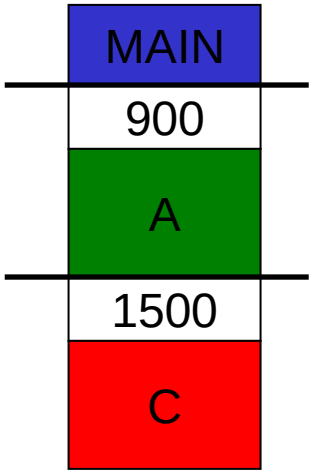
- Optimization: Method Inlining
  - typical approach: using call graph
    - instrument method calls with *<caller-callee>* pair counts
    - construct call-graph at certain intervals
      - timer or profile counter
    - invoke dynamic optimization system
      - analyze methods that should be inlined
        - trigger when call count exceeds threshold (e.g., 1500)
  - alternative approach: using stack frame
    - goal: reduce analysis overhead of call graph creation
      - decide based on localized view
    - include profile counter in stack frames
    - when threshold reached
      - *walk* backwards in stack → active portion of call graph

High-Performance Emulation

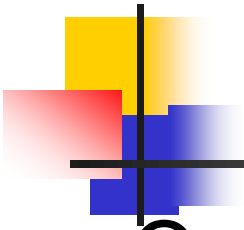
- Optimization: Method Inlining
  - e.g., using call-graph and stack frames
    - threshold: 1500 invocations



with call graph



via stack frame

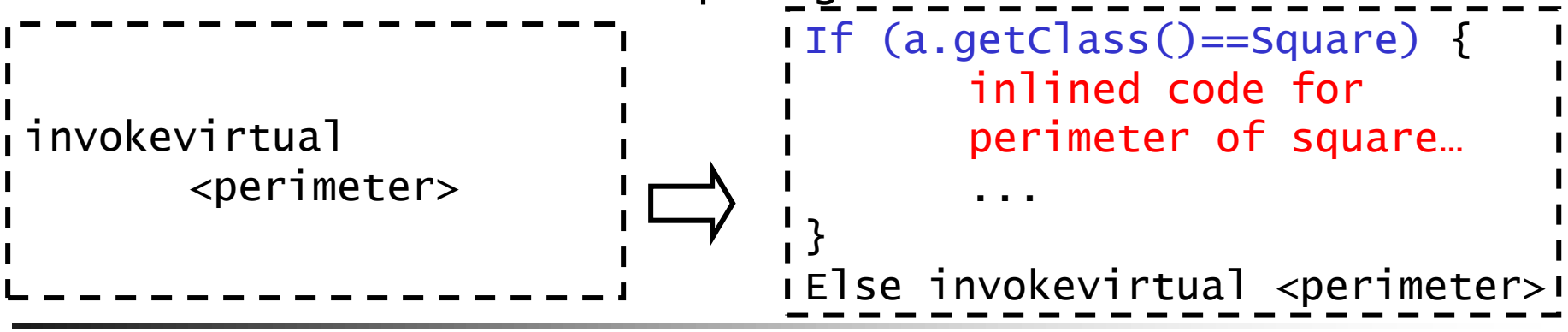


High-Performance Emulation

- Optimization: Method Inlining
  - advantages
    - relative overhead of calling sequence decreases
      - passing parameters, managing stack frame, control transfer
      - less operations and branches, improved code cache behavior
    - code analysis scope expands
      - more optimizations are then applicable to the inlined code
  - disadvantages
    - beneficial in most of cases for small methods
    - with large methods:
      - reduced benefits due lower relative size of calling sequence
      - if inlined in too many places
        - can lead to code explosion
        - poor cache behavior, performance losses
    - cost-benefit analysis needed, may be complex

High-Performance Emulation

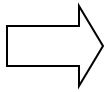
- **Optimization: Virtual Method Calls**
  - inlining OK in static methods, but most methods are *virtual*
    - target of *invokevirtual* can change dynamically (polymorphism)
    - resolved at run time via dynamic method table lookup
    - **Q: which code should be inlined?**
  - insight: often the target does not change much, or at all
  - approach: **guarded inlining** or **polymorphic inline caching**
    - inline code for most common case (guarded inlining)
    - inline code for small set of cases (PIC)
    - use method table lookup for general cases



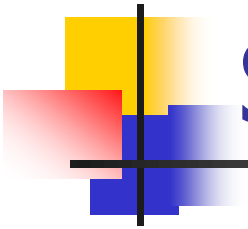
High-Performance Emulation

- Optimization: Low-Level
  - array bound checking very frequent within loops
    - repetitive overhead, possibly twofold memory accesses
    - i.e., check array metadata and compare index and length
  - often,
    - bound/range check always gives invariant result (e.g., OK)
  - optimize: hoist invariant check, guard general case
    - check code executed only once in common case

```
for (int i=0;i<j;i++) {
  sum += A[i]; <range check A>
}
```



```
if (j < A.length)
then for (int i=0;i<j;i++) {
 sum += A[i]; <no range check>
}
else for (int i=0;i<j;i++) {
 sum += A[i]; <range check A>
}
```



Summary

- HLL VM Implementation Overview
- Implementing Security
  - sandobox model, intra-process protection
  - bytecode correctness, security policy enforcement
- Garbage Collection
  - mark-and-sweep, compacting GC, copying GC
  - generational GC
  - incremental and concurrent GC
- High Performance Emulation
  - optimization framework
  - optimizations: inlining, virtual methods, short-lived objects, array checks

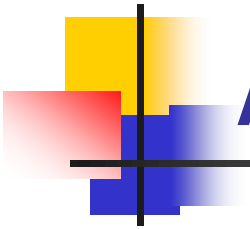


Cloud Platform-as-a-Service *(Platforms and Programming Models)*

Prof. Luís Veiga
IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

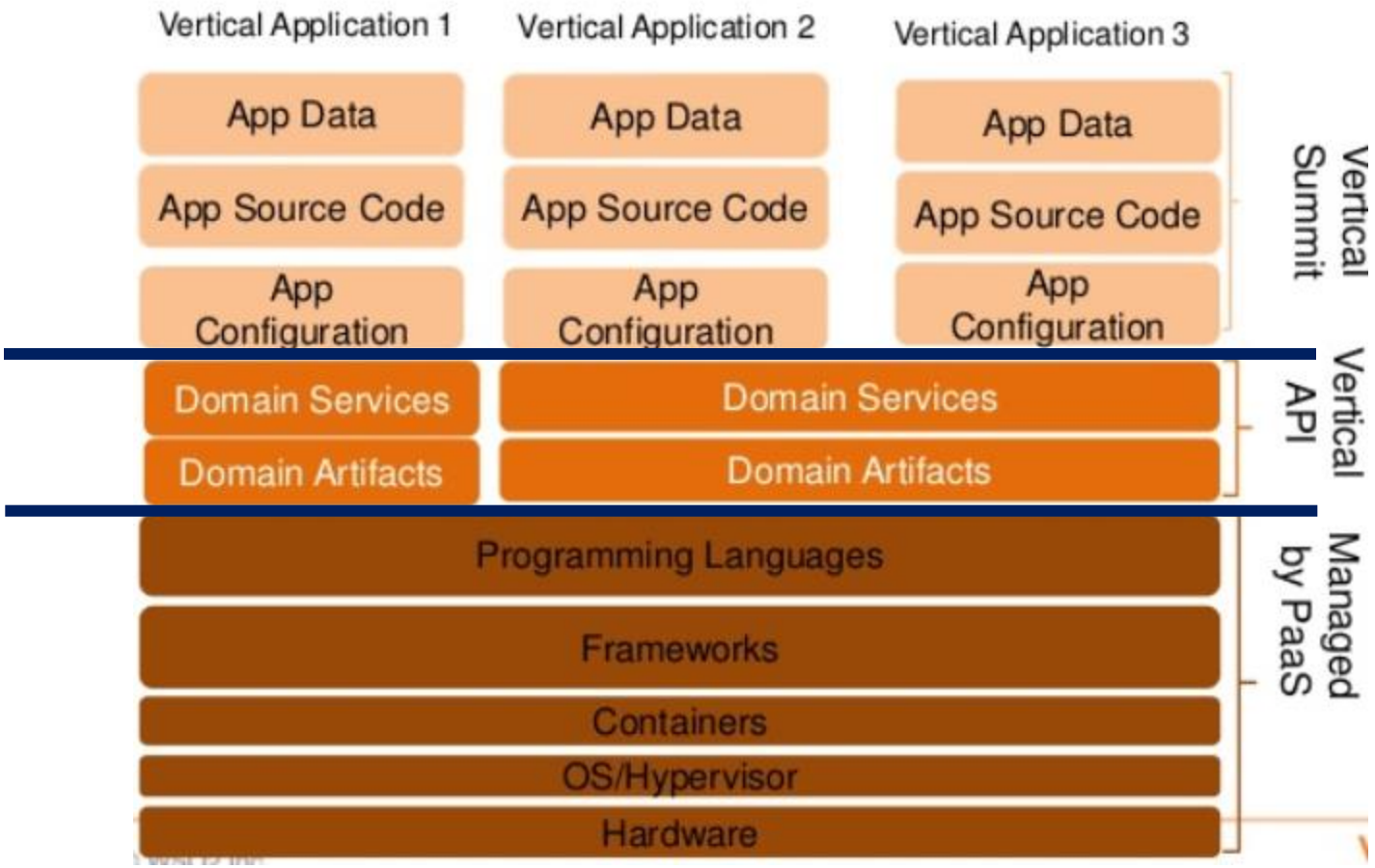
© LV, JG 2015-24

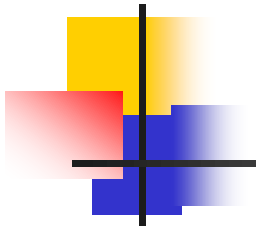


Agenda

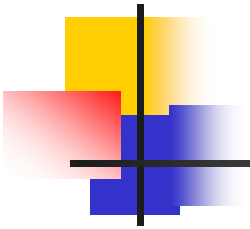
- Cloud Execution Platforms and Programming Models (Cloud PaaS)
  - Google App Engine
    - overview, platform architecture, application structure
    - GAE services: e.g. distributed caching
    - scalability
  - Microsoft Azure
    - overview, applications
    - web roles and worker roles, service model
  - Integration overview
    - message queues, messaging services
    - job management
    - Google Pub/Sub

Cloud PaaS Stack



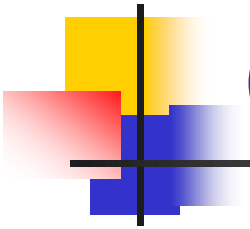


Google App Engine



Overview: Google Cloud

- Google provides a large set of services:
  - Google Search
  - Gmail
  - Google Drive
  - Google Calendar
  - Google Groups
  - Google Image Search
  - Google News
  - Google Scholar
  - Google Maps
  - Google Chat
  - Google Reader
  - Google Analytics
  - Picasa Web Albums
  - Google Docs
  - ……
- Plus many acquisitions: Youtube, Blogger, …



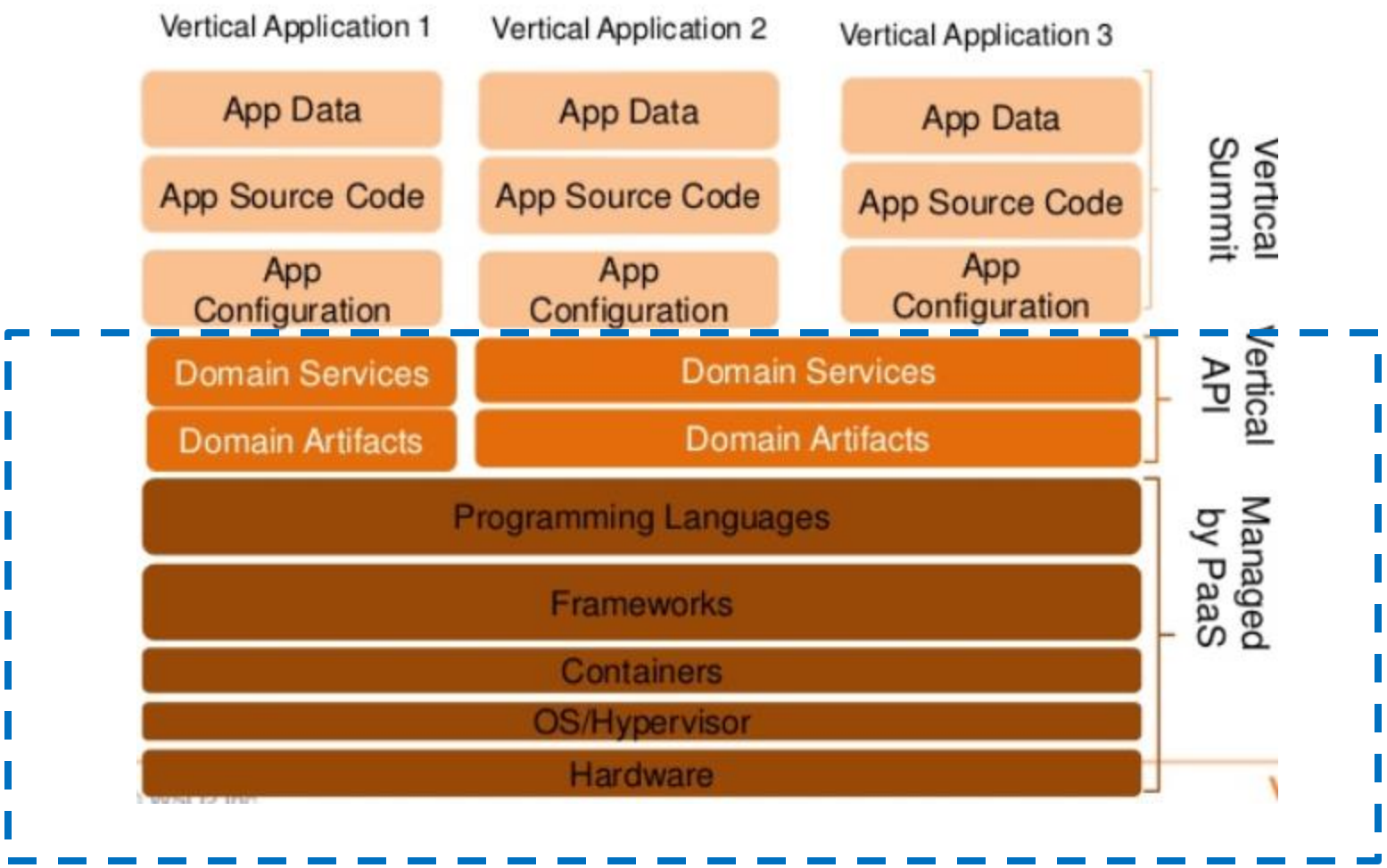
Overview: Google Infra-structure

- Most services use a set of common infra-structural cloud services:
  - GFS: Google File System
  - BigTable
  - MapReduce
- These services are not openly available.
  - partly justify Apache open-source clones:
    - HDFS, HBase, Hadoop M/R.

Overview: Google App Engine

- Google's cloud app programming framework/platform.
  - all apps web-based, initially written in Java or Python.
    - Today: Java, Python, Node.js, Ruby, PHP, Go
  - Google App Engine mostly processes HTTP(S) requests.
- Application configuration is quite simple:
  - With limited access to performance tuning/configuration.
- The framework is designed primarily for scalability:
  - **Goal**: to reduce interactions so as not to limit number of apps, requests per second, storage space…
  - The APIs are as simple as possible.
- The cloud infra-structure (H/W and systems level) is almost completely hidden from the programmer and the users.

Overview: PaaS Stack



Overview: GAE Control assumptions



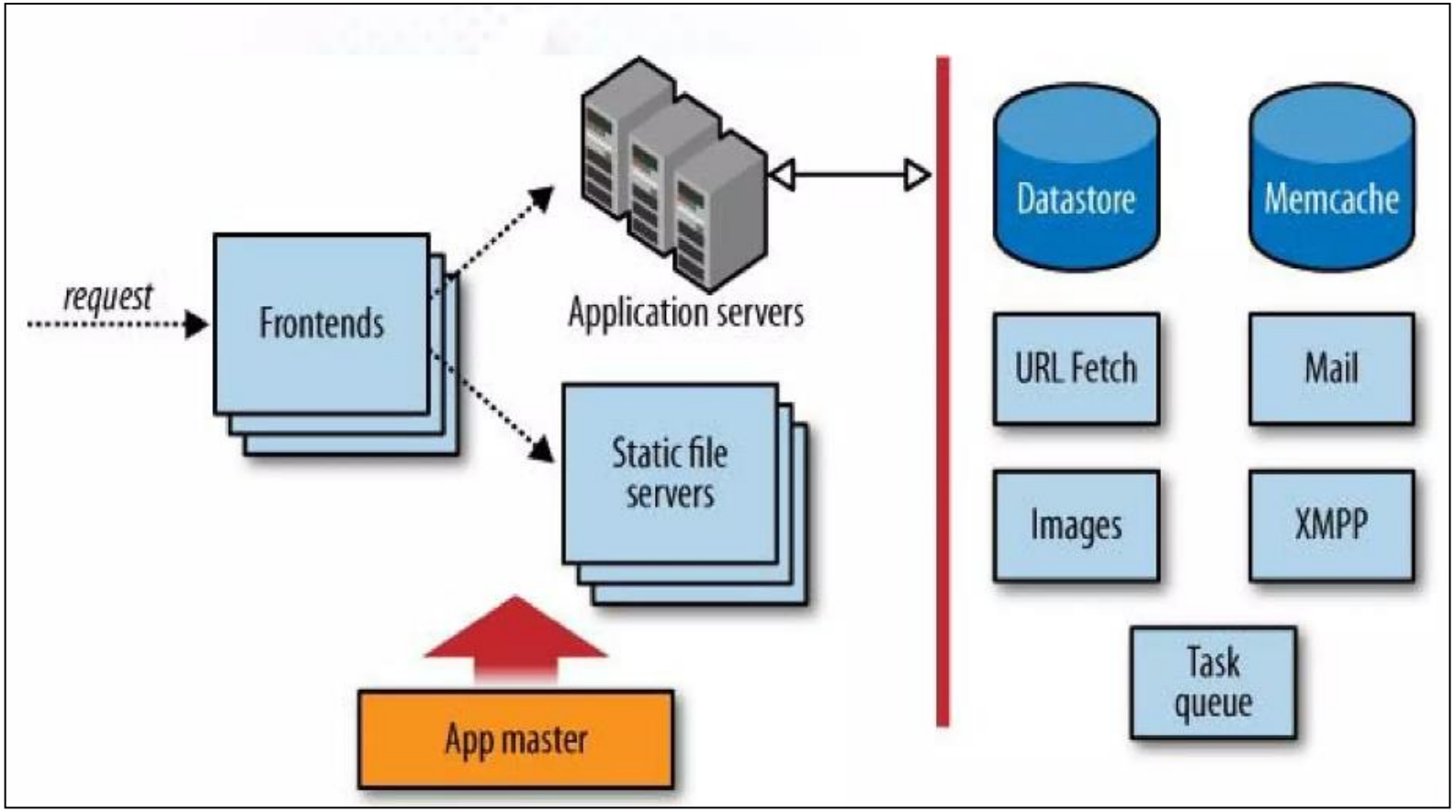
- HW configuration
- Operating system
- Language (Java,Python,..)
- Middleware stack (data-base, messaging,…)
- Architecture
- Performance (Quota : CPU, Mem, Net,..)
- Data center location (Region)

You



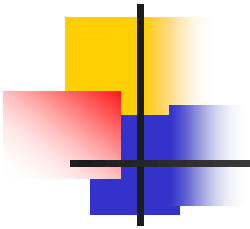
- Application code
- Application configuration
- Select the middleware stack:
  - from a predefined list.

Google App Engine Architecture



GAE Architecture: Front-end

- Identifies the application based on the URL.
  - common approach employed in web apps (*endpoints)*
- Separates requests for static and dynamic content.
- May redirect request to Google's authentication page.
- Applications are configured using YAML files:
  - **app.yaml:** index of the site: servlets, jsp, authentication,…
    - web app config: sessions, services, resource scaling policy,…
- Additional configuration:
  - **cron.yaml:** issue of periodic tasks/requests for repeating tasks
  - **dispatch.yaml:** allows override request routing rules
    - e.g. routing URLs with regexp to services



GAE Architecture: Static Content

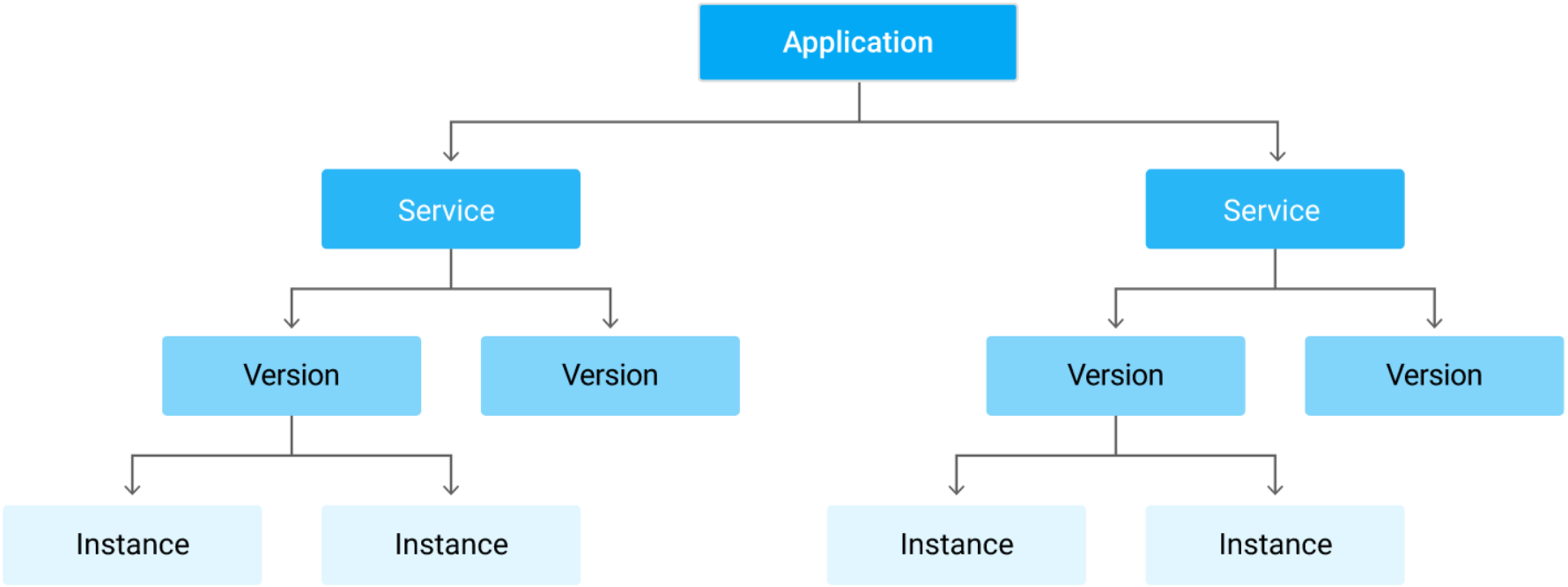
- GAE Static File Servers:
  - Serve the static content of the websites.
  - By default:
    - everything outside the application's *war* file WEB-INF folder (next).
  - The file extensions that are static/dynamic are configurable.

GAE Architecture: Back-end

- Runs work in the background.
- Accessible web addresses, e.g.:
  - http://<instance>.<serviceID>.<appID>.appspot.com
  - https://<versionID>.<serviceID>.<projectID>.<regionID>. r.appspot.com
- Different classes of machines, for:
  - backends/modules/services, versions, instances
- Can be: **dynamic** (request-driven) or
  - **resident** (continuous execution, manual start)
- Configurable: using console or command line tool.

GAE Architecture: Back-end

- Different classes of machines, for:
  - backends/modules/services, versions, instances



GAE Arch.: Execution Environment

- GAE standard environment:
  - Applications execute within a (strict) sandbox.
- By default, it is **not** possible to:
  - Process a request for more than 60 seconds:
    - this is a problem: Accessing the Datastore may be slow.
  - Write to files (except /tmp).
  - Open sockets.
  - Create threads or processes.
  - Make system calls.
  - ….

Google App Engine Services

- Apps can call GAE's APIs for:
  - Blobstore: key-value storage for large objects.
  - DataStore: Datastore access.
  - Images: perform image manipulation.
  - Memcache: memory cache management.
  - Mail: sending emails.
  - URL Fetch: access other web apps.
  - Integration with Google's account/login system.
  - Exchange messages via queues
  - ……

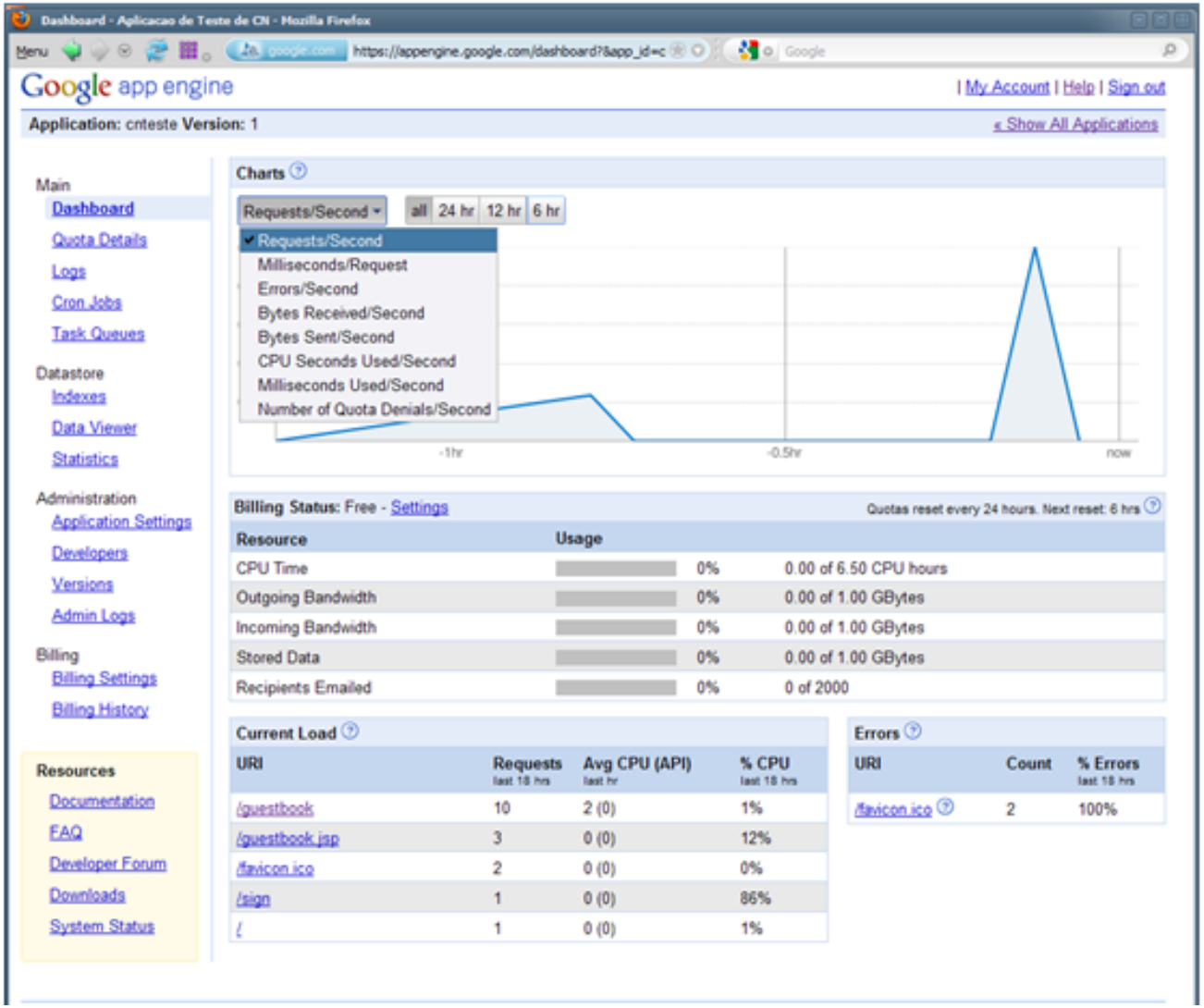
Services: Memory Cache (memcache)

- Distributed memory cache (memcache) – key/value store (KVS)
  - allows faster access to stored data,
  - from any application instance.
- Data are stored as <key,value> pair:

```
MemcacheService memcache =
   MemcacheServiceFactory.getMemcacheService();
memcache.put("headlines", headlines);
headlines = (List<String>) memcache.get("headlines");
memcache.delete("headlines");
```

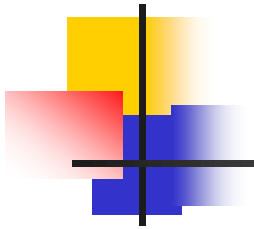
- A data item can be an instance of any serializable class.
- The memcache is not stable or persistent
  - Memcache entries can be eliminated by the runtime.
- Memcache can be configured to store data for limited time.
- Each entry can have up to 1 MB.

GAE Administration Console



GAE: Scalability

- Scalability is mainly achieved at three levels.
  - 1. Number of frontend and application servers is
    - scaled automatically by Google App Engine.
  - 2. Web apps are kept in memory.
  - 3. Web requests are sent to the server(s)
    - where the application is already running.



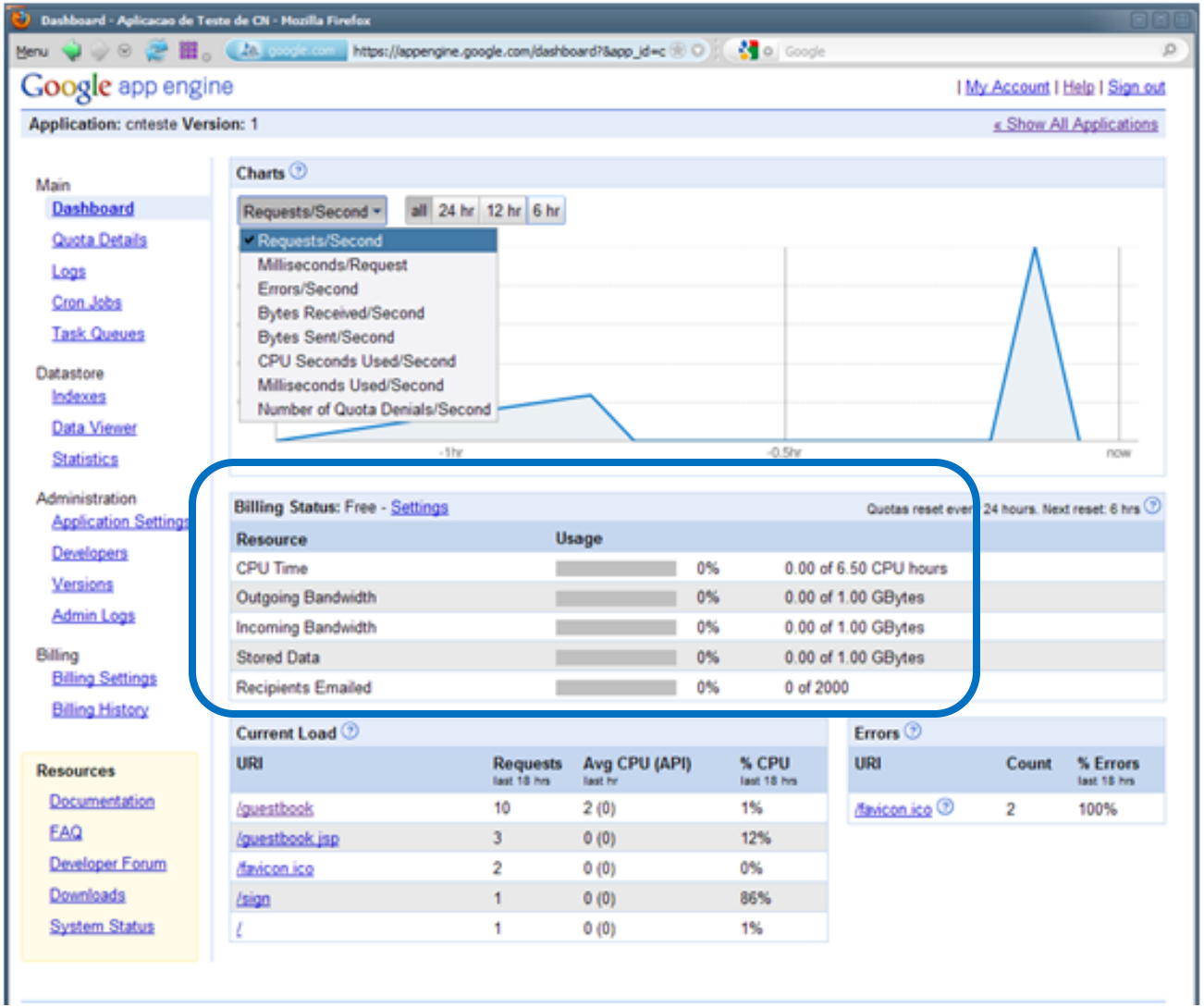
GAE: Scalability

- Scalability Management:
  - Applications are replaced in servers:
    - using a LRU (*least recently used*) policy.
  - Static variables must not contain application state.
    - only configurations, temporary values, etc.
  - Storage scales automatically:
    - based on Datastore/BigTable/Google File System.

Google App Engine SLAs

- Main features:
  - Payment is independent from the number of servers (automatic decision up to GAE).
  - User is charged on *real* usage:
    - CPU usage time,
    - data bandwidth,
    - data storage, and
    - number of API calls (e.g., mail, pubsub)

Google App Payment e.g



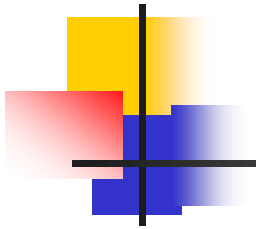
GAE: Application-Defined Scaling

- Application-defined Scaling Policies:
  - Automatic scaling in app.yaml config
- Key Parameters for each instance running:
  - target-cpu-utilization (0.5 – 0.95)
    - target CPU load meet spawn new instance.
  - max-concurrent-requests (def.:10, max:80)
    - max simultaneous load before spawning new instance.
  - min/max-pending-latency (0.01-15)
    - minimum and maximum seconds for requests waiting in the scheduler queue before spawning new instance.

GAE: Application-Defined Scaling

- Key Parameters for each instance running:
  - min/max-instances
    - minimum/maximum number of instances running for each service.
    - size of your cluster.
  - min/max-idle-instances
    - minimum and maximum number of instances to be kept running even while idle.
    - determines responsivity of the cluster to spikes in request load and how to release unused resources.

```
<automatic-scaling>
  <target-cpu-utilization>0.65</target-cpu-utilization>
  <min-instances>5</min-instances>
  <max-instances>100</max-instances>
  <max-concurrent-requests>50</max-concurrent-requests>
</automatic-scaling>
```

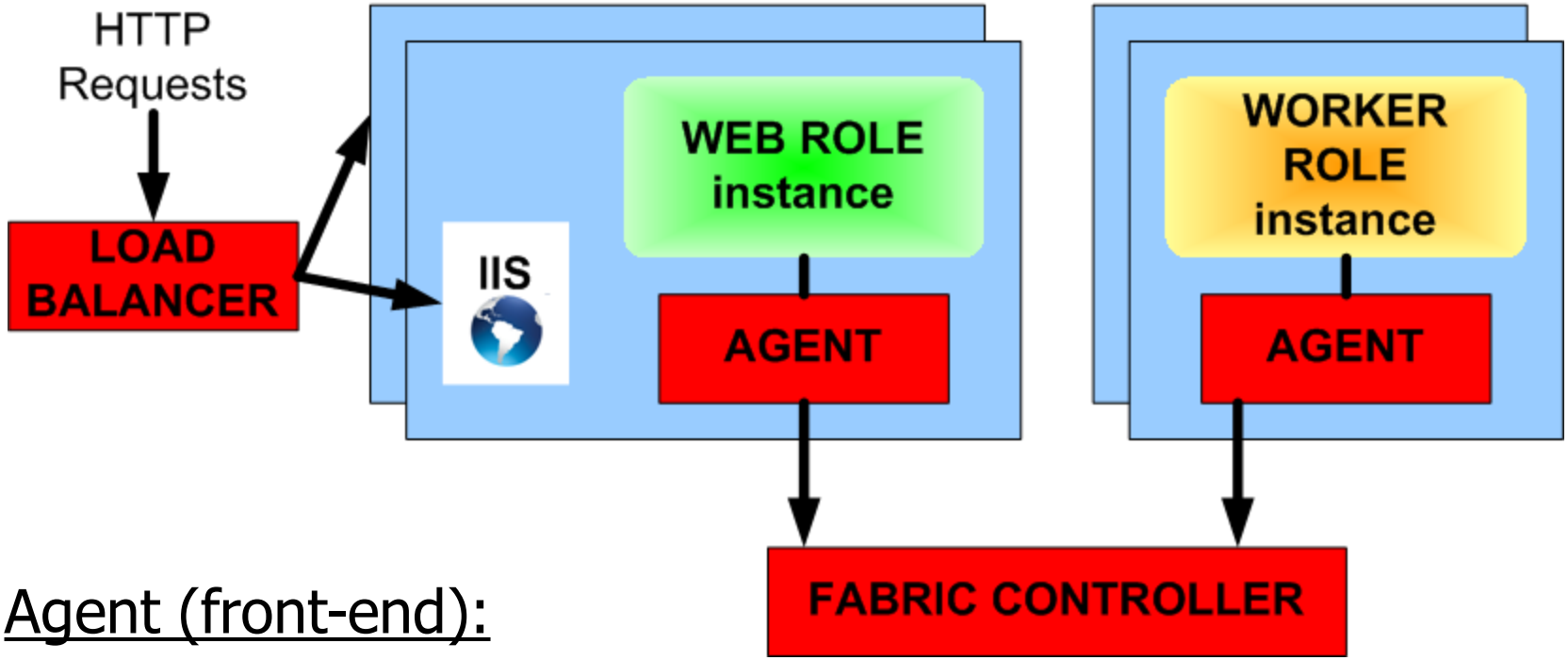


Azure Programming

Azure Applications

- A Microsoft Azure application
  - is made of Web Roles and Worker Roles:
- **Web Role:** Web Service running in IIS on Windows VM.
  - An .**aspx** page deployed on a user-chosen **URL**,
  - e.g. `myApp.cloudapp.net`, and associated to a virtual IP address (VIP).
- **Worker Role:** Services without outside interface
  - running on a Windows VM.
  - C# class derived from `RoleEntryPoint`.
- Configured by a **service model** (example later)

Architecture: Agents and Fabric



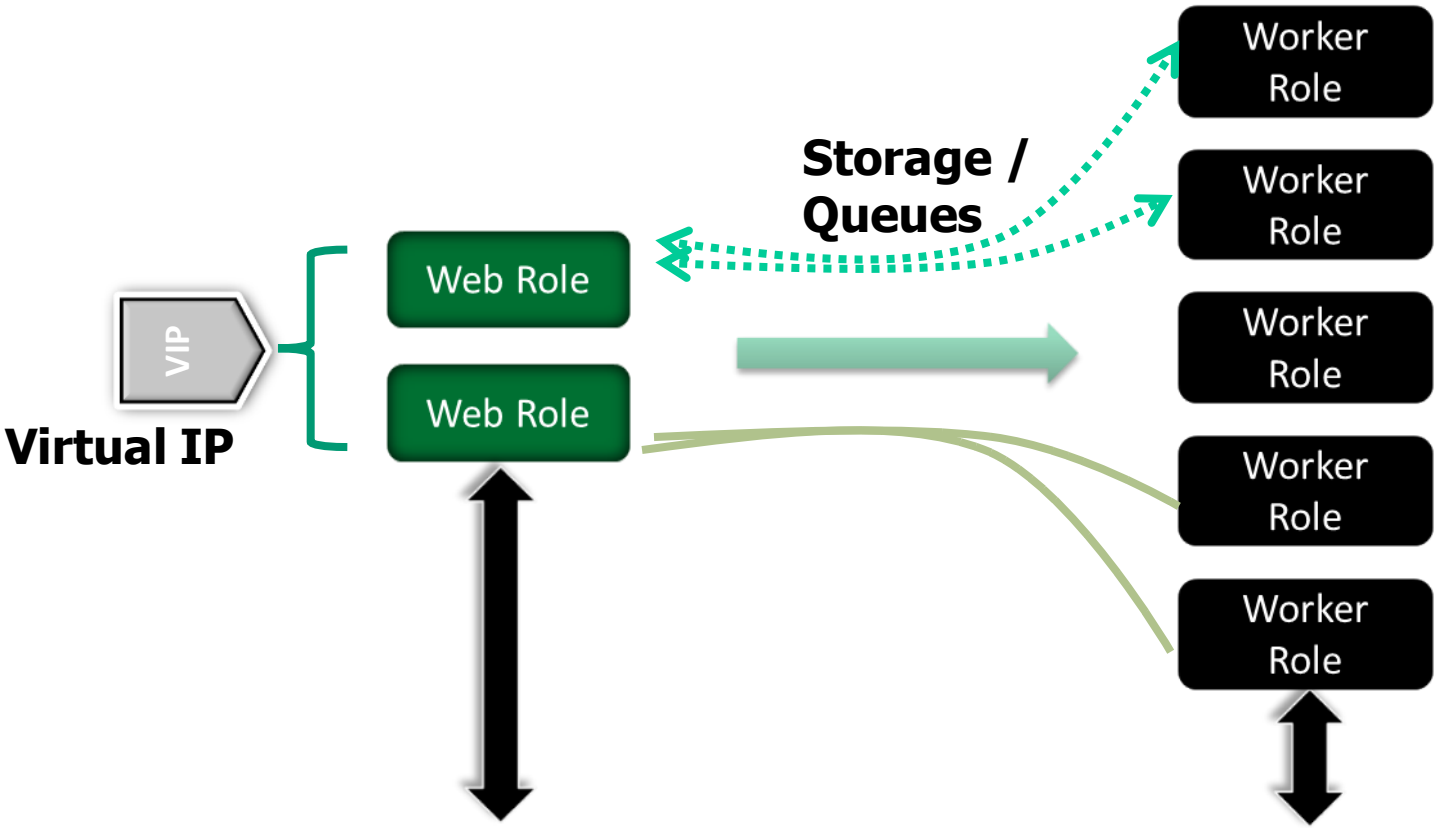
Agent (front-end):

- Exposes the API
- Monitors the failure conditions of the application

Fabric (back-end):

- Allocates resources according to configuration file
- Detects and restarts failed web roles and workers

Service Model: Web Role + Worker Role



Web Role (1): aspx web page

```
<%@ page language="C#" CodeFile="CodeBehind_cs.aspx.cs"
    Inherits="CodeBehind_cs_aspx" %>
    <html>
     <head><title>ASP.NET CodeBehind Pages</title></head>
     <body>
    <form runat="server"> <h1>Welcome to ASP.NET 2.0!</h1> <b>Enter Your
    Name:</b> <asp:TextBox ID="TextBox1" Runat="server"/> <asp:Button
    ID="Button1" Text="Click Me" OnClick="Button1_Click"
    Runat="server"/> <asp:Label ID="Label1" Text="Hello“
    Runat="server"/></form>
     </body>

    </html>
```

Web Role (2): C# Code Behind

```
using System;

public partial class CodeBehind_cs_aspx : System.Web.UI.Page {
    protected void Button1_Click(object sender, EventArgs e) {
      Label1.Text = "Hello " + TextBox1.Text;
      }
    }
```

Worker Role Instances (1)

- Communication with *web roles*
  - Typically done using Storage or Queues.
- Programming Model:
  - **`OnStart`** method:
    - used for application initialization.
    - called before the application is running.
    - run by the load balancer.
  - **`OnStop`** method:
    - called when an application is stopped.
  - **`Run`** method:
    - contains the *worker role*'s service logic.

Worker Role (2): Example Code

```
public class MyWorkerRole : RoleEntryPoint    {
      public override bool OnStart()       {
         DiagnosticMonitor.Start("DiagnosticsConnectionString");
         return base.OnStart();
      }

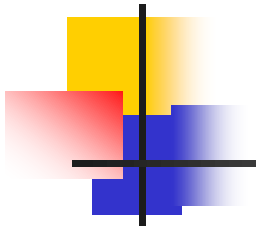
 public override void Run()       {
         var dayTimeServer = new
    DayTimeTcpServer(RoleEnvironment.CurrentRoleInstance.InstanceEndpoin
    ts["DayTime"].IPEndpoint);
         dayTimeServer.Start();
         int count = 0;
         for ( ; ; )          {
            count ++;
            Trace.WriteLine(String.Format("Message {0}", count));
            Thread.Sleep(TimeSpan.FromSeconds(10));
         } } } }
```

Worker Role Instances (3)

- Resource-Life/cycle Management:
  - All *worker roles* have to start a thread
    - that does not finish in order for the *worker role* not to be terminated.
  - If the **`Run`** method returns,
    - the application instance is recycled.
  - **`Run`** method of the **`RolePointEntry`** base class
    - starts thread that performs an infinite loop.
- Deployment:
  - Once worker created in Azure administration portal:
    - It is installed via Visual Studio or command line tool.

Service Model Description

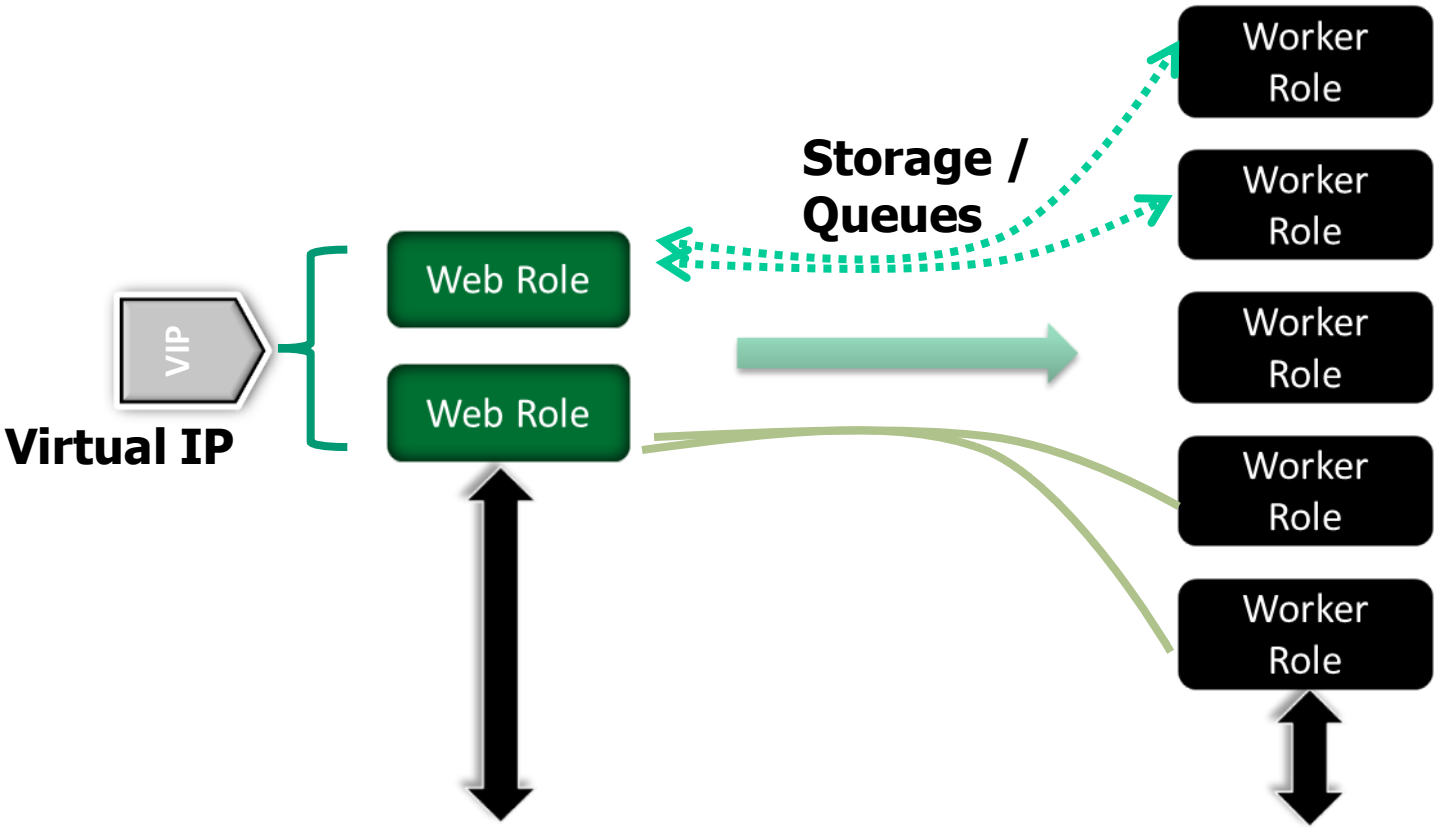
- The service model describes the app's characteristics:
  - *(similar purpose to .yaml configuration files in GAE)*
  - Which roles and how many.
  - Communication endpoints and their protocols.
  - Which interfaces are exposed.
  - Number of *fault domains*:
    - in same data center, on how many racks will the app run.
  - Number of *update domains*:
    - if app is updated, how many instances updated at once.
- Described in an XML files .csdef and .cscfg
  - derived mostly from Visual Studio configurations.



Service Integration

PaaS App Component/Service Integration

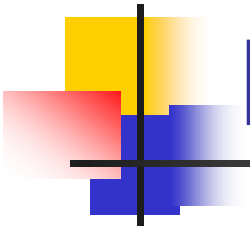
- Recall typical Azure app setting.



PaaS App Component/Service Integration

- Approach: Messaging Services

- Why?
  - Participants can be weakly connected:
    - in time, space, technology, e.g.
      - No network connection.
      - No simultaneous execution.
      - No binary compatibility.
      - No direct addressing (knowledge,ID,address) among senders and receivers

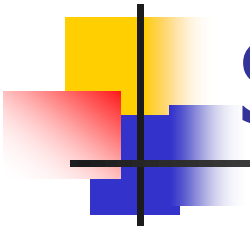


Messaging Services

- Very useful for:
  - connecting heterogeneous/legacy systems.
  - workflow systems.
    - processes can be manipulated by adding/removing/replicating messages.
- E.g.,
  - SQS: Amazon Simple Queue Service
  - Microsoft Azure Queues
  - Google Pub/Sub

SQS: Amazon Simple Queue Service

- Amazon Simple Queue Service
- Message delivery triggered by receiver.
  - no delivery times are guaranteed.
- Fault Tolerance and Scalability
  - message queues are replicated
  - when messages read:
    - they become invisible for a period
    - to avoid frequent double processing
    - need explicit deletion after consumption
    - otherwise, made visible again for reading
    - (to ensure liveness/completeness and promote efficiency)



SQS: Programming Guidelines

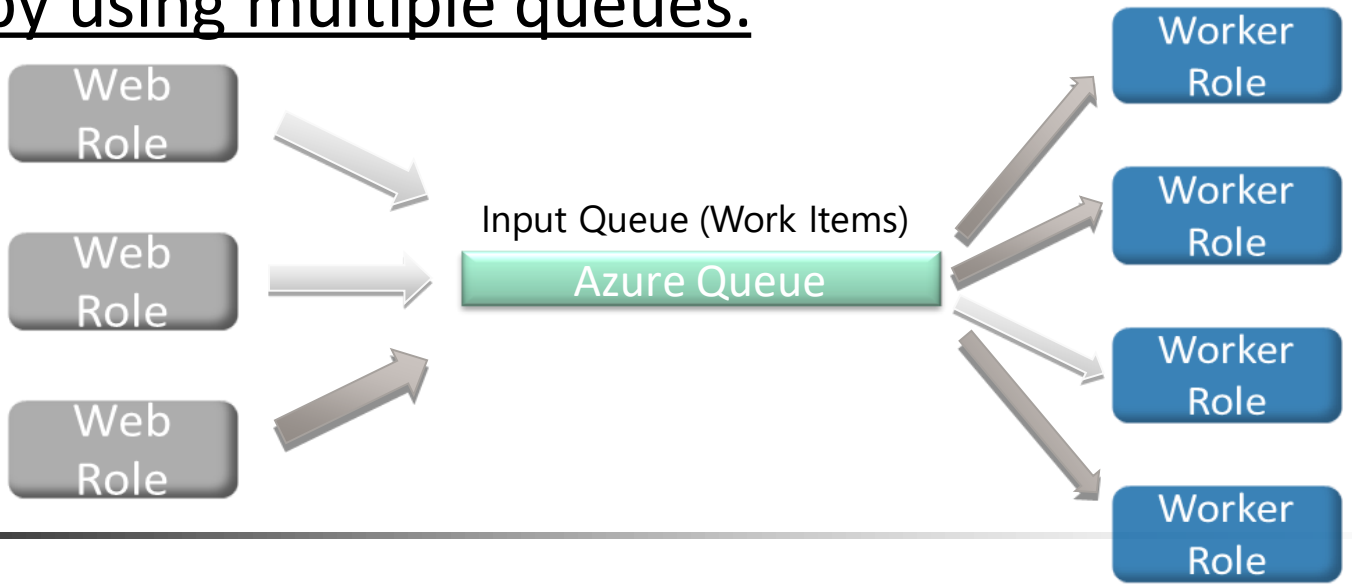
- Limited message size:
  - 8KB/64KB/256KB
- Use a idempotent message protocol:
  - i.e. don't design operations that assume a particular application state, e.g.:
    - Choose `SetValue(v,i)` and not `IncrementValue(v,i)`.
    - Choose `NewPosition(x,y)` and not `MoveForward()`.
  - (to ensure correctness – no erroneous duplicate actions)
- Don't use message queues for applications with timing restrictions,
  - e.g. transactional systems.

Microsoft Azure Queues

- Reliable and persistent messaging service.
- Unlimited:
  - queues per account and
  - messages per queue.
- Limited size per message (8KB/64KB)
- Fault tolerance mechanism (as in SQS):
  - when messages are read
    - they become invisible for a period
    - to avoid frequent double processing
    - need explicit deletion after consumption
    - otherwise, made visible again for reading
    - (recall liveness/completeness and efficiency)

Web+Worker Integration using Queues

- Typical situation:
  - **Web Role**: places messages in the queue.
  - **Worker Role**: blocks on the queue waiting.
- Queues can be scaled:
  - by offloading data to *blobs*
  - by using multiple queues.



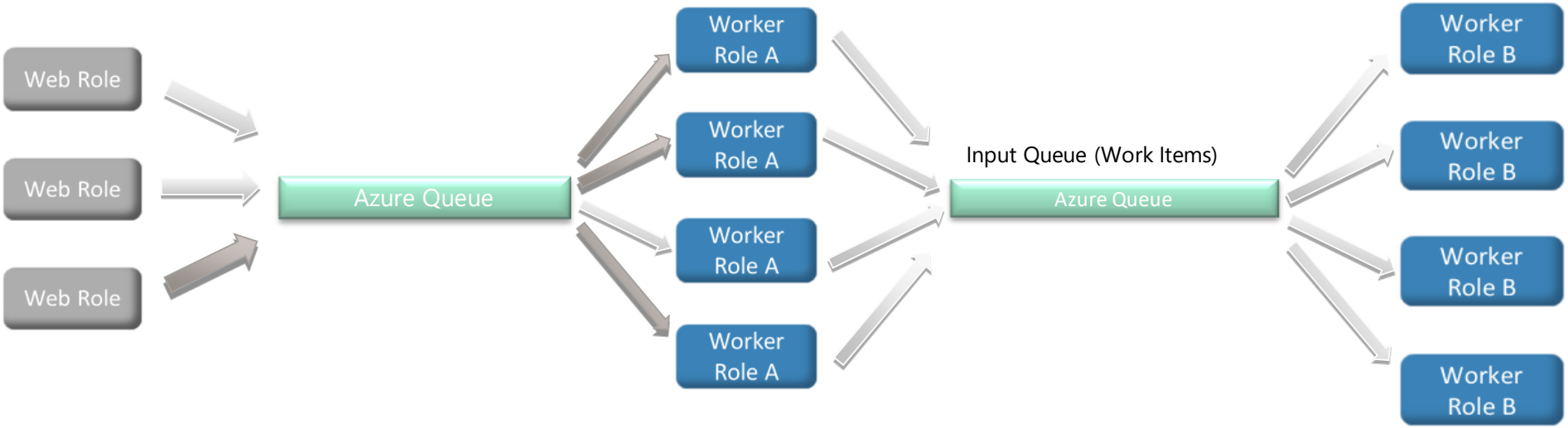
Web Role Writing to Queue

```
// Retrieve storage account from connection string.
CloudStorageAccount storageAccount =
    CloudStorageAccount.Parse(CloudConfigurationManager.GetSetting(
    "StorageConnectionString"));
// Create the queue client.
CloudQueueClient queueClient =
    storageAccount.CreateCloudQueueClient();
// Retrieve a reference to a queue.
CloudQueue queue = queueClient.GetQueueReference("myqueue");
// Create the queue if it doesn't already exist.
queue.CreateIfNotExists();
// Create a message and add it to the queue.
CloudQueueMessage message = new CloudQueueMessage("Hello,
    World");
queue.AddMessage(message);
```

Worker Role Reading from Queue

```
// Retrieve storage account from connection string
CloudStorageAccount storageAccount =
   CloudStorageAccount.Parse(CloudConfigurationManager.GetSett
   ing("StorageConnectionString"));
// Create the queue client
CloudQueueClient queueClient =
   storageAccount.CreateCloudQueueClient();
// Retrieve a reference to a queue
CloudQueue queue = queueClient.GetQueueReference("myqueue");
// Get the next message
CloudQueueMessage retrievedMessage = queue.GetMessage();
//Process msg in >30 seconds, and then delete it
   queue.DeleteMessage(retrievedMessage);
```

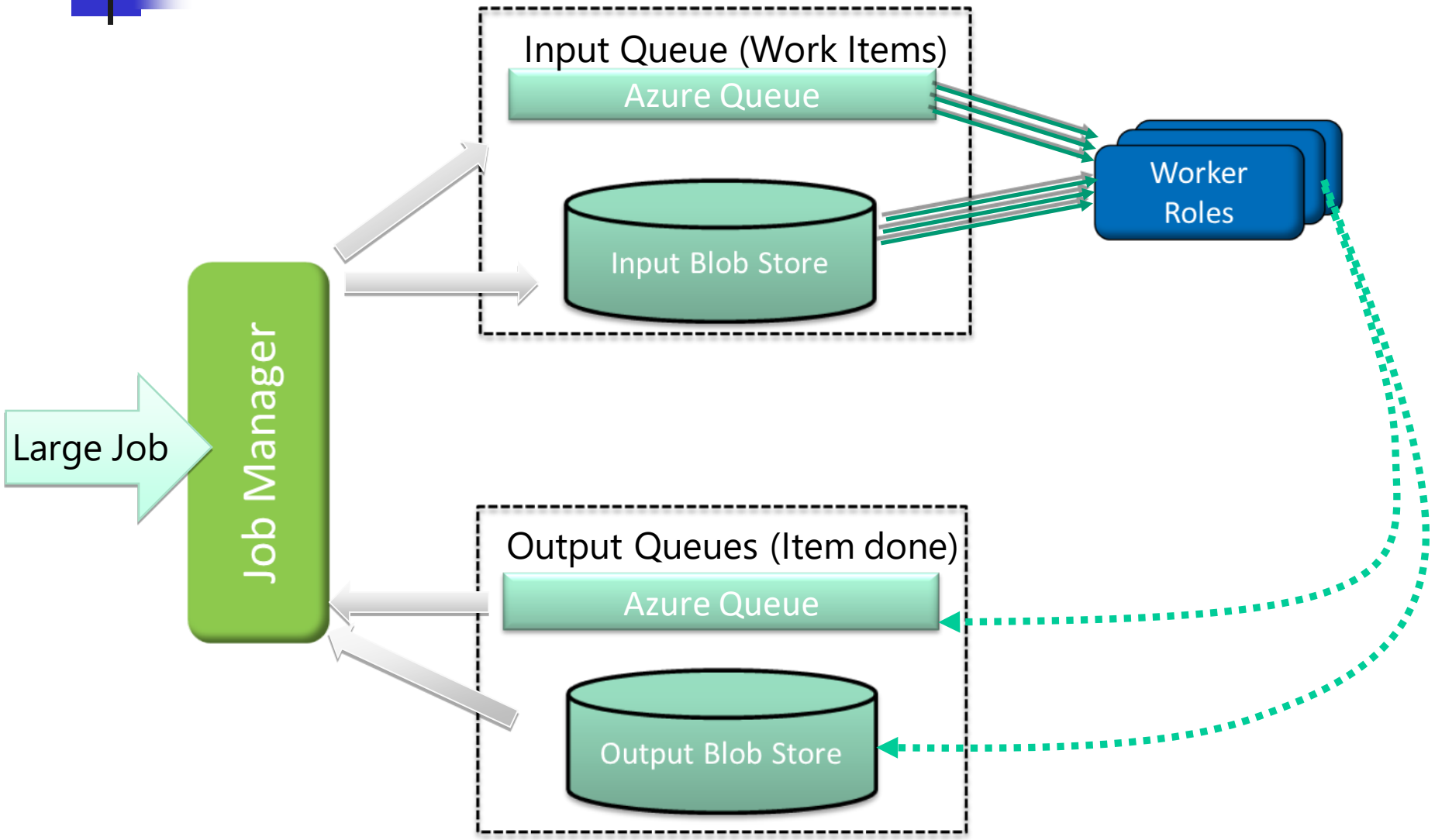
Web+Worker Integration Example



Worker Role for Managing the Workflow: managing instance number and queues

Job Manager

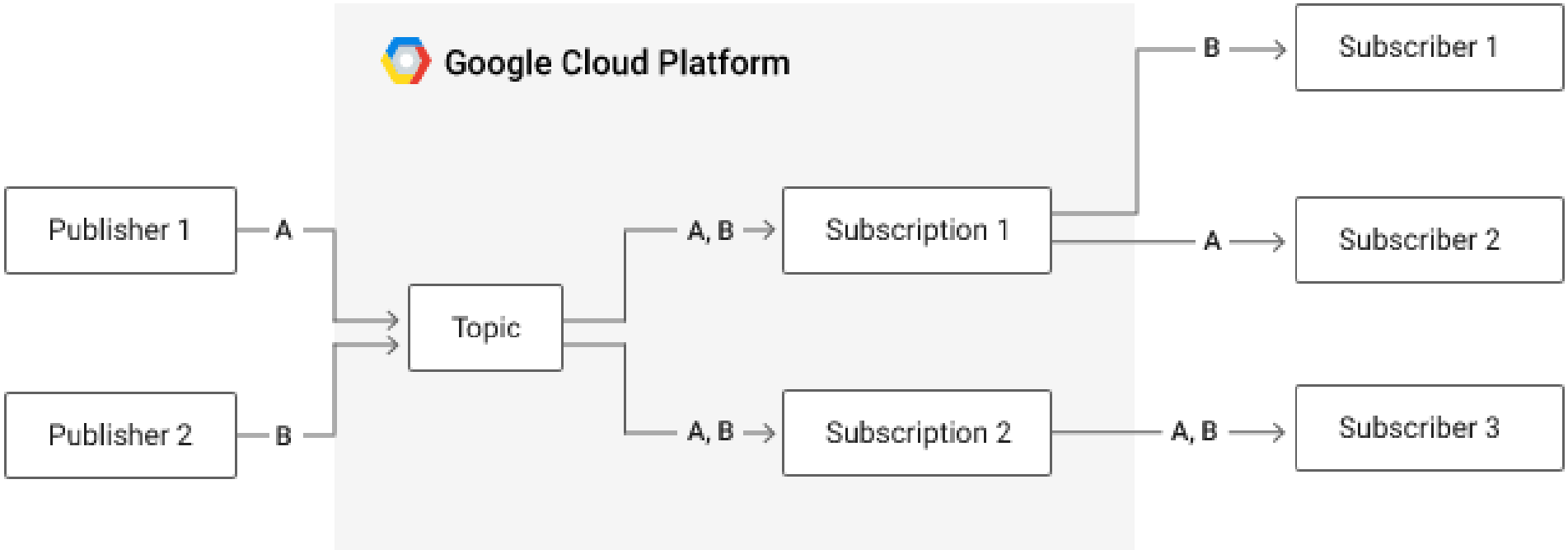
Example: Job Manager



Google Pub/Sub

- Google Messaging service
  - allows services to communicate asynchronously
  - latencies on the order of 100 milliseconds
- E.g. usage:
  - streaming analytics
  - data integration pipelines (ingestion, distribution)
- Multiple function:
  - messaging-oriented middleware (service integration)
  - queue to parallelize tasks
  - information dissemination with several patterns
- Pub/Sub guarantees:
  - synchronous replication of all data to at least two zones
  - best-effort replication to a third additional zone

Google Pub/Sub: Architecture



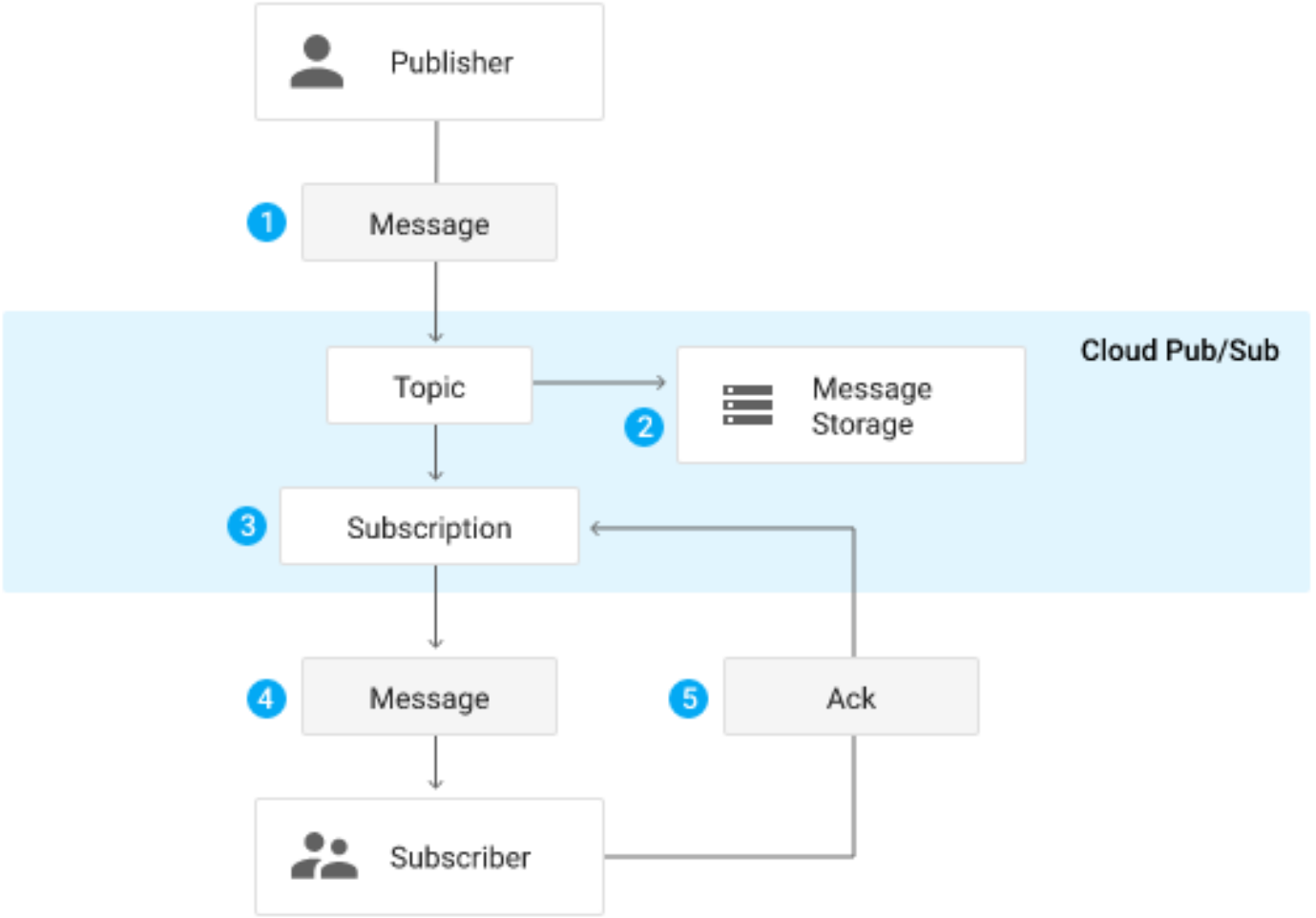
Google Pub/Sub: Core Concepts

- Topic:
  - named resource to which messages are sent.
- Subscription:
  - named resource representing the stream of messages from a single, specific topic, to be delivered to the subscribing application.
- Message:
  - data and (optional) attributes that a publisher sends to a topic and is eventually delivered to subscribers.
- Message attribute.
  - key-value pair that publisher can define for a message.

Google Pub/Sub: Core Concepts

- Publisher:
  - app that creates/sends messages to topic(s).
- Subscriber:
  - app with subscription to topic(s) to receive messages from it.
- Acknowledgement (or "ack"):
  - signal sent by a subscriber to Pub/Sub after it has received a message successfully.
  - *Acked* messages removed from subscription's queue.
- Message delivery methods:
  - subscriber receives messages either by Pub/Sub **pushing** them to the subscriber's chosen endpoint/receiver
  - or by the subscriber **pulling** them from the service.

Google Pub/Sub: Message Life-cycle



Google Pub/Sub: e.g. Create Topic

```
public static void createTopicExample(String projectId,
   String topicId) throws IOException {
…
    TopicAdminClient topicAdminClient =
                    TopicAdminClient.create();
    TopicName topicName = TopicName.of(projectId, topicId);
    Topic topic = topicAdminClient.createTopic(topicName);

    System.out.println("Created topic: " + topic.getName());
..
}
```

Google Pub/Sub: e.g. Publishing

```
…
String message = "Hello World!";
ByteString data = ByteString.copyFromUtf8(message);

Publisher publisher = Publisher.newBuilder(topicName).build();

PubsubMessage pubsubMessage =
   PubsubMessage.newBuilder().setData(data).build();

// Once published, returns a server-assigned message id (unique
   within the topic)
ApiFuture<String> messageIdFuture =
   publisher.publish(pubsubMessage);

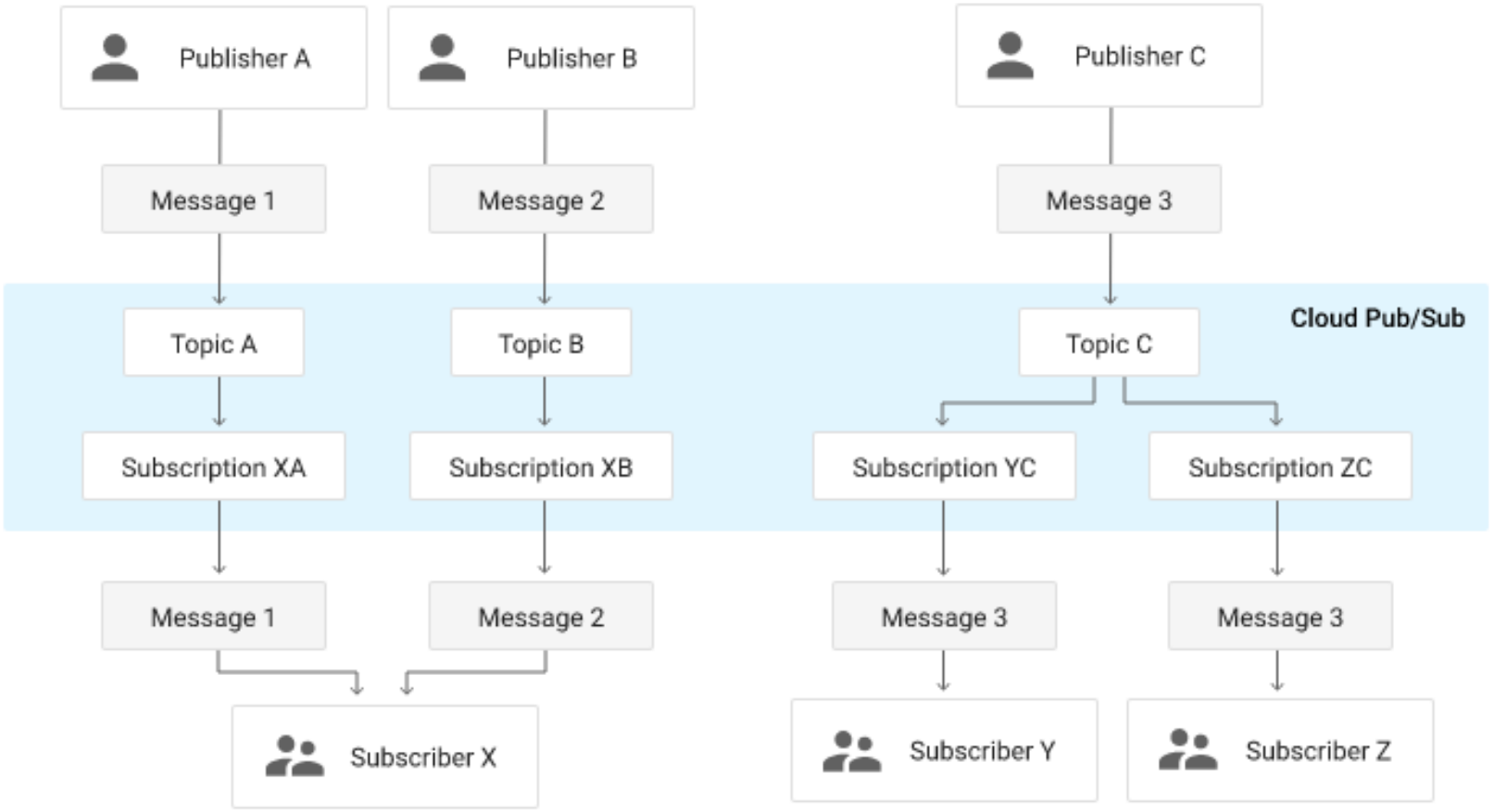
String messageId = messageIdFuture.get();
….
```

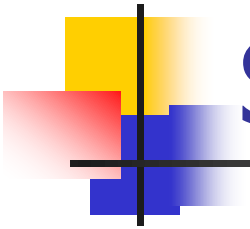
Google Pub/Sub: e.g. Receiving

```
// Instantiate an asynchronous message receiver.
MessageReceiver receiver =
  PubsubMessage message, AckReplyConsumer consumer) -> {
  // Handle incoming message, then ack the received message.
  System.out.println("Id: " + message.getMessageId());
  System.out.println("Data: "+ message.getData().toStringUtf8());
  consumer.ack();
  };
…
  Subscriber subscriber =
     Subscriber.newBuilder(topicName, receiver).build();
  // Start the subscriber.
  subscriber.startAsync().awaitRunning();
```

Google Pub/Sub: Real-life deployments

- All publisher-subscriber relationships:
  - one-to-many (fan-out)
  - many-to-one (fan-in)
  - many-to-many





Summary

- Cloud Execution Platforms and Programming Models (Cloud PaaS)
  - Google App Engine
    - overview, platform architecture, application structure
    - GAE services: e.g. distributed caching
    - scalability
  - Microsoft Azure
    - overview, applications
    - web roles and worker roles, service model
  - Integration overview
    - message queues, messaging services
    - job management
    - Google Pub/Sub

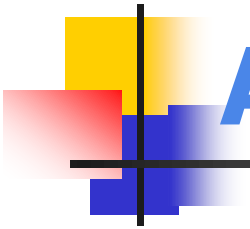


Cloud Function-as-a-Service
(Platforms and Programming Models)

Prof. Luís Veiga
IST / INESC-ID Lisboa

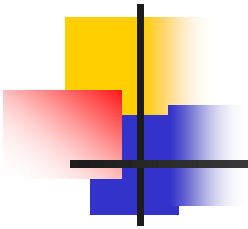
https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© RB, LV 2021-24



Agenda

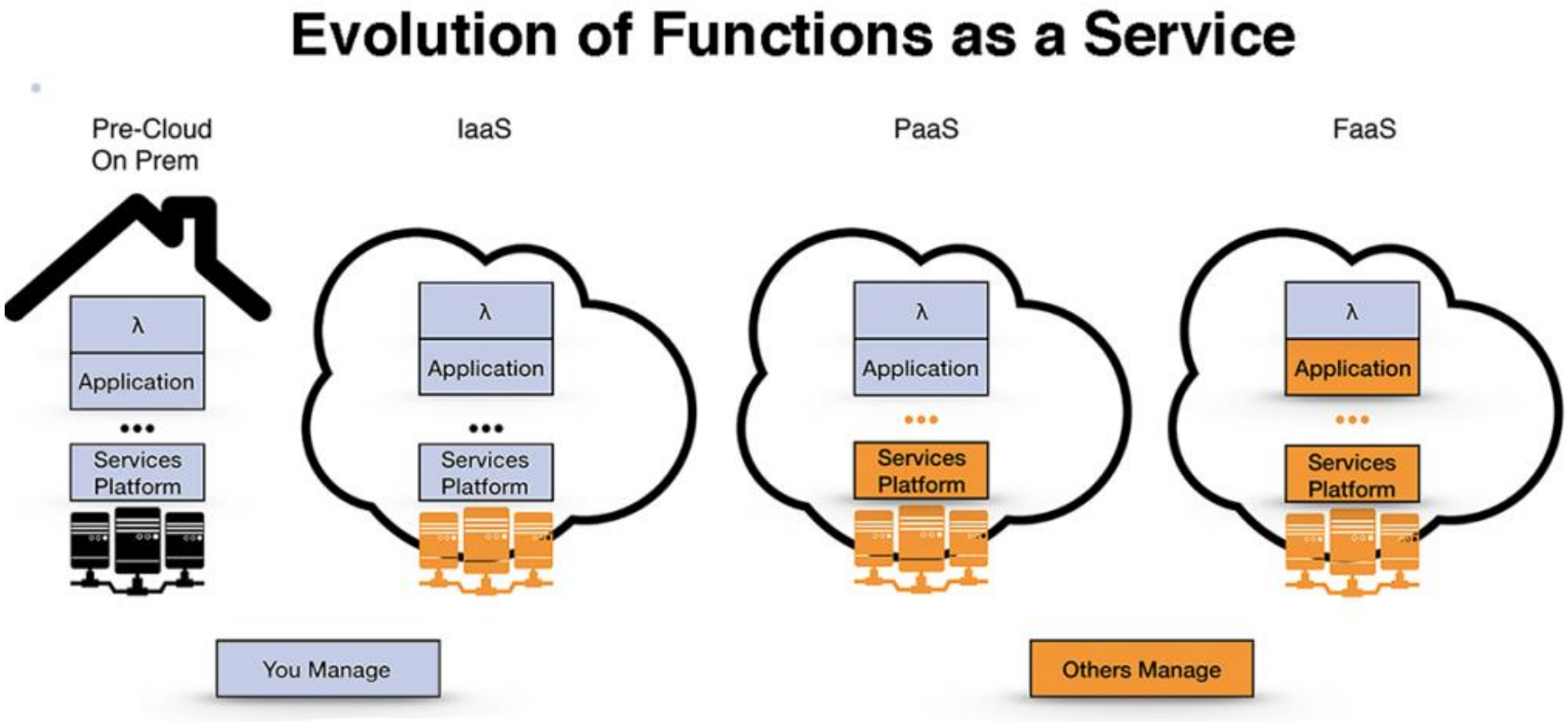
- **Function-as-a-Service**
  - Application model
  - Execution model
- Serverless Platforms
- Design Challenges
- Advanced FaaS Applications
  - Video Encoding (ExCamera)
  - Data Analytics (Lambada)
- FaaS Optimizations
  - Function co-execution
  - AOT Compilation
  - Sharing code and profiles



Function-as-a-Service

- FaaS is a recent cloud computing programming paradigm
  - also often loosely referred to as Serverless
- Technology generating a lot of interest
  - companies see it as
    - vehicle for automatic scalability and fault-tolerance
    - *pay as you go* is now *pay as you use*
  - researchers see it as
    - the next generation of cloud computing
- Technology is still maturing
  - extremely active research topic (lots of papers)

FaaS - Application Model



FaaS - Application Model

- Application is expressed as
  - set of logic units, functions
  - set of triggers (i.e., how to activate a function)
- One invocation, one task
  - e.g., check login information, convert image format, ...
  - concurrency/scalability *__not__* managed by developer:
    - function code is (in many cases) sequential.
    - simple code, less bugs, faster development.
- Fine(r) granularity:
  - as compared to microservices and monolithic services.
  - independent function development.
  - faster development cycles.

FaaS - Application Model

- Applications become event-based
  - easier to scale up, just add more FaaS nodes.
  - events could be web requests, storage update, etc.
- Only business logic is managed by the developer
  - improvement, as developer is the most expensive component
- Serverless can be used to implement most applications
  - no LB,
  - no auto scaler,
  - no web server
  - all provided transparently!

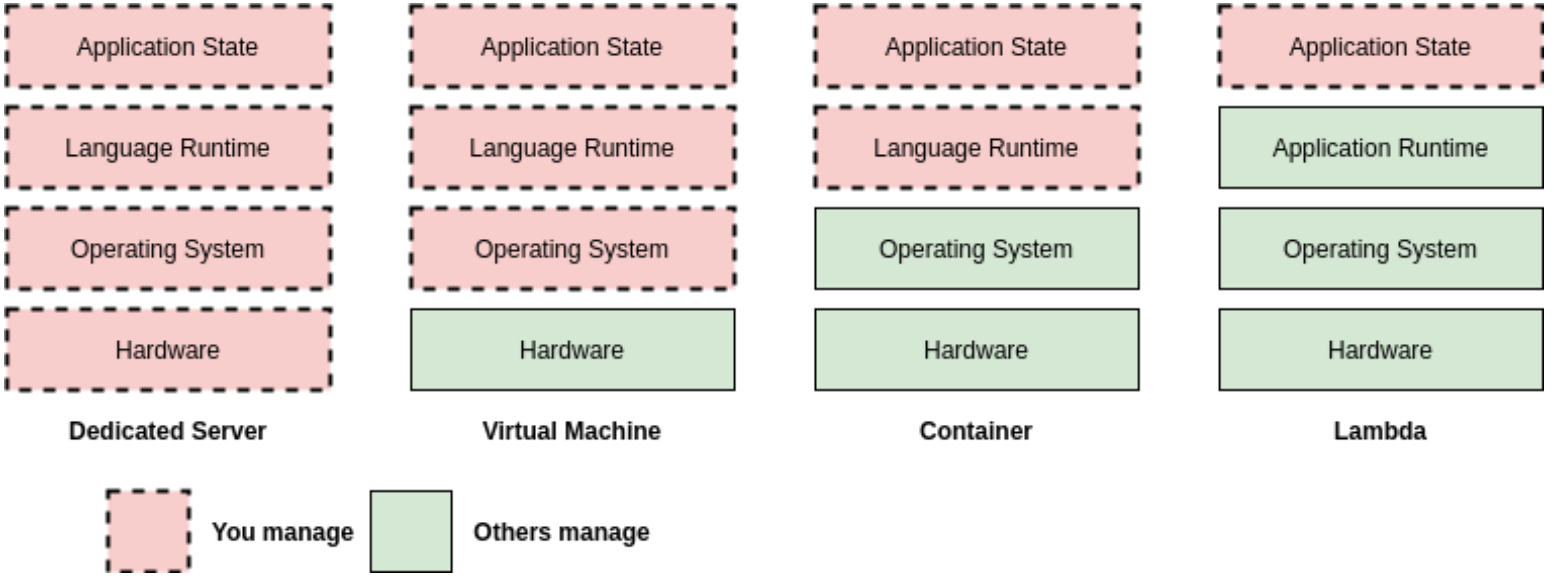
FaaS - Application Model

- Typical FaaS event-driven application



FaaS - Execution Model

- User vs Cloud Provider responsibilities:



FaaS - Serverless Platforms

- Amazon Lambda
  - launched in 2015
  - most popular Serverless
    - (and FaaS) platform
  - users upload code (e.g. .jar/ *.py/ *.js),
    set triggers, and set required memory
  - code is executed upon a trigger (e.g., HTTP request)
  - Lambdas
    - have a maximum 15 min timeout
    - execute inside Firecracker microVM
    - cannot modify the underlying system
    - cannot communicate directly with each other
  - Lambdas communicate via: storage, queues, db, …

> all these can generate triggers

FaaS - Serverless Platforms

- Amazon Lambda overview



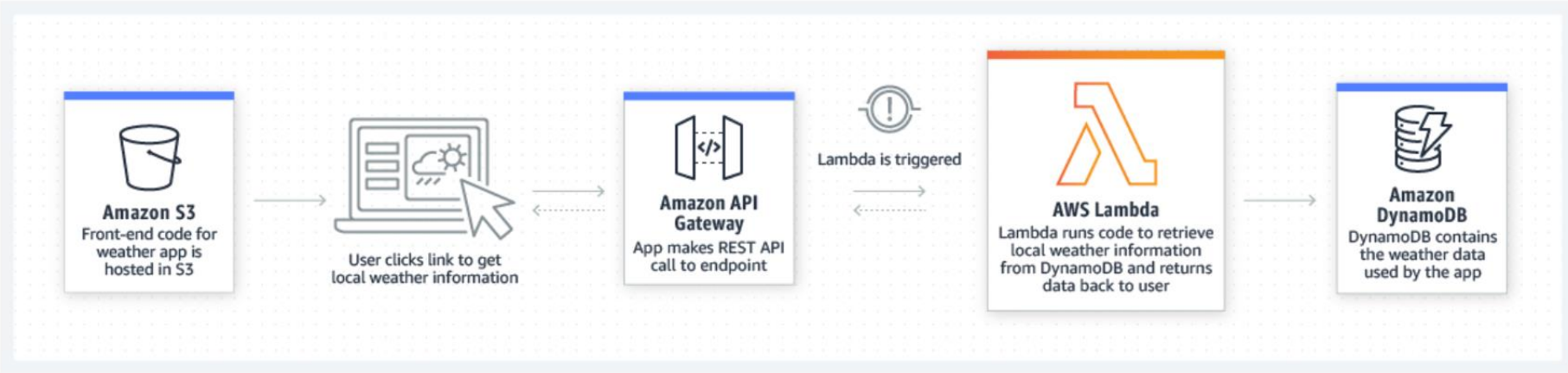
FaaS - Serverless Platforms

- Amazon Lambda Java example

```
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.LambdaLogger;
import java.util.Map;
public class Handler implements RequestHandler<Map<String, String>,
Integer> {
    private int factorial(int n ) {
                int fact = 1;
                for(int i = 1; i <= n; i++){
                    fact = fact * i;
                }
                return fact;
    }
    @Override
    public Integer handleRequest(Map<String,String> event, Context context){
        LambdaLogger logger = context.getLogger();
        logger.log("Input: " + event.get("number"));
        return factorial(Integer.valueOf(event.get("number")));
    }
}
```

FaaS - Application Model

- Amazon Lambda e.g., handling user updates
  - E.g. generated via web app.



FaaS - Application Model

- Amazon Lambda e.g., handling streaming data



FaaS - Serverless Platforms

- Amazon Lambda Python example
  - (in 20 lines of code)

```python
import os
import logging
import jsonpickle
import boto3
from aws_xray_sdk.core import xray_recorder
from aws_xray_sdk.core import patch_all

logger = logging.getLogger()
logger.setLevel(logging.INFO)
patch_all()

client = boto3.client('lambda')
client.get_account_settings()

def lambda_handler(event, context):
    logger.info('## ENVIRONMENT VARIABLES\r' + jsonpickle.encode(dict(**os.environ)))
    logger.info('## EVENT\r' + jsonpickle.encode(event))
    logger.info('## CONTEXT\r' + jsonpickle.encode(context))
    response = client.get_account_settings()
    return response['AccountUsage']
```

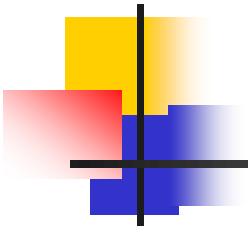
FaaS - Application Model

- Function:
  - a resource that you can invoke to run your code in Lambda.
  - a function has code to process events.
- Trigger:
  - a resource or configuration that invokes a Lambda function.
  - triggers include:
    - AWS services configured to invoke function (e.g. storage update)
    - event source mappings (e.g., queues, streams to functions)
- Event:
  - JSON document with data for a Lambda function to process.
  - Example custom event – weather data:

```
{
  "TemperatureK": 281,
  "WindKmh": -3,
  "HumidityPct": 0.55,
  "PressureHPa": 1020
}
```

FaaS - Application Model

- Concurrency:
  - number of requests that your function is serving at any given time.
- Execution lifecycle:
  - when function invoked, Lambda provisions instance to process event.
  - when function code finishes running, it can handle another request.
  - if invoked again while a request is still being processed:
    - another instance provisioned, increasing function's concurrency.
- Concurrency is subject to quotas at the AWS Region level:
  - configurable, and min and max thresholds can be set.
  - *prevents excess spending but may incur in idle costs.*



FaaS - Serverless Platforms

- Amazon Lambda: Pricing
  - cost: number of invocations and memory usage
  - **single knob:** memory
  - invocation has fixed price
    - e.g. per million invocations
  - (temporary/ephemeral storage also charged)
  - price rounded to ms
    - exec time * ms * (memory_MB_rate)
  - ~ 0.1 vCPU / 128 MB
    - Lambdas are IO dominant
    - 10 lambdas per CPU!

FaaS - Serverless Platforms

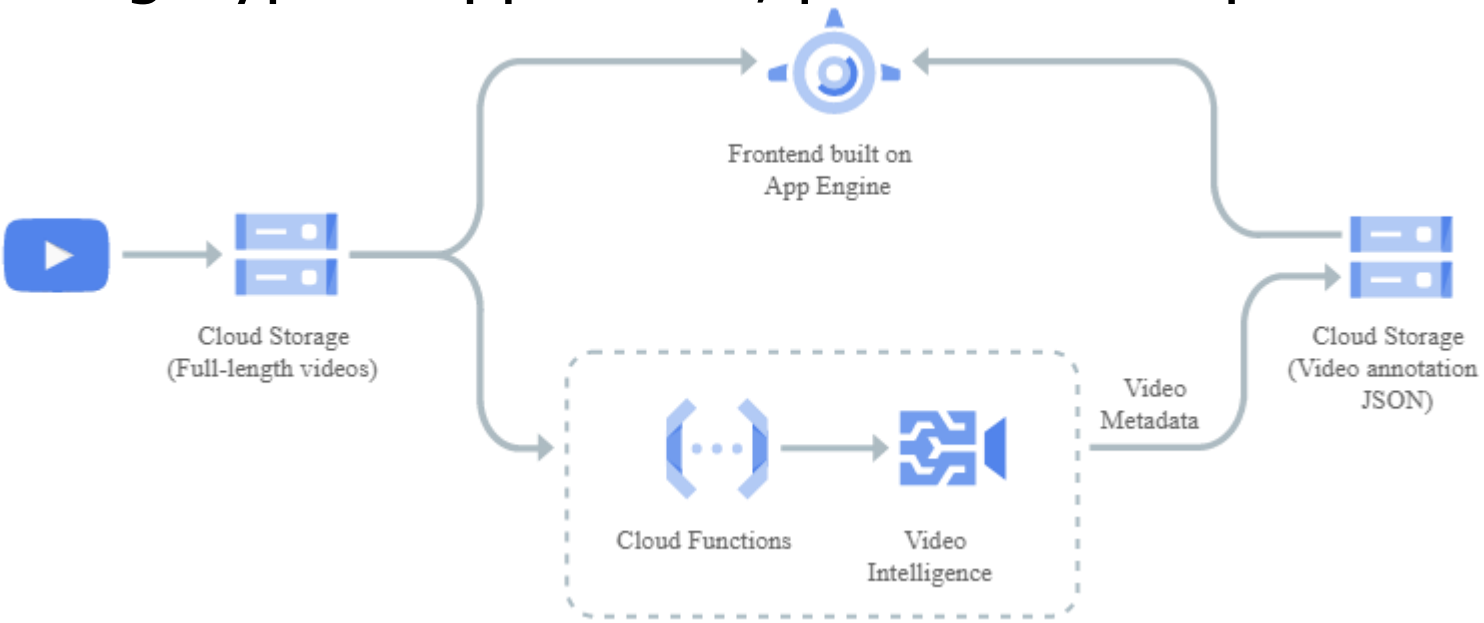
Amazon Lambda: Pricing

$0.20 per 1M requests

Memory (MB) (with larger memory)	Price per 1ms
128	$0.0000000021
512	$0.0000000083
1024	$0.0000000167
1536	$0.0000000250
2048	$0.0000000333
3072	$0.0000000500
4096	$0.0000000667
5120	$0.0000000833
6144	$0.0000001000
7168	$0.0000001167
8192	$0.0000001333
9216	$0.0000001500
10240	$0.0000001667

FaaS - Serverless Platforms

- Google Cloud Functions:
  - Google pricing similar to Amazon Lambda
    - e.g. number of invocations and memory usage
  - E.g. typical application, parallel video processing



FaaS - Serverless Platforms

- Google Cloud Functions, e.g.

```
package functions;

import com.google.cloud.functions.HttpFunction;
import com.google.cloud.functions.HttpRequest;
import com.google.cloud.functions.HttpResponse;
import java.io.BufferedWriter;
import java.io.IOException;

public class HelloWorld implements HttpFunction {
  // Simple function to return "Hello World"
  @Override
  public void service(HttpRequest request, HttpResponse response)
      throws IOException {
    BufferedWriter writer = response.getWriter();
    writer.write("Hello World!");
  }
}
```

FaaS - Serverless Platforms

- Google Cloud Functions: pricing

Memory	vCPU[1]	Price/100ms (Tier 1 Price)
128MB	.083 vCPU	$0.000000231
256MB	.167 vCPU	$0.000000463
512MB	.333 vCPU	$0.000000925
1024MB	.583 vCPU	$0.000001650
2048MB	1 vCPU	$0.000002900
4096MB	2 vCPU	$0.000005800
8192MB	2 vCPU	$0.000006800

Invocations per month	Price/million
First 2 million	Free
Beyond 2 million	$0.40

FaaS - Serverless Platforms

- Azure Functions
  - similar to Amazon Lambda
    - but with specific pricing in Premium plan
  - e.g. Python example

```python
import logging

import azure.functions as func


def main(req: func.HttpRequest) -> func.HttpResponse:
    logging.info('Python HTTP trigger function processed a request.')

    name = req.params.get('name')
    if not name:
        try:
            req_body = req.get_json()
        except ValueError:
            pass
        else:
            name = req_body.get('name')

    if name:
        return func.HttpResponse(f"Hello, {name}. This HTTP triggered function executed successfully.")
    else:
        return func.HttpResponse(
             "This HTTP triggered function executed successfully. Pass a name in the query string or in the r
             status_code=200
        )
```

FaaS - Serverless Platforms

- Azure Functions: pricing
  - Pay-as-you-go Plan
    - payment per number of invocations and memory usage

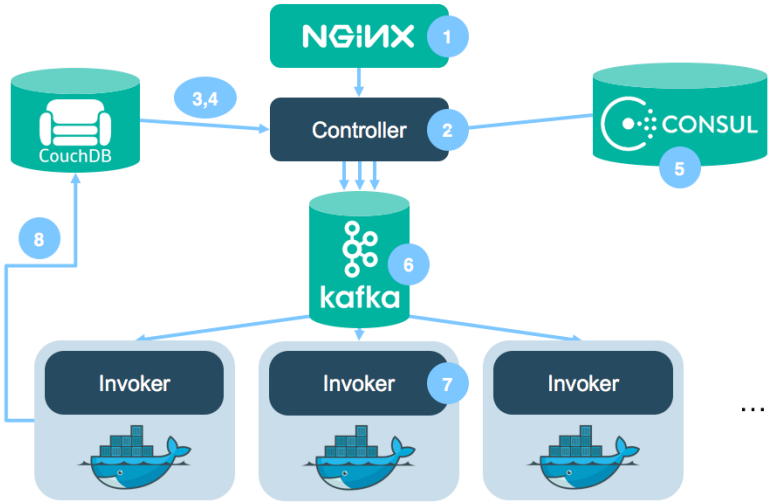
Meter	Free Grant (Per Month)	Pay as you go
Execution Time*	400,000 GB-s	$0.000016/GB-s
Total Executions*	1 million executions	$0.20 per million executions

  - Premium Plan
    - additional payment per each VCPU usage time

Meter	Price
vCPU duration	vCPU: $0.173 vCPU/hour
Memory duration	Memory: $0.0123 GB/hour
Price per million executions	× $0.20

FaaS - Serverless Platforms

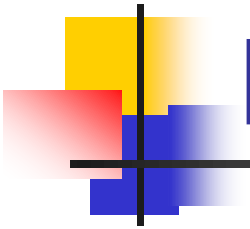
- IBM OpenWhisk
  - open source Serverless platform
  - components:
    - NGINX, the load balancer
    - Controller, lambda scheduler
    - CouchDB, keeps most state, history,
    - KV store, keeps active nodes, etc
    - Kafka, pushes actions to invokers
    - Invoker, manages a local lambda
    - Docker, lambda execution environment



FaaS - Serverless Platforms

- IBM OpenWhisk: e.g. resource and service limits

limit	description	configurable	unit	default
timeout	a container is not allowed to run longer than N milliseconds	per action	milliseconds	60000
memory	a container is not allowed to allocate more than N MB of memory	per action	MB	256
logs	a container is not allowed to write more than N MB to stdout	per action	MB	10
instances	an action is not allowed to have more containers than this value (**new scheduler only**)	per action	number	namespace concurrency limit
concurrent	no more than N activations may be submitted per namespace either executing or queued for execution	per namespace	number	100
minuteRate	no more than N activations may be submitted per namespace per minute	per namespace	number	120
codeSize	the maximum size of the action code	configurable, limit per action	MB	48
parameters	the maximum size of the parameters that can be attached	not configurable, limit per action/package/trigger	MB	1
result	the maximum size of the action result	not configurable, limit per action	MB	1



Function-as-Service

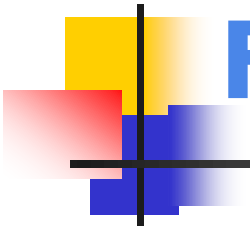
- Design and Implementation Challenges:

FaaS - Execution Model

- Where to run function invocations (i.e. lambdas)?
  - in a dedicated VM / container? Resource waste would be enormous...
    - applications are event-driven, idle most of the time.
    - starting a VM / container takes 100s of ms.
      - more than the function takes to execute?

- **Solution**: function runs *"with no dedicated server"*, i.e., Serverless
  - users don't select a VM nor OS type
    - users select:
      - language runtime (JVM, Python, JS, CLR, etc.), and
      - required memory
  - providers can decide where to host invocation
    - -> use 'leftovers' from IaaS
    - no persistent state (lambdas should not contain state)

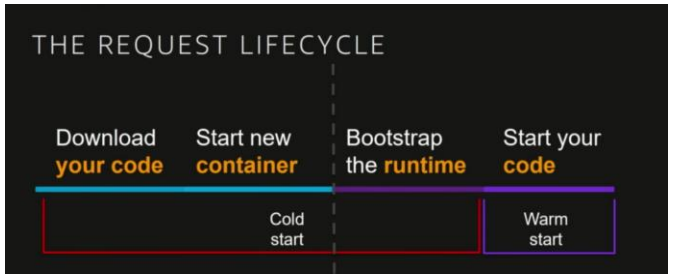
  - Lambdas, functions' execution environment, are lightweight
    - extreme elasticity and billing per request per millisecond

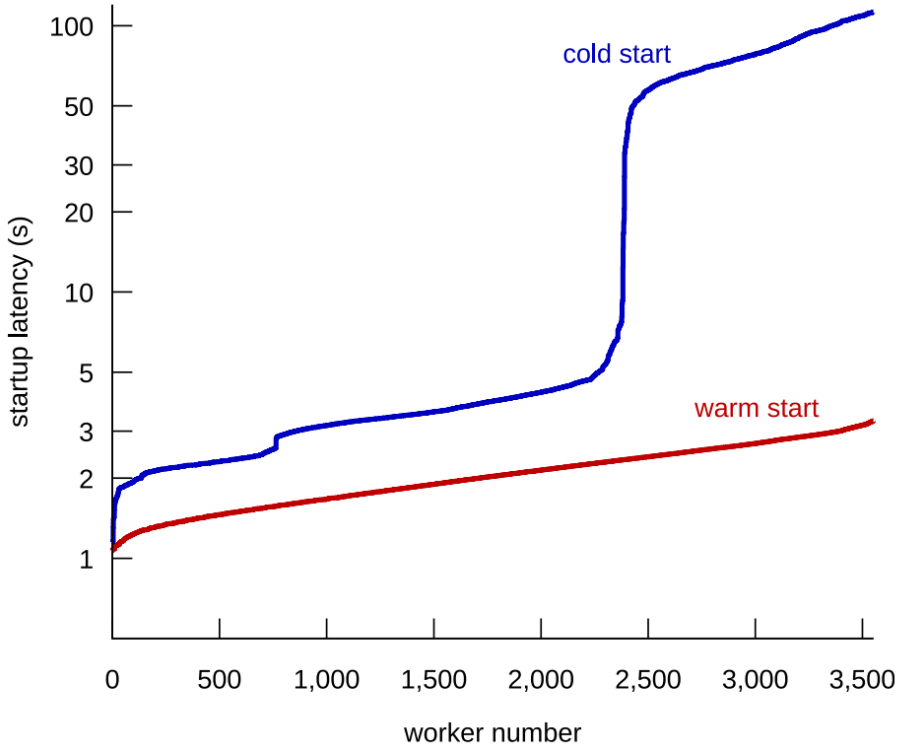


FaaS - Serverless Platforms

- Lambda Startup (>> 100 ms)
  - download code and VM/container image
  - start VM/container
  - start runtime
- Lambda execution ([few ms, few mins])
  - handle application request

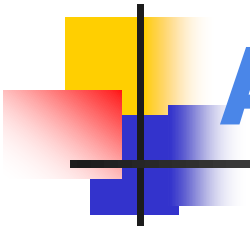
FaaS - Serverless Platforms





FaaS - Serverless Platforms

- Solution: reuse already started environment
- (widely used optimization)
  - number of cold-starts (lambda startup) << number of warm starts (lambda execution)
  - function code does not control if execution starts from a cold or warm start
  - warm containers are kept in memory until evicted
    - (timeout or memory required)



Agenda

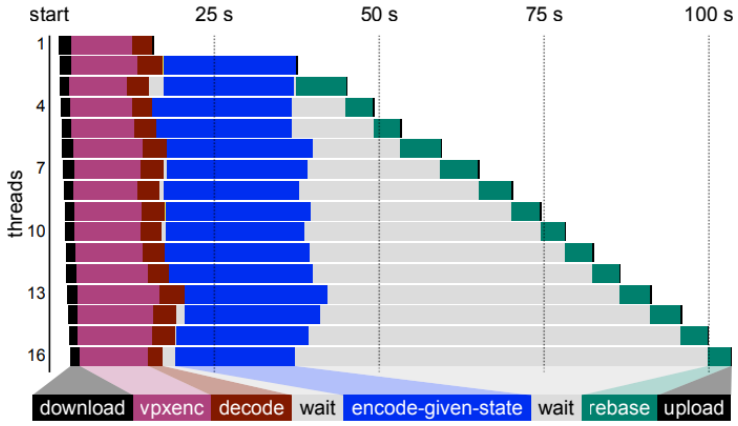
- Function-as-a-Service
  - Application model
  - Execution model
  - Serverless Platforms
- **Advanced FaaS Applications**
  - Video Encoding (ExCamera)
  - Data Analytics (Lambada)
- FaaS Optimizations
  - Function co-execution
  - AOT Compilation
  - Sharing code and profiles

Advanced FaaS Applications

- ExCamera - low latency video encoding tool based on FaaS
  - Problem: video encoding is computationally intensive...
    - Parallel computation, e.g. 3600 threads to process chunks of video
      - 3600 threads in EC2 for 1 second costs 20$ (coarse grained billing granularity)
      - 3600 threads in Lambda for 1 second costs 10¢ (fine grained billing granularity)
  - Solution:
    - upload video to cloud storage (S3)
    - start thousands of lambdas that consume a chunk of video
    - merge the output (encoded video)

Advanced FaaS Applications

- ExCamera:- low latency video encoding tool based on FaaS



14.8-minute 4K Video @20dB	
vpxenc Single-Threaded	453 mins
vpxenc Multi-Threaded	149 mins
YouTube (H.264)	37 mins
ExCamera[6, 16]	2.6 mins

- **Lambdas can be used for massively parallel bursts of computation**

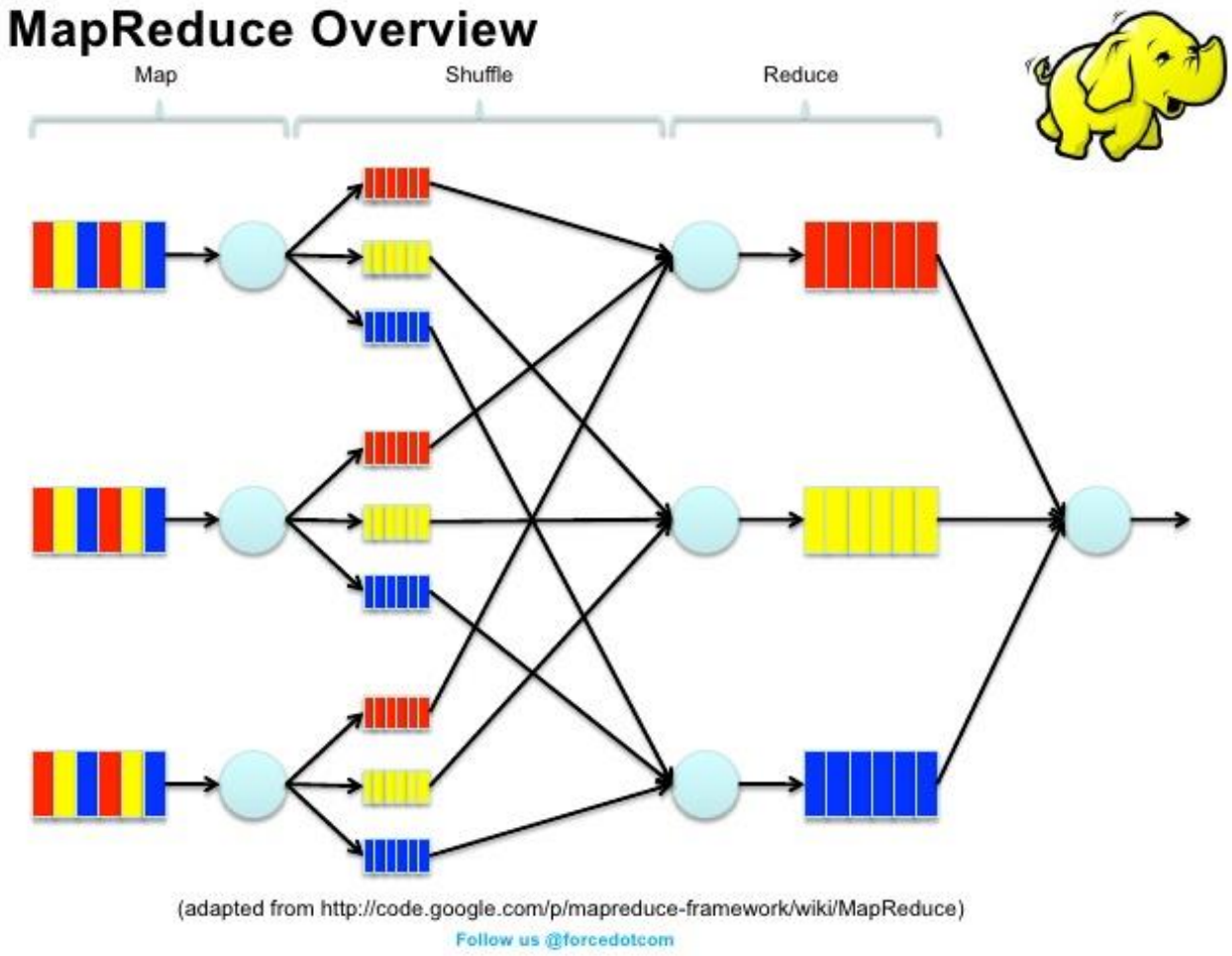
Source: Encoding, Fast and Slow: Low-Latency Video Processing Using Thousands of Tiny Threads

Advanced FaaS Applications

- Lambada: low latency data analytics
  - Question: can we use FaaS for low latency data analytics?
    - E.g., running Mapreduce apps
  - Problem: data needs to be shuffled
    - (Lambdas cannot directly communicate with each other)

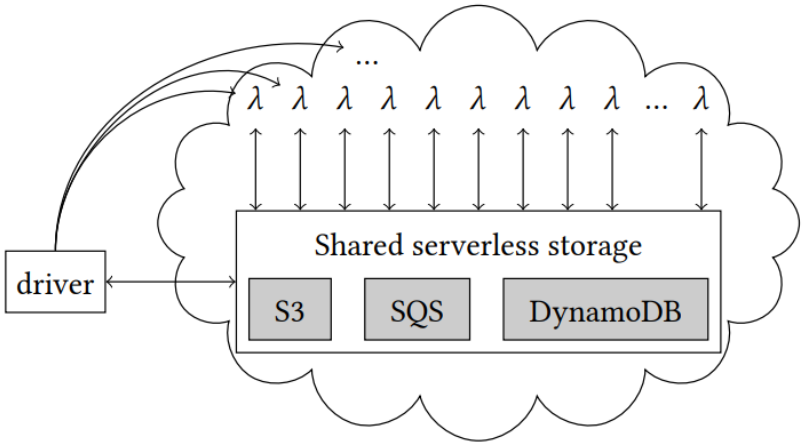
Source: Lambada: Interactive Data Analytics on Cold Data Using Serverless Cloud Infrastructure

Advanced FaaS Applications



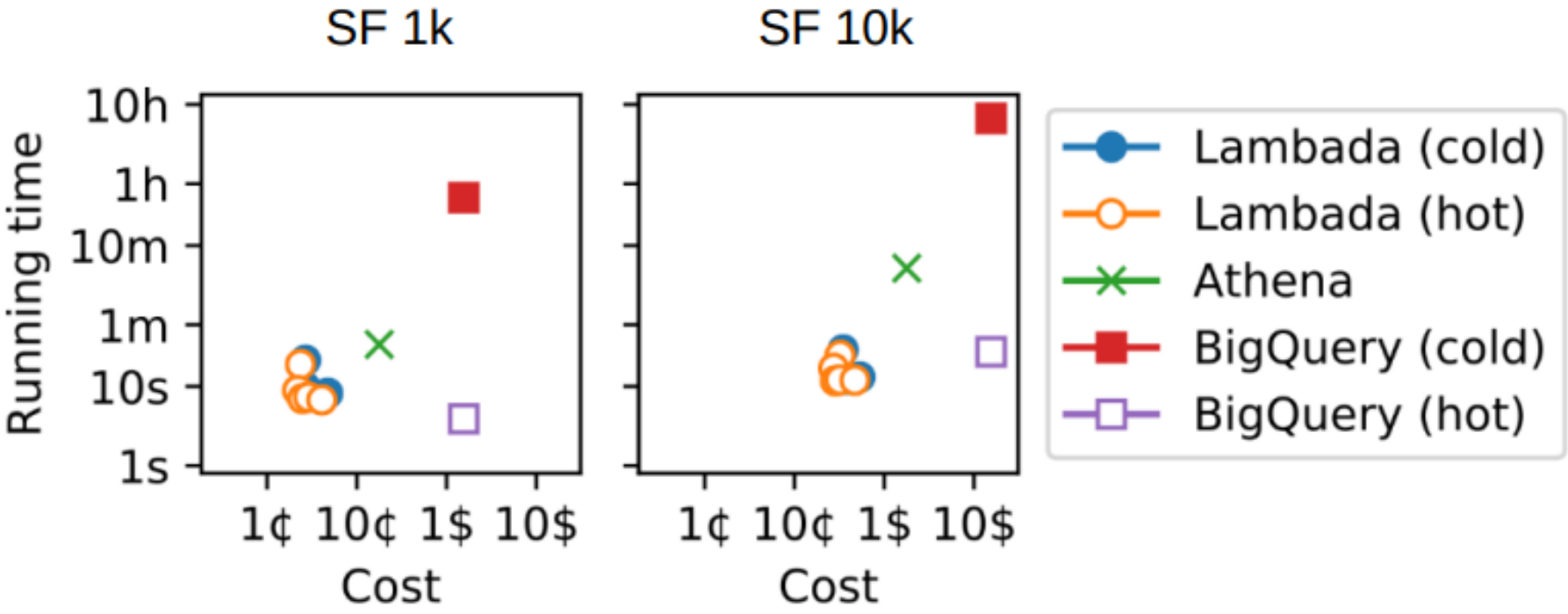
Advanced FaaS Applications

- Lambada: low latency data analytics
  - Sub-question: how to efficiently shuffle data?



Advanced FaaS Applications

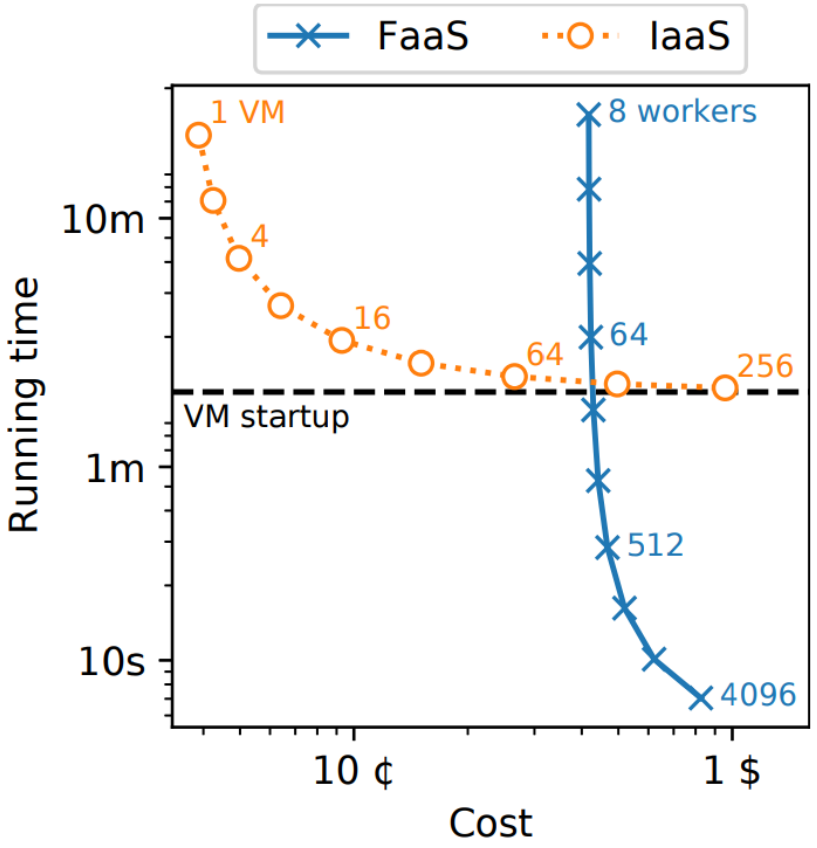
- Lambada: results
  - SF 1k: 320 workers, SF 10k: 3200 workers

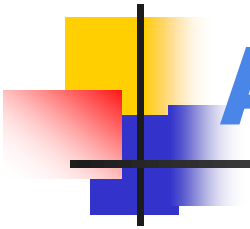


Advanced FaaS Applications

- Lambada: is FaaS the solution to run data analytics?
  - No so fast…

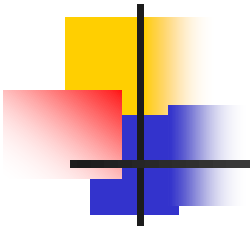
Only for interactive data analytics on cold data





Agenda

- Function-as-a-Service
  - Application model
  - Execution model
  - Serverless Platforms
- Advanced FaaS Applications
  - Video Encoding (ExCamera)
  - Data Analytics (Lambada)
- **FaaS Optimizations**
  - Function co-execution
  - AOT Compilation
  - Sharing code and profiles

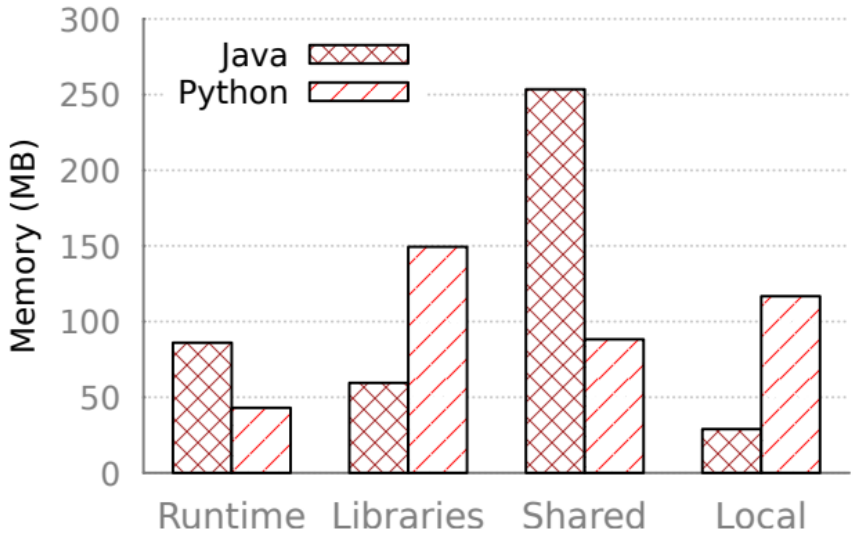


FaaS Optimizations

- FaaS / Serverless open problems
  - Cold start latency
    - long tail latency due to slow VM/container + runtime startup
  - Resource redundancy
    - leads to high memory footprint
      - VM/container + runtime may account for most memory footprint
  - Long warmup (low throughput)
    - JIT compilers and interpreters optimize the code over long periods of time.
      - runtimes were not designed for the serverless use-case
      - how many invocations are needed to fully warmup the runtime?

FaaS Optimizations

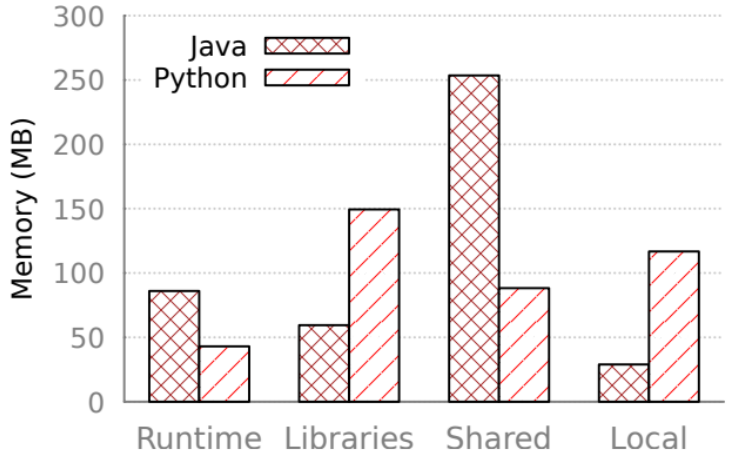
- Function Co-execution
  - Idea: can we execute multiple **concurrently** lambdas **in** the same runtime?
    - runtime can be shared
    - libraries can be shared
    - global state can be shared
      - E.g., ML models
    - invocation-private state is low

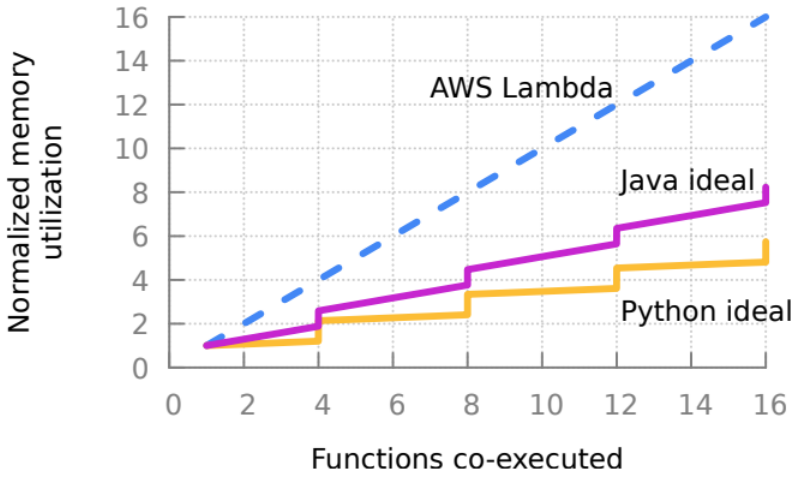


Source: Photons: Lambdas on a diet

FaaS Optimizations

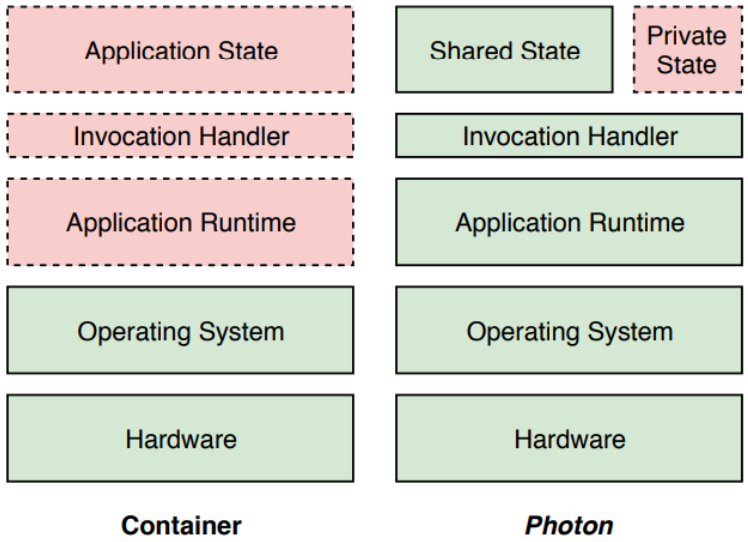
- Function Co-execution
  - Idea: can we execute multiple **concurrently** lambdas **in** the same runtime?





FaaS Optimizations

- Function Co-execution - Photons
  - # Photons >> # Runtimes
  - Photons benefit from a single:
    - cold start
    - runtime warmup
  - Photons reduce overall memory utilization
  - Problem: what about global state?
    - Requires data isolation!



Source: Photons: Lambdas on a diet

FaaS Optimizations

- Function Co-execution - Photons

```
static Bar bar;

foo () {
    bar = new Bar(...);
    bar.baz();
}
```

What if multiple Photons execute at the same time? Need to separate state!

FaaS Optimizations

- Function Co-execution - Photons

```
static Bar bar;

foo () {
    bar = new Bar(...);
    bar.baz();
}
```



```
static WeakHashMap<PhotonID, Bar>
bar;

foo () {
    bar.put(photonId, new Bar(...));
    bar.get(photonId).baz();
}
```

Automatic Bytecode transformation at bytecode loading time using Javassist!

FaaS Optimizations

- Function Co-execution - Photons

```
static Bar bar = new Bar(...);
```

How about now? Is this a problem?

FaaS Optimizations

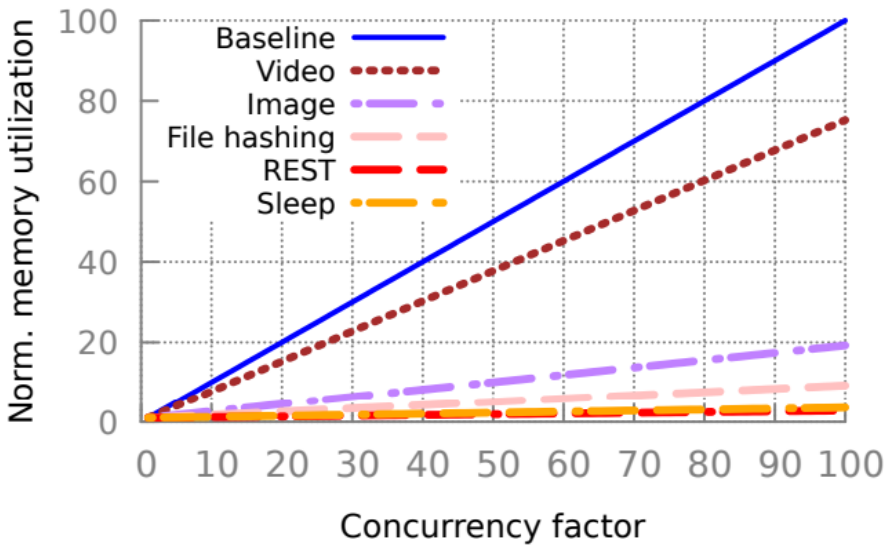
- Function Co-execution - Photons

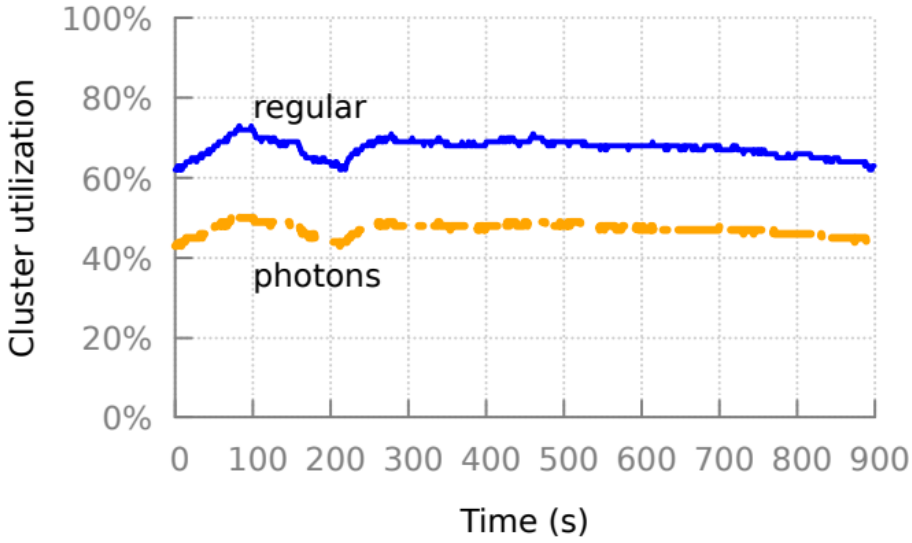
```
static Bar bar = new Bar(...);  ──────►  __static_initializer() {
                                             bar.put(photonId, new Bar(...));
                                         }
```

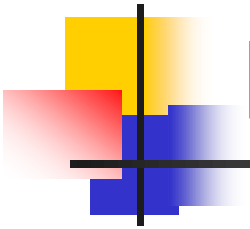
Static initializers are executed when Photons are started!

FaaS Optimizations

- Function Co-execution - Photons





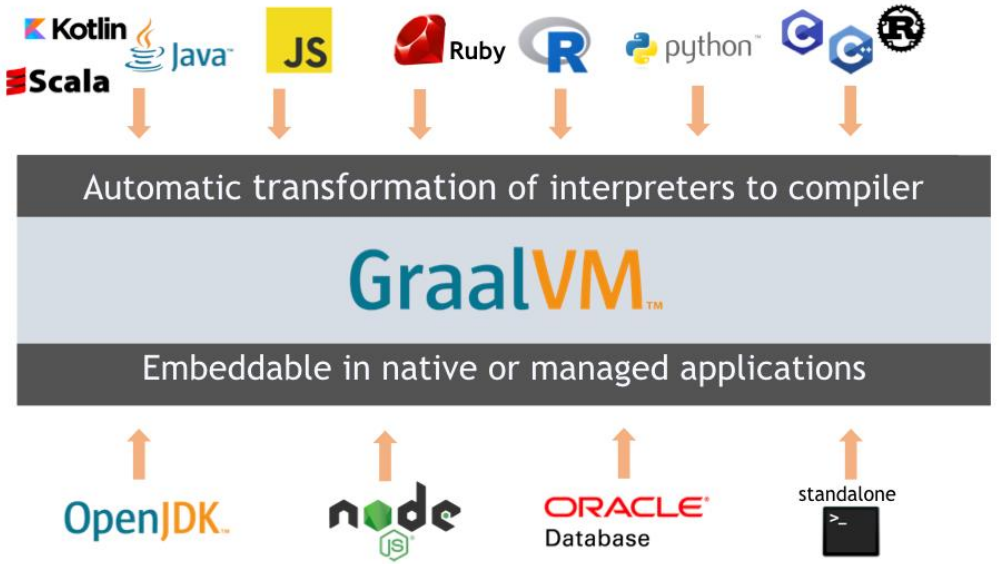


FaaS Optimizations

- AOT: Ahead-of-time compilation (and initialization)
  - Idea: can we compile high level apps into binaries?
    - such binary would not contain:
      - any JIT compiler
      - any interpreter
    - such binary would reduce:
      - number of cold starts (no runtime warmup)
      - memory footprint (smaller)

FaaS Optimizations

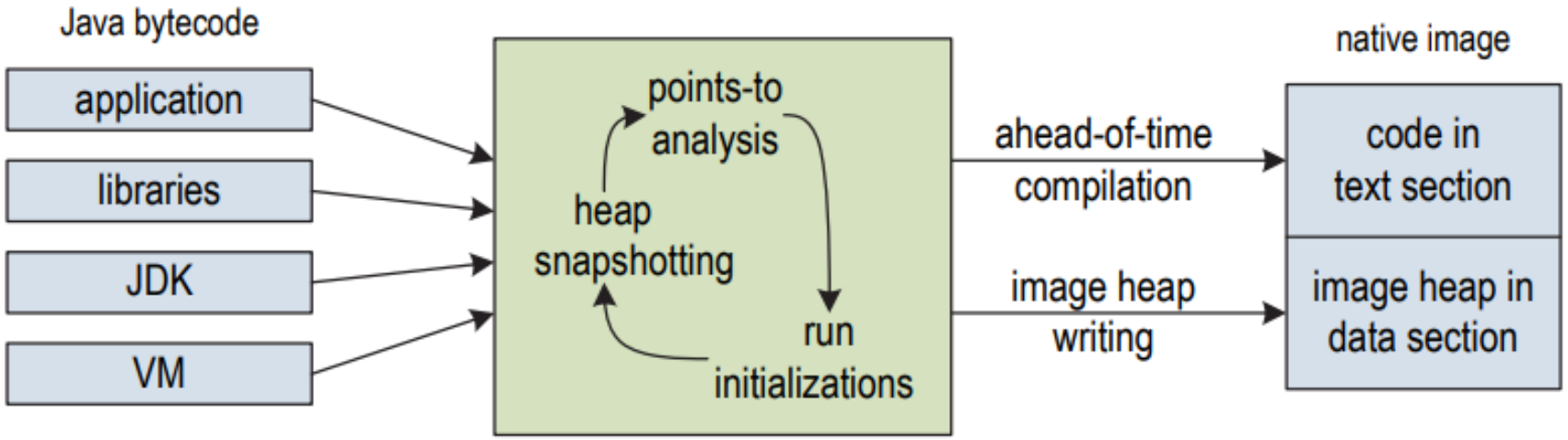
- GraalVM Native Image
  - GraalVM is a JDK containing:
    - a powerful JIT compiler (Graal)
    - a language implementation framework (Truffle)
    - an AOT tool for Java (Native Image)



Source: Initialize Once, Start Fast: Application Initialization at Build Time

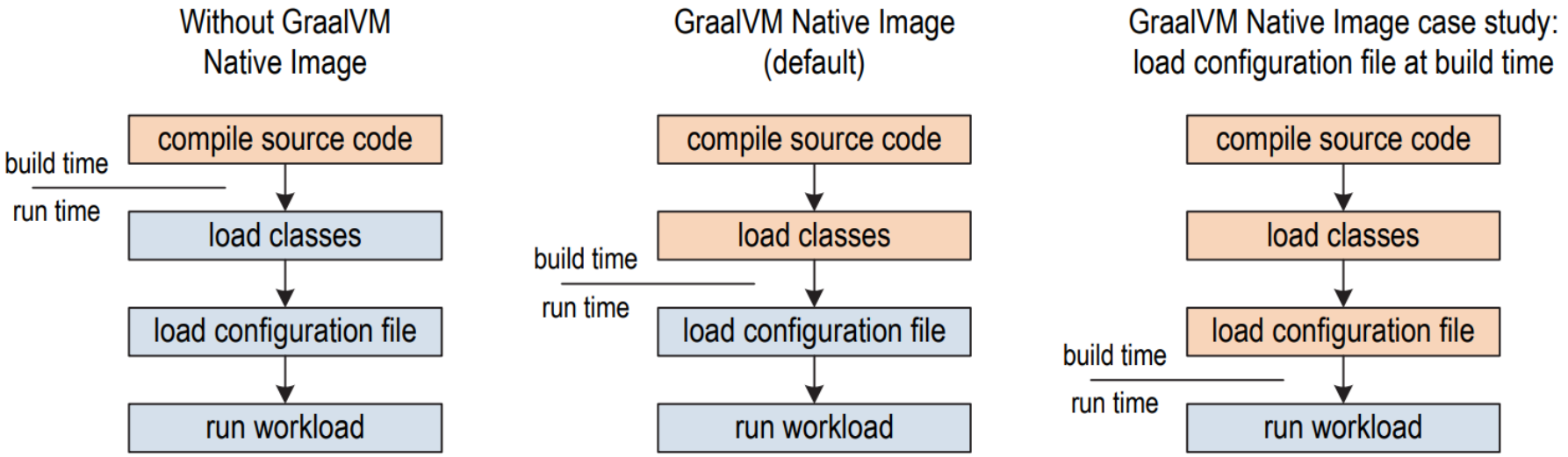
FaaS Optimizations

- GraalVM Native Image:



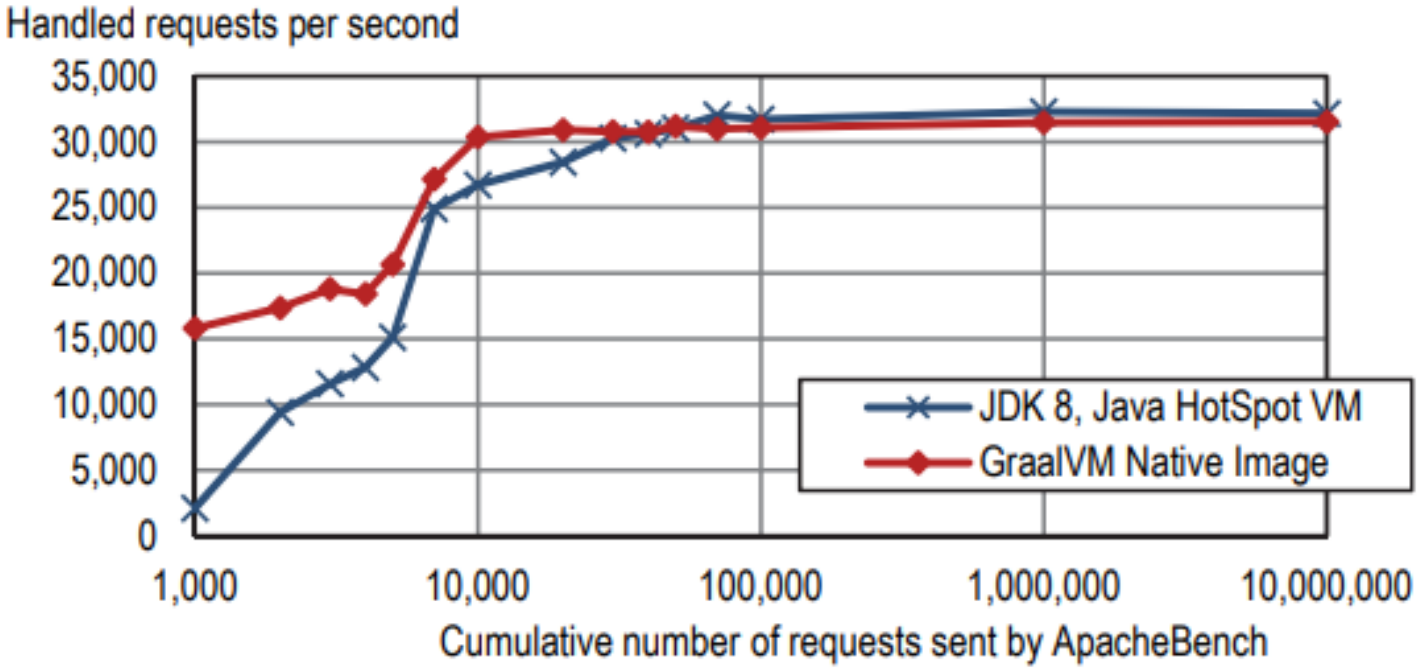
FaaS Optimizations

- GraalVM Native Image:



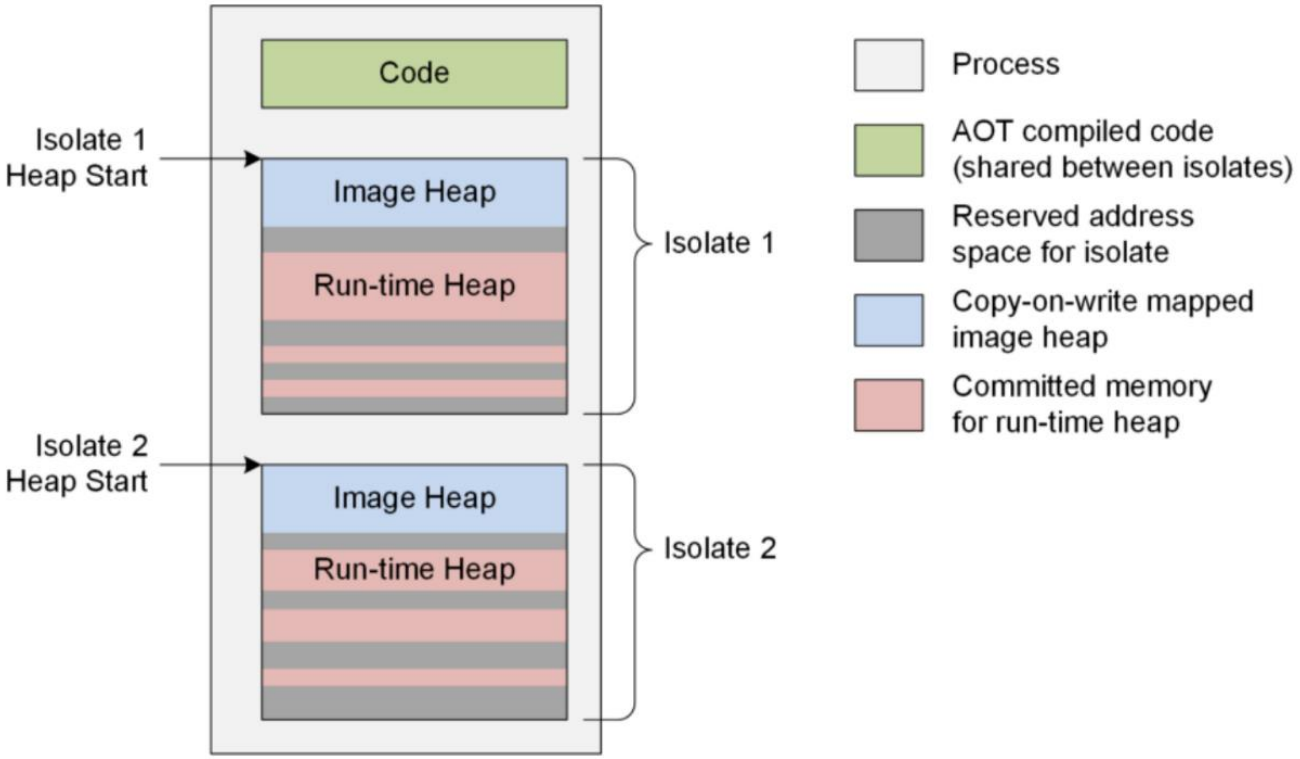
FaaS Optimizations

- GraalVM Native Image:



FaaS Optimizations

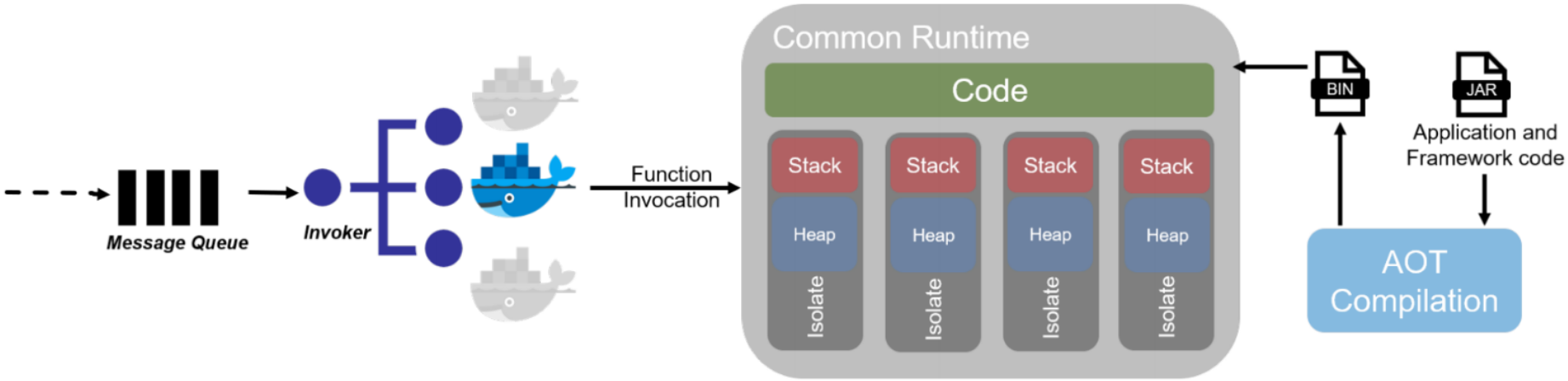
- GraalVM Native Image Isolates
  - Sharing runtime and code safely across instances



Source: Isolates and Compressed References: More Flexible and Efficient Memory Management via GraalVM

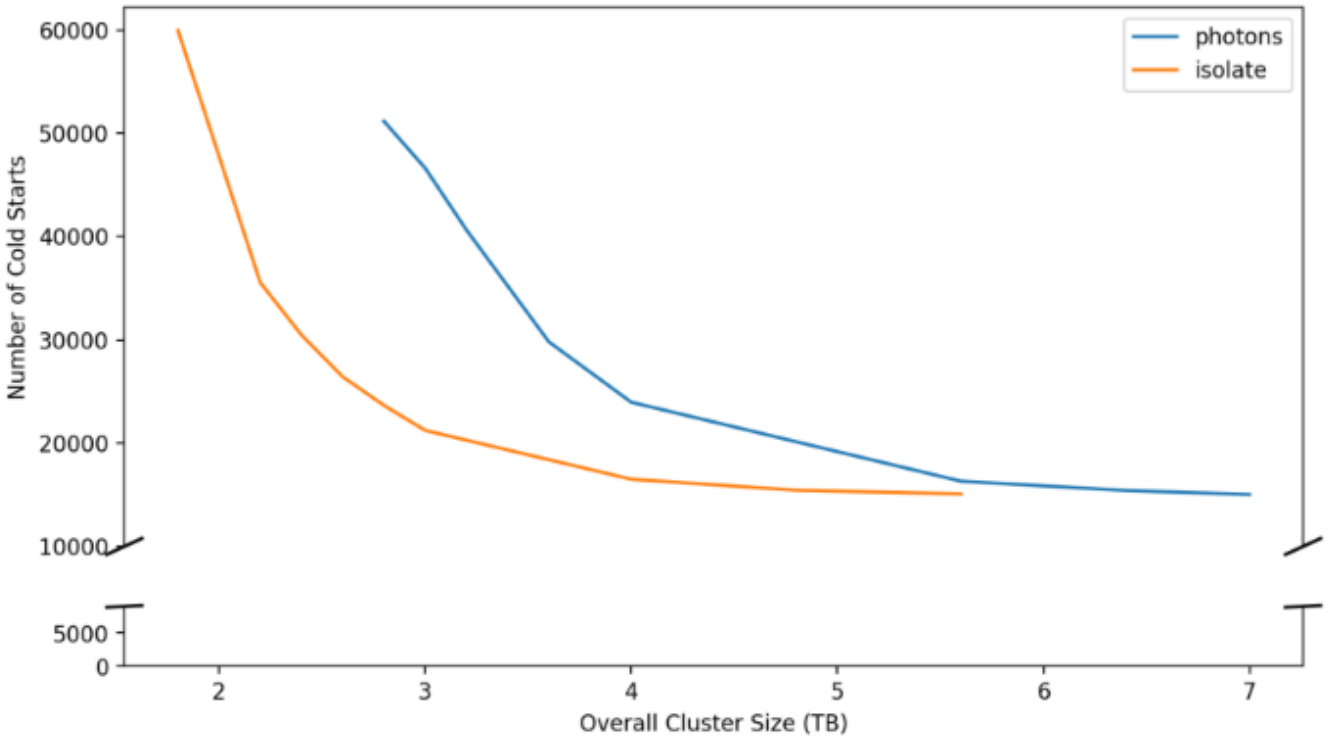
FaaS Optimizations

- FaaS with GraalVM Native Image Isolates



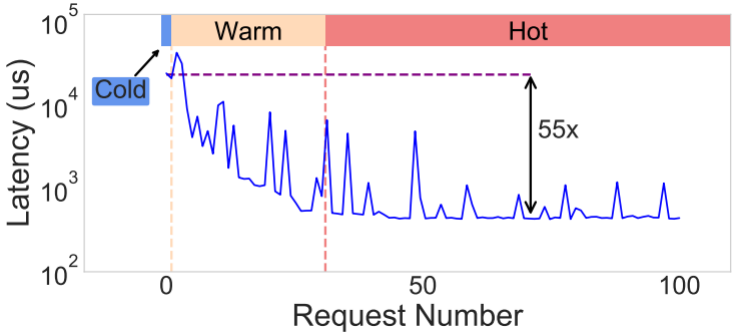
FaaS Optimizations

- FaaS with GraalVM Native Image Isolates

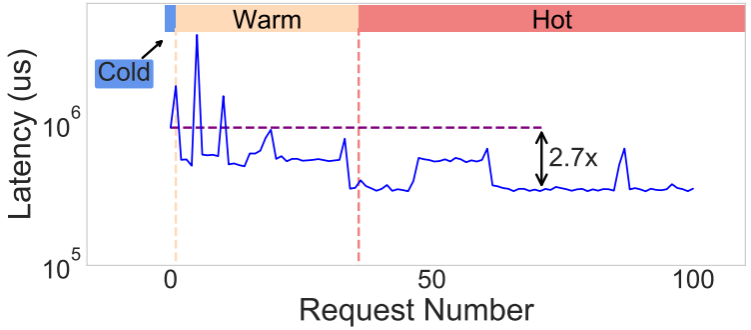


FaaS Optimizations

- Runtime code profiles and compiled code sharing
  - Problem 1: every time a new runtime is launched, it needs to profile and compile the code
    - functions may experience a long warmup period
  - Problem 2: all code profiling and code compilation is lost when the runtime is recycled



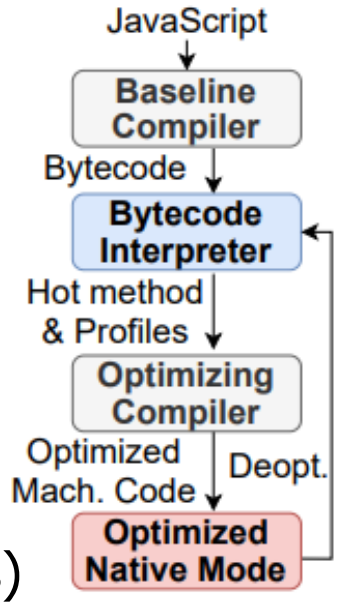
WordCount.java



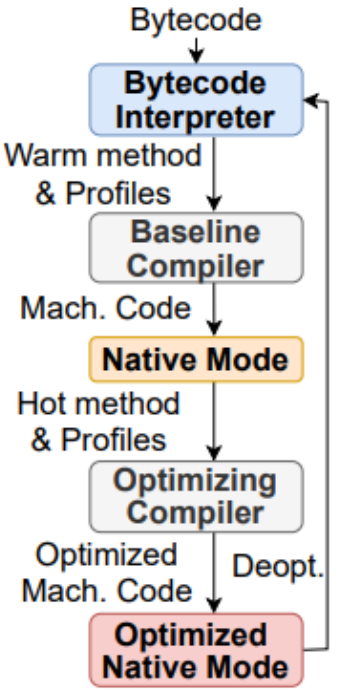
WordCount.js

FaaS Optimizations

- Runtime code profiles and compiled code sharing
  - Code flows through compilation along pipelines
    - interpreter (JVM only)
      - code profiling enabled
    - baseline compiler (basic optimizations)
      - code profiling enabled
    - optimizing compiler (advanced opts)
  - Profiling and compiling costs CPU cycles
    - Idea: can we cache and share them?



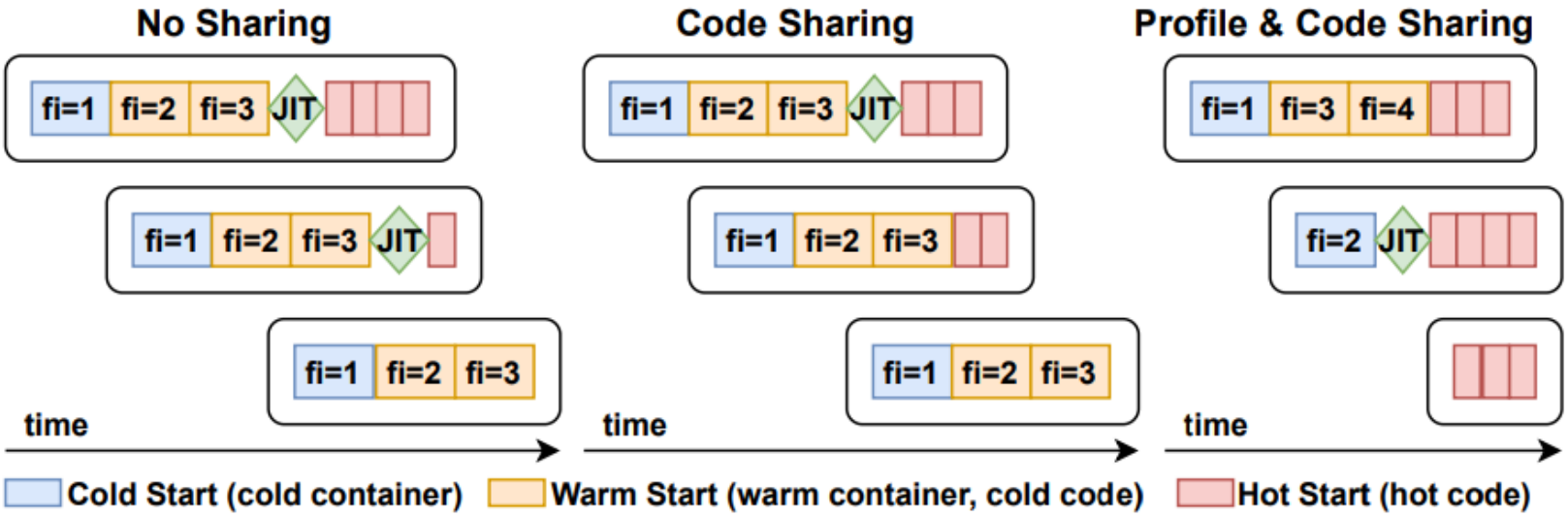
Google V8



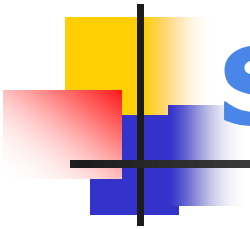
OpenJDK JVM

FaaS Optimizations

- Runtime code profiles and compiled code sharing (Ignite)



Source: From Warm to Hot Starts: Leveraging Runtimes for the Serverless Era



Summary

- Function-as-a-Service
  - Application model
  - Execution model
- Serverless Platforms
- Design Challenges
- Advanced FaaS Applications
  - Video Encoding (ExCamera)
  - Data Analytics (Lambada)
- FaaS Optimizations
  - Function co-execution
  - AOT Compilation
  - Sharing code and profiles

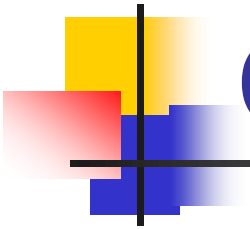


Cloud Storage (Part A)

Prof. Luís Veiga
IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© LV, JG 2015-24



Contents

- Cloud Storage:
  - overview: types of cloud storage, storage products
- Dynamo system:
  - overview, consistency (review)
  - system design
  - consistency
  - coordination
  - fault tolerance
  - synchronization
- Amazon table storage: DynamodDB
- *Reference:* Amazon object storage:
  - S3 – Simple Storage System

Overview: Storage Types

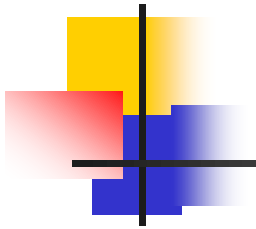
- A range of search, streaming and indexing variants.

- Object/Byte Storage:
  - allow direct Program <-> Storage interaction
  - support only Object ID indexing
  - e.g. Amazon S3 (Dynamo), Azure Blobs

- (Cloud) File System:
  - hierarchical organization, files, permission, streaming data,...
  - e.g. Hadoop File System / Google File System

- Tables (no-SQL DB):
  - records and tables
  - search, range-scan
  - no (full) relational model
  - e.g. Apache HBase / Google BigTable (AppEngine Datastore)
  - Amazon DynamoDB, Simple DB
  - MS Azure Tables
  - Facebook/Apache Cassandra

- Relational Databases:
  - full relational model
  - conventional services
  - e.g. Amazon RDS, SQL Azure



Cloud Object Store: Dynamo

(powering DynamoDB and Amazon Simple Storage System - S3)

Amazon S3 (on Dynamo)

- Amazon's persistent object storage system.
- Implementation based on the Dynamo system (SOSP, 2007).
- Amazon storage: S3 – Simple Storage System
  - Dynamo system
    - overview, consistency (review)
    - system design: API, decisions, partitioning, replication
    - consistency: vector clocks, data versions, reconciliation
    - coordination: read and write operations, quorums,
      - fault tolerance
    - synchronization: update processing, state convergence

Dynamo: Insight (CAP theorem)

- CAP Theorem (by Brewer):
  - **C**onsistency, **A**vailability and **P**artition tolerance
  - You can only pick two, impossible to ensure all always
- At Amazon: Availability = Client's trust
  - Cannot be sacrificed.
- In large data centres faults are going to be frequent :
  - The possibility of a partition has to be taken into account
- Most data services tolerate small inconsistencies:
  - Relaxed consistency ==> Eventual consistency.

Consistency Models (Review)

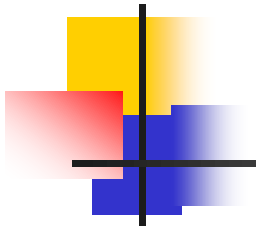
- *Strong Consistency*: Once a write operation is finished for the requester, any subsequent read will return the value that was written.
- *Weak Consistency*: The system does not guarantee that subsequent accesses return the written value.
  - Some condition must be verified for the written value to be returned (a time interval, an access to a synchronization variable,…).
  - The period between the write finishing and the value visibility is called the inconsistency window.
- **Eventual Consistency***:* The system guarantees that, if there are no more writes, the updates will become visible for all clients
  - (e.g. DNS): a DNS name update is propagated between zones until all clients *eventually* see the new value.

Dynamo: System Assumptions

- Interaction Model:
  - Total/full/complete reads and writes with unique IDs.
  - Binary objects with up to 5GB.
  - No operations on multiple objects.

- ACID properties (Atomicity, Consistency, Isolation, Durability):
  - Atomicity/Isolation: total writes of an object.
  - Durability: replicated write.
  - Only the consistency isn’t strong.

- Efficiency:
  - Optimized for the 99,9 percentile.



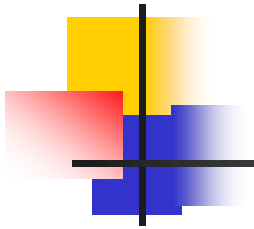
Dynamo: API

- Two operations:

- `put(key, context, object)`
  - key: object ID.
  - context: vector clocks and object's history.
  - object: data to be written.
- `get(key) -> (object versions, context)`
  - returns single object from key, or
  - list of conflicting versions and context

Dynamo: Design Decisions

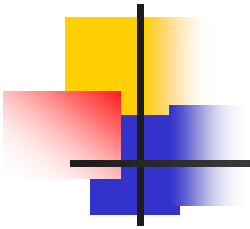
Problem	Technique	Advantage
Partitioning	Consistent *Hashing*	Incremental Stability
Write Availability	Vector clocks and conflict resolution of writes	Version size does not depend on the update rate
Temporary Faults	Relaxed quorum and *hinted handoff*	High availability and durability
Permanent Faults	Anti-entropy with Merkle trees	Synchronizes replicas asynchronously
Membership and Fault detection	Gossip based membership protocol	Maintains symmetry and avoids requiring a centralized directory



Dynamo: Design Decisions

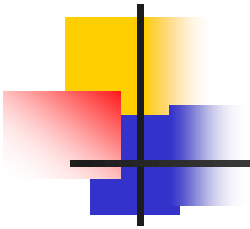
- Incremental Scalability:
  - Adding nodes has to be simple.
- Load balancing and support for heterogeneity:
  - The system must distribute/balance the requests.
  - And support nodes with different characteristics.

- Solution: nodes in a Chord-like organization.



Dynamo: Design Decisions

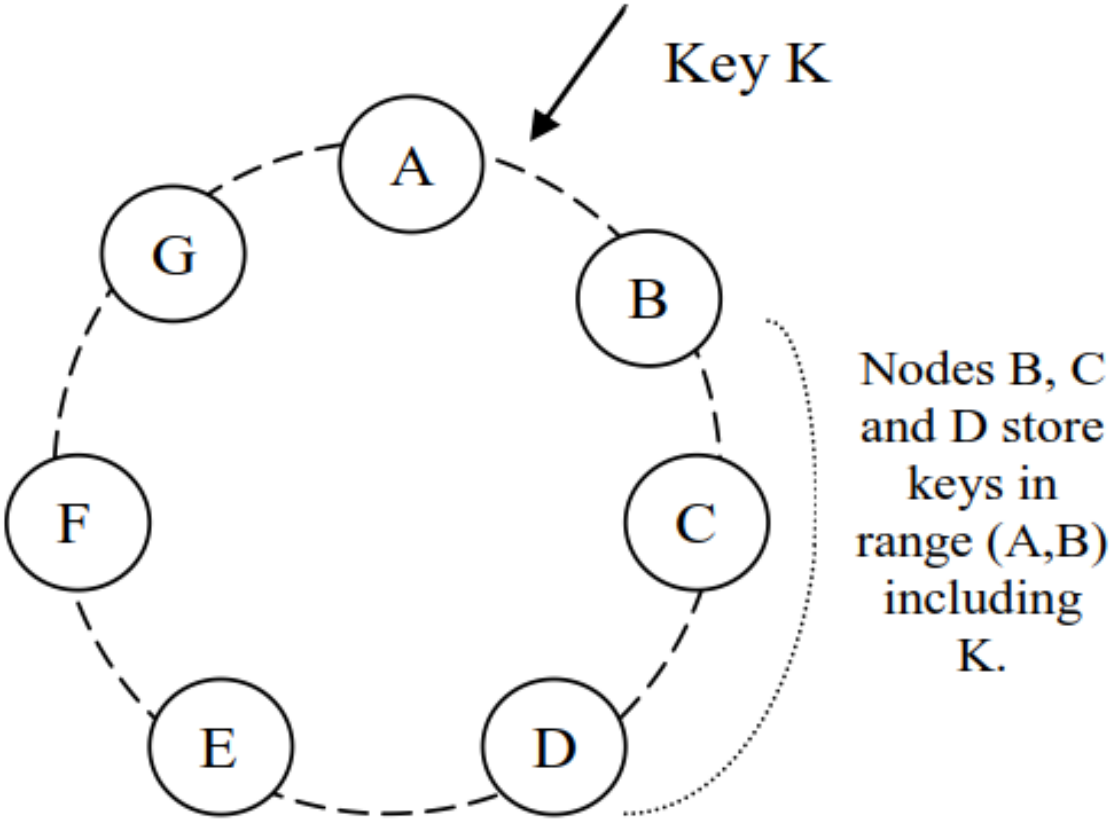
- Solution: nodes in a Chord-like organization
  - Peer-to-peer (P2P)
  - Distributed Hash Table (DHT)
- Symmetry:
  - All nodes are equally responsible peers.
- Decentralization:
  - Avoid single points of failure.



Partitioning and Replication

- Uses consistent hashing.
- Inspired by Chord:
  - Each node has an ID in the key space.
  - Nodes are arranged in a ring.
  - Data items are stored in the node with the lowest key that is larger than the object's
    - employing Key-Based Routing inspired in P2P systems
- Replication:
  - All objects are replicated in the N nodes that follow the (home) node associated with the object.

Partitioning and Replication
The Chord Ring with Replication



Consistency: Data Versions

- Nodes for writing and reading are selected based on load.

- So, we can only have eventual consistency:
  - There may be different versions written on different replicas.
  - Conflict resolution is made when reading and not when writing.
    - promoting full availability and reduced latency in writes.

- Semantic Reconciliation:
  - The user/programmer must decide, while divergence is uncommon.
  - For all read operations:
    - 99.94% - 1 version;
    - 0.00057% - 2 versions; 0.00047% - 3 versions; 0.00009% - 4 versions.

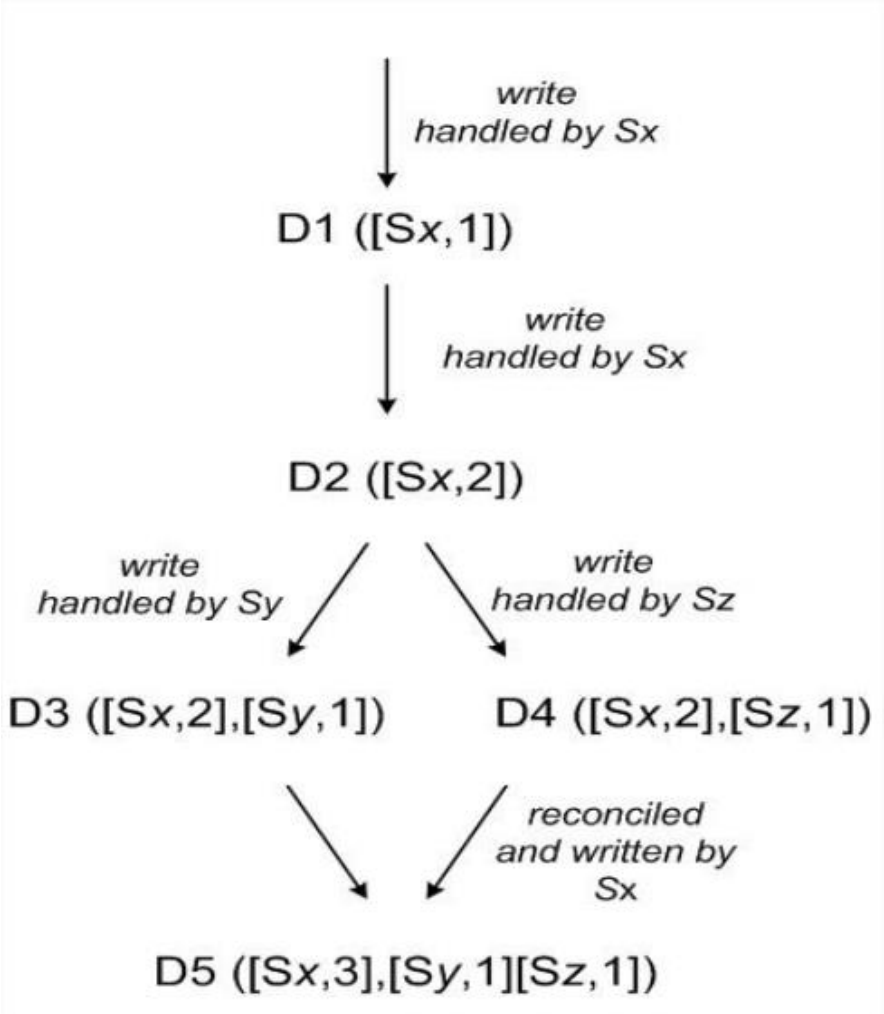
- Timeout:
  - After a number of generations without writing, versions are discarded.

Consistency: Vector Clocks (1)

- Represents time in a distributed system without clock synchronization.
  - replaces physical time with causality.
  - vector clock is a list of (node, counter) pairs.
- *If all positions of the vector clock time of an event A are smaller or equal (at least one smaller) than those of another event B,*
  - then A *happened before* B.
  - i.e., there is a causal event chain from A to B.

Consistency: Object Versions

- If we assign a vector clock timestamp to all object versions (update operations)
- we can detect divergent replicas and conflicts between versions/updates
- Example:
  - X, Y e Z are servers with replicas of object D.
  - D5 is the result of a semantic reconciliation performed/ decided by the user.



Coordination: Read/Write Operations

- Dynamo supports writing and reading using a quorum model.
  - allows not waiting for all replicas during an operation.
- Consider R and W are the number or read and write replicas that must synchronously take part in an operation.
- If R + W > N we have a quorum based system, then
  - the set of replicas used for writing always overlaps with the set of read replicas:
  - It is impossible to read an object without retrieving the latest written object version in some of the replicas read.
- Latency is determined by the slowest node in the R (or W) set.
  - Therefore, to improve performance, one lowers R or W.

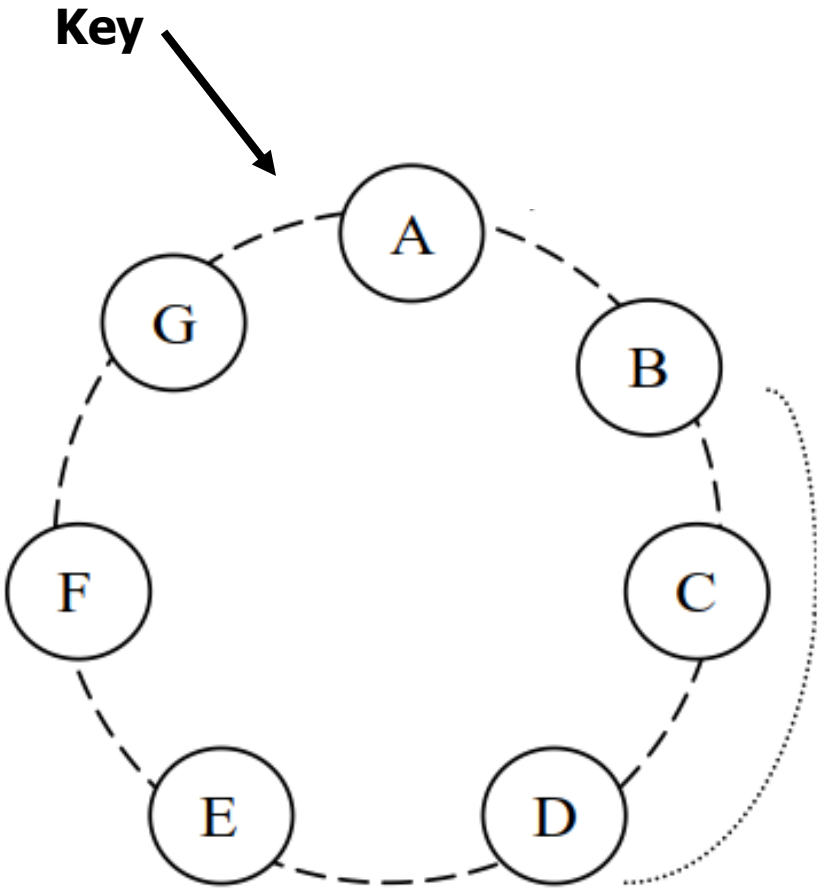
Coordination: Sloppy Quorum

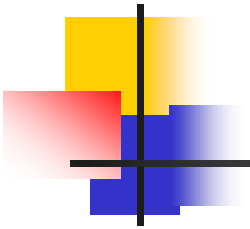
- To ensure availability (i.e. never block writes),
  - Dynamo uses a *"sloppy quorum".*
- Each data item is stored on N nodes:
  - of list spanning multiple machines and data centers (*preference list*).
- Operations are performed:
  - not on the N existing replicas but on…
  - the first healthy N nodes on the preference list.

Tolerating Temporary Faults: *Hinted Handoff*

- Assuming N = 3.
- If A is unavailable or fails when we write, send a replica to D.
- D marks the replica as temporary and returns the data to A as soon as it recovers.

- Replicas are chosen from a preference list of nodes.
- Preference lists always span multiple datacenters for fault tolerance.



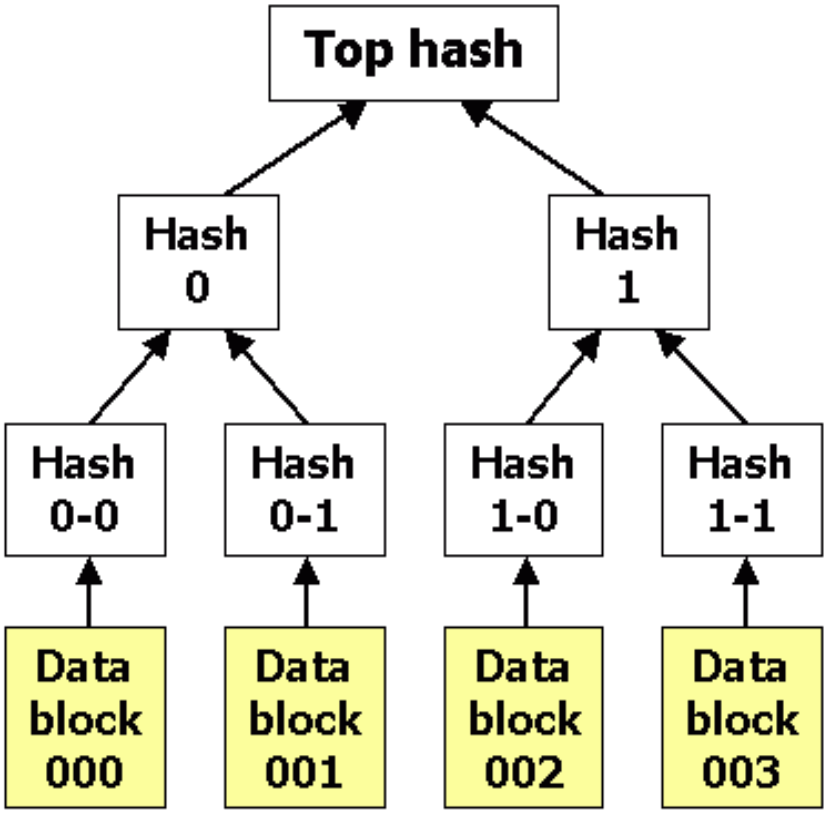


Handling Permanent Faults

- When a hinted replica (that has write-ops belonging to another replica) is considered permanently failed:
  - Data is synchronized with the new replica using Merkle trees.

Convergence: Merkle Trees

- Accelerates synchronization between nodes by comparing trees of hashes.
- very easy and fast to identify what needs to be exchanged.
- Each node has hash of children.
- Replicas exchange top periodically.
- recursively exchange hash of lower nodes.
- until modified blocks are found and sent
- The update can be asynchronous:
  - an out-of-date tree is not a serious issue.



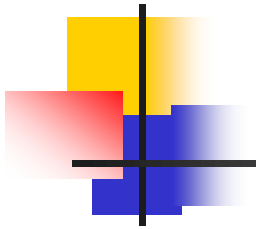


Table Storage System: Amazon DynamoDB

DynamoDB

- Hierarchical data storage.
- Based on Dynamo.
- Adds multiple attributes, indexing and queries.
- *Ad hoc* data storage:
  - No schema.
  - Automatically scalable.
  - Efficient for data where read operations dominate
    - due to eventual consistency.
    - e.g. forums, metadata, backups.
    - if conflicts are too common,
      - system becomes inefficient.
- *Missing features:*
  - No ordering, no joins.

DynamoDB: Data Model

- Table:
  - Identified by a string.
- Item:
  - Identified by a key.
  - Unlimited number per table.
  - Indexed by a hash key
    - (for unique selection in queries)
  - Additionally a range key can be added
    - for range queries.
  - limited to 400KB
- Attribute:
  - Presented like columns.
  - In practice are <key, value> pairs in Dynamo object.

DynamoDB: Queries

- **Select**: allows querying the domain for specific hashes or ranges within hashes.

- Supports conditional modifications/updates:
  - Boolean: ATTRIBUTE_EXIST, CONTAINS e BEGINS_WITH
  - Comparison: =, <>, , =, BETWEEN e IN
  - Logic: NOT, AND e OR.

- Supports atomic increment and decrement of scalar values.
  - work as single-row transactions (read, modify, update)

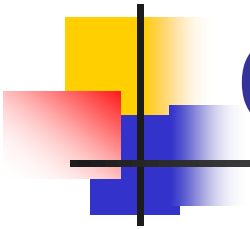


Cloud Storage (Part B)

Prof. Luís Veiga
IST / INESC-ID Lisboa

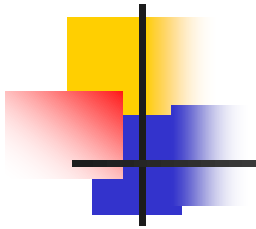
https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© LV, JG 2015-24

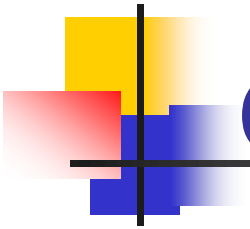


Contents

- Cloud File Systems: GFS/HDFS
  - architecture, operations, fault-tolerance
- Table storage: Big Table
  - overview, data model, operations,
  - architecture, tablets, updates, fault-tolerance
- Table Storage Systems:
  - Google Datastore, transactions with Datastore/Megastore
- *Reference:*
  - Azure storage: overview, blobs, Azure tables
  - summary of cloud storage

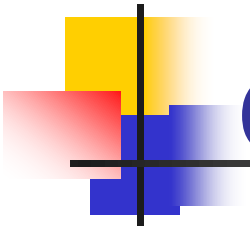


Cloud File Systems



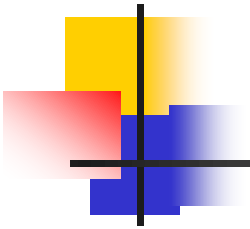
Cloud File System: HDFS/GFS

- (Very Large-Scale) Distributed File System
  - Reimplementation of the Google File System (GFS).
  - Runs on clusters of generic machines.
- HDFS is tuned for:
  - Very large files.
  - Streaming (sequential) access.
  - Most writing is append-based.
  - Generic hardware.
- Key to Scalability:
  - data operations don't go through the central server.



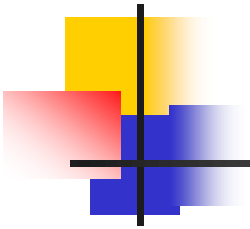
Cloud File System: HDFS/GFS

- Architecture
  - Block structure
  - Namenode, Datanode
- Operations
  - Reading, Writing with fault-tolerance



Blocks

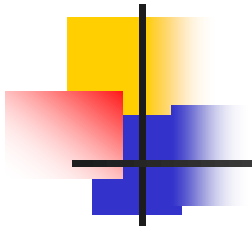
- Simplify space management:
  - allocation and replication
  - a file may grow almost indefinitely.
- Evolution:
  - Disk blocks: 512 bytes
  - File system blocks: 2,4,8 kB
  - HDFS blocks: 64MB
    - To eliminate seek steps: contiguous 64MB.
    - A file smaller than one block does not occupy 1 block.
      - smaller local FS file is used



Namenode

- Manages the file system name space:
  - folder hierarchy, name uniqueness,…
- Maintains the folder tree and the metadata in 2 files:
  - *namespace image* and *edit log*.
- HDFS cannot operate without the *namenode*.

- Files can be written, read, renamed and deleted.
- It is not possible to:
  - write in the middle of the file.
  - write concurrently to the same file.
- Fault tolerance mechanism:
  - atomic replication to another machine.



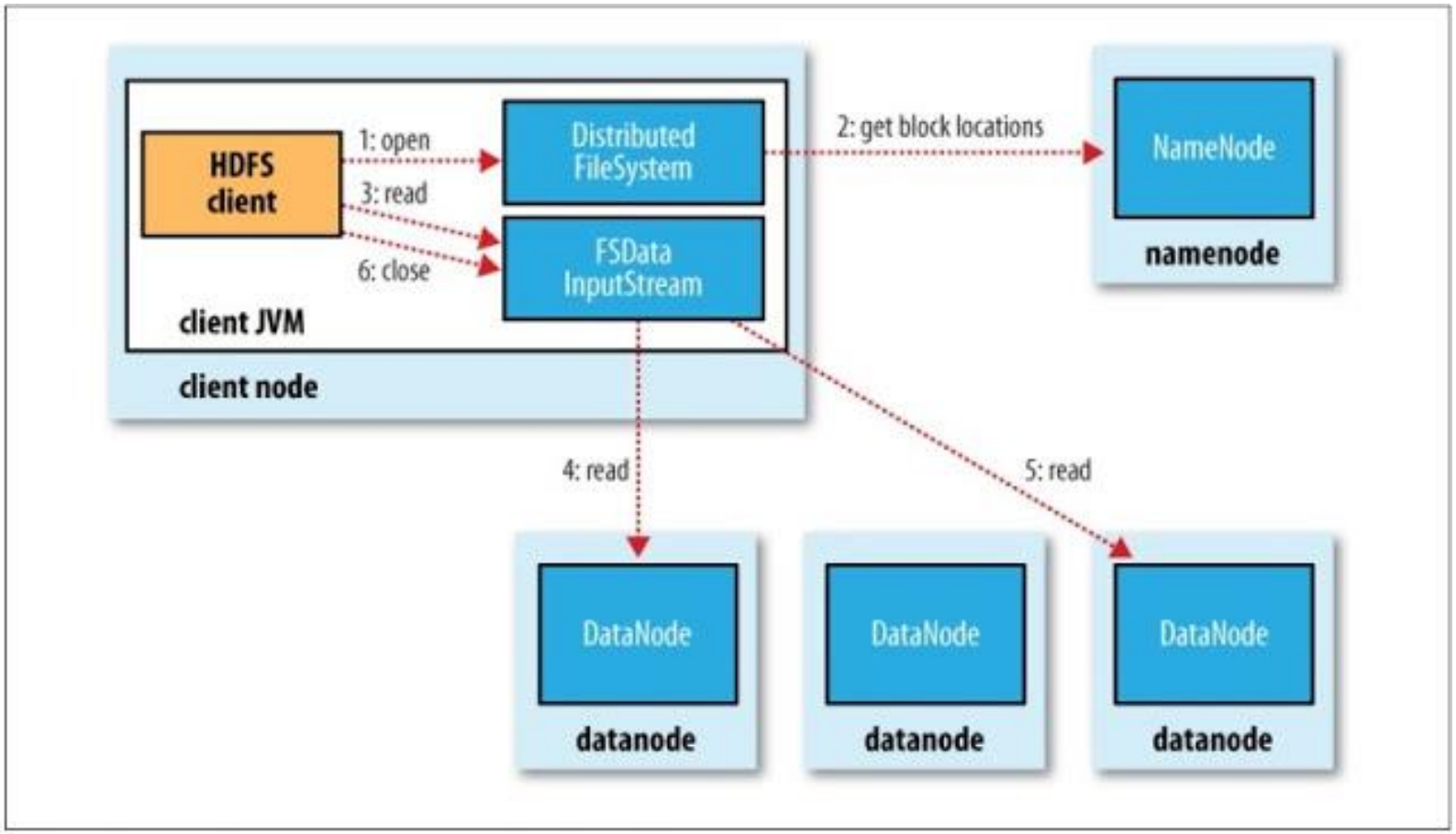
Datanode

- Manages a set of blocks.
- Process clients' or *namenode's* requests:
  - reading and writing, management.
- Periodically:
  - notifies the *namenode* of the blocks it holds
- Block replication management:
  - when replica number drops below configured value:
    - then, a new replica is created.

Reading

- Client contacts the *namenode* to:
  - get list of *datanodes* with the file's blocks
    - (stored in memory).
  - selects/prefers *datanodes* located in same:
    - node, rack, data center, other DC.
- Receives a `FSDataInputStream` that:
  - opens and closes connections to *datanodes*,
  - requests blocks from the *datanodes*,
  - repeats operations when necessary
  - logs failed *datanodes*.

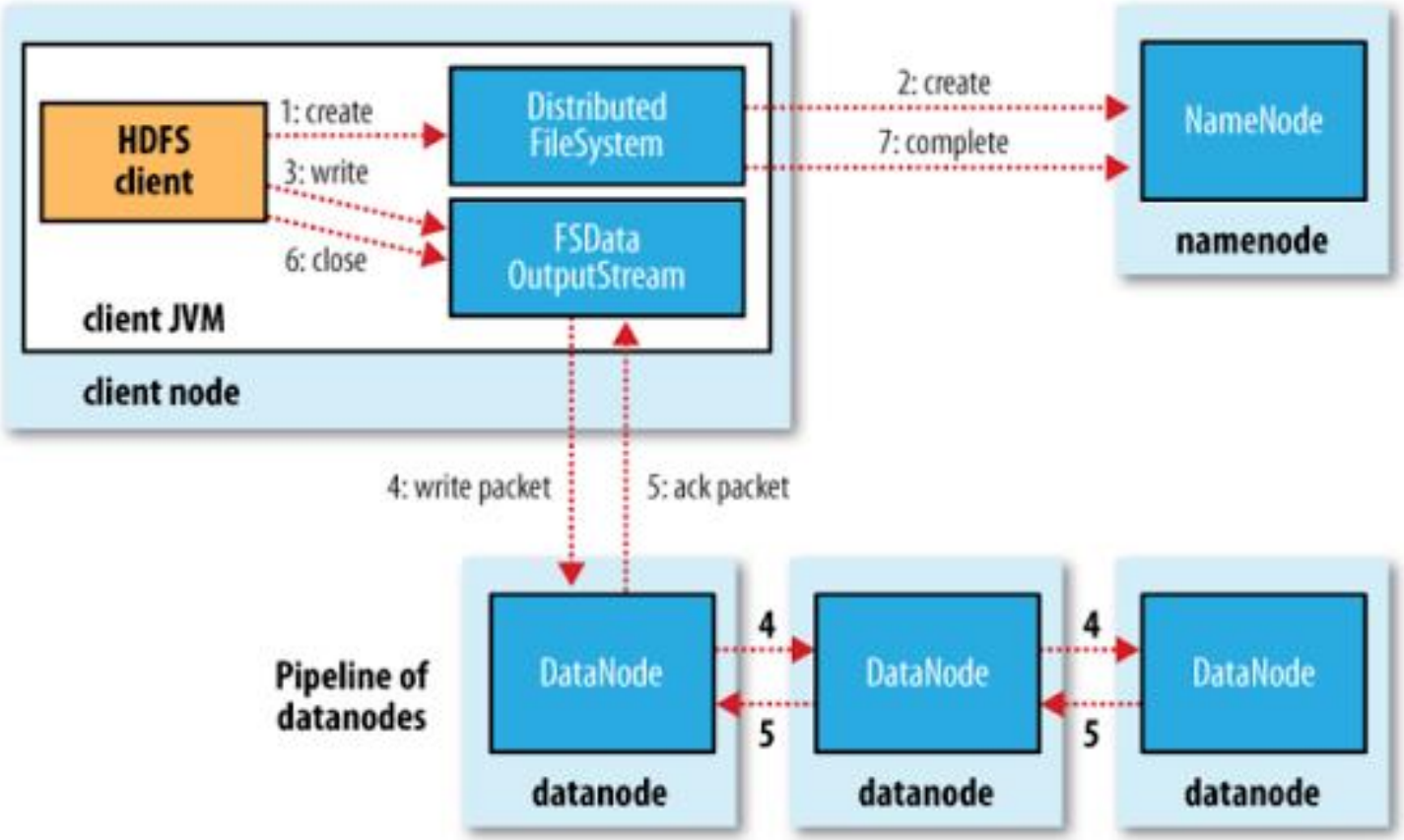
Reading


HDFS client
1: open
3: read
6: close
Distributed FileSystem
FSData InputStream
client JVM
client node
2: get block locations
NameNode
namenode
4: read
5: read
DataNode
datanode
DataNode
datanode
DataNode
datanode

Writing (+ creating)

- Client requests a new file to the *namenode* checking permission and uniqueness.
  - If it succeeds, it receives a `FSOutputStream`.
- *Namenode* provides:
  - a set of *datanodes* for replication.
    - *chain replication* employed.
- Block write requests are kept in a *data queue*.
- Unconfirmed block write requests are kept in an *ack queue*.

Writing


HDFS client
1: create
3: write
6: close
Distributed FileSystem
FSData OutputStream
client JVM
client node
2: create
7: complete
NameNode
namenode
4: write packet
5: ack packet
Pipeline of datanodes
DataNode
DataNode
DataNode
4
5
4
5
datanode
datanode
datanode

Writing (fault-tolerance)

- In case the *datanode* fails:
  - the client changes the block ID
    - so that the corrupted replica is deleted later.
- By default:
  - if one of the replicas is successfully written:
    - the writing is considered done.
    - other replicas are written to asynchronously.

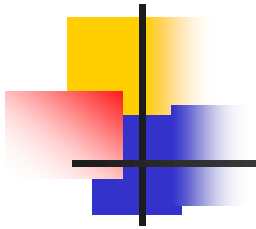
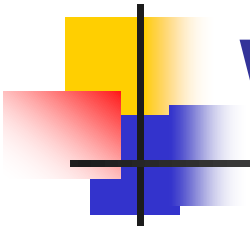


Table Storage Systems: Google BigTable



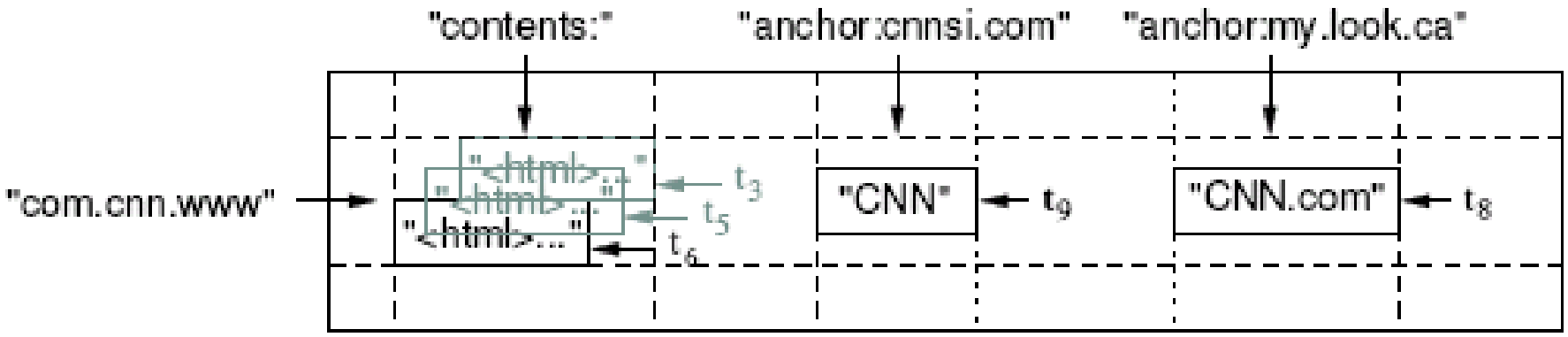
Why Cloud Table Storage?

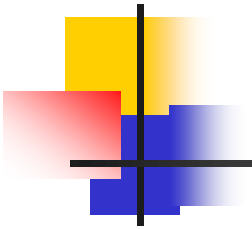
- Motivation:
  - The concept of a record (a row) is essential for most object representations.
  - Having an indexed data structure is essential for queries.
  - Data model matches well DB concepts.

  - **But , still,**
  - **ACID transactions on a conventional relational model do not scale.**

Google BigTable

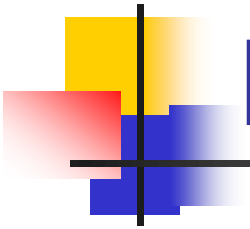
- Google large-scale replicated table storage service.
  - used, e.g., for crawling the entire web.
  - Google Datastore is based on BigTable.
  - stores all persistent state on top of Google FS (~HDFS).





Data Model

- A **table** has **entities** (records, rows,…).
- A table has **column families**:
  - Created statically (e.g. family = “anchor”)
  - A column family is like a property.
- Each column family has **columns**:
  - Created dynamically (e.g. column= “anchor:cnnsi.com”)
  - A column is an instance of a property.
- Each column is **time stamped**:
  - An entity can have several versions with different timestamps.
  - Stored in reverse chronological order (newer first!).

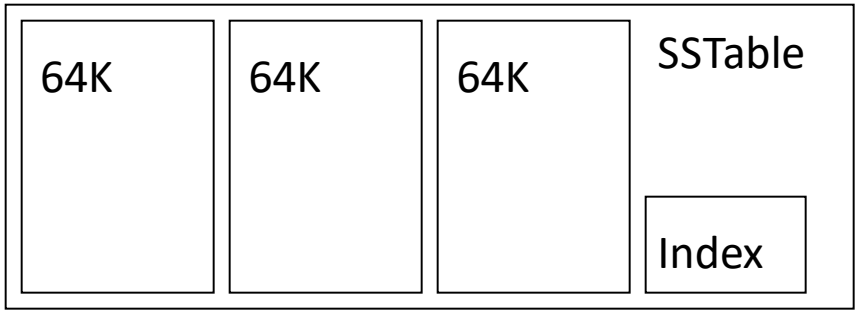


Entities

- Entities are ordered **alphabetically**.
- Entities can be:
  - written, deleted,
  - read as:
    - single row (single key selection),
    - scanned (key-prefix selection),
    - range-scanned (key-range/interval selection)
- ACID transactions only on a single entity.
- Sequences of **entities** are stored in **tablets**.
- The choice of key determines which entities are nearby in the tablet!!

Arch.: Tablets

- Each tablet contains:
  - a contiguous sequence of entities.
- Tablets are stored in:
  - SSTable files within GFS
  - (or HDFS in Hadoop / HBase).
- SSTable, file format:
  - Index of <key, value> pairs + 64kB data blocks.



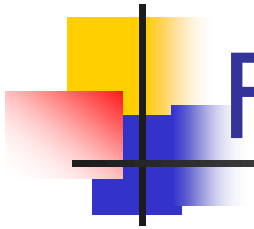
Arch.: Components

- Client application uses library to interact w/service.
- *Master* performs operations on:
  - Tables: creation.
  - Column families: creation.
  - Startpoint for locating of entities (rows) and ranges
  - Tablets: allocation, elimination,
    - load balancing tablets among tablet servers.
- *Tablet Servers*:
  - performs reading, writing directly with clients
  - partitioning for tablets that are too large (> 200MB).
    - employing tree with of levels of **metadata tablets**.
    - leaves are tablets with actual **data tablets**.

Updates (Writing)

- Writing is done on two tablet server structures:
  - In a redo log (one per server) and
  - In a memory temporary table, memtable.

- Compaction of the memtable into SSTables:
  - Minor: when the memtable reaches a certain size, it's converted into an SSTable.
    - Reduces log and memory occupation.
  - Merge: Groups SSTables from minor compactions.
  - Major: Converts SSTables to a minimum by filtering entity removals (no longer including removed entities).



Fault Tolerance

- At startup *master*:
  - checks list of live servers
  - compares tablets they claim to have with metadata
  - updates tablet distribution.

- Every time a tablet server dies:
  - the master is notified.

  - lock of tablet server with the tablet metadata has expired.

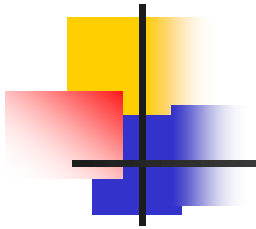
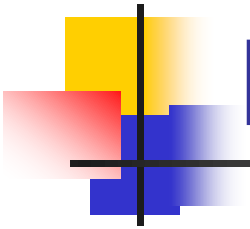


Table Storage Systems:

- Google Datastore/Megastore

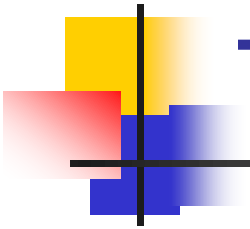
Datastore: BigTable for Programmers

- NoSQL table storage over BigTable:
  - Record = entity.
  - Each entity is accessible by a single application.
  - Each entity has a system unique ID that includes:
    - app ID and *kind* ID.
  - *Kind* is a namespace for entities.
  - Each entity has an app-wide ID.
  - Data items are stored in the entities columns.
  - Columns have a name and a value.



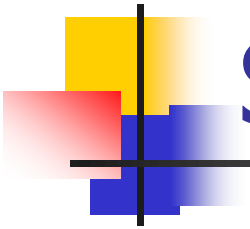
Datastore: DB-like support

- An intermediate layer (**Megastore**):
  - executes queries
  - builds indexes:
    - kind index
    - single-property index
    - composite index (creation before app run)
  - performs multi-record transactional support.



Transactions

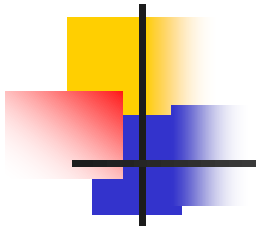
- BigTable supports only transactions on one entity.
- Datastore supports transactions on several entities.
- Datastore transactions (originally limited):
  - transactions can only operate on one server
    - (i.e. on entities of the same *entity group*).
  - no distributed transactions.
    - (due to the usual relational DB scalability issues…)
  - cancelled at the end of the servlet if not confirmed
    - (recall: GAE is web app oriented).



Summary

- Cloud File Systems: GFS/HDFS
  - architecture, operations, fault-tolerance
- Table storage: Big Table
  - overview, data model, operations,
  - architecture, tablets, updates, fault-tolerance
- Table Storage Systems:
  - Google Datastore, transactions with Datastore/Megastore

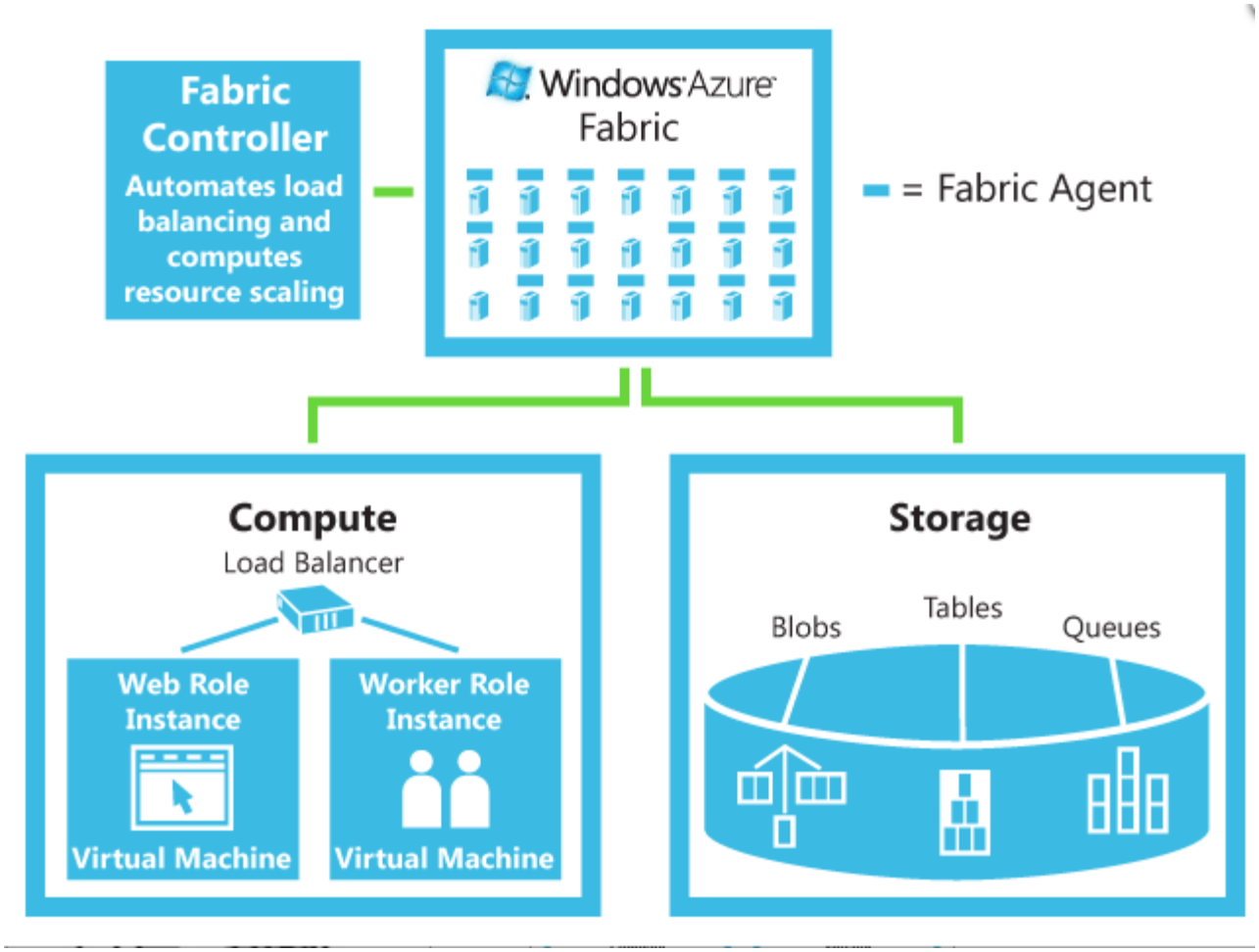
- *Reference:*
  - Azure storage: overview, blobs, Azure tables
  - summary of cloud storage



(Reference)

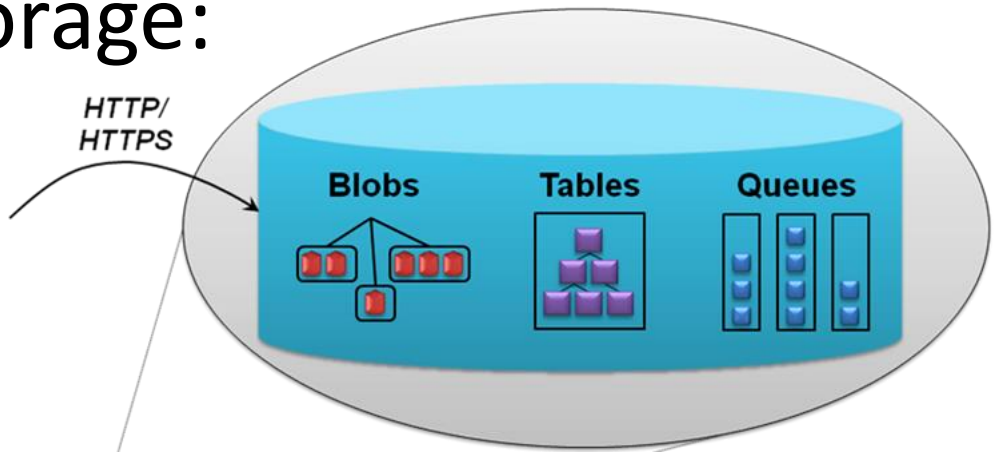
Azure Storage Overview

Windows Azure



Azure Storage: Overview (1)

- Windows Azure Storage:
  - Blobs (objects)
  - Tables



- Service is accessible via Web Services
  - or libraries on top of these (C#, VB, Java).

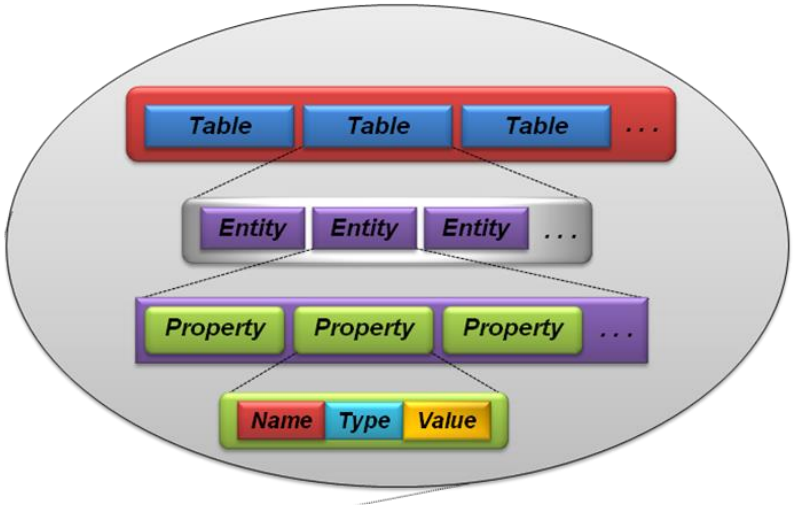
- Partition and Replication
  - Partitions (<= 1GB) used to store and shard data
  - Replication factor of 3

Azure Storage: Overview (2)

- Blobs:
  - a blob is a <name, object> pair as in Dynamo/S3.
  - allows storage of objects up to 50GB.
- Operations (atomic):
  - Put: creating
  - Get: reading
  - Set: updating
  - Delete: eliminating
  - *Lease: 1 minute locking.*

Azure Storage: Overview (3)

- Azure Tables:
  - Similar to BigTable



- Storage hierarchy:
  - Table -> Entity -> Property ⍰ <name, type, value>
- Integration with .NET languages (e.g., C#)
- Optimistic concurrency control.
- Entities have two keys:
  - RowKey: unique identification (e.g. ID card)
  - PartitionKey: to drive partitioning (e.g. last name)

Cloud Storage: Review Map

	AWS	Microsoft Azure	Google / Hadoop
Tables	DynamoDB	Tables	Datastore [BigTable] / HBase Percolator
Objects/Blocks	S3	Blobs	(GFS) / HDFS
Queues	Simple Queue Service (SQS)	Queues	Task Queues Google PubSub



Big-Data Processing (MapReduce)

Prof. Luís Veiga
IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© LV, JG 2015-24



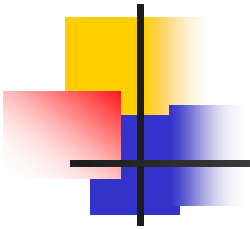
Agenda

- Map Reduce Paradigm:
  - What is it?, concepts
  - Examples:
    - WordCount,
    - PageRank
- Map Reduce Platform:
  - Architecture
  - Data splitting
  - Task Management
  - Resource Management, Scheduling
  - Fault-tolerance

MapReduce: What is it?

- Typical Challenges in Parallel and Distributed Processing:
  - How to assign tasks to the workers?
  - What if we have more tasks than workers?
  - What if the workers need to share partial results?
  - How do we aggregate partial results?
  - How do we know whether the workers have finished?
  - What if the workers fail?

- One solution: MapReduce
  - Functional Programming + Distributed Processing
  - Combined in a single distributed platform



Origins: Functional Programming

- An old idea from the ’50s.
- What is functional programming?
  - Computing as the composition/sequencing of a set of functions.
  - The theoretical foundations are lambda calculus.
- What is the difference to imperative programming?
  - The concepts of data and instructions are blended.
  - The flow of data is implicit.
  - Execution order less relevant in the program design.
    - promoting parallelism and easing concurrency

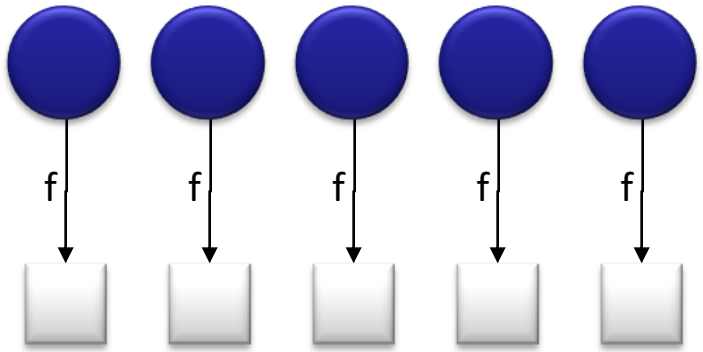
M/R and functional programming

- What does it have to do with MapReduce?
  - Functional languages, e.g. Lisp and Scheme are strongly modelled on list processing.

- They use two basic concepts from functional programming:
  - *Map* : apply the same operation to all the elements in a list.
  - *Fold* : use an operator to combine all the elements of a list.

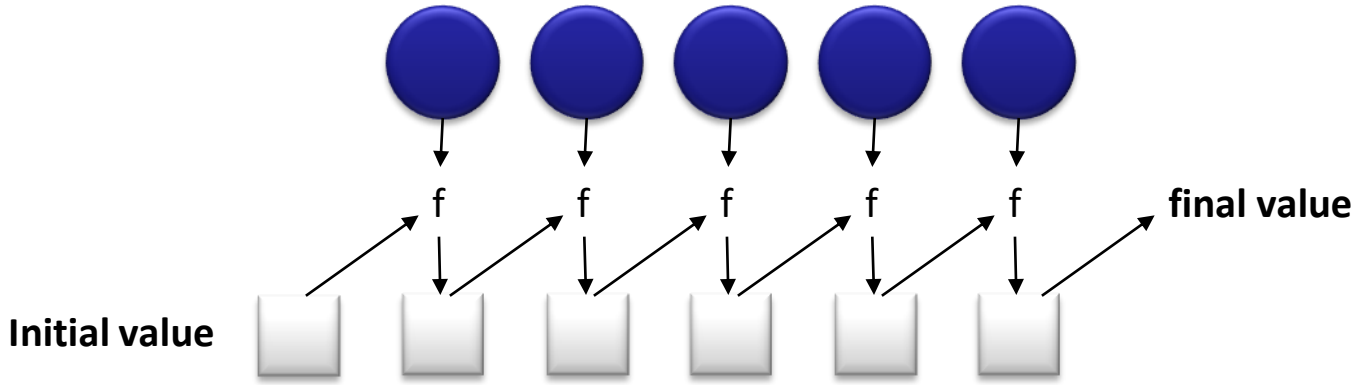
Map

- *Map* is a second order function:
  - It receives another function as a parameter.
- It works by:
  - Applying the *parameter function* to each and every one of the *elements* of a list.
  - Thereby generating a *new list*.



Reduce

- *Fold (Reduce)* is also a second order function.
- It works by:
  - Initializing an *accumulator*.
  - Applying the *parameter function* to the *accumulator* and the *first element* of the list.
  - The result is stored in the accumulator.
  - The operation is repeated for each element of the list.
  - The result is the final value of the accumulator.



Map/Fold Example

- Simple *map* example:

```
(map (lambda (x) (* x x))
     '(1 2 3 4 5))
 → '(1 4 9 16 25)
```

- Simple *fold* example:

```
(fold + 0 '(1 2 3 4 5)) → 15
(fold * 1 '(1 2 3 4 5)) → 120
```

- Sum of squares:

```
(define (squares-sum v)
  (fold + 0 (map (lambda (x) (* x x)) v)))
(squares-sum '(1 2 3 4 5)) → 55
```

MapReduce

- *Map+fold* over lists of `<key, value>` pairs.

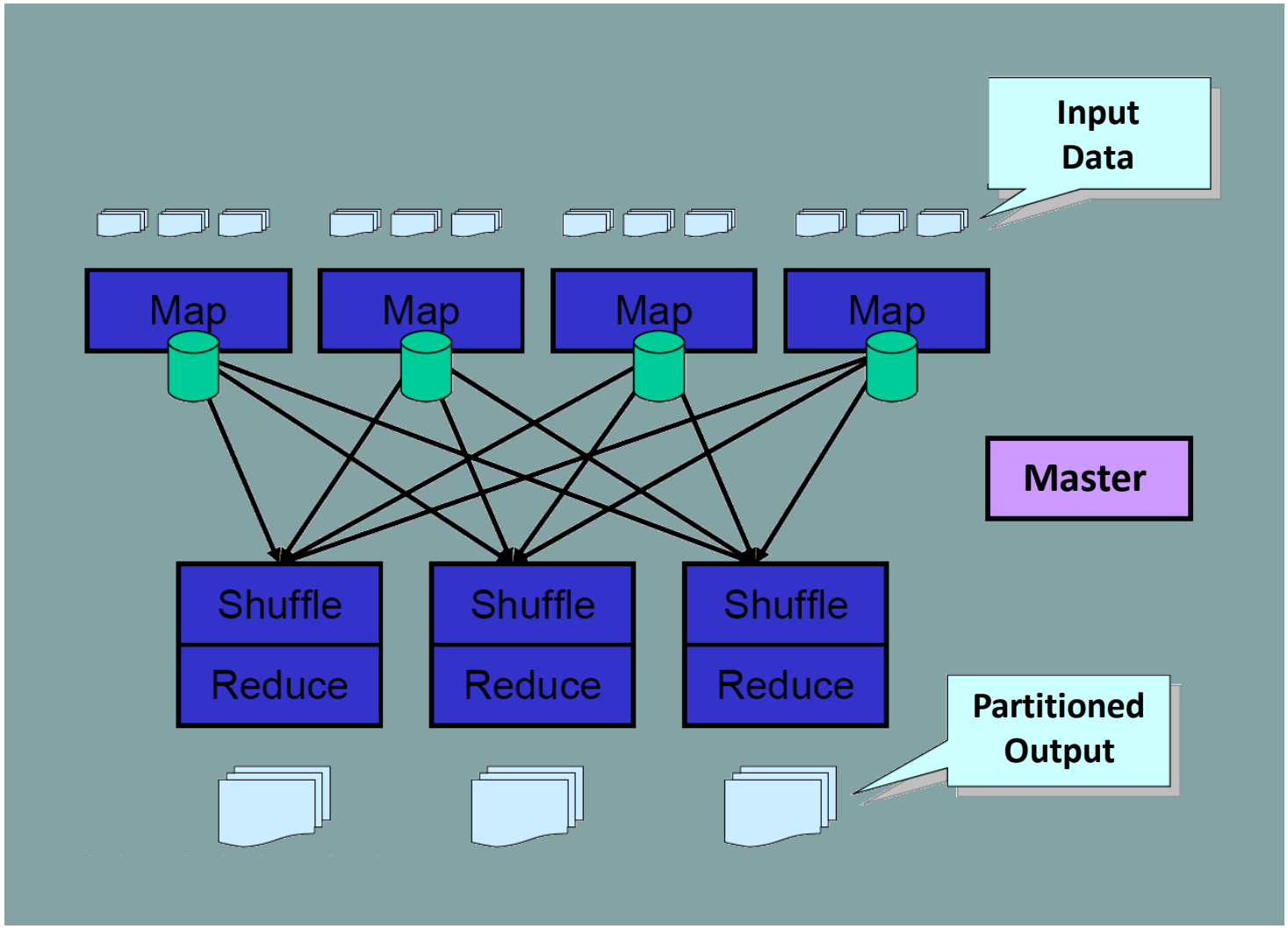
- *Map*:

  operates on `<key1, value1>` pairs resulting in lists of `<key2, value2>` pairs.

- *Reduce (fold)*:

  receives all `<key2, value2>` for a specific key2 and generates `<key3, value3>` pairs.

MapReduce


Input
Data
Map
Map
Map
Map
Master
Shuffle
Reduce
Shuffle
Reduce
Shuffle
Reduce
Partitioned
Output

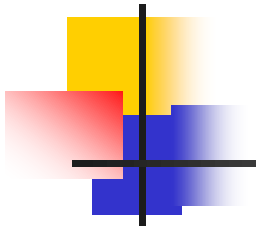
M/R - Real Examples

- Web crawling and ranking
- Distributed search (logs, grep, including web)
- Web page access statistics
- Reverse web link graphs
- Inverted word index for web search
- Implementing scalable learning algorithms.
- Graph algorithms:
  - e.g. PageRank, *travelling salesman*.
- Detecting similarities in large sets of images.

Hadoop: Map/Reduce

- Hadoop
  - Apache project reimplementing Google's cloud components, e.g. MapReduce.

- Examples:
  - Word Counting
    - the *HelloWorld* of big-data processing
  - PageRank
    - Google original innovative algorithm



Wordcount

Wordcount: Map

```
public class Map extends MapReduceBase implements Mapper {
  private final static IntWritable one =
    new IntWritable(1);
  private Text word = new Text();

  public void map(WritableComparable key, Writable values,
                  OutputCollector output, Reporter reporter)
    throws IOException {
    String line = values.toString();
    StringTokenizer tokenizer = new StringTokenizer(line);
    while (tokenizer.hasMoreTokens()) {
      word.set(tokenizer.nextToken());
      output.collect(word, one);
  }}}
```

Wordcount: Reduce

```
public class Reduce extends MapReduceBase implements
   Reducer {

   @Override
   public void reduce(WritableComparable _key,
     Iterator _values, OutputCollector output,
     Reporter reporter) throws IOException {
      Iterator<IntWritable> values =
        (Iterator<IntWritable>) _values;
      int sum = 0;

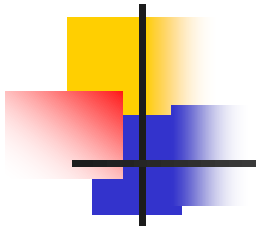
     while (values.hasNext()) {
        sum += values.next().get();
      }
      output.collect(_key, new IntWritable(sum));
   }}
```

Wordcount: Main

```
public static void main(String[] args) {
 Configuration conf = new Configuration();
    String[] otherArgs = new GenericOptionsParser(conf,
   args).getRemainingArgs();
    if (otherArgs.length != 2) {
      System.err.println("Usage: wordcount <in> <out>");
      System.exit(2);
    }
    Job job = new Job(conf, "word count");

    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(otherArgs[0]));
    FileOutputFormat.setOutputPath(job, new Path(otherArgs[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

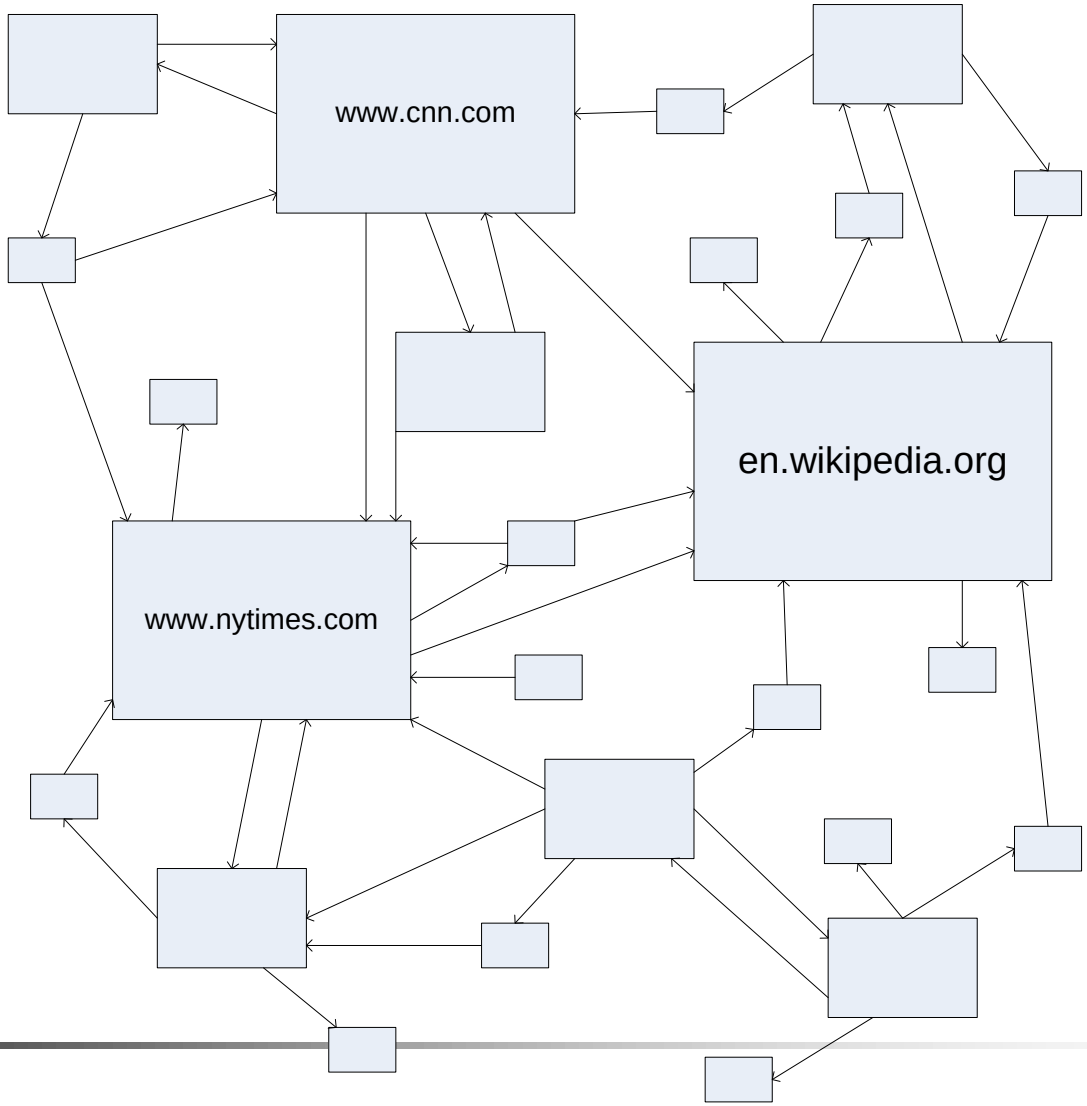


PageRank Algorithm

PageRank: Random Walks in the Web

- PageRank is Google's famous web indexing algorithm.
- Rationale: If a user starts on a random web page and starts clicking what is the probability that she will reach a given page?
- High probability signals an important page.
- This is the basic principle of *PageRank*:
  - More "pointed to" pages get more points.

PageRank: Visually



PageRank: Formula

Given a page A, and other pages T_1 up to T_n with links to A, PageRank is defined as:

$$PR(A) = (1-d) + d\ (PR(T_1)/C(T_1) + \ldots + PR(T_n)/C(T_n))$$

C(P) is the cardinality of P (number of outbound links)

d is called the dampening of the randomness of the page choice (normally 0.85).

It is the probability of choosing a page without coming from a link.

PageRank: Intuition

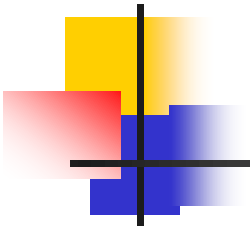
- The calculation is iterative: PR_{i+1} is based in PR_i
- Each page distributes its PR_i evenly to all the pages it is linked to.
- The pages that receive fragments of PR add all the received fragments to generate their PR_{i+1}

PageRank: Challenges

- Will PageRank converge?
- How fast?

- What is the correct value of d and how sensitive is the algorithm to this value?

- What is the most efficient algorithm for PageRank?



PageRank: First Implementation

- Create two tables (current and next) with the PageRank of each page.
- Fill the current table with the initial values of PR and the adjacent pages.

- Traverse the graph distributing current PR among the next PR elements.
- current := next; next := new_table ();

- Iterate again or stop.

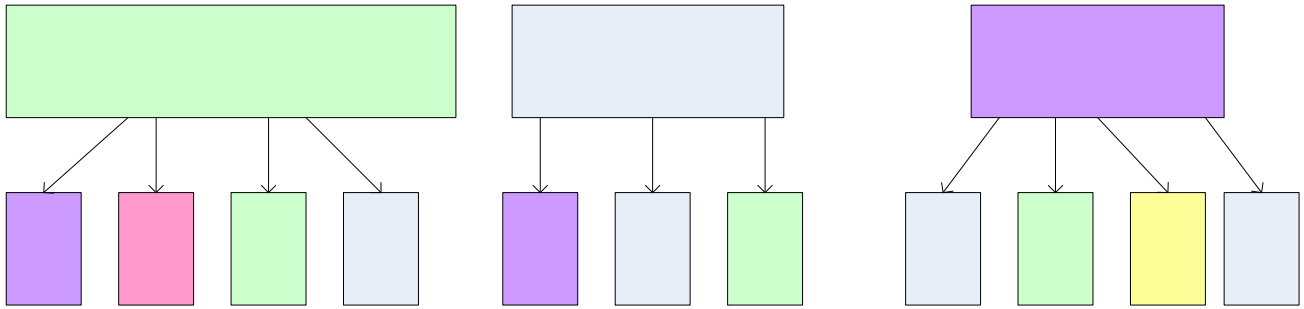
Algorithm Distribution (1)

- Parallelization intuition:
  - Each line of the next table depends on the current table but not on other lines of the next table.
  - Individual lines of the adjacency tables, the tables of neighbours, can be processed in parallel.
  - The lines in sparse matrices are relatively small compared to the number of nodes (pages).

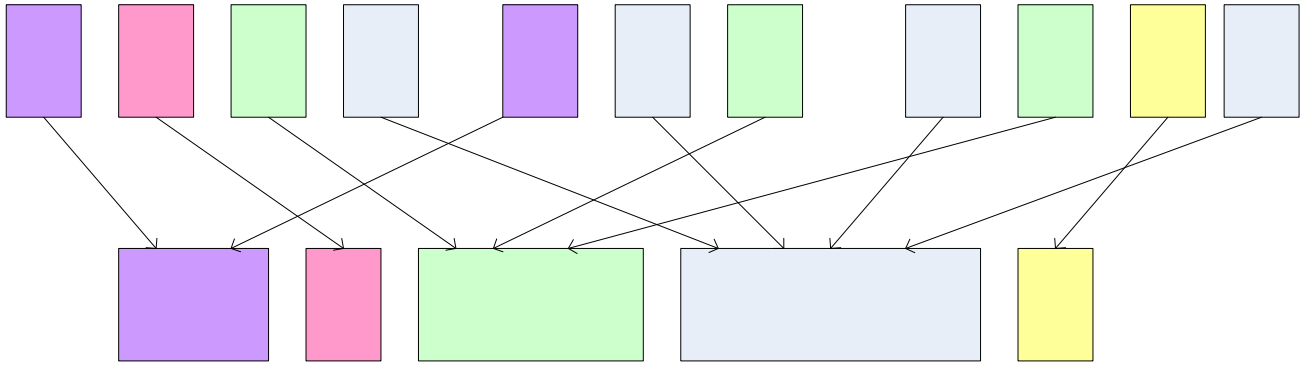
Algorithm Distribution (2)

- Consequences of this approach:
  - We can *map* each line of the current table thereby generating PR fragments to be distributed to all pages pointed by the page.
  - Fragments can be *reduced* (added) to a single value of PR.

Map step: break page rank into even fragments to distribute to link targets



Reduce step: add together fragments into next PageRank



Iterate for next step...

Phase 1: Reading the HTML

- Map task reads (URL, page-content) and transforms it into (URL,(PR_{init}, url-list))
  - PR_{init} is the initial PageRank value for the particular URL.
  - url-list contains all the pages to which the page points.

- In this iteration, the reducer is the identity function (pass-through reducer).

Phase 2: PageRank Distribution (i)

- Map reads (URL, (current_pr, url-list))
  - For each u in url-list, output(*u*, current_pr/|url-list|)
  - Output also (URL, url-list) to pass the graph arcs to the next iterations.

$$PR(A) = (1-d) + d\ (PR(T_1)/C(T_1) + \ldots + PR(T_n)/C(T_n))$$

Phase 2: PageRank Distribution (ii)

- Reduce receives (URL, url-list) and many (URL, PR_frag):
  - Adds all vals and adjusts the calculation with d.
  - Generates (URL, (new_PR, url-list))

$$PR(A) = (1-d) + d\ (PR(T_1)/C(T_1) + \ldots + PR(T_n)/C(T_n))$$

- A non-parallel component decides whether the algorithm has converged.
  - (fixed number of iterations?)
  - (while fraction of pages with PR modified beyond some treshold)
  - (until all pages w/ PR modified below treshold)

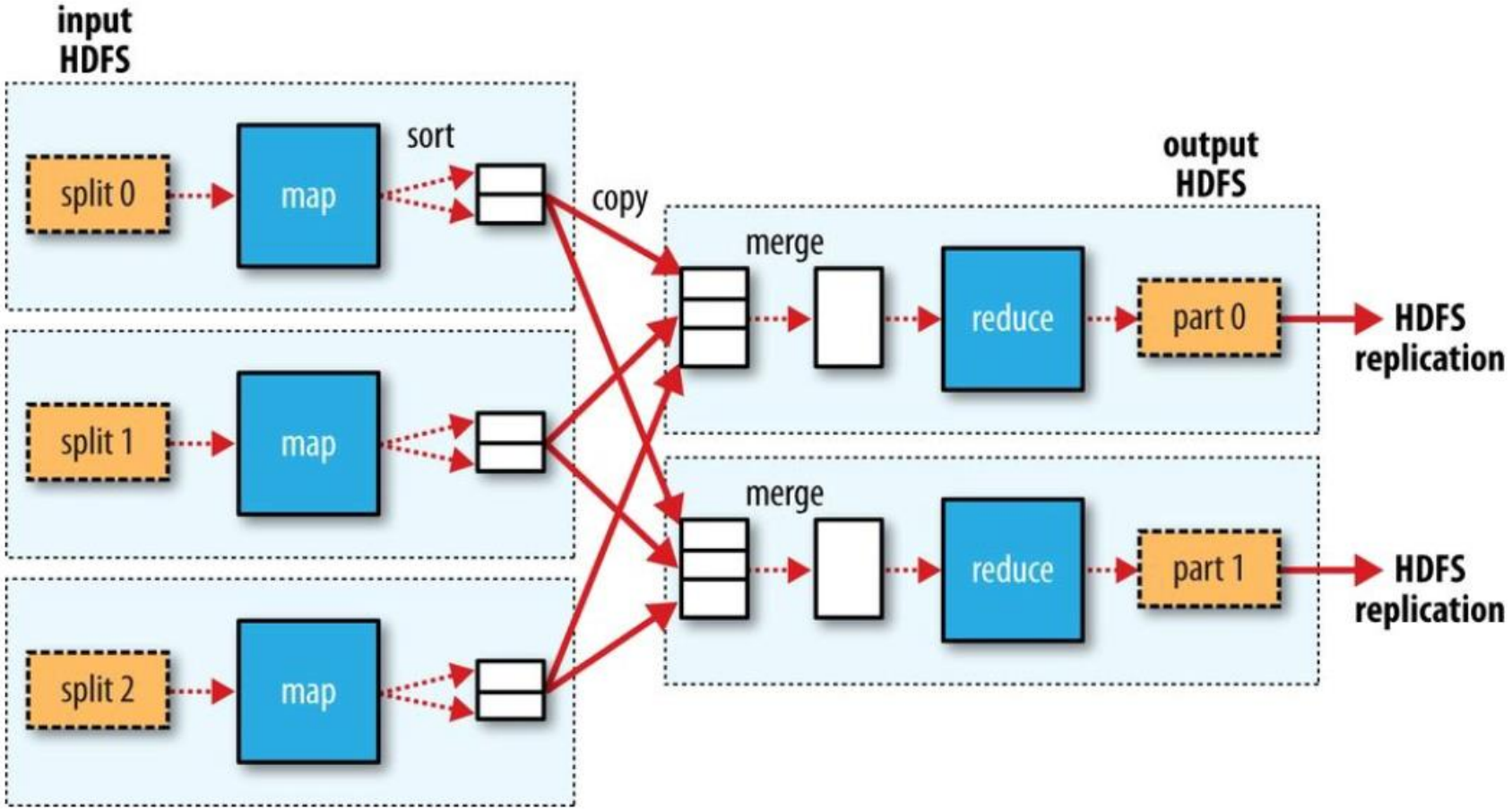
Recap M/R - Original Challenges…

- Scheduling:
  - How to assign *map* and *reduce* to the workers.
- Distribution:
  - How workers are moved to the data.
- Synchronization:
  - Gathering, ordering and distributing intermediate results.
- Fault Tolerance:
  - Detection and restart of failed tasks.

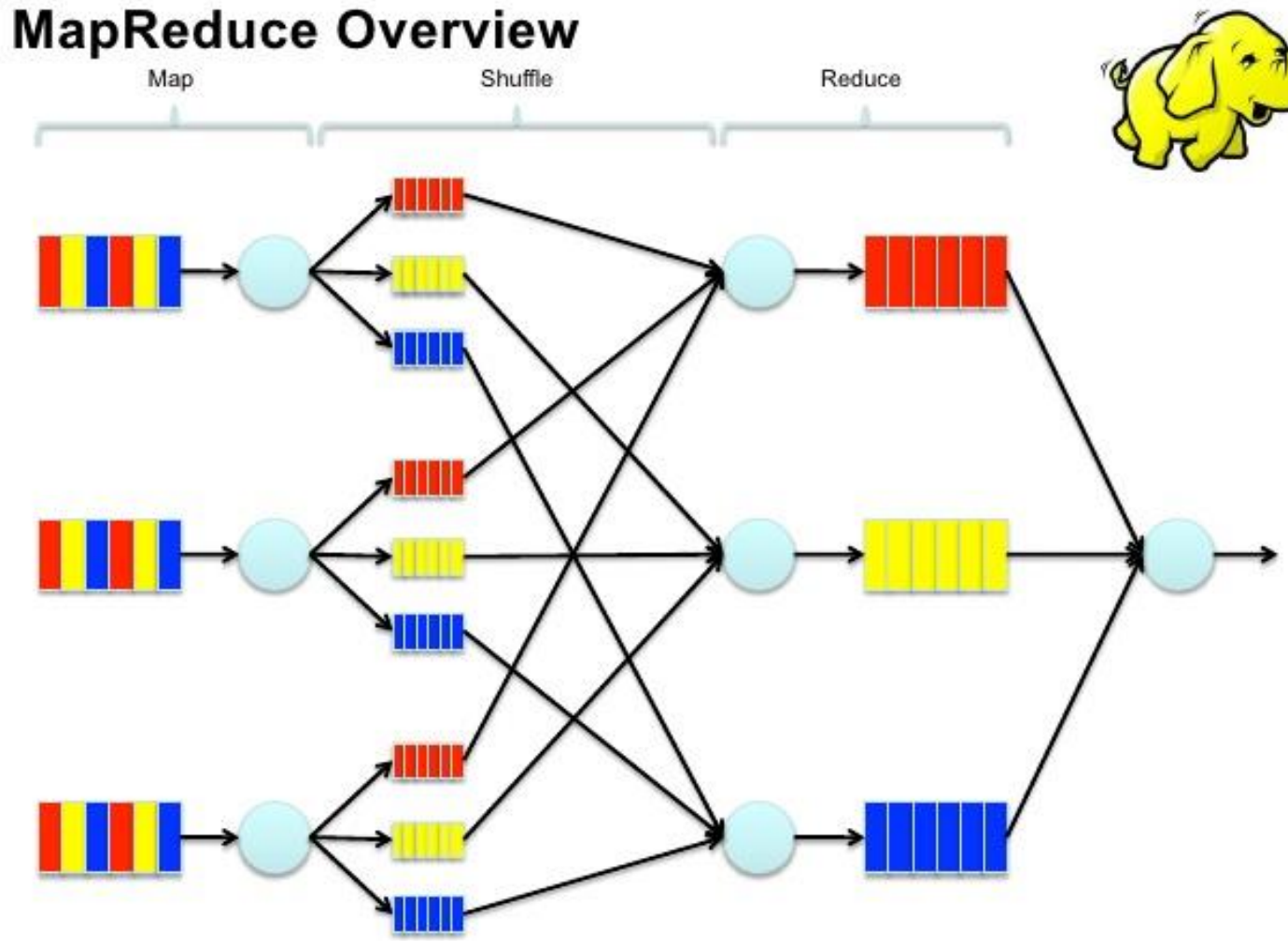
Map Reduce Platform

- Architecture
  - M/R workflow, master, workers
- Data splitting
- Task Management
  - number of tasks, mappers, reducers,
  - intermediate data partitioning,
  - optimizations: combiners
- Resource Management
  - Job tracker, task tracker
  - Scheduling,
- Fault Tolerance
  - handling faults

Architecture: M/R workflow



Architecture: M/R workflow



Architecture (1)

- Execution is controlled by the **master** process:
  - Input data are split into 64MB blocks.
  - Tasks are assigned to worker processes dynamically.
  - In the *reduction* phase there are as many calls to reduce as the output files of the map phase
    - (but only 1 reducer process by default).
  - A typical Google run: *Mappers*=200,000; *Reducers*=4,000; Workers=2,000.

- The master assigns each map task (1 split) to one **worker**:
  - Worker reads the input from closest location.
    - e.g. data local (ideal), rack-local, off-rack (worst option)
  - produces R local files with `<key,value>` pairs.

Architecture (2)

- Master assigns each reduce task to a reducer:
  - The reducer reads the intermediate files with key pairs from the mappers.
  - The reducer orders the `<key,value>` pairs and applies the reduce function.

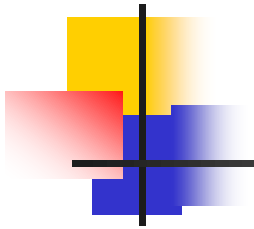
- The user may specify partitions to control which values each reducer gets.

Architecture (3)

- Shuffling:
  - i.e., sort, copy, merge
  - input to the Reducer is the output **sorted** by the mappers.
  - the framework **fetches (copy)** the relevant partition of the output from all the mappers
  - fetch and merge are overlapped for efficiency
    - while map-outputs are being **fetched** they are **merged**.

Data Splitting *(Splits)*

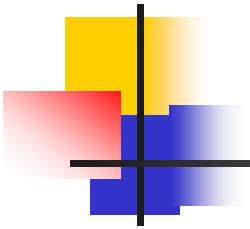
- Input data provided to mappers in ***splits***.
  - A split is a block of input data.
  - HDFS blocks are 64MB big (by default).
- In particular, Hadoop tries to create splits containing data local only to one node.
  - otherwise, on same rack preferably.
- A mapper then runs on the node where the *split* was created.



Task Management

- The number of mappers is determined based on size of input data.
- By default there is only 1 reducer process.

- But both are configurable:
  - `conf.setNumMapTasks(int num);`
  - `conf.setNumReduceTasks(int num);`



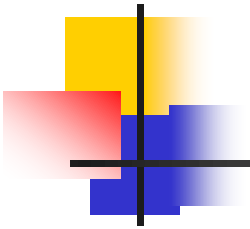
TM : Partitions

- Partitioners partition the intermediate data before being submitted to the reducers.
- By default, the partition is determined by the `hashCode` function of the intermediate (map output) key class.
- You can change it by reimplementing the key's `hashCode()` method or by replacing the partitioner with a new one.

TM : Optimization (Combiners)

- Reduce amount of data at mappers output by aggregating intermediate results from one node.
- E.g., in Wordcount, we can aggregate mappers output before shuffling them into the reducers.
  - i.e., aggregate local intermediate counts for words found by mappers in each mapper process, e.g. (*the*, *12345*)

- By default there is no combiner.
- Combiners are implemented like a *(local)* reducer class and are added to the workflow by calling `conf.setCombinerClass();`

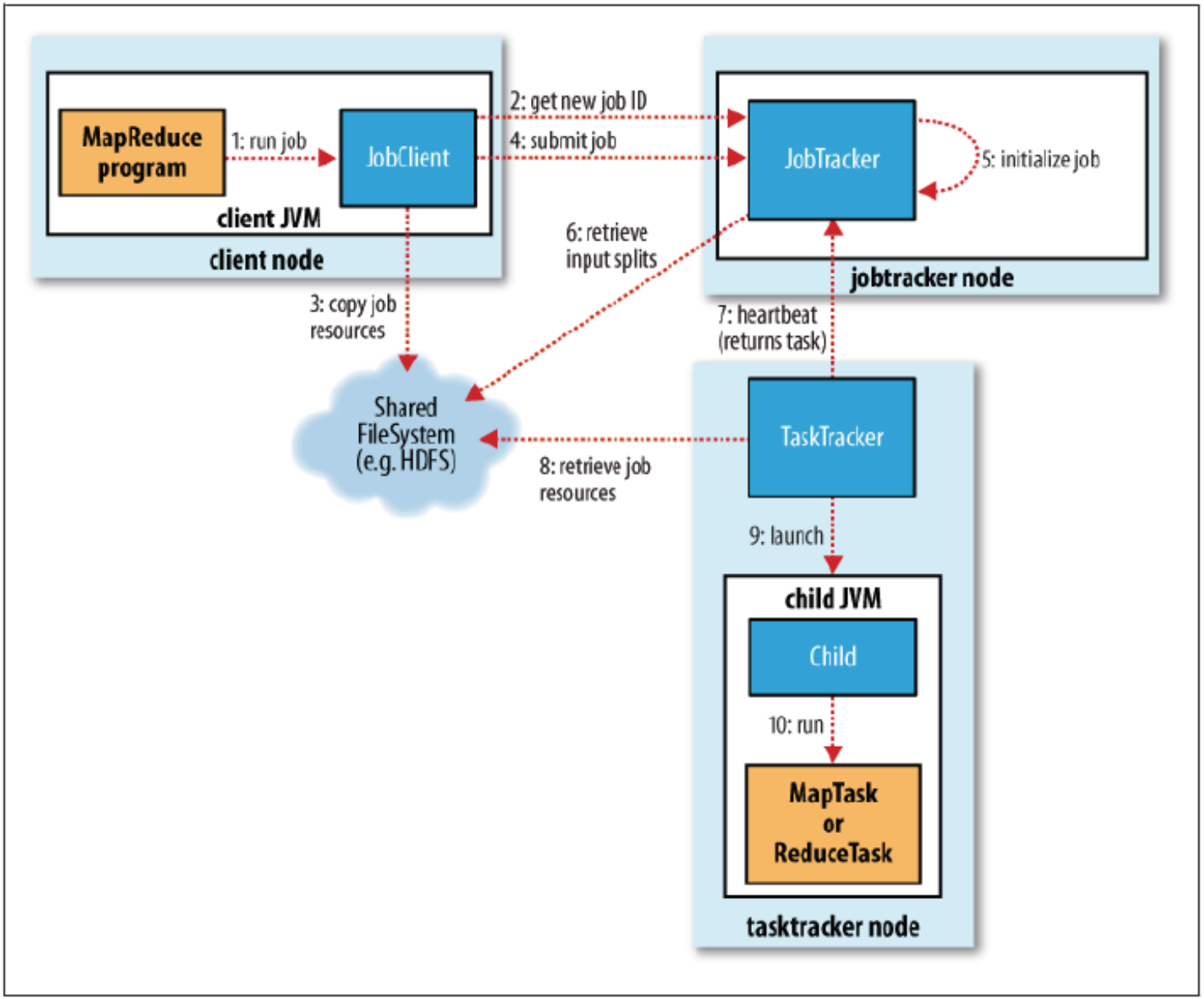


Resource Management

Hadoop MapReducer Servers

- **Jobtracker**
  - Java class.
  - It's the central manager of a MapReduce job.
- **Tasktracker**
  - Java class.
  - It's the process responsible for starting local map and reduce workers.
  - Periodically notifies jobtracker that it's still alive (optionally reporting progress w/ counters)

Execution of MapReduce Programs



Scheduling (1)

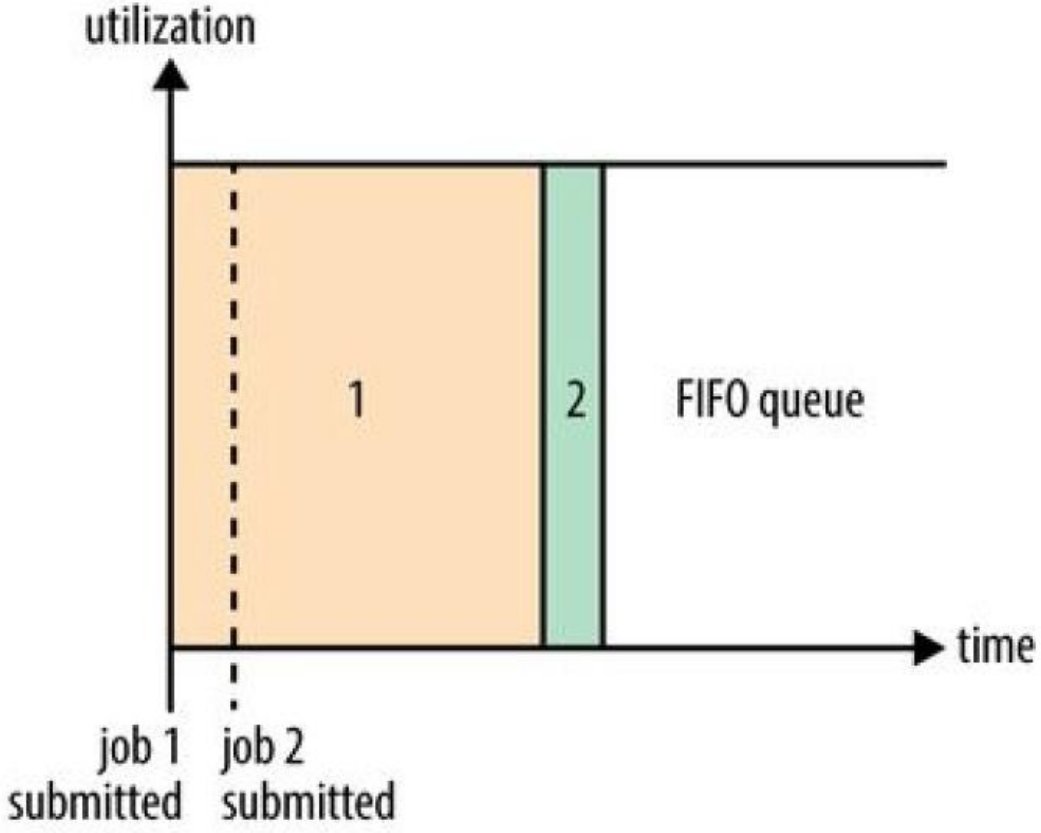
- Hadoop MapReduce scheduling has evolved to ensure greater evenness:

- **`FIFO Scheduler`** (default):
  - Originally, MR jobs scheduled in FIFO order.
  - short jobs may have to wait for very long jobs
  - Later, FIFO with priorities `(VERY_HIGH, HIGH, NORMAL, LOW, VERY_LOW)` without preemption.

- Later, scheduler was transformed into a plugin.
  - schedulers incorporated in YARN resource manager

Scheduling (2)

- **e.g. FIFO Scheduler:**

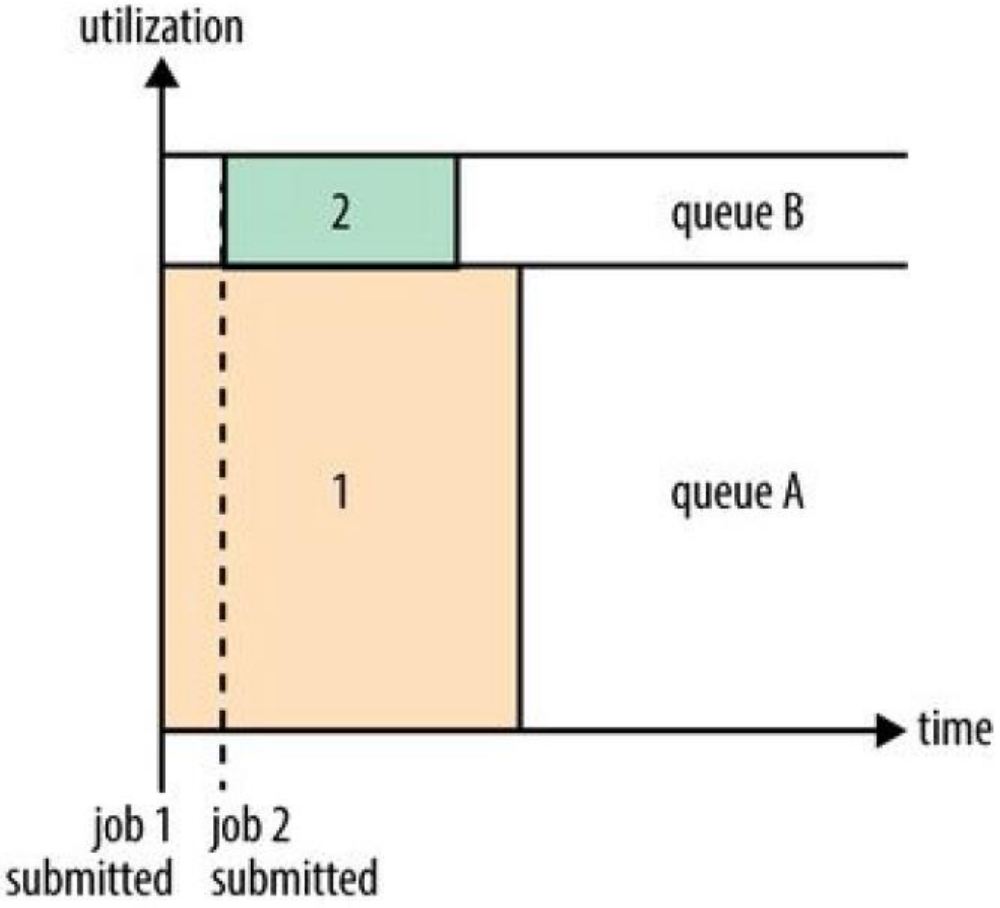


Scheduling (3)

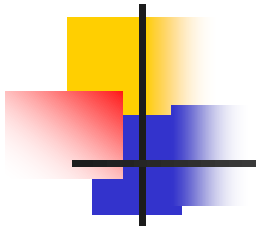
- **`CapacityScheduler`** (Yahoo):
  - Keeps a set of separate queues.
    - each one with its capacity in reserved slots
    - queue capacity reserved for jobs in that queue.
  - guarantees resources for shorter jobs
  - at the cost of overall cluster utilization
    - large job may finish later than with FIFO

Scheduling (4)

- **e.g. CapacityScheduler:**



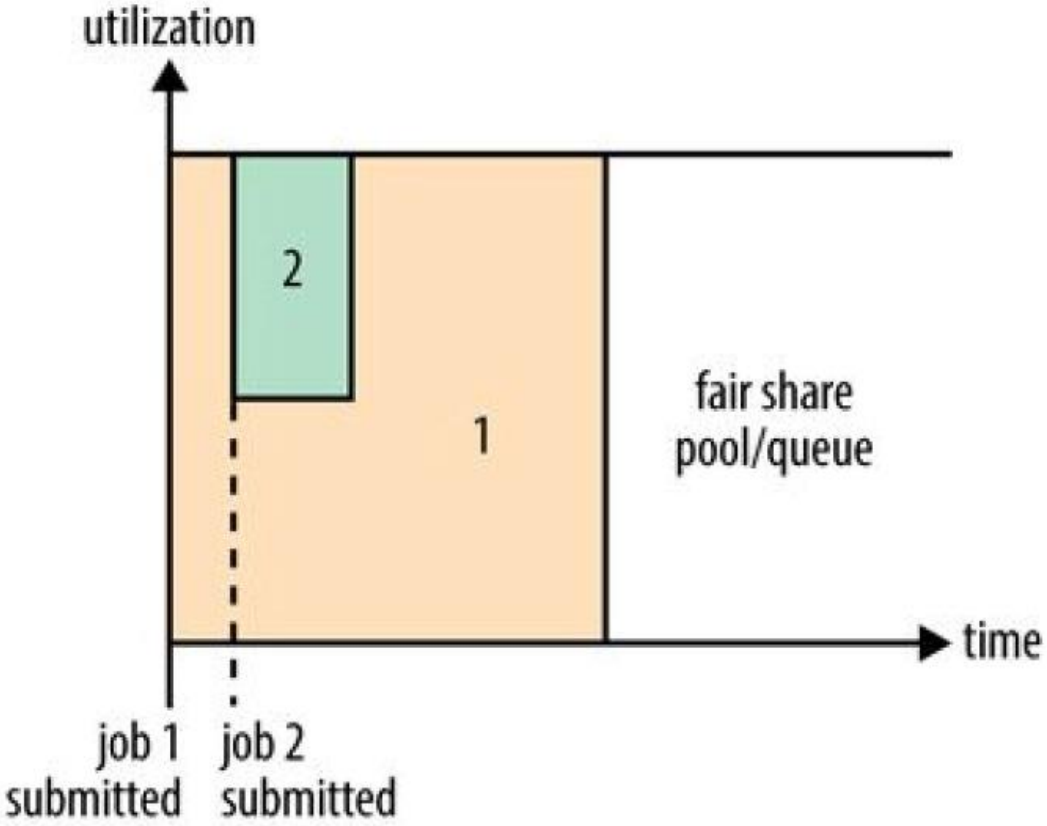
Scheduling (5)

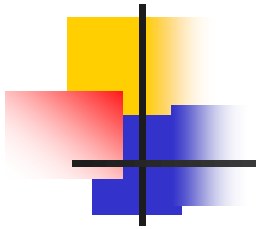


- **`FairScheduler`** (Facebook)
  - no need to reserve capacity per queue
  - free slots distributed evenly between all groups.
  - it will dynamically balance resources between all running jobs.
    - e.g., first (large) job starts, gets all resources in cluster
    - when second (small) job starts, it is allocated half of the cluster resources (and proportionally with more jobs).
  - thus, each job is using its *fair share* of resources.

Scheduling (6)

- **e.g. FairScheduler:**

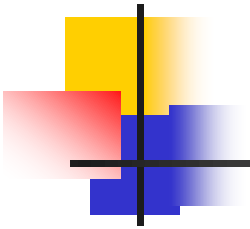




Fault Tolerance

- Master failure:
  - The master state is checkpointed in a distributed file system.
  - When a failed master is restarted, it reads the saved state and continues from that point on.

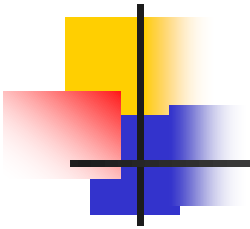
- Worker faults:
  - Faults are detected by periodic pings.
  - Ready or ongoing tasks that fail are redone.
  - Ongoing reduce tasks that fail are redone.
  - Finished tasks are reported to the master.



FT: Server Faults

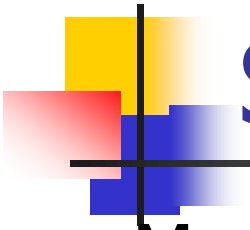
- If the **tasktracker** fails, or is
  - too slow reporting to the jobtracker,
  - its tasks are restarted at other nodes.

- If the **jobtracker** fails:
  - all work of specific job is lost.
  - it’s main Hadoop problem.



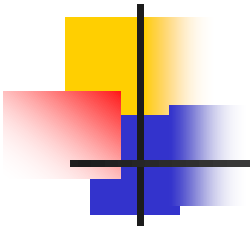
Conclusion

- MapReduce makes us think differently.
- But makes possible to do quite sophisticated parallelizations and distribution almost seamlessly
- It enables us to leverage computing infrastructures of massive scale (improve resource utilization)
- Still, it is fundamental to think about resource efficiency: the amount of transmitted information.
  - minimize intermediate data, e.g., using combiners



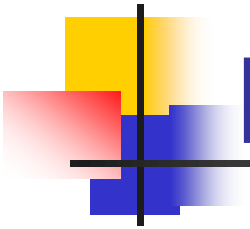
Summary

- Map Reduce Paradigm:
  - What is it?, concepts
  - Examples:
    - WordCount,
    - PageRank
- Map Reduce Platform:
  - Architecture
  - Data splitting
  - Task Management
  - Resource Management, Scheduling
  - Fault-tolerance



Reference: Input Formats

- Controlled by `conf.SetInputFormat`
- `FileInputFormat` (default option): 64MB blocks (for files 64MB) per split (and per map).
- `CombineFileInputFormat`: groups files on the same node into the same split.
- `KeyValueTextInputFormat`
- `NLineInputFormat`



Reference: Output Formats

- Controlled by `conf.SetOutputFormat`.
- `TextOutputFormat`: one file per reducer (default option).
- `SequenceFileOutputFormat`: compressed output. Normally used to feed other MapReduce cycles.
- `MultipleOutputFormat`: allows control over the amount and name of output files.
- It is also possible to output to databases.

Reference: FT - Worker Faults

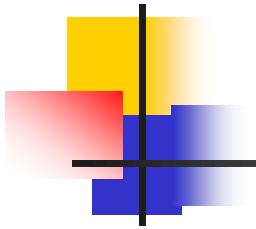
- Situations where workers notify the jobtracker:
  - Exceptions; Abrupt termination of the JVM.
- A worker is failed if it spends longer than `mapred.task.timeout` without reporting to the jobtracker.
  - failed workers are retried 4 times on different nodes.
- Map (intermediate) output files are in local file system and not HDFS.
  - it is not efficient to replicate output data (mappers can be repeated).
- mappers and reducer can deal with corrupted data.
  - after the 2nd retry, they change to skipping mode:
  - all input records that cause errors are skipped.
  - job can make progress processing rest of (big) data.



Big-Data Processing II (Dataflows with Stream Processing)

Prof. Luís Veiga
IST / INESC-ID Lisboa

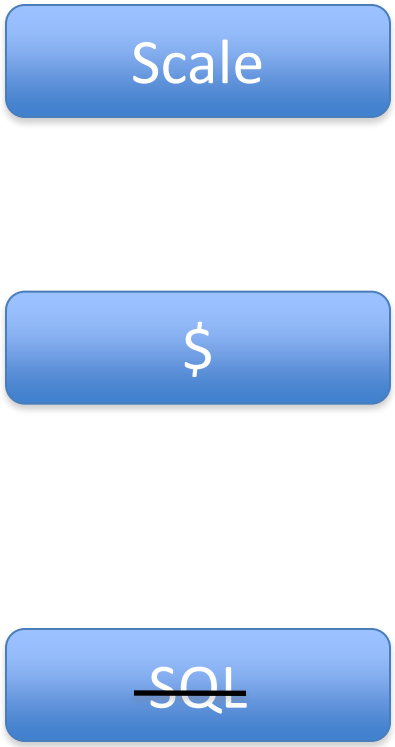
https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/



Agenda

- Map-Reduce analysis
- Spark: overview, RDDs
  - programming model, examples, RDD operations
  - fault-tolerance, performance
- Spark Streaming
  - overview, discretized stream processing
  - windows, sliding windows, micro-batching
- Flink
  - overview, windowing
  - tumbling windows, sliding windows, custom windows
  - time-based windows, watermarks
  - state management, distributed snapshots
  - fault tolerance, execution semantics

M/R revisited: Advantages



Scalable due to simpler design

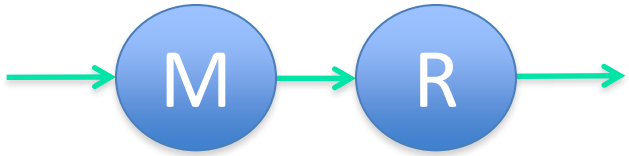
- Only parallelizable operations
- No transactions

Runs on cheap commodity hardware

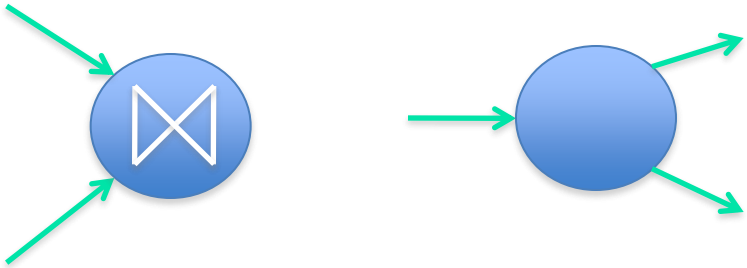
Procedural Control- a processing “pipe”

M/R revisited: Disadvantages

1. Extremely rigid data flow

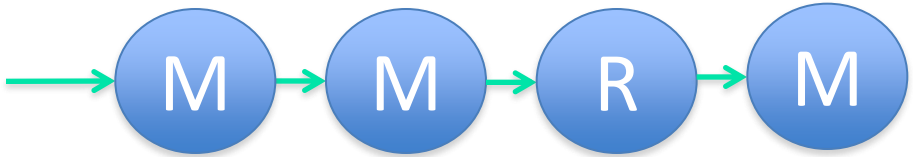


Other flows constantly hacked in



Join, Union

Split



Chains

2. Common operations must be coded by hand
   - Join, filter, projection, aggregates, sorting, distinct

3. Semantics hidden inside map-reduce functions
   - Difficult to maintain, extend, and optimize

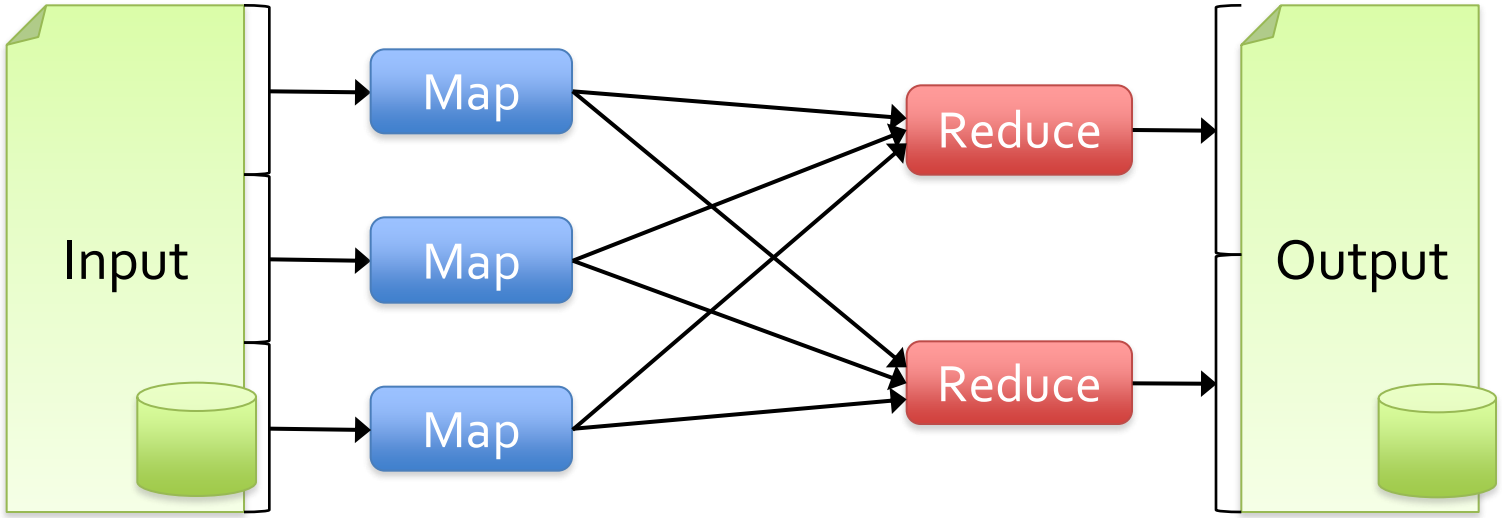
Spark



2022 SIGMOD Systems Award

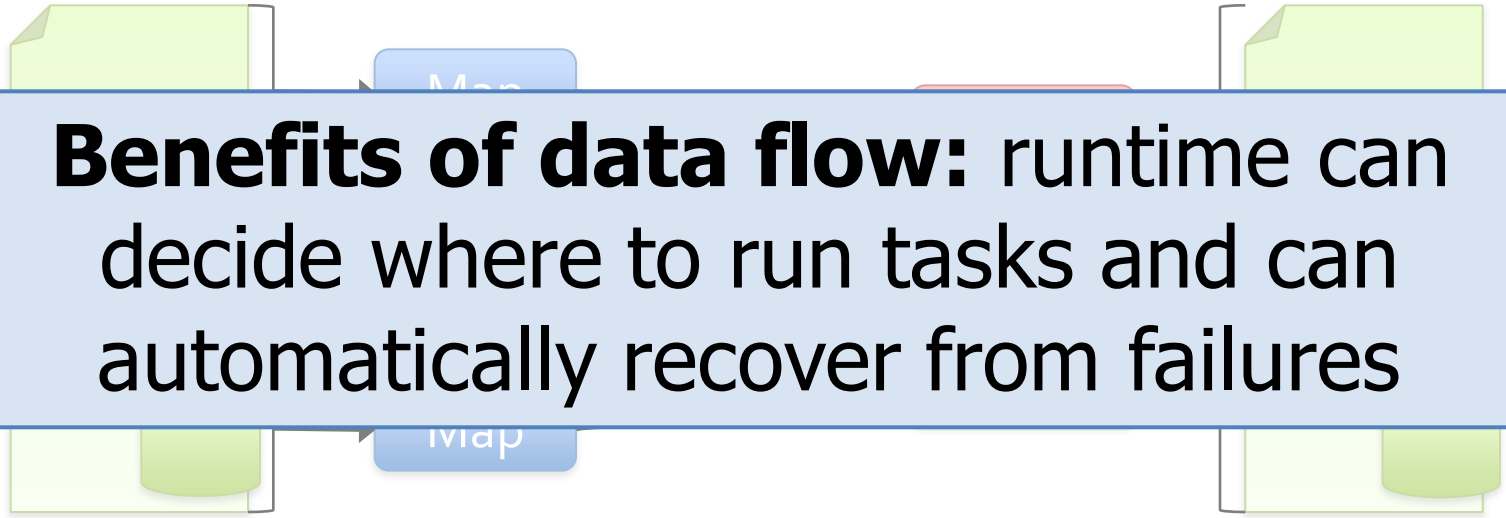
Motivation

- Current popular programming models for clusters transform **data flowing** from stable storage to stable storage
- E.g., MapReduce, Pig, …, Dryad, Pregel, etc.



Motivation

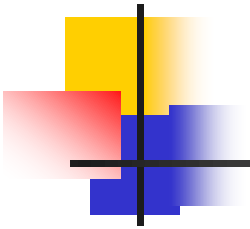
- Current popular programming models for clusters transform **data flowing** from stable storage to stable storage
- E.g., MapReduce, Pig, …, Dryad, Pregel, etc.



Benefits of data flow: runtime can decide where to run tasks and can automatically recover from failures

Motivation

- Acyclic data flow is a powerful abstraction, but is not efficient for applications that **repeatedly reuse a *working set*** of data:
  - **Iterative** algorithms (many in machine learning)
  - **Interactive** data mining tools (R, Excel, Python)
- MapReduce does not leverage memory of workers across processing steps
- Spark makes working sets a first-class concept to efficiently support these apps



Spark Goal

- Provide **distributed memory abstractions** for clusters to support apps with *working sets*
- Retain the attractive properties of MapReduce:
  - Fault tolerance (for crashes & stragglers)
  - Data locality
  - Scalability

Solution: augment data flow model with "resilient distributed datasets" (RDDs)

Programming Model

- Resilient distributed datasets (RDDs)
  - Immutable collections partitioned across clusters that can be rebuilt if a partition is lost.
  - Created by **RDD transformations**:
    - with data flow operators
    - e.g. map, filter, group-by, …
    - input from stable storage or other RDD transformations
  - Can be ***cached*** across parallel operations

- **Parallel operations** on RDDs
  - Reduce, collect, count, save, …

E.g. Word Count in Spark

```
val lines = spark.textFile(“hdfs://...”)

val counts = lines.flatMap(_.split(“\\s”))
                  .reduceByKey(_ + _)

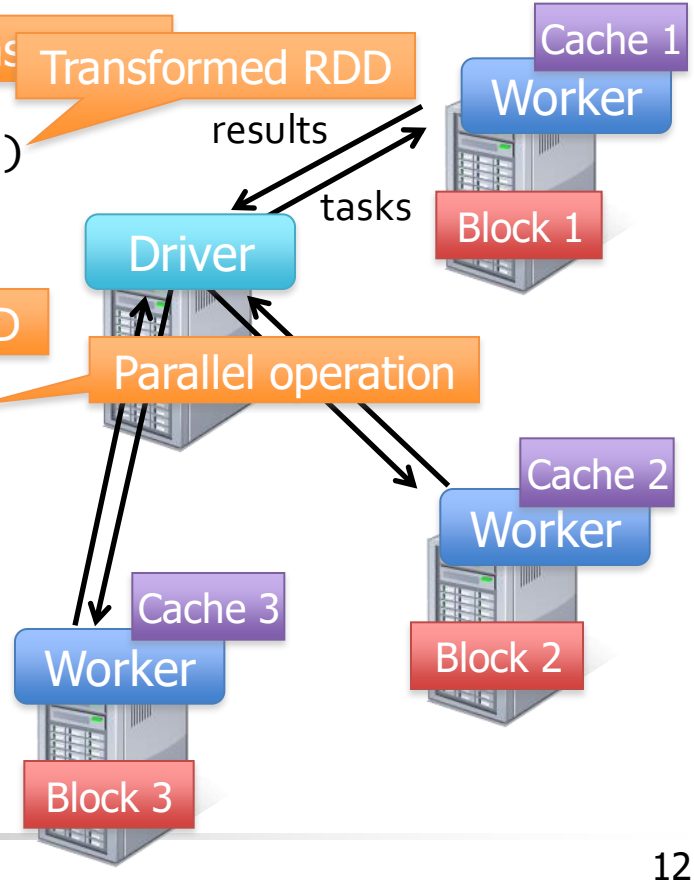
counts.save(“hdfs://...”)
```

E.g. Log Mining

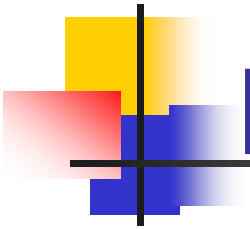
- Load error messages from a log into memory, then interactively search for various patterns

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(_.startsWith("ERROR"))
messages = errors.map(_.split('\t')(2))
cachedMsgs = messages.cache()

cachedMsgs.filter(_.contains("foo")).count
cachedMsgs.filter(_.contains("bar")).count
. . .
```



Result: full-text search of Wikipedia in <1 sec (vs 20 sec for on-disk data)



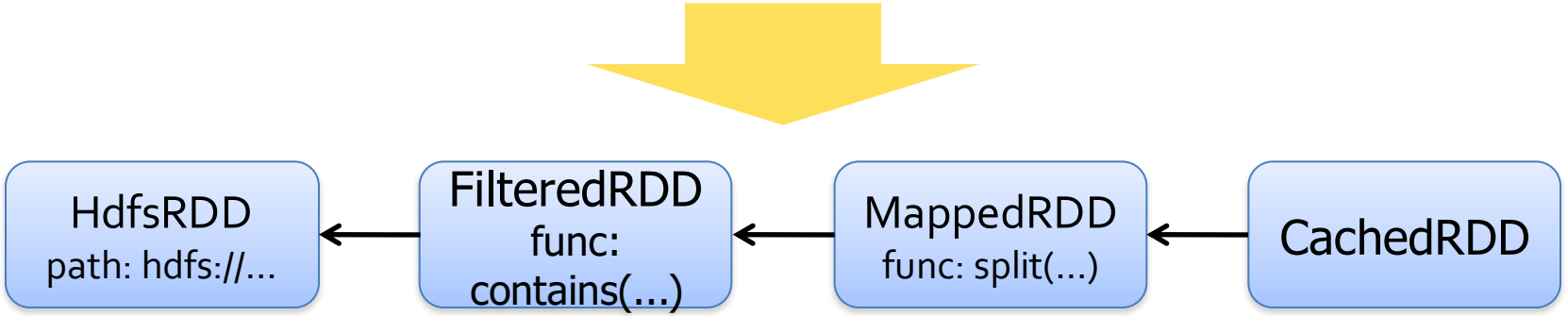
RDDs in More Detail

- An RDD is an immutable, partitioned, logical collection of records
  - needs not be materialized on disk
  - contains enough information to allow rebuilding a dataset from stable storage
- Partitioning can be based on a key in each record (using hash or range partitioning)
- Built applying transformations on other RDDs
- Can be cached for future reuse

RDD Fault Tolerance

- RDDs maintain *lineage* information that can be used to reconstruct lost partitions
  - i.e., track data dependencies in the data flow
- Ex:

```
cachedMsgs = textFile(...).filter(_.contains("error"))
                          .map(_.split('\t')(2))
                          .cache()
```

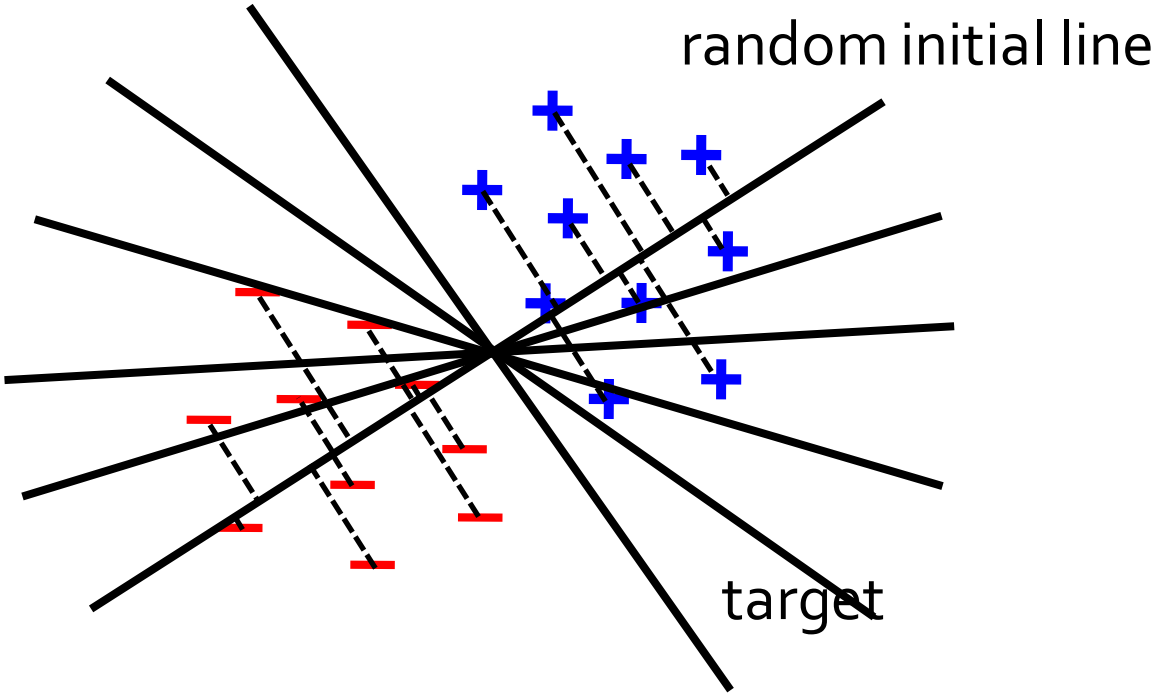


Benefits of RDD Model

- Consistency is easy due to immutability
- Inexpensive to maintain fault tolerance
  - (log lineage dependency information
  - rather than replicating/checkpointing data)
  - **Q:** what about recovery?
- Locality-aware scheduling of tasks on partitions
- Despite being restricted
  - (not as expressive as queries)
  - model seems applicable to a broad variety of applications

E.g. Logistic Regression

- Goal: find best line separating two sets of points iteratively



E.g., Logistic Regression Code

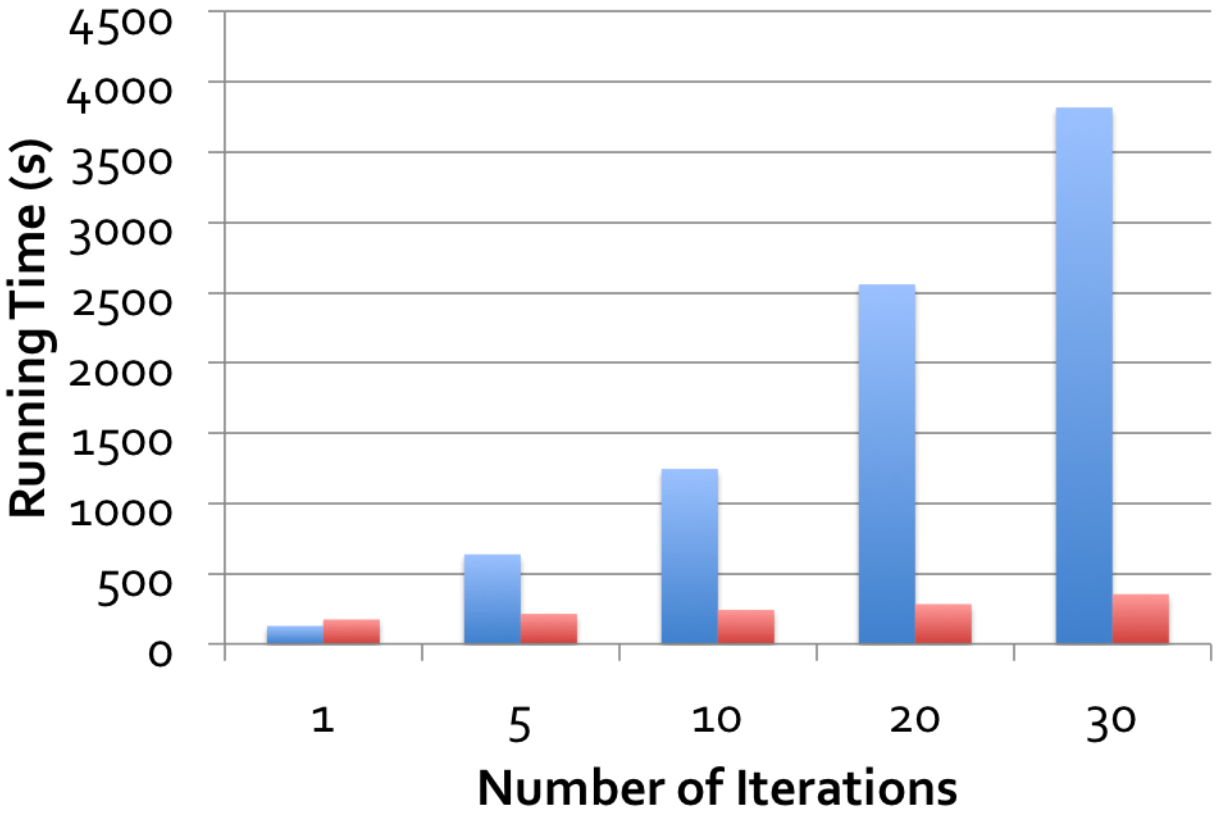
```
val data = spark.textFile(...).map(readPoint).cache()

var w = Vector.random(D)

for (i <- 1 to ITERATIONS) {
  val gradient = data.map(p =>
    (1 / (1 + exp(-p.y*(w dot p.x))) - 1) * p.y * p.x
  ).reduce(_ + _)
  w -= gradient
}

println("Final w: " + w)
```

Logistic Regression Performance



127 s / iteration

Hadoop

Spark

first iteration 174 s
further iterations 6 s

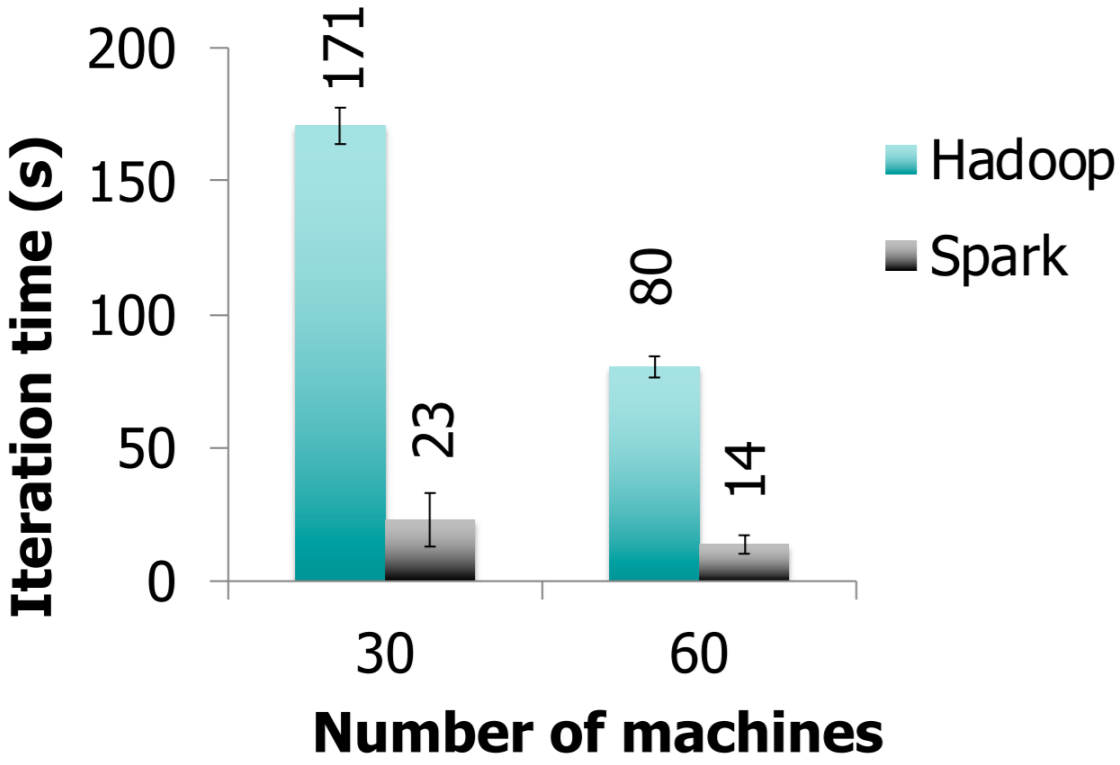
E.g. Page Rank

```
val links = // load RDD of (url, neighbors) pairs
var ranks = // load RDD of (url, rank) pairs

for (i <- 1 to ITERATIONS) {
  val contribs = links.join(ranks).flatMap {
    case (url, (links, rank)) =>
      links.map(dest => (dest, rank/links.size))
  }
  ranks = contribs.reduceByKey(_ + _)
                  .mapValues(0.15 + 0.85 * _)
}

ranks.saveAsTextFile(...)
```

Page Rank Performance

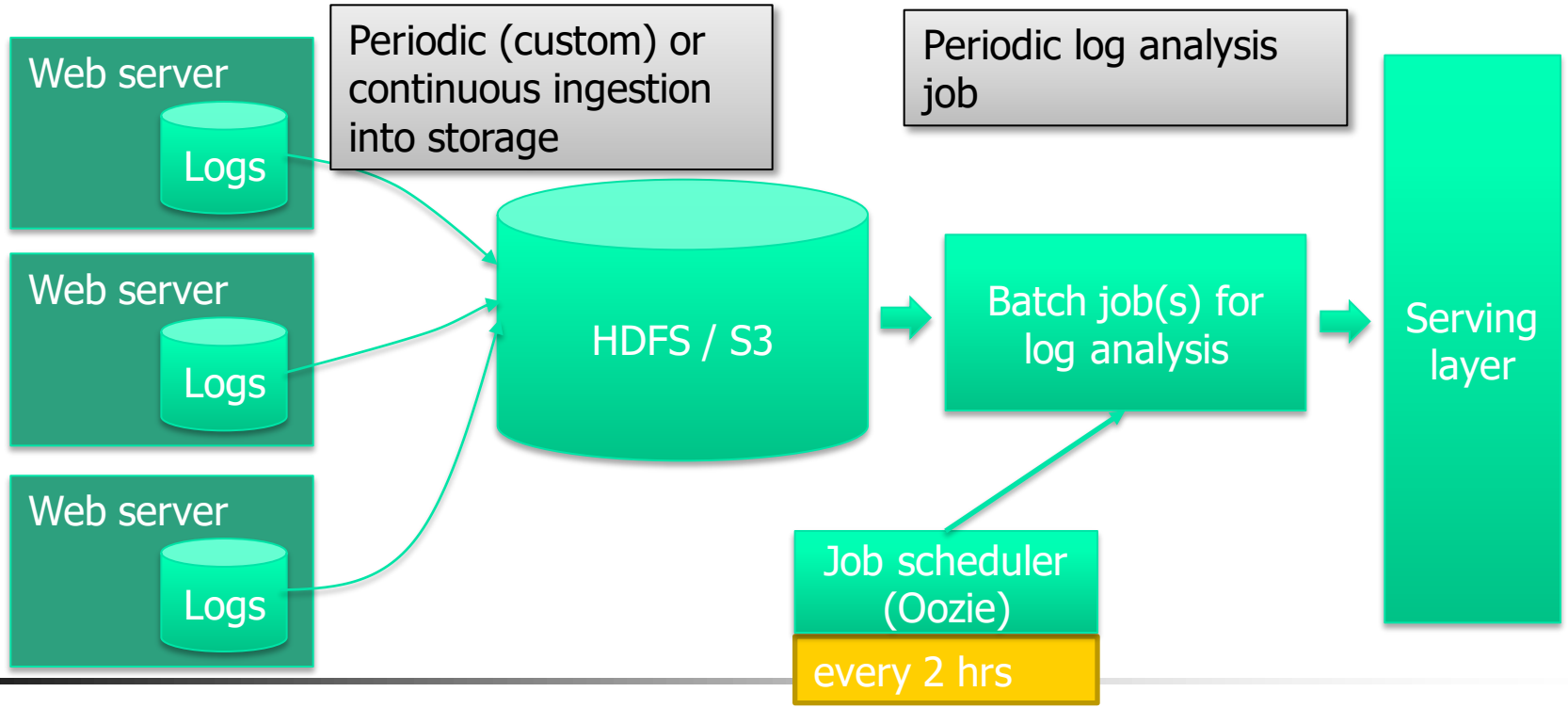




Spark Streaming

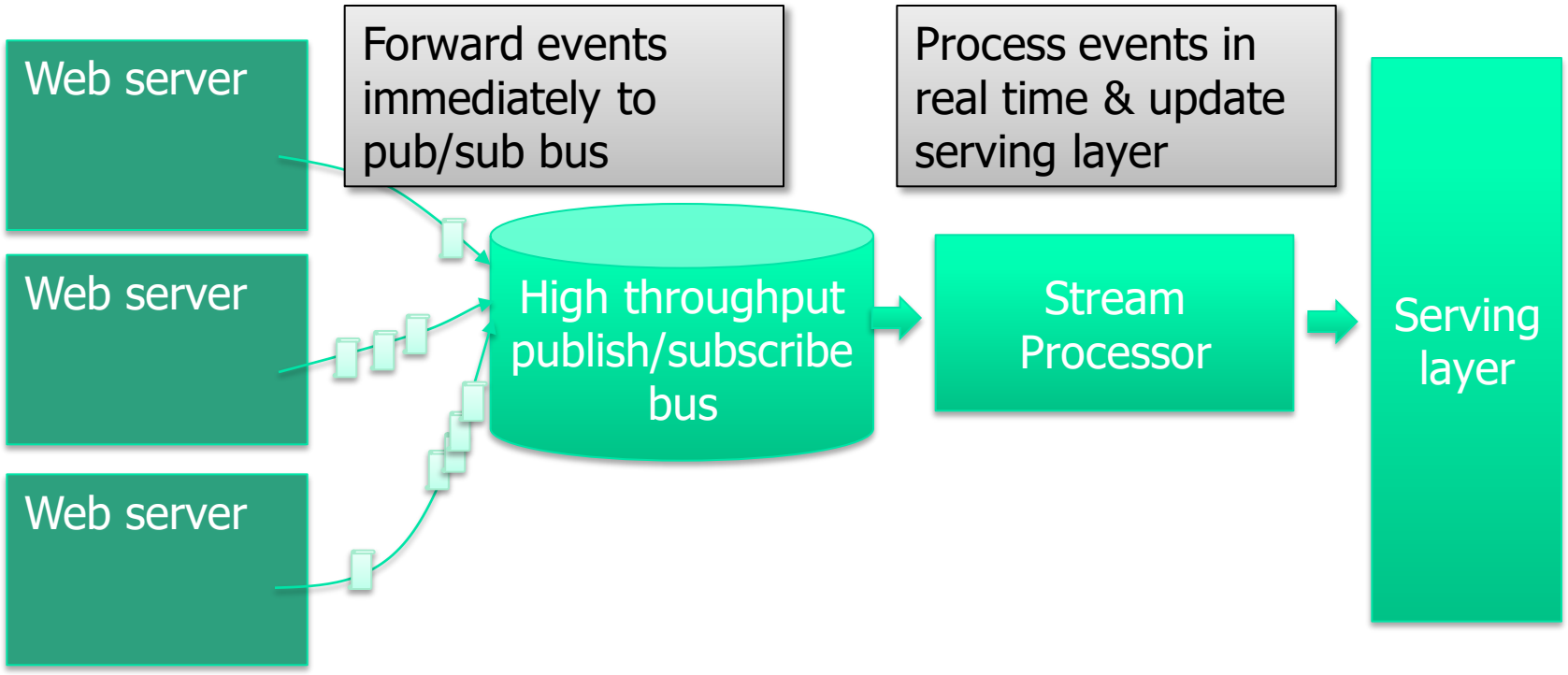
Traditional data processing

- E.g., log analysis example using a batch processor
- **Latency** from log event to serving layer usually in the **range of hours**



Log event analysis using stream processor

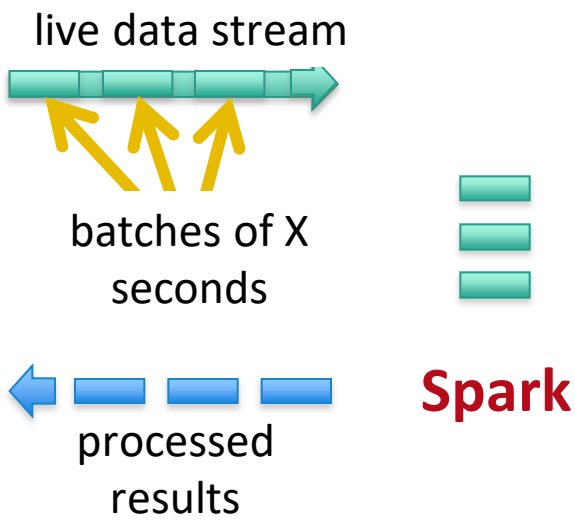
- Stream processors allow to analyze events with **sub-second latency**.



Discretized Stream Processing

Run a streaming computation as a **series of very small, deterministic batch jobs**

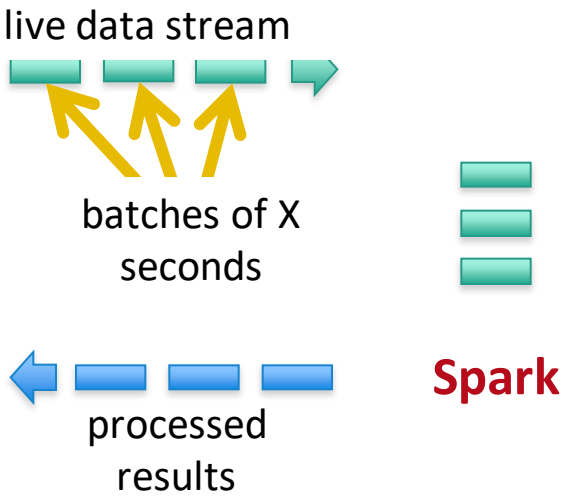
- Chop up the live stream into *micro*-batches of X seconds
- Spark treats each batch of data as RDDs and processes them using RDD operations
- Finally, the processed results of the RDD operations are returned in batches



Discretized Stream Processing

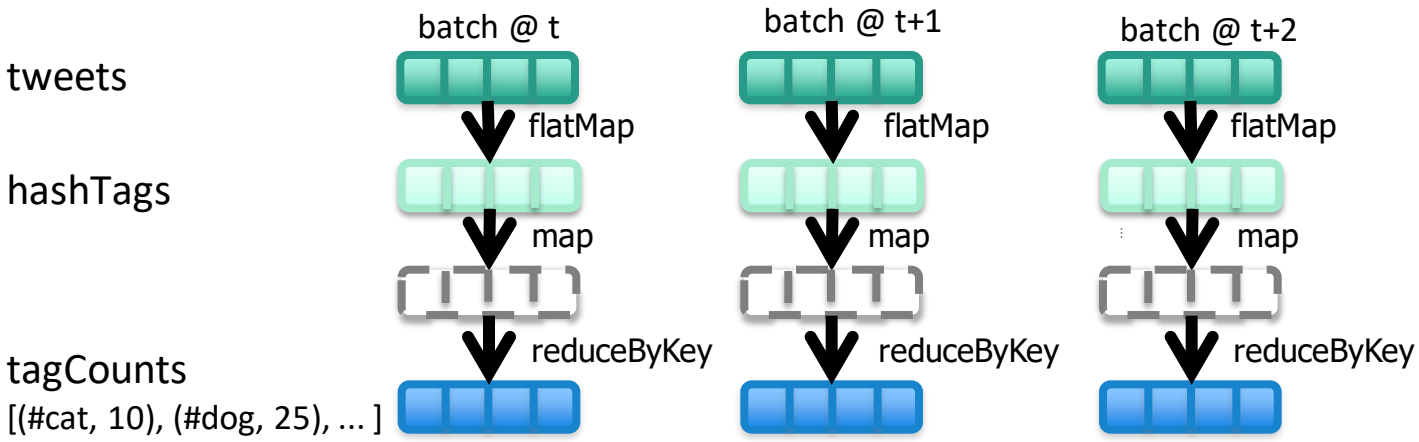
Run a streaming computation as a **series of very small, deterministic batch jobs**

- Batch sizes as low as ½ second, latency ~ 1 second *(micro-batching)*
- Potential for limited combination of batch processing and stream processing in the same system



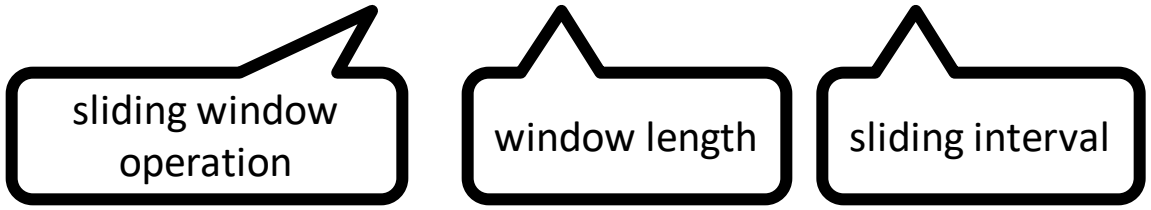
Example – Count hashtags

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
val hashTags = tweets.flatMap (status => getTags(status))
val tagCounts = hashTags.countByValue()
```



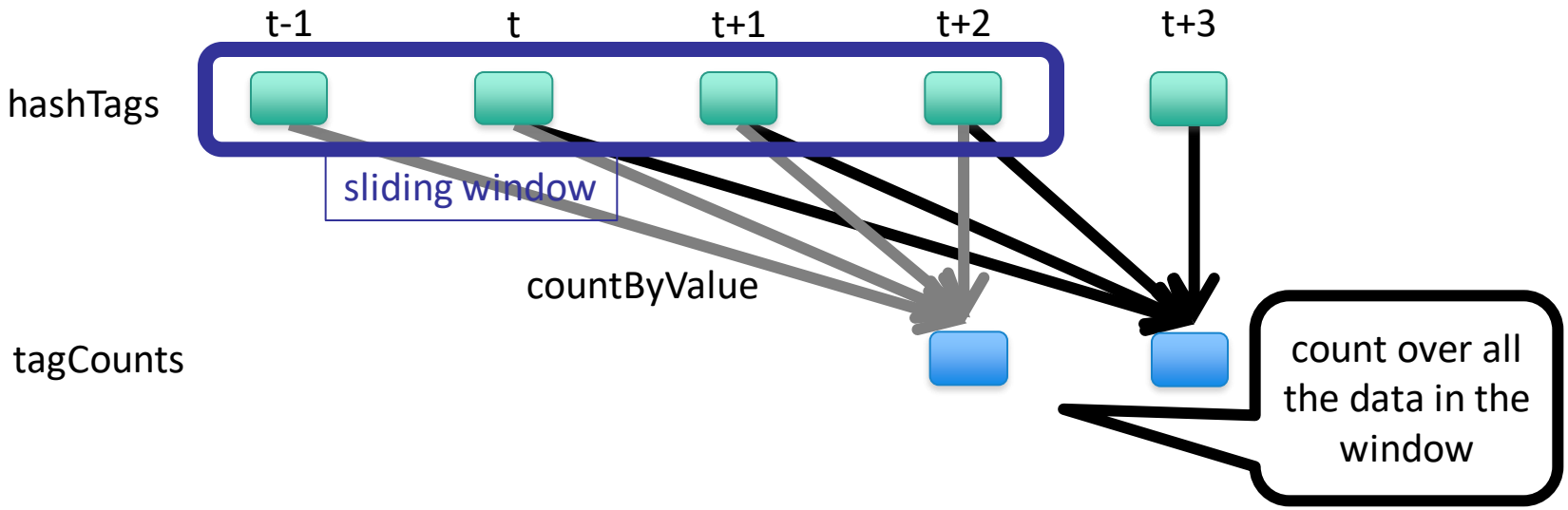
Example – Count hashtags over last 10 mins

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
val hashTags = tweets.flatMap (status => getTags(status))
val tagCounts = hashTags.window(Minutes(10), Seconds(1)).countByValue()
```



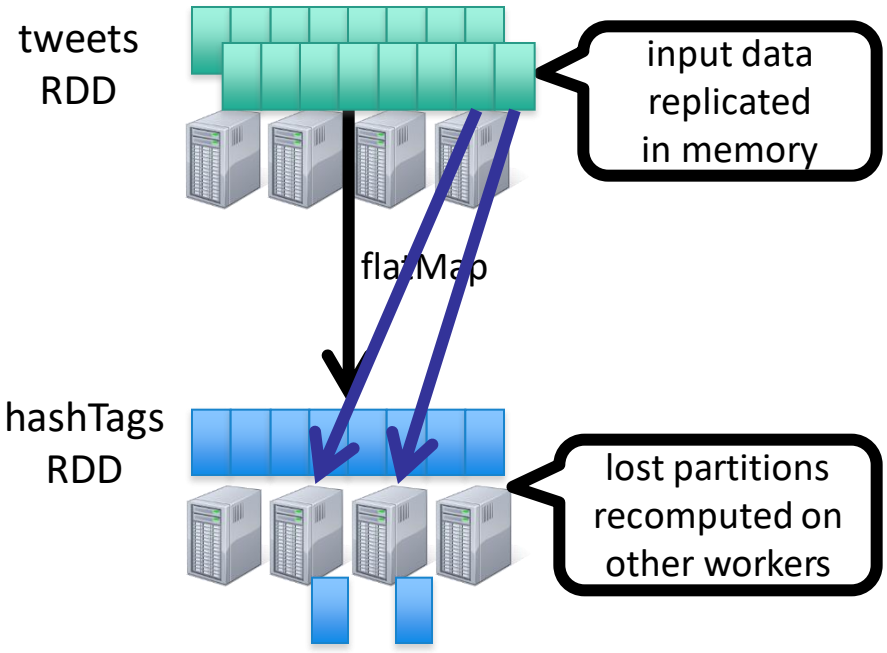
Example – Count hashtags over last 10 mins

```
val tagCounts = hashTags.window(Minutes(10), Seconds(1)).countByValue()
```



Fault-tolerance

- RDDs remember the sequence of operations/transformations (dataflow) that created it from the original fault-tolerant input data
- Batches of input data are replicated in memory of multiple worker nodes, therefore fault-tolerant
- Data lost due to worker failure, can be recomputed from input data



Flink



2023 SIGMOD Systems Award

Apache Flink

- Apache Flink is an open source stream processing framework
  - Low latency
  - High throughput
  - Stateful
  - Distributed



- Developed at the Apache Software Foundation,
- Used in production

Apache Flink

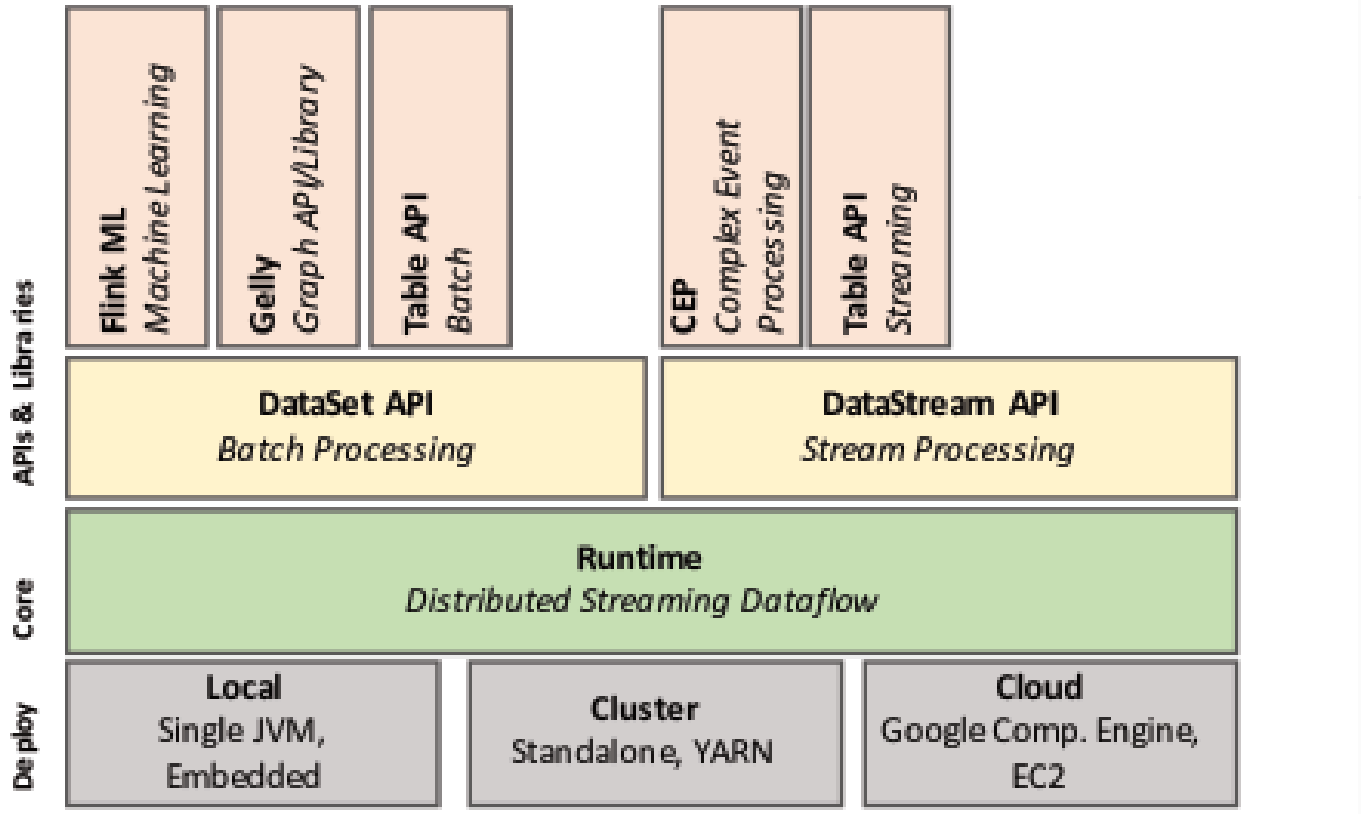
Real-world data is produced in a continuous fashion.

Systems like Flink embrace streaming nature of data.

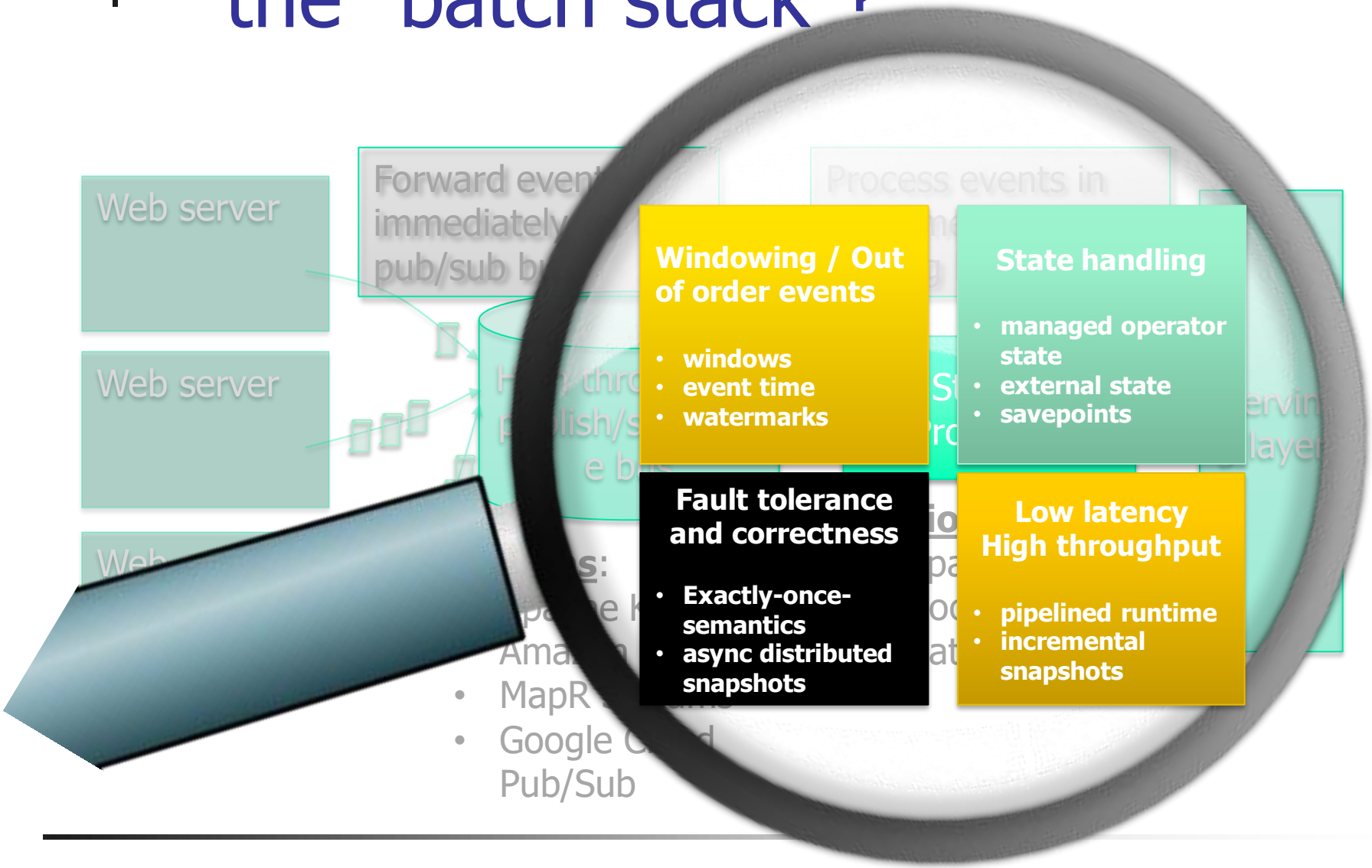


Apache Kafka:
reliable message queue/feed broker

Overview of Flink Architecture



What do we need for replacing the "batch stack"?



Building windows from a stream

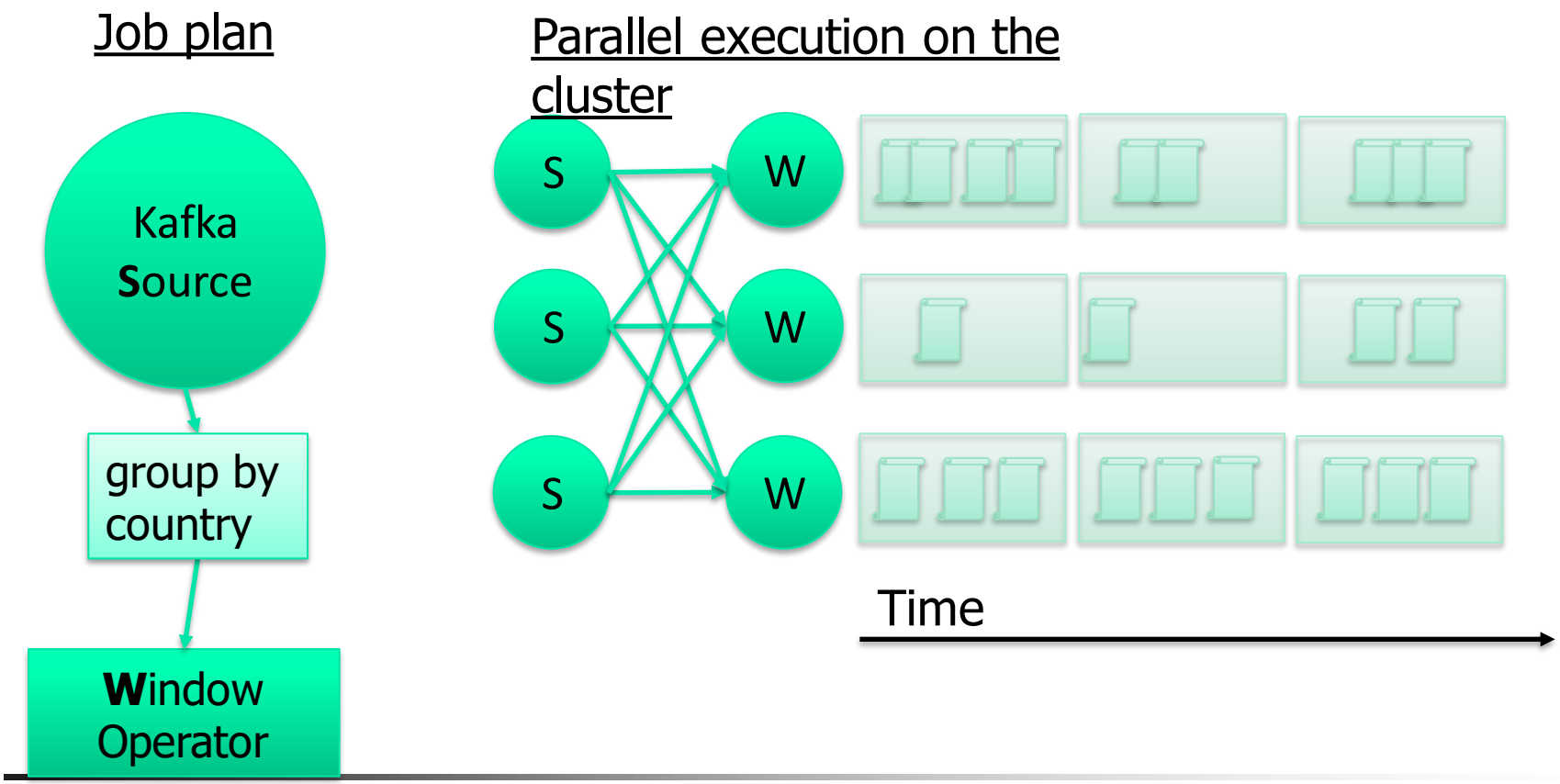


- "*Number of visitors in the last 5 minutes per country*"

```
// create stream from Kafka source
DataStream<LogEvent> stream = env.addSource(new KafkaConsumer());
// group by country
DataStream<LogEvent> keyedStream = stream.keyBy("country");
// window of size 5 minutes
keyedStream.timeWindow(Time.minutes(5))
// do operations per window
.apply(new CountPerWindowFunction());
```

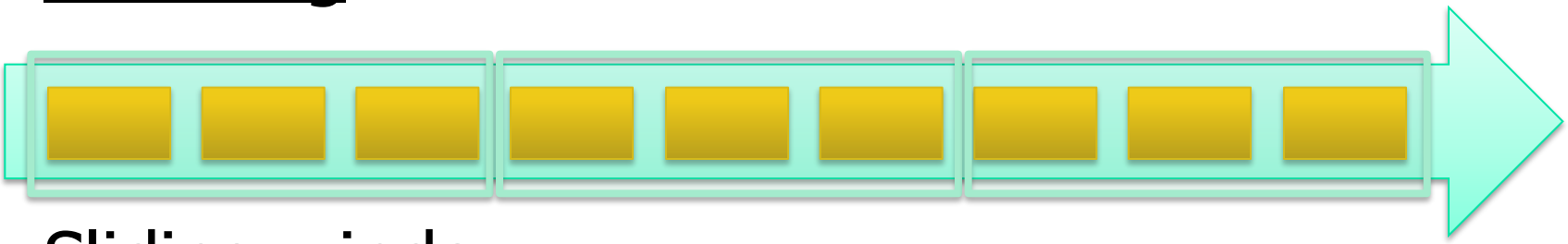
Building windows: Execution

```
// window of size 5 minutes
keyedStream.timeWindow(Time.minutes(5));
```

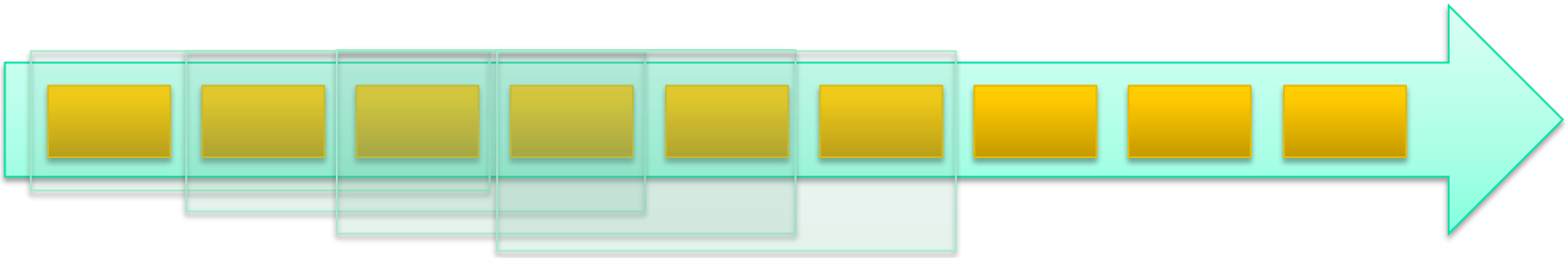


Window types in Flink

- Tumbling windows

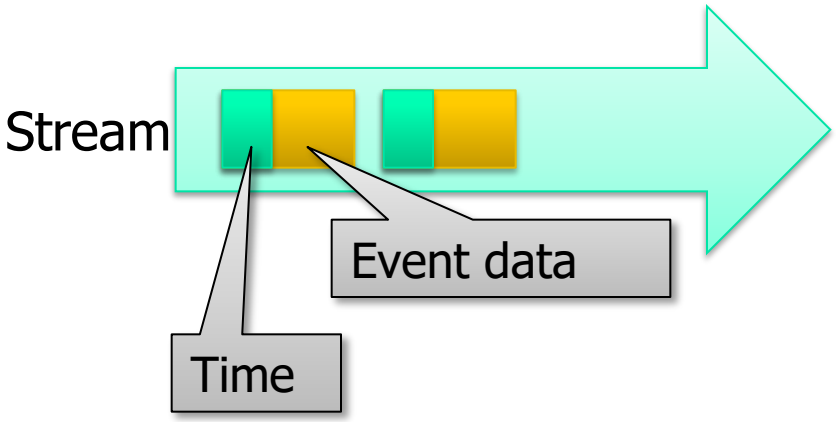


- Sliding windows



- Count windows: based on number of events
- Time-based: based on time (event, ingestion)
- Custom windows with window assigners, triggers and evictors

Time-based windows

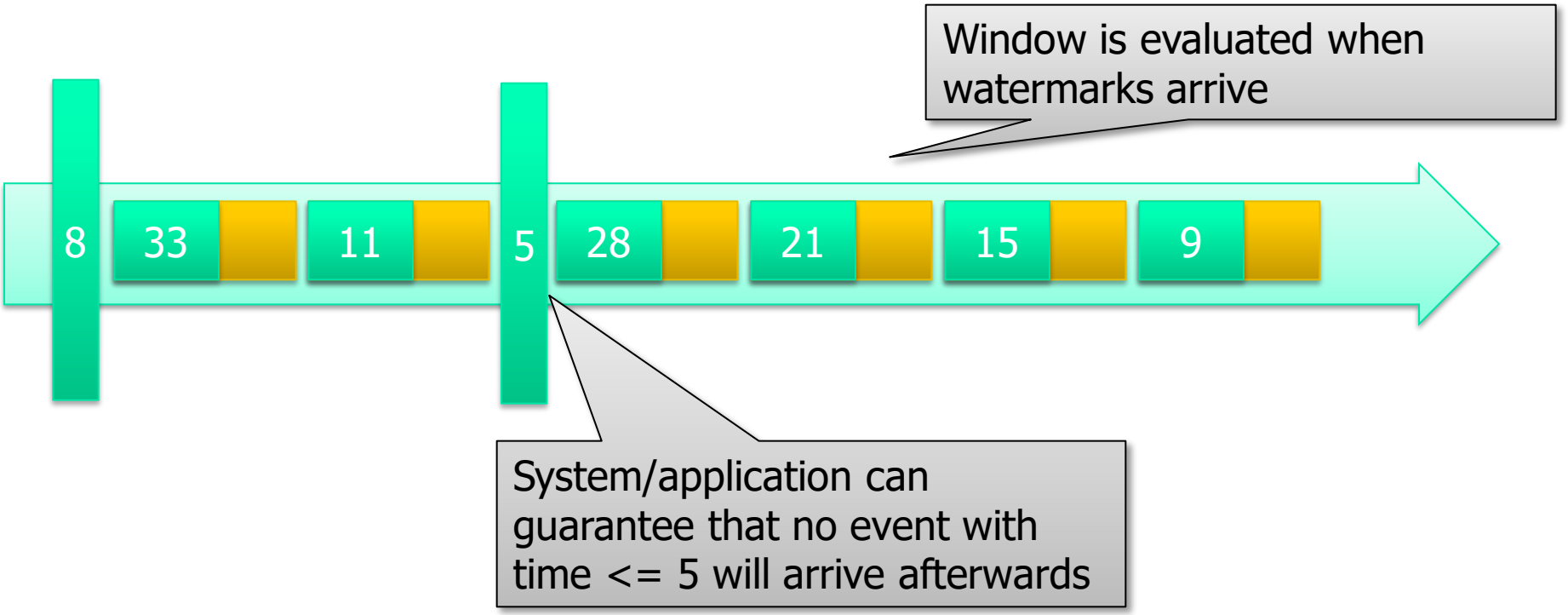



```
{
   "accessTime": "1457002134",
   "userId": "1337",
   "userLocation": "UK"
}
```

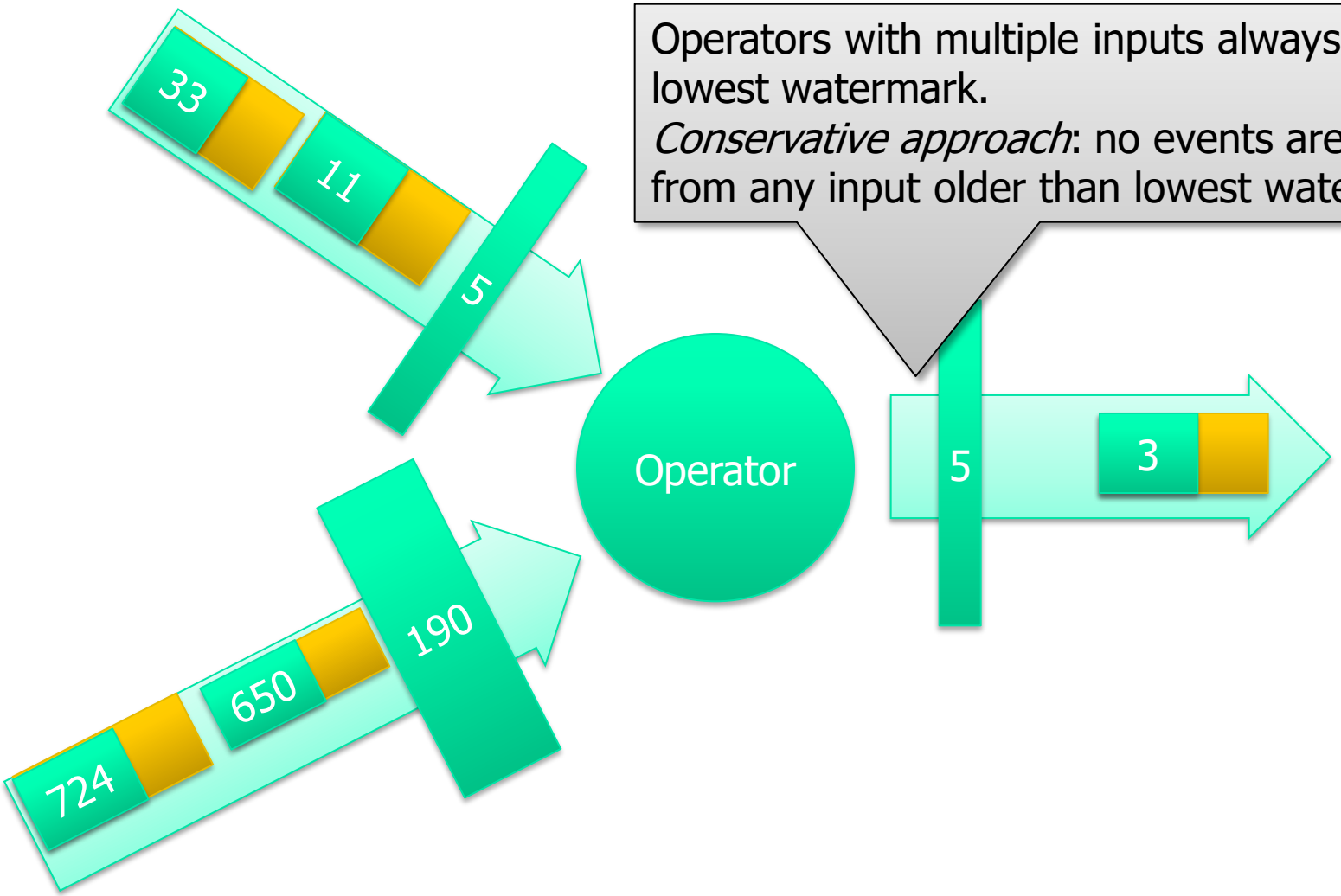
- Three notions of *Time* considered in Flink:
  - Processing time: wall clock time when window is being built and processed
  - Ingestion time: time when event entered Flink, at beginning of dataflow (possible network delays)
  - Event time: *out-of-order* handling of time when event was truly created, e.g., sensor emitting data

Time: Low Watermarks

- Periodically, low-watermarks are sent through system
  - to indicate *true* progression of event time
  - bound timestamps of upstream *out-of-order* events
  - allow avoiding waiting *forever* for delayed events



Time: Low Watermarks

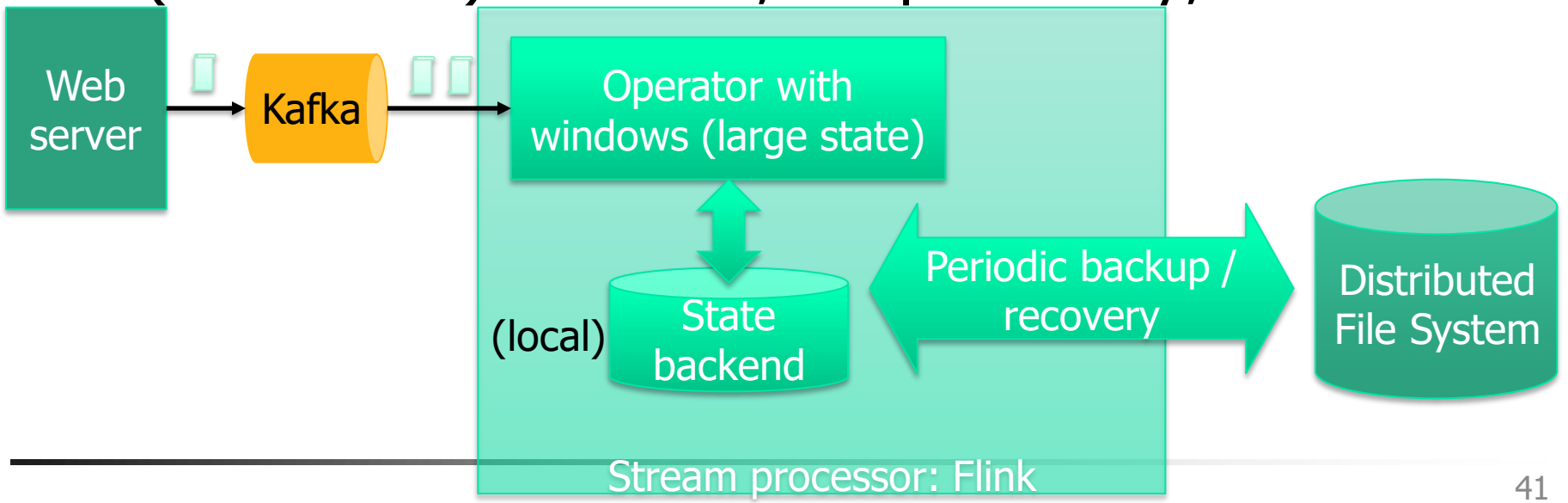


Operators with multiple inputs always forward the lowest watermark.
Conservative approach: no events are expected from any input older than lowest watermark

Earlier work on: “MillWheel: Fault-Tolerant Stream Processing at Internet Scale” by T. Akidau et. al.

State: Managed state in Flink

- Windows can have internal *keyed* processing state
  - e.g. counters, max, min of all tuples for each key
- Flink automatically backups and restores state
- State can be larger than the available memory
- State backends:
  - (embedded) RocksDB, heap memory, custom



Managing the state

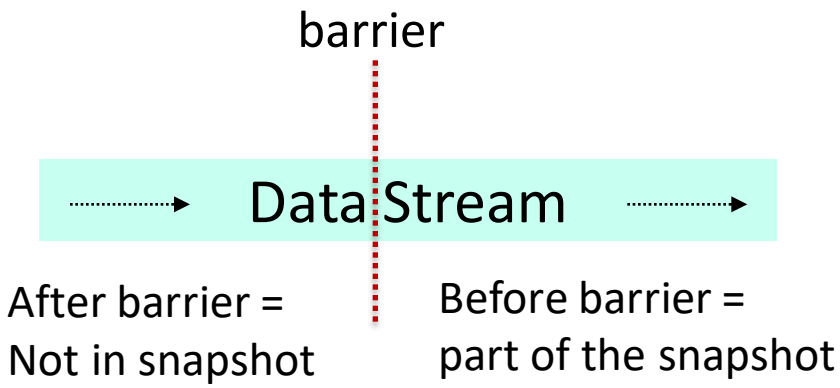


- How can we operate such a pipeline 24x7?
- Losing state (by stopping the system) would require replay of possibly many past events.

- Need a way to checkpoint the state!

Async Distributed Snapshots

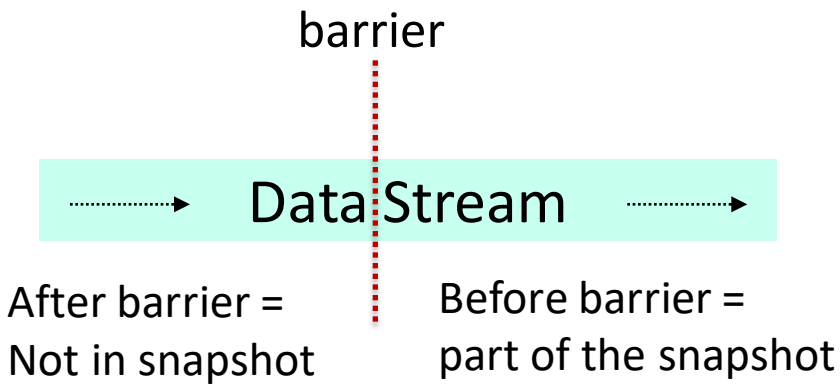
- Lightweight approach of storing the state of all operators without pausing the execution
- Implemented using barriers flowing through the DAG topology being executed



Reference work on: "Lightweight Asynchronous Snapshots for Distributed Dataflows" by Carbone et. al.

Async Distributed Snapshots

- A Distributed Snapshot is a consistent snapshot of:
  - application (window) state
  - the position (cursor) in input stream(s)
  - optimized with incremental snapshots
    - only diferences between snapshots are recorded



Reference work on: “Lightweight Asynchronous Snapshots for Distributed Dataflows” by Carbone et. al.

Fault tolerance & correctness

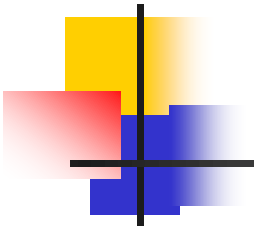


- How do we ensure results are always correct?
  - e.g., number of visitors
  - e.g., match of *advert clicks* and *page visits*
  - wrong results may mean lost revenue or penalties

- Failures should not lead to data loss or incorrect results because of duplicates

Fault tolerance & correctness

- **Execution Semantics:**
- **at least once:** ensure all operators see all events
  - Storm (another stream processor):
    - Replay stream in failure case (*acking* of individual records)
- **Exactly once:** ensure that operators do not perform duplicate updates to their state
  - **Flink: Distributed Snapshots**
  - Spark: Micro-batches of RDDs on batch runtime



Summary

- Map-Reduce analysis
- Spark: overview, RDDs
  - programming model, examples, RDD operations
  - fault-tolerance, performance
- Spark Streaming
  - overview, discretized stream processing
  - windows, sliding windows, micro-batching
- Flink
  - overview, windowing
  - tumbling windows, sliding windows, custom windows
  - time-based windows, watermarks
  - state management, distributed snapshots
  - fault tolerance, execution semantics

Reference: E.g. Spark RDD functions

Transformations (define a new RDD)	**Parallel actions/operations** (return a result to driver)
map filter sample union groupByKey reduceByKey join cache …	reduce collect count countByKey save …

Reference: E.g. M/R in Spark

- MapReduce data flow can be expressed using RDD transformations

```
res = data.flatMap(rec => myMapFunc(rec))
          .groupByKey()
          .map((key, vals) => myReduceFunc(key, vals))
```

Including combiners:

```
res = data.flatMap(rec => myMapFunc(rec))
          .reduceByKey(myCombiner)
          .map((key, val) => myReduceFunc(key, val))
```



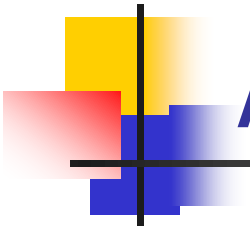
Cloud Datacenter Design and Operation

Scale and Energy Efficiency

Prof. Luís Veiga
IST / INESC-ID Lisboa

https://fenix.tecnico.ulisboa.pt/disciplinas/AVExe23/2023-2024/2-semestre/

© JG,LV 2015-24 partial source:J.Hamilton



Agenda

- Cloud Data center overview
  - Issues with Scale and Energy
- Energy Efficiency
  - Power usage efficiency
- Cooling
  - Chillerless DCs
- Optimizing Servers
  - Cooperative Expendable Micro-Slice (CEMS)
- Load handling
  - Load variations, Spot instances

Scale Matters

- When you scale from 1000 to 50000 CPUs prices change

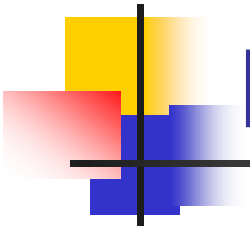
Technology	Cost in Medium-Sized DC	Cost in Very Large DC	Ratio
Network	$95 per Mbit/sec/month	$13 per Mbit/sec/month	7.1
Storage	$2.20 per GByte/month	$0.40 per Gbyte/month	5.7
Administration	≈ 140 Servers / Administrator	> 1000 Servers / Administrator	7.1

Huge DCs 5-7X as Cost Effective as Medium-Scale DCs

- 1000 scale (company):
  - Largest cost is personnel
    - (100 CPUs per sysadmin).
  - Goal: consolidation of services in integrated services.
- 50000 scale (cloud):
  - Largest cost is hardware. Followed by energy and cooling.
    - Cloud data centers measured in MW.
  - Personnel drops to under 10% (1000 CPUs per sysadmin).
  - Problem: Entry costs: 15MW cloud = 200M$.

The Cloud Scale

- The cloud scale shifts the focus of data center design and implementation:
- Before:
  - the main cost was **personnel**.
- At cloud scale:
  - the main cost is **hardware** and **cooling**.
- Therefore, the main metrics become:
  - **work/$** and
  - **work/joule**.

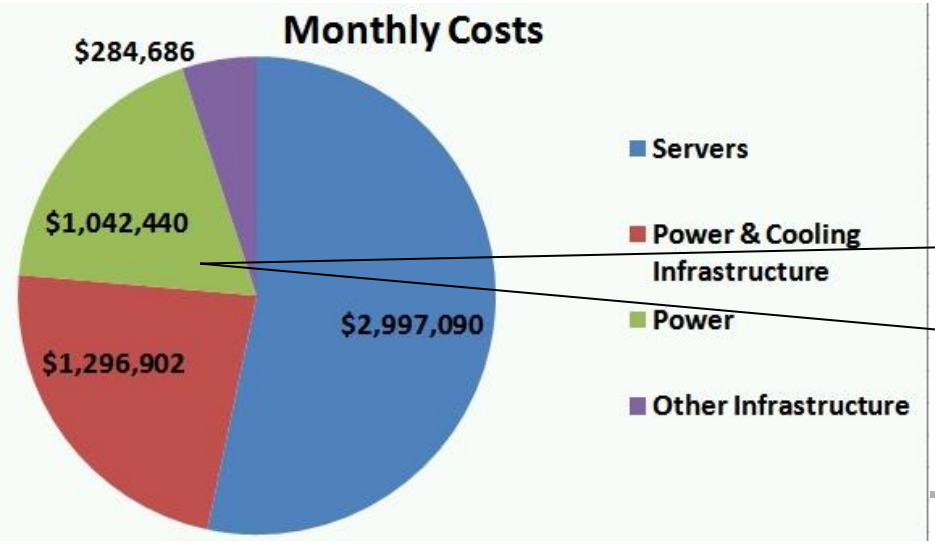


Energy

- Energy costs are a major concern.
- Energy is spent in:
  - Power conversion
  - Networking
  - Computation
  - Cooling
- Market players focus on:
  - general efficiency and
  - CPU power/load proportionality.

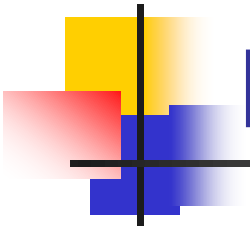
Energy's Impact on Costs

- How much does a cloud datacenter cost per year?
- Assumptions (prices change over time):
  - Infrastructure: 200 M$ for a 15MW cloud (amortized over 15 years).
  - Servers: 1.5k$/each, 50000 (amortized 3 years).
  - Power consumption at 30% of CPU load: 80% of max. specification.
  - Energy costs: 0,07$/kWh.



•2,3M$/month of energy related costs and it will get worse:

- *Energy prices rising.*
- *HW prices are decreasing.*



Energy's Impact on DC Location

- Typical criteria for data center placement, e.g. Google:
  - Large volumes of cheap electricity.
  - Green energy.
    - Focuses on renewable power sources.
  - Proximity to rivers and lakes.
    - They use a large amount of water for cooling purposes.
  - Large areas of land.
    - Allows for more privacy and security.
  - The distance to other Google data centers
    - (for fast connections between data centers).
  - Tax incentives.

Power Supply



High Voltage



115kV

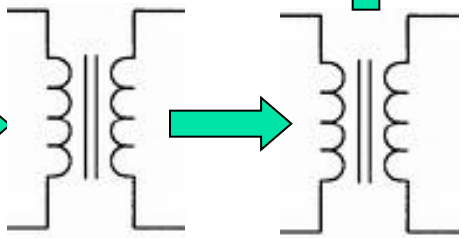
Medium Voltage



13,2kV

Three-phase Power





480V

208V



Diesel UPS

Efficiency of the Electrical Installation

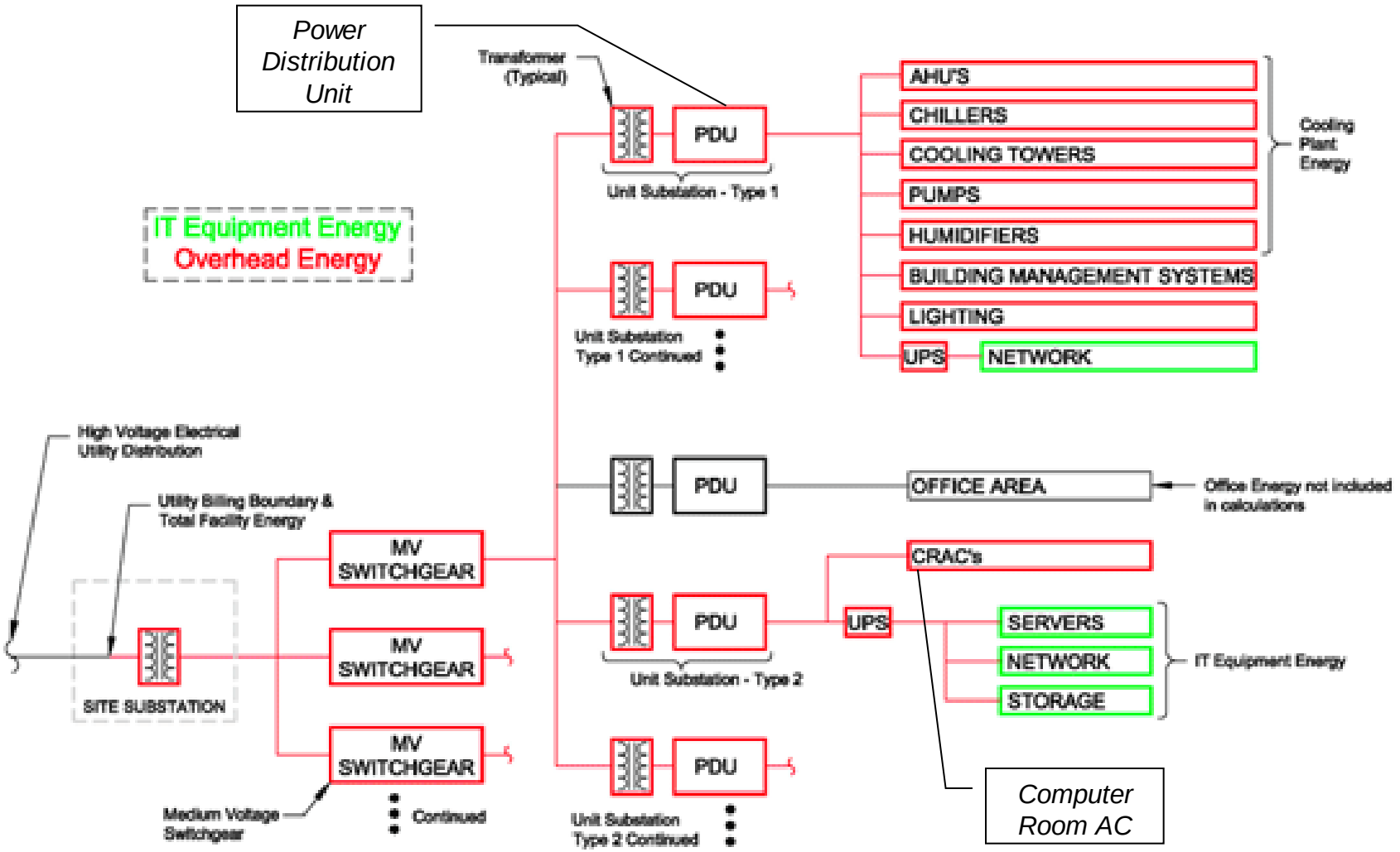
- Losses are very low!
  - From 115kV to 13.2kV: 0,3%
  - At the UPS/stabilization: 6%
  - From 13.2kV to 480V: 0,3%
  - From 480V to 208V: 0,3%
  - In the installation: 1%

  - Total: 0.997^3*0.94*0.99=**92.2%**

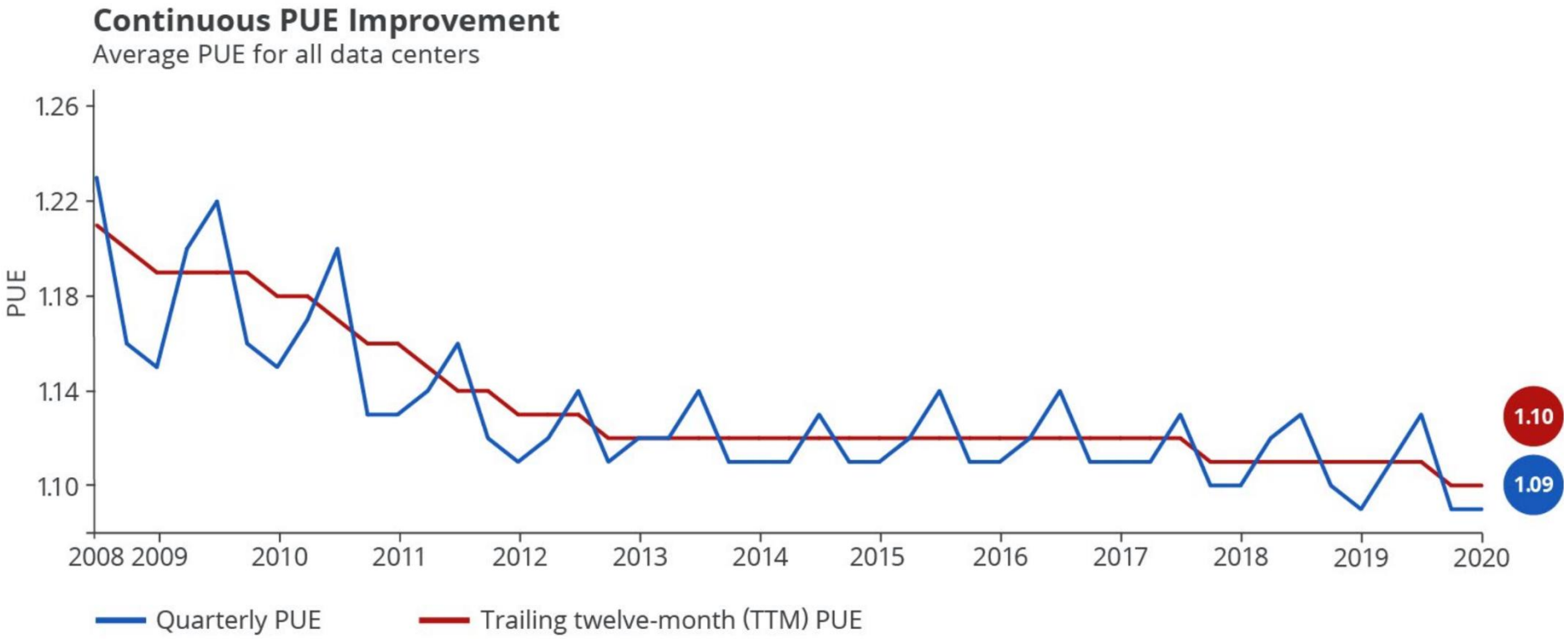
Energetic Efficiency

- 2 common metrics:
  - **PUE** (Power Usage Effectiveness):
    - PUE = Total Power / Power supplied to the servers
  - **DCiE** (Data Center Infrastructure Efficiency):
    - DCiE: inverse of PUE
  - Problems:
    - Where was total power measured? At what voltage?
    - The *box game* : Level at which server power is measured.
      - At the rack? At the server? What about the power supply? What if the fans are only in the rack?
- The ideal metric **tPUE**:
  - tPUE= Total Power / Energy supplied to critical components
    - (CPU, RAM, disk)
  - Not easy to implement/measure, but it can be an ideal metric

PUE Components at Google

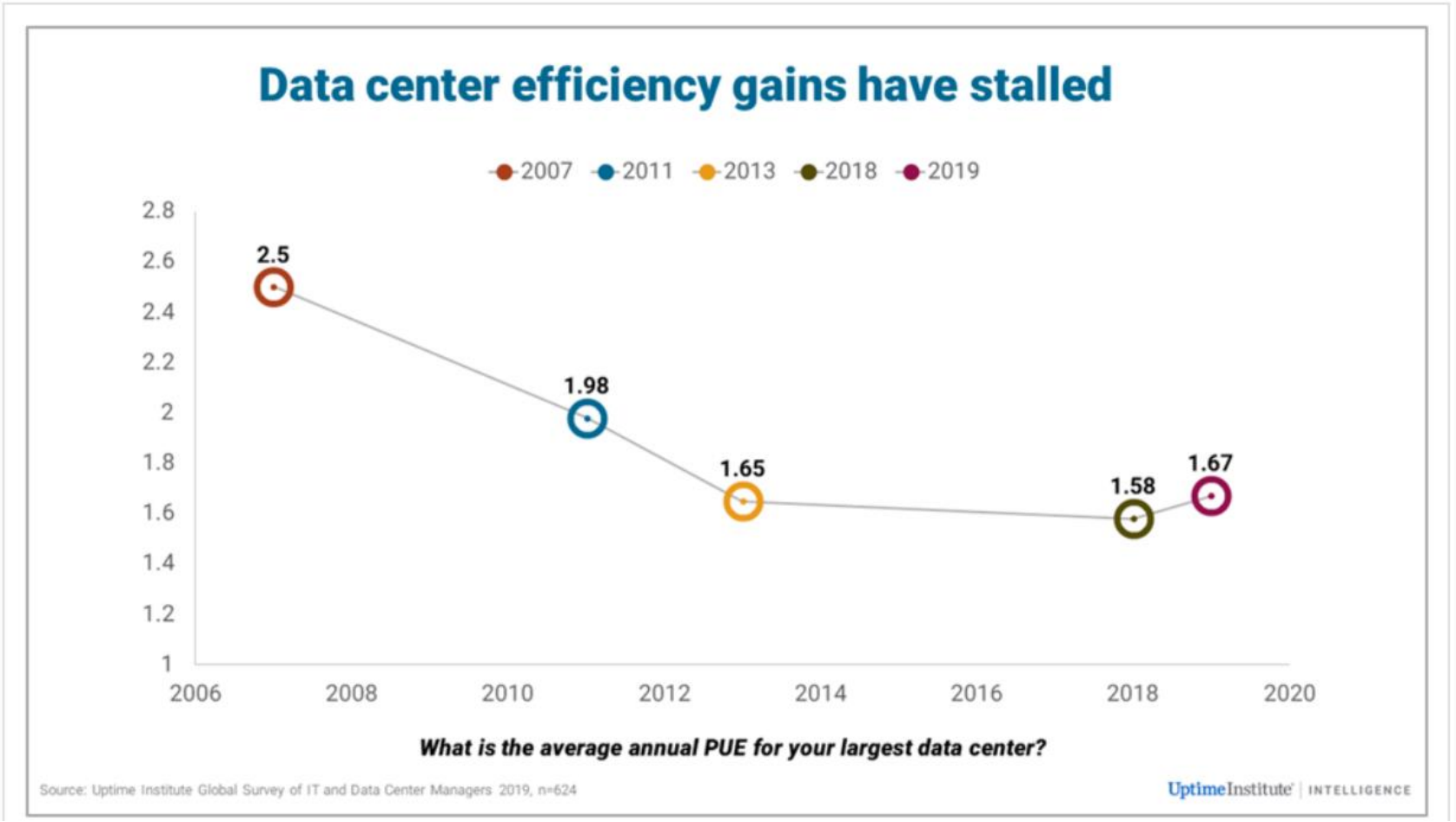


Historic Data From Google



- Q: Why the quarterly variations ?

Historic Data Overall Industry



- Overall PUE average still > 1.5 (i.e. 33% waste)

Where is the power going to?

- E.g. with average datacenter PUE ~1,7 [1.1–2.8]:
  - means only 59% DCiE
  - for each Watt that reaches the server:
    - 0.7 W are spent on power installation and on cooling.

- Measuring the power to non-computing electrical components is easy:
  - about 8% (100% minus 92.2% shown earlier)
  - remaining of wasted energy due to cooling
    - ~33% of total (100% - 59% - 8%)

Chiller-**less** Datacenters

- Chiller:
  - machine that uses electricity to remove heat from a liquid using the vapor-compression cycle.
    - e.g. fridge, freezer, air conditioning.

- **Chiller-less DC, e.g:**
  - Google's datacenter in Saint-Ghislain, Belgium.
  - DeepGreen DC, Lake Walensee, Switzerland
    - approaches followed by Microsoft, Amazon,…
      - e.g. Microsoft chiller-less DC in Dublin, Ireland.

E.g. Google's DC, Saint-Ghislain

- Key selection criteria:
  - low average year-round temperature.
  - close to civilization (maintenance, jobs, etc.)
  - on average, only 7 days per year above 26ºC.
  - cool refrigeration water with outside air.
  - when datacenter overloads:
    - Google moves load to other locations.

E.g. DeepGreen DC, Lake Walensee

- **Cool refrigeration water with ouside water.**

1. Intake pipe pulls 6ºC cold water from a depth of 60m from Lake Walensee, Switzerland

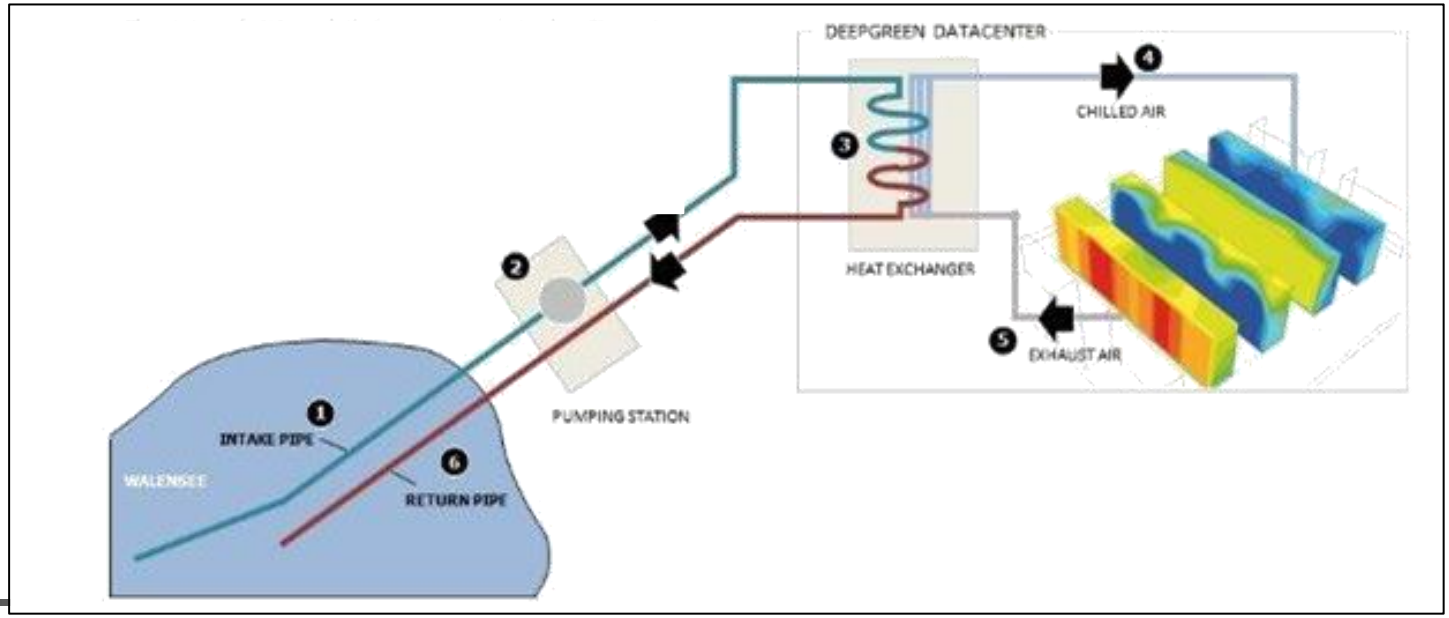
2. Lake water is filtrated and pumped through two redundant pipes to the DeepGreen datacenter.

3. Heat exchangers chill internal chilled water loop and take rejected heat back into return pipes.

4. Air cooled through computer room air handling units and ventilated into cold aisles on DC floor.

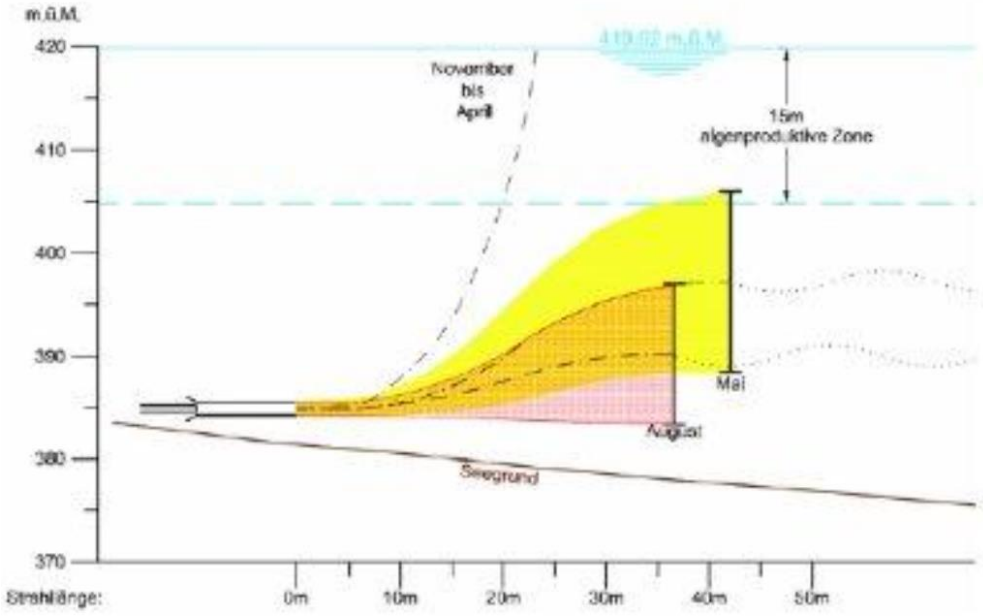
5. Hot return air is strictly separated and moved back to the computer room air handling units.

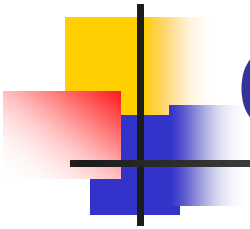
6. Heated return lake water pumped back at 14ºC and injected into Lake Walensee at 30m depth.



E.g. DeepGreen DC, Lake Walensee

- No impact on biological system:
  - two studies assessed negligible impact, possibly positive
  - heated return water run through mixing pipe to cool off
  - ejected at 14ºC at depth of 30m
    - below algae productive zone
    - only a cone of 45m sees temperature increase





Optimizing servers

- Memory speed has kept up with CPU speeds.

- In small installations:
  - aim is **consolidation**
    - (work/CPU because of CPU/sysadmin ratio)
  - **fast CPUs are used**.

- In large cloud datacenters:
  - goal is to **optimize work/J** and **work/$**:
    - **fast, expensive, hot CPU may not be ideal option.**

Optimizing Servers: CEMS

- E.g. of alternative servers:
  - **C**ooperative **E**xpendable **M**icro-**S**lice Servers (Amazon/Rackable project):

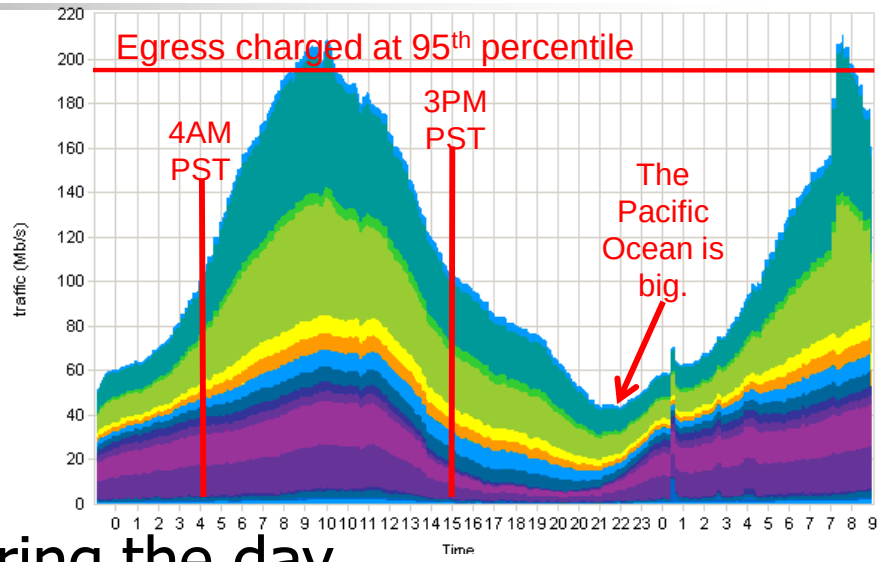
	Conventional	CEMS v3	CEMS v2	CEMS v1
Load	**56%**	57%	57%	61%
Req./sec. (RPS)	**95.9**	75.3	54.3	17.0
Price	$2371	**$500**	$685	$500
Power Cons.	295	60	39	**33**
RPS/$	0,04	**0,15**	0,08	0,03
RPS/joule	0,33	1,25	**1,39**	0,52
RPS/rack	1918,4	**18062,4**	13024,8	4080

Optimizing Servers: CEMS

- CEMS optimizes *work per J* and *work per $*, but:
  - obviously assumes services are fully partitionable.
    - suitable to current emerging micro-services, FaaS era.
  - but some are difficult to partition, e.g. relational DB.
  - software waste significant in OS load and licensing.
  - globally, average server load is low: 15% estimate.

- What's needed:
  - better occupation with dynamic resource management.
  - continue to aim for power/load proportionality:
    - ~0-10% loaded servers often pull 60% of power!

Load Handling: load variations

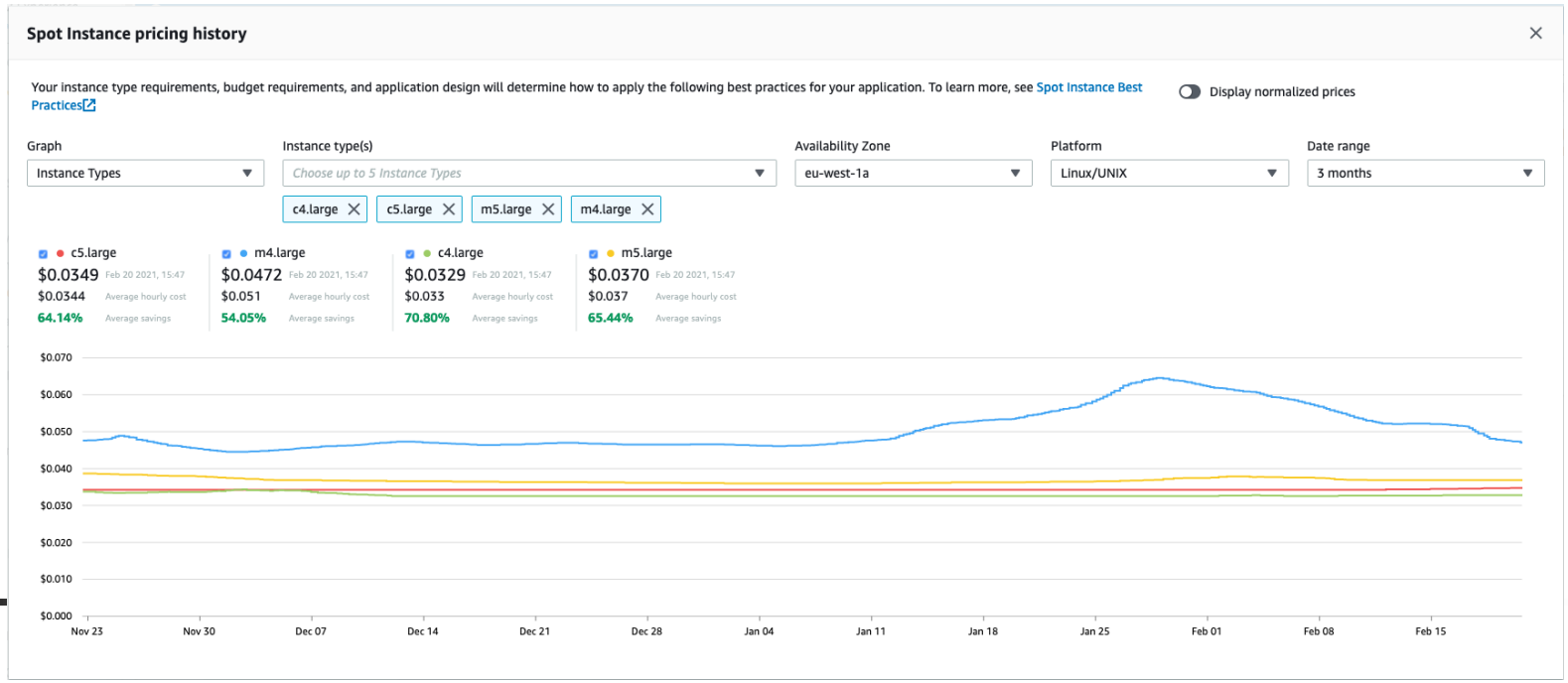


- Load has significant variations during the day.
  - base and peak load values determine power supply cost.

- Essential to push the "mountains" into the "valleys":
  - replicate data to other data centers during low load.
  - perform jobs without dealines during low load periods.
  - diferentiate prices base on urgency.
  - switch off or suspend machines during low load.

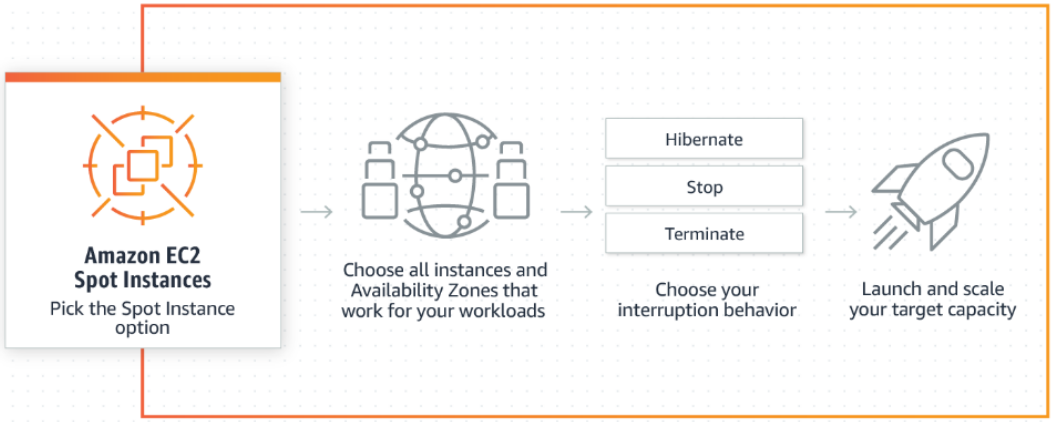
Spot Instances

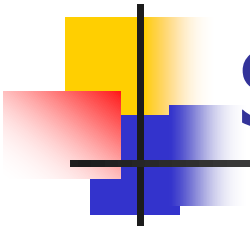
- AWS EC2 maintains Spot Instance exchange:
  - prices set by AWS adjusted gradually based on long-term trends in supply and demand for Spot Instance capacity.
- Spot instance requests submitted with a target price:
  - when offered price is met, instance is run.
  - often, 50-70% down from on-demand prices.



Spot Instances

- Spot Instances are subject to interruption:
  - when Spot price becomes higher than that paid by user.
  - when AWS capacity is no longer enough.
- Users specify instance interruption behaviour:
  - terminate: all CPU and storage resources freed.
  - stop: only CPU is freed, instance can be rebooted later.
  - hibernate: only CPU is freed, instance can be restored later and resumed faster without need of full reboot.
  - instances to stop/terminate are notified with 2-minute prior notice:
    - *spot instance interruption notice*
    - allows handling termination correctly/ gracefully.

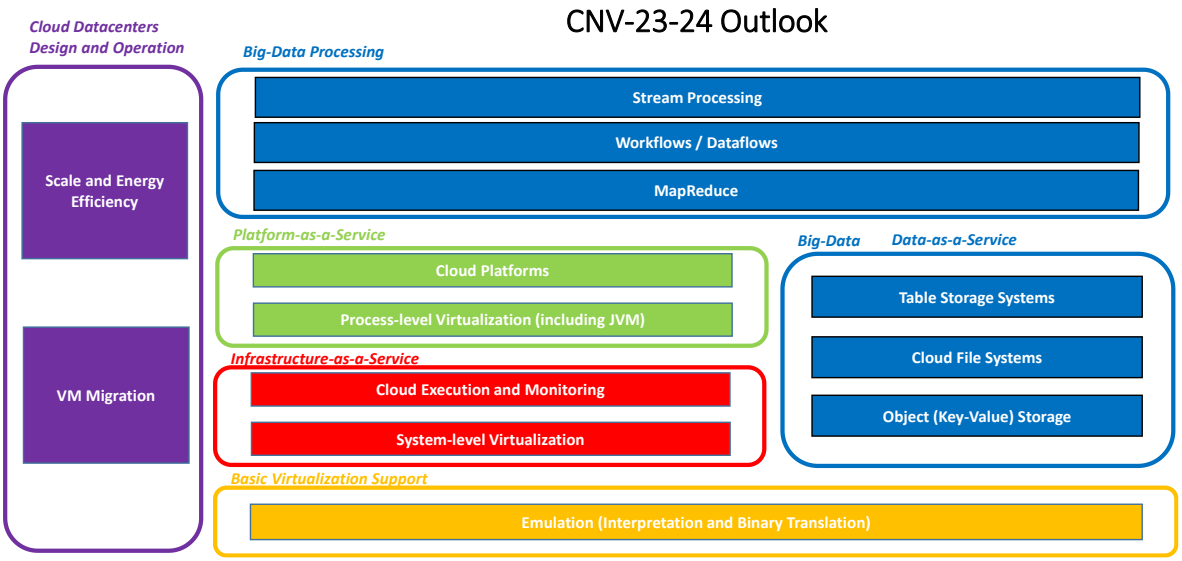




Summary

- Cloud Data center overview
  - Issues with Scale and Energy
- Energy Efficiency
  - Power usage efficiency
- Cooling
  - Chillerless DCs
- Optimizing Servers
  - Cooperative Expendable Micro-Slice (CEMS)
- Load handling
  - Load variations, Spot instances

Cloud Computing and Virtualization - 2023-2024
(MEIC/METI/MECD)
Quick Study Guide (*Incrementally Updated*)



This quick study guide recalls the course organization that was presented in lectures when each new topic was introduced. For each lecture, a small motivation is presented (*Why)* and the topics addressed (*What)* are listed.

For your convenience, there are also references where the topics are addressed in the books (CCB: Cloud Computing: Theory and Practice by Marinescu; VMB: Virtual Machines by Smith and Nair), lecture papers; and additional references when topics are more recent or not as detailed in the books, and also to refer to dedicated (e.g. initial, reference, practice) publications on some topics.

Note: All the content addressed in the lectures is included in the slides made available. They cover the entire topics discussed in lecture or given as individual post-class homework. *Numbers are omitted as they vary on each campus due to holidays.*

= 01: Introduction to Cloud Computing and Virtualization and Core Emulation Techniques =

Part A: Introduction to Cloud Computing and Virtualization

Why: All journeys have a beginning.

What: The cloud enablers, drivers, ecosystem overview; and the key architectural elements of the virtualization technology underneath.

Addressed in:

CCB: Sections 1-1.4; 4.10; 10.2;
above-the-clouds-berkeley.pdf paper.
VMB: Sections 1-1.2; 1.3.4; 1.4 – 1.4.1.

Part B: Core Emulation Techniques

Why: At the heart of any virtual machine in cloud computing there is an execution engine that is virtualizing an architecture or platform over the physical hardware (or even another virtualization layer), whose core task is performing emulation. This needs to be correct and fast to enable widespread usage.

What: The basic interpretation techniques and their improvements; binary translation for relevant performance and optimization of the predominant remaining overhead; comparison regarding key metrics.

Addressed in:

VMB: Sections 2-2.3; 2.5; 2.6.3; 2.7.1; 2.10.

= 02: System Virtualization (Cloud Resource Virtualization) =

Why: An instance deployed in an Infrastructure-as-a-Service cloud with full access to resources is a fully fledged virtual server, i.e., a guest VM in a System-level virtualization environment (System VM) that should strive for near native speed.

What: How the Virtual Machine Monitor multiplexes several guest VMs over the same physical resources; how the main resources of CPU and memory are virtualized for all software, including guest OS in each guest VM executing without the

required privileges, as each one no longer controls the hardware directly/exclusively; VM optimizations and CPU hardware support for near native performance.

Addressed in:

CCB: Sections 10.3 – 10.5 (until pp. 377).
VMB: Sections 8.1; 8.2 – 8.2.3; 8.3 – 8.3.2; 8.5.1; 8.7 – 8.7.3.

= 03. IaaS Cloud Execution and Monitoring (part A) =

Why: Infrastructure-as-a-Service (IaaS) is the predominant and most flexible cloud delivery model. Amazon Web Services were the first widespread offering of this approach. We may as well start with it.

What: The AWS Cloud Ecosystem and its main services; the Elastic Compute Cloud (EC2) public cloud service to provision and execute virtual service instances; support services for load balancing of requests and monitoring and auto-scaling of EC2 cloud resources (CloudWatch, Elastic Load Balancing, AutoScale).

Addressed in:

CCB: Sections 2.1 – 2.4; A.1 – A.4.

Additional references:

AWS:

https://docs.aws.amazon.com/AWSEC2/latest/APIReference/API_RunInstances.html
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/cloudwatch_concepts.html
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-cloudwatch-metrics.html
https://docs.aws.amazon.com/autoscaling/ec2/userguide/create-asg-launch-configuration.html
https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scale-based-on-demand.html

= 04. IaaS Cloud Execution and Monitoring (part B) =

Why: There are more clouds besides AWS, both public (e.g., Azure) as well as private, driven by open-source platforms (such as OpenStack) whose architectural details and algorithms are available. Furthermore, overall, there needs to be agreement on how to divide the responsibilities between providers and users/developers and to monitor and act upon their (un)fulfillment.

What: Overview of the Azure public cloud architecture; Overview of the OpenStack open-source private cloud architecture, its main services and how VM allocation and scheduling is carried out (Nova); Case-study on how to make use of public clouds to handle flash crowds, for different types of service deployments, without significant infrastructure investment upfront.

Addressed in:

CCB: Sections 2.6; 2.9; 2.10;
handling-flashcrowds-from-garage-atc.pdf paper.

Additional references:

OpenStack and Nova:

https://docs.openstack.org/contributors/en_GB/common/introduction.html
https://docs.openstack.org/nova/latest/admin/architecture.html
https://docs.openstack.org/nova/latest/admin/scheduling.html

AWS SLA:

https://aws.amazon.com/compute/sla/

= 05. High-level Language VMs Architecture (Java VM Architecture) =

Why: Developers design systems and services as application developers. An influential programming language is Java with its managed runtime (Java VM and its Specification) allowing for increased application portability and robustness. Runtimes such as the Java VM provide Process-level virtualization as basis for Platform-as-a-Service cloud delivery.

What: Overview of Java Virtual Machine Architecture and Specification; its data types; its byte-code instruction set and the core of bytecode emulation; its class organization; how errors, exceptions and synchronization are handled.

Addressed in:

VMB: Sections 5 - 5.1; 5.3 – 5.3.5.

Additional references:

Java Virtual Machine Instruction Set:
https://docs.oracle.com/javase/specs/jvms/se11/html/jvms-6.html

= 06. High-level Language VMs Implementation =

Why: The JVM specification is just a specification on paper; it needs to be implemented so that Java applications can run on it. Emulating a (byte-code) instruction-set that is not directly supported by any widely available hardware, and provide robustness and performance/efficiency, are key aspects that determined Java widespread adoption.

What: The core components of the Java VM implementation, its mechanisms, techniques, and algorithms: Security, Automatic Memory Management (Garbage Collection), the Emulation Engine and its optimizations in the JIT (*Just-in-Time*) compiler inside more advanced Java VM implementations.

Addressed in:

VMB: Sections 6.2 – 6.2.2; 6.3-6.3.5; 6.3.7; 6.5; 6.6 (except On-Stack Replacement, Optimizing GC).

= 07. Cloud Execution Platforms and Programming Models (Cloud PaaS) =

Why: Deploying and managing complete systems to the cloud is flexible but many developers just want/need to deploy application code to the cloud and want to manage as little as possible the underlying resources. In Platform-as-a-Service (PaaS) cloud delivery, providers leverage Process-level virtualization (e.g., instances of Java VM) to deploy and execute application components (e.g., classes, services, server pages, servlets, micro-services) in a managed way.

What: The Google App Engine application: model, services, scalability and billing; Microsoft Azure application model and its scalability; Integration of application instances using AWS Simple Queue Service, Azure Queues and the Google Pub/Sub.

Addressed in:

CCB: Sections 2.5; 2.6.

Additional references:

Google App Engine:
https://cloud.google.com/appengine/docs/standard/java/an-overview-of-app-engine
https://cloud.google.com/appengine/docs/standard/java11/how-requests-are-routed
https://cloud.google.com/appengine/docs/standard/java/config/webxml
https://cloud.google.com/appengine/docs/standard/java/config/appref#scaling_elements
https://cloud.google.com/appengine/docs/standard/java/how-instances-are-managed#scaling_dynamic_instances

Azure:
https://docs.microsoft.com/en-us/azure/security/fundamentals/isolation-choices#the-azure-fabric-controller
https://docs.microsoft.com/en-us/azure/azure-government/azure-secure-isolation-guidance
http://msdn.microsoft.com/en-us/magazine/dd727504.aspx
https://msdn.microsoft.com/en-us/library/microsoft.windowsazure.serviceruntime.roleenvironment.currentroleinstance.aspx

AWS Simple Queue Service:
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-basic-architecture.html
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/standard-queues.html
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html
https://docs.aws.amazon.com/code-samples/latest/catalog/javav2-sqs-src-main-java-com-example-sqs-SQSExample.java.html

Azure Queues:
https://docs.microsoft.com/en-us/azure/storage/queues/storage-queues-introduction
https://docs.microsoft.com/en-us/azure/storage/queues/storage-java-how-to-use-queue-storage?tabs=java
https://docs.microsoft.com/en-us/azure/storage/queues/storage-tutorial-queues?tabs=dotnet%2Cenvironment-variable-windows

Google Pub/Sub:
https://cloud.google.com/pubsub/pricing
https://cloud.google.com/pubsub/architecture
https://cloud.google.com/pubsub/docs/reference/libraries#client-libraries-install-java

= 08. Cloud Execution Platforms and Programming Models (Cloud FaaS) =

Why: The ability to deploy application components in a managed way, instead of complete virtual servers, is a major leap in simplicity for developers, and it allows more degrees of freedom for the providers to manage the infrastructure efficiently. But what if we could go even further? Function-as-a-Service (FaaS) allows deploying functionality in a more fine-grained manner, with the provider handling the scalability of each single application function (e.g. a lambda) individually, promising near-zero start-up times, fast and limitless scalability, in a true serverless fashion, even if for a price.

What: Function-as-a-Service cloud delivery model; Faas application and execution model; Main serverless platforms (AWS Lambda, Google Cloud Functions, Azure Functions); Case-studies of advanced usage of FaaS (e.g., video encoding and analytics); Optimization of FaaS platforms (cold-starts, saving memory, code efficiency across cluster of FaaS nodes).

Not yet addressed in the books.

Additional references:

AWS Lambda:
- https://aws.amazon.com/lambda/
- https://aws.amazon.com/lambda/getting-started/
- https://docs.amazonaws.cn/en_us/lambda/latest/dg/gettingstarted-concepts.html
- https://docs.aws.amazon.com/lambda/latest/dg/welcome.html
- https://docs.aws.amazon.com/lambda/latest/dg/lambda-dg.pdf#API_Reference
- https://aws.amazon.com/lambda/pricing/

Google Cloud Functions:
- https://cloud.google.com/functions
- https://cloud.google.com/functions/docs/create-deploy-http-java
- https://cloud.google.com/functions/pricing

Azure Functions:
- https://docs.microsoft.com/en-us/azure/azure-functions/functions-reference-python
- https://issuehint.com/issue/Azure/azure-functions-python-worker/849
- https://azure.microsoft.com/en-us/pricing/details/functions/

OpenWhisk:
- https://openwhisk.apache.org/documentation.html
- https://github.com/apache/openwhisk/blob/master/docs/about.md#how-openWhisk-works
- https://github.com/apache/openwhisk/blob/master/docs/reference.md#system-limits

Advance Case-Study papers:
- Encoding, Fast and Slow: Low-Latency Video Processing Using Thousands of Tiny Threads
- Lambada: Interactive Data Analytics on Cold Data Using Serverless Cloud Infrastructure
- Photons: Lambdas on a diet
- Initialize Once, Start Fast: Application Initialization at Build Time
- Isolates and Compressed References: More Flexible and Efficient Memory Management via GraalVM

= 09. Cloud Storage =

Part A: Introduction and Dynamo-based Cloud Storage

Why: Historically, computing platforms are only as good as the applications they support. Today, most relevant Internet-wide applications and services are defined by the increasing scale of the amounts of data they store, vast numbers of users accessing it, and the need to insert and retrieve data with good performance (i.e., latency, throughput). We start with AWS storage and its breakthrough Key-Value storage and products built upon it.

What: Cloud storage overview; cloud storage types; Dynamo system; AWS Simple Storage System; AWS DynamoDB.

Addressed in:

CCB: Sections 6.7; 6.11; A.5

Additional references:

Dynamo paper and DynamoDB:
- https://www.dynamodbguide.com/the-dynamo-paper/
- http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html

Part B: Part A: Google File System and Big-Table

Why: There are more cloud storage providers (Google and Azure). Moreover, in the cloud, the well-known file system and tabular (not fully relational) semantics are still useful and widely employed, but their implementation must be radically overhauled to provide unlimited scalability and competitive performance (i.e., latency, throughput). We continue with breakthrough Google File System/Hadoop File System and tabular storage (some built on top of it) with additional limited support for transactions.

What: Google File System/Hadoop File System; Google Big-Table tabular storage; Google Datastore/Megastore; Azure storage (reference).

Addressed in:

CCB: Sections 6.5; 6.6; 6.9; 6.10.

Additional references:

Hadoop File System:

https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html
Hadoop Definitive Guide, Tom White., O'Reilly.

BigTable:

http://usenix.org/event/osdi06/tech/chang/chang.pdf

Google Datastore:

https://cloud.google.com/datastore/docs/concepts/cloud-datastore-transactions
https://cloud.google.com/datastore/docs/cloud-datastore-best-practices

Azure storage:

https://docs.microsoft.com/en-us/java/api/com.azure.resourcemanager.storage.fluent.blobcontainersclient?view=azure-java-stable
https://docs.microsoft.com/en-us/java/api/com.azure.resourcemanager.storage.fluent.blobcontainersclient.lease?view=azure-java-stable
https://docs.microsoft.com/en-us/rest/api/storageservices/Understanding-the-Table-Service-Data-Model

= 10. Big-Data Processing: MapReduce =

Why: With vast amounts of data stored comes the possibility of carrying out relevant data processing at massive scales (not just inserting and retrieving) to generate new higher-value data. The Map-Reduce paradigm was the first large-scale data processing paradigm, developed by Google to process very large, complete, sequential file-based data (e.g. web crawls) in order to index the web, rank web pages, and perform searches on the web index.

What: Map-Reduce paradigm; Key example programs such as WordCount, PageRank; The Map-Reduce/Hadoop platform (architecture, distribution, task management, resource management, fault tolerance).

Addressed in:

CCB: Sections 7.5; 7.6 (up to pp. 259).

Additional references:

Map-Reduce/Hadoop:

https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html
https://research.google.com/archive/mapreduce-osdi04.pdf
Hadoop Definitive Guide, Tom White., O'Reilly.

= 11. Big-Data Processing: Dataflows with Stream Processing =

Why: Processing Big-data with Map-Reduce is powerful but with limited expressiveness and reuse. Furthermore, it cannot process data incrementally, as new input arrives, nor iteratively without resorting to disk operations. Spark improves expressiveness and supports iterative and interactive data processing by leveraging caching in memory. Spark Streaming supports incremental processing with micro-batching of arriving events. Flink provides streaming with low latency and improved window and event handling semantics.

What: Apache Spark; Apache Spark Streaming; Apache Flink (inspired in Google MillWheel).

Addressed in:

CCB: Sections 8.11; 12.6 (up to pp. 461)

Additional references:

Spark:

https://www.usenix.org/event/hotcloud10/tech/full_papers/Zaharia.pdf
https://www.usenix.org/system/files/conference/nsdi12/nsdi12-final138.pdf
https://spark.apache.org/docs/latest/rdd-programming-guide.html

Spark Streaming:

https://www.usenix.org/system/files/conference/hotcloud12/hotcloud12-final28.pdf
https://spark.apache.org/docs/latest/streaming-programming-guide.html

Flink:

https://asterios.katsifodimos.com/assets/publications/flink-deb.pdf
https://arxiv.org/pdf/1506.08603.pdf
https://research.google/pubs/pub41378.pdf

https://flink.apache.org/news/2015/12/04/Introducing-windows.html
https://nightlies.apache.org/flink/flink-docs-release-1.15/docs/concepts/time/
https://nightlies.apache.org/flink/flink-docs-release-1.15/docs/concepts/stateful-stream-processing/

https://asterios.katsifodimos.com/assets/publications/flink-deb.pdf
https://arxiv.org/pdf/1506.08603.pdf
https://research.google/pubs/pub41378.pdf

https://flink.apache.org/news/2015/12/04/Introducing-windows.html
https://nightlies.apache.org/flink/flink-docs-release-1.15/docs/concepts/time/
https://nightlies.apache.org/flink/flink-docs-release-1.15/docs/concepts/stateful-stream-processing/

= 12. Data Center Design: Scale and Energy Efficiency =

Why: At the giant cloud scale, everything changes. The cost model changes, and concerns about energy usage, cost and specifically energy spent in cooling become highly relevant. Energy efficiency can be assessed and improved resorting to innovative energy management, cooling, specially designed hardware, and managing load across peak and off-peak periods.

What: Cloud data center overview; Issues with scale and energy; Energy efficiency assessment; Cooling; Cooperative Expendable Micro-Slice hardware nodes; Handling and leveraging load variations.

Addressed in (not yet addressed in the books):

cooperative-expendable-micro-slice-servers.pdf paper

Additional references:

https://www.pingdom.com/blog/map-of-all-google-data-center-locations/
https://www.google.com/about/datacenters/locations/st-ghislain/
https://perspectives.mvdirona.com/2010/11/46mw-with-water-cooling-at-a-pue-of-1-10/
https://aws.amazon.com/ec2/spot/pricing/